Data Science sous R

Notes de cours — Ingé3 Big Data & Digital Marketing (MAJ101)

ESME — 2026-2027

Semestre S8

Table des matières

Avant-propos **5**
Objectifs du module . . . 5
Comment utiliser ces notes . . . 5
Logiciels et packages . . . 6
Les données du cours . . . 6
Bibliographie . . . 6

1 Introduction à la science des données **7**
1.1 Qu'est-ce que la science des données ? . . . 7
1.2 Science des données véridique . . . 9
1.3 Le cadre PCS : prédictibilité, calculabilité, stabilité . . . 10
1.4 Le cycle de vie d'un projet de science des données . . . 12
1.5 Résumé . . . 13
1.6 Exercices . . . 13

2 Introduction au langage R **19**
2.1 R : un peu d'histoire . . . 19
2.2 Introduction à RStudio . . . 19
2.3 Premiers pas : R comme calculatrice . . . 22
2.4 Variables et assignation . . . 23
2.5 Types de données . . . 24
2.6 Structures de données . . . 26
2.7 Opérateurs . . . 29
2.8 Accéder aux éléments et les modifier . . . 31
2.9 Structures algorithmiques . . . 33
2.10 Fonctions et programmation modulaire . . . 35
2.11 Packages et aide . . . 37
2.12 Résumé . . . 37
2.13 Exercices . . . 38

3 Manipulation de données avec R **45**
3.1 Les données de ce chapitre . . . 45
3.2 Importer et exporter des données . . . 47
3.3 Structure d'un tableau de données . . . 48
3.4 Analyse univariée . . . 49
3.5 Analyse bivariée . . . 52
3.6 Manipuler des données avec `dplyr` . . . 58
3.7 Données rangées (*tidy data*) . . . 69
3.8 Valeurs manquantes . . . 70
3.9 Générer des données . . . 71

3.10 Résumé . . . 73
3.11 Exercices . . . 73

4 Représentations graphiques et régression 93
4.1 Pourquoi visualiser ? . . . 93
4.2 Graphiques avec R de base . . . 95
4.3 Graphiques avec `ggplot2` . . . 96
4.4 Régression linéaire simple . . . 110
4.5 Régression linéaire multiple . . . 113
4.6 Régression logistique . . . 115
4.7 Résumé . . . 118
4.8 Exercices . . . 118

5 Analyse en composantes principales, arbres de décision et forêts aléatoires 131
5.1 Analyse en composantes principales (ACP) . . . 131
5.2 Arbres de décision . . . 140
5.3 Forêts aléatoires . . . 145
5.4 Résumé . . . 153
5.5 Exercices . . . 153

6 Classification automatique, éthique et R Shiny 161
6.1 Classification non supervisée . . . 161
6.2 Les k-moyennes (*k-means*) . . . 162
6.3 Classification ascendante hiérarchique (CAH) . . . 163
6.4 Choisir le nombre de classes . . . 165
6.5 Décrire les classes . . . 167
6.6 L'éthique dans la science des données . . . 170
6.7 Interfaces graphiques avec R Shiny . . . 172
6.8 Résumé . . . 174
6.9 Exercices . . . 174

7 Projet et évaluation des acquis 177
7.1 Déroulé d'un projet : le DSLC en pratique . . . 177
7.2 Le rendu : un notebook reproductible . . . 178
7.3 Grille d'auto-évaluation . . . 178
7.4 Réussir la soutenance . . . 179
7.5 Où trouver des jeux de données . . . 179
7.6 Fiche de révision . . . 180
7.7 Questions de révision . . . 181

Avant-propos

Ces notes accompagnent les supports de cours et les séances pratiques du module **Data Science sous R**. Elles ne remplacent pas le cours : elles servent à **revoir**, **approfondir** et **pratiquer** ce qui a été vu en séance. Les chapitres suivent l'organisation des supports de cours.

Objectifs du module

À la fin du module, vous devez être capable de :

- **AAV1** — nommer et reconnaître les types de données et les structures algorithmiques de R ;
- **AAV2** — lire, écrire, transformer et décrire des données de formats variés, et écrire du code R modulaire et lisible ;
- **AAV3** — produire des graphiques avec R (graphiques de base et `ggplot2`) ;
- **AAV4** — mettre en œuvre une analyse en composantes principales (ACP) et des arbres de décision ;
- **AAV5** — choisir une méthode adaptée à un problème de classification ;
- **AAV6** — évaluer et comparer des méthodes d'analyse factorielle, et décrire une application web R Shiny ;
- **AAV7** — concevoir et évaluer une chaîne de traitement complète pour répondre à un problème d'analyse de données.

Évaluation : contrôle continu 30 %, projet 50 %, examen final 20 %.

Comment utiliser ces notes

- Les blocs de code sont **exécutables** : recopiez-les dans RStudio et modifiez-les. On apprend la science des données en pratiquant.
- Les sorties de R sont précédées de `#>`.
- Chaque chapitre se termine par des **exercices**, avec des solutions repliables dans la version web.
- Trois types d'encadrés reviennent dans le texte :

> **À retenir** — les idées essentielles à savoir restituer.

> **Attention** — les pièges classiques et les erreurs fréquentes.

> **Exercice** — à faire pendant ou après la séance.

–> –> –> –> –> –> –>

Logiciels et packages

Installez R (version 4.3 ou plus récente) puis RStudio Desktop (voir le chapitre 2). Les packages utilisés dans le module s'installent en une seule commande :

```
install.packages(c(
  "tidyverse",      # dplyr, tidyr, readr, ggplot2, ...
  "questionr", "readxl", "nycflights13", "ISLR",
  "FactoMineR", "factoextra",
  "rpart", "randomForest", "pROC", "cluster", "shiny"
))
```

Les données du cours

Le dossier `datasets/` contient :

- `phbirths.dat` : poids de naissance de 1 115 bébés à Philadelphie ;
- `boutique_clients.csv`, `boutique_commandes.csv`, `boutique_categories.csv` et `boutique_campagne.csv` : les données **simulées** d'une boutique en ligne fictive (2 000 clients, environ 9 500 commandes en 2025, un test A/B d'emailing), fil rouge marketing des chapitres 3 à 6.

Ces fichiers sont téléchargeables depuis la version web des notes (section « Les données du cours »).

Les autres jeux de données proviennent de packages : `hdv2003` et `rp2018` (`questionr`), `nycflights13`, `Carseats`, `OJ` et `Caravan` (`ISLR`), `decathlon` (`FactoMineR`).

Bibliographie

- Bin Yu et Rebecca L. Barter, *Veridical Data Science: The Practice of Responsible Data Analysis and Decision Making*, MIT Press, 2024 (en accès libre sur vdsbook.com).
- Hadley Wickham, Mine Çetinkaya-Rundel et Garrett Grolemund, *R for Data Science*, 2e éd., O'Reilly, 2023 (r4ds.hadley.nz).
- Myriam Maumy-Bertrand et Frédéric Bertrand, *Initiation à la statistique avec R*, 3e éd., Dunod, 2018.
- Gilbert Saporta, *Probabilités, analyse des données et statistique*, 3e éd., Technip, 2011.
- François Husson, Sébastien Lê et Jérôme Pagès, *Analyse de données avec R*, 2e éd., Presses universitaires de Rennes, 2016.
- Henri Laude, *Data scientist et langage R*, ENI, 2016.
- Julien Barnier, *Introduction à R et au tidyverse* (juba.github.io/tidyverse).

Chapitre 1

Introduction à la science des données

Objectifs du chapitre

- Comprendre ce qu'est la science des données et à quoi elle sert, notamment en marketing.
- Savoir pourquoi un résultat issu de données doit être évalué avec un regard critique.
- Connaître le cadre PCS (prédictibilité, calculabilité, stabilité) et savoir découper des données en ensembles d'entraînement, de validation et de test.
- Connaître le cycle de vie d'un projet de science des données.

1.1 Qu'est-ce que la science des données ?

1.1.1 Une pratique ancienne, un nom récent

Utiliser des données pour répondre à des questions concrètes n'est pas nouveau : on parlait autrefois d'« analyse de données » ou de « statistique appliquée ». Le traitement automatisé des données commence avec les cartes perforées d'Herman Hollerith, conçues pour dépouiller le recensement américain de 1890 ; sa société est l'une des quatre qui fusionneront pour former IBM. Pour une histoire de la discipline, lire l'article de David Donoho, *50 Years of Data Science* (2017).

1.1.2 Définition

> La science des données unifie la statistique, l'analyse de données, l'informatique et leurs méthodes associées pour **comprendre et analyser des phénomènes réels à partir de données**.

Elle se situe à l'intersection de trois domaines :

- l'**informatique** : techniques de calcul, programmation, stockage ;
- les **mathématiques et la statistique** : modèles, inférence, incertitude ;
- l'**expertise du domaine** : connaissance du contexte de la question.

Un · e *data scientist* écrit du code, l'associe à des connaissances statistiques pour extraire de l'information des données, puis **communique** ses résultats à des publics variés afin d'éclairer une décision.

Deux exemples.

- *Santé* : un hôpital veut prédire quels patients risquent d'être réadmis après leur sortie. Le modèle transmet cette information aux médecins, qui décident de la sortie.

- *Marketing* : un opérateur de téléphonie veut repérer les abonnés qui risquent de résilier (*churn*). Le modèle produit une liste de clients à risque ; l'équipe de fidélisation décide à qui proposer une offre, et laquelle.

Dans les deux cas, le but final n'est pas « d'avoir un modèle », mais d'**aider une décision réelle**, prise par des personnes.

1.1.3 Quatre grands types d'analyse

Type	Question	Techniques	Exemple marketing
Descriptive	Que s'est-il passé ?	tableaux, statistiques, graphiques	tableau de bord des ventes par canal
Diagnostique	Pourquoi est-ce arrivé ?	exploration, corrélations, fouille de données	pourquoi les ventes ont-elles baissé en mars ?
Prédictive	Que va-t-il se passer ?	régression, arbres, apprentissage automatique	quels clients vont résilier ? quelles ventes à Noël ?
Prescriptive	Que faut-il faire ?	optimisation, simulation, recommandation	quel budget publicitaire par canal ? quel produit recommander ?

Ce module couvre surtout l'analyse descriptive (chapitres 3 et 4), les méthodes exploratoires (ACP et classification, chapitres 5 et 6) et quelques méthodes prédictives (régression, arbres de décision).

1.1.4 La science des données en marketing

Le marketing est l'un des domaines qui utilisent le plus les données :

- **connaissance client** : segmentation, valeur client (*customer lifetime value*), analyse RFM (récence, fréquence, montant) ;
- **acquisition** : ciblage publicitaire, attribution des ventes aux canaux (moteur de recherche, réseaux sociaux, emailing…) ;
- **fidélisation** : prédiction de l'attrition (*churn*), programmes de fidélité ;
- **offre** : recommandation de produits, prix, analyse du panier ;
- **expérimentation** : tests A/B (comparer deux versions d'un email, d'une page web, d'une offre sur des clients tirés au hasard).

Le fil rouge marketing de ces notes est une **boutique en ligne fictive** dont les données (clients, commandes, campagne d'emailing) sont simulées. Elles sont décrites en détail au chapitre 3, où elles sont utilisées pour la première fois.

1.1.5 Les projets réels sont exigeants

Un projet réel commence souvent par une **question floue** (« pourquoi nos ventes baissent-elles ? »), à laquelle on doit répondre avec des **données imparfaites** : ambiguës, erronées, et presque jamais collectées pour votre question. Les données d'un site de vente sont par exemple collectées pour facturer, pas pour comprendre les clients. L'analyse suit un chemin sinueux et aboutit parfois à des résultats peu concluants. C'est normal : l'enjeu est de rendre ces difficultés visibles plutôt que de les cacher.

Les projets échouent souvent pour les raisons suivantes :

1. décalage entre la vision métier et la vision technique ;
2. objectifs flous ;
3. données de mauvaise qualité ;
4. traitement des données incorrect ;
5. méthodes inadaptées à la question.

1.2 Science des données véridique

1.2.1 Les trois mondes

Yu et Barter (2024) proposent de distinguer trois « mondes » :

1. **la réalité**, que l'on cherche à comprendre ;
2. **les données**, qui n'en sont qu'une *approximation* (mesures partielles, bruitées, collectées dans un contexte donné) ;
3. **le monde des constructions mentales** : nos hypothèses, nos analyses et nos algorithmes.

Un résultat calculé à partir de données vit dans le troisième monde. Pour qu'il nous apprenne quelque chose sur la réalité, il faut justifier chaque passage d'un monde à l'autre. C'est pourquoi un résultat issu de données est une **preuve** (*evidence*) en faveur d'une conclusion, pas une **démonstration**.

Exemple : la colonne `nb_visites` d'un outil d'audience web ne compte pas des *personnes*, mais des sessions de navigateur. Une même personne sur son téléphone et son ordinateur compte deux fois ; une personne qui refuse les cookies ne compte pas. Les données approximent la réalité, avec des écarts qu'il faut connaître.

> **Science des données véridique** : mener des analyses en assumant les jugements humains qu'elles contiennent et en s'appuyant sur la connaissance du domaine, afin d'extraire et de communiquer des informations **utiles et fiables** pour résoudre un problème réel.

1.2.2 Pensée critique : les bonnes questions à se poser

Sur le problème

- La question formulée correspond-elle vraiment à ce qui nous intéresse ?
- La question est-elle éthique ?

Sur les données

- D'où viennent-elles ? La source est-elle fiable ?
- Sont-elles pertinentes pour la question et pour les situations où le résultat sera utilisé ?
- À quoi correspond, dans le monde réel, chaque valeur du tableau ?

Sur l'analyse et les résultats

- Que signifient les résultats dans la réalité ? Sont-ils plausibles ?
- Quelles hypothèses la méthode fait-elle ? Sont-elles raisonnables ?
- Les résultats répondent-ils à la question posée ?
- La façon de les communiquer est-elle fidèle ?

Quand c'est possible, allez **sur le terrain** (le *shoe leather* de D. Freedman) : parlez aux personnes qui produisent les données (vendeurs, service client), observez le processus de collecte.

1.2.3 Se méfier des biais

Biais cognitifs de l'analyste :

- **biais de confirmation** : on vérifie moins un résultat qui correspond à nos attentes ;
- **biais de désirabilité** : on cherche le résultat que l'on pense attendu par les autres (client, responsable, enseignant…).

On ne peut pas éliminer ces biais, même avec de l'entraînement. On peut en revanche cultiver un **scepticisme sain** envers *tous* les résultats, qu'ils confirment ou non nos attentes.

Biais dans les données, très fréquents en marketing :

- **biais de survie** : une enquête de satisfaction envoyée aux clients *actuels* ignore ceux qui sont partis, justement parce qu'ils étaient insatisfaits ;
- **biais de sélection** : les avis en ligne sont surtout laissés par des clients très satisfaits ou très mécontents ; leur moyenne ne représente pas l'ensemble des clients ;
- **biais de mesure** : un changement de l'outil de suivi du site peut créer une « hausse » des visites qui n'existe pas.

1.2.4 Étude de cas : les feux de brousse australiens (2019–2020)

Pendant la saison 2019–2020, des incendies ont ravagé plus de 42 millions d'acres (environ 17 millions d'hectares) en Australie. Le professeur Chris Dickman (Université de Sydney) a estimé que « plus d'un milliard d'animaux ont péri ». D'où vient ce chiffre ?

- **Les données** : il provient d'un rapport de 2007 pour le WWF sur l'impact du défrichement en Nouvelle-Galles du Sud. Le chiffre inclut mammifères (hors chauves-souris), oiseaux et reptiles, mais ni les grenouilles, ni les insectes, ni les autres invertébrés.
- **L'analyse** : on a estimé des densités d'animaux par hectare dans une zone limitée, puis multiplié par la surface brûlée. Cette extrapolation ignore les différences de densité entre régions et leur évolution dans le temps.

Le chiffre n'est pas « faux » : c'était la meilleure estimation possible avec les données disponibles, produite par des personnes de bonne foi. Mais il est **très incertain**, et cette incertitude doit être communiquée.

La même démarche s'applique à un chiffre marketing comme « notre campagne a généré 2 millions d'euros de ventes » : quelles ventes sont comptées ? Celles qui auraient eu lieu sans la campagne sont-elles exclues ? Comment a-t-on extrapolé ?

Cette étude de cas est approfondie dans l'exercice 1.3 en fin de chapitre.

1.3 Le cadre PCS : prédictibilité, calculabilité, stabilité

Le cadre PCS donne des lignes directrices pour **évaluer empiriquement la fiabilité** des résultats à chaque étape d'un projet. Prédictibilité et stabilité fournissent des *indices* de fiabilité ; elles ne la prouvent pas, car il faudrait pour cela tester tous les scénarios possibles. L'esprit est proche de la **réfutabilité** de Karl Popper : un résultat gagne en crédibilité quand il résiste à des tentatives sérieuses de le mettre en défaut.

1.3.1 Prédictibilité

Un résultat est **prédictible** s'il se retrouve dans de nouveaux contextes pertinents. La meilleure preuve consiste à le vérifier sur des données **futures ou externes**. À défaut, on simule ces données futures en découpant le jeu de données :

- **entraînement** : explorer les données et ajuster les modèles ;
- **validation** : comparer les modèles et choisir les réglages ; elle joue le rôle de « données futures » ;
- **test** : évaluation finale, une seule fois, sur des données jamais vues.

Le découpage doit refléter la relation entre les données actuelles et celles auxquelles le résultat sera appliqué :

Situation	Découpage	Exemple marketing
Observations interchangeables	**aléatoire**	visiteurs d'un site tirés au hasard pour étudier ce qui mène à un achat
Données ordonnées dans le temps	**temporel** : le passé pour entraîner, le plus récent pour valider et tester	prévoir les ventes : entraîner sur 2023–2024, valider sur début 2025, tester sur fin 2025
Groupes naturels, application à un nouveau groupe	**par groupes** : un groupe entier va dans un seul ensemble	prévoir les ventes d'un **nouveau magasin** : chaque magasin est entièrement en entraînement, en validation ou en test

En R, un découpage aléatoire simple (60 % / 20 % / 20 %) s'écrit :

```
set.seed(2026)                  # rend le tirage reproductible
n <- 100                        # nombre d'observations
ensemble <- sample(c("entrainement", "validation", "test"),
                   size = n, replace = TRUE, prob = c(0.6, 0.2, 0.2))
table(ensemble)
```

```
#> ensemble
#> entrainement         test   validation
#>           63           19           18
```

Fuite de données (*data leakage*) : de l'information sur la réponse ou sur les données de test « fuit » dans l'entraînement. Les performances mesurées sont alors trop optimistes, et le modèle déçoit une fois déployé. Exemples :

- prédire la résiliation d'un abonnement avec la variable « nombre d'appels au service résiliation », qui n'est connue… qu'une fois le client parti ;
- normaliser les données avec la moyenne calculée sur **tout** le jeu de données, test compris ;
- explorer le jeu de test avant de choisir le modèle.

Faites le découpage avant toute exploration, et vérifiez que chaque variable explicative est réellement disponible **au moment où la prédiction sera faite**.

Data snooping : tester un grand nombre d'hypothèses ou de modèles sur les mêmes données jusqu'à « trouver » un résultat. Si l'on compare 20 objets d'email sans réelle différence, l'un d'eux paraîtra « significativement » meilleur au seuil de 5 % dans près de deux cas sur trois ($1 - 0{,}95^{20} \approx 0{,}64$), par pur hasard. Pour l'éviter : formuler les questions à l'avance, garder un jeu de test intact, et rapporter tout ce qui a été essayé.

1.3.2 Calculabilité

La **calculabilité** concerne le calcul lui-même : l'analyse doit être réalisable avec les ressources disponibles, **reproductible** (même code et mêmes données donnent le même résultat) et docu-

mentée. En pratique :

- écrire le code dans des scripts ou des notebooks, jamais seulement dans la console ;
- fixer la graine des tirages aléatoires (`set.seed()`) ;
- noter les versions de R et des packages (`sessionInfo()`) ;
- utiliser des chemins relatifs et des projets RStudio (chapitre 2).

1.3.3 Stabilité

Un résultat est **stable** s'il change peu lorsqu'on applique des **perturbations raisonnables** à chaque étape :

- à la collecte (autre échantillon : rééchantillonnage *bootstrap*) ;
- au nettoyage (autre façon de traiter les valeurs manquantes ou aberrantes) ;
- à la modélisation (autre algorithme, autres paramètres).

On distingue l'incertitude **aléatoire** (liée à l'échantillonnage) et l'incertitude **épistémique** (liée à nos choix et à nos modèles).

Exemple : une segmentation de clientèle qui change complètement quand on relance l'algorithme avec une autre graine, ou quand on exclut les 1 % de clients aux plus gros montants, ne peut pas servir de base à une stratégie marketing.

1.4 Le cycle de vie d'un projet de science des données

1.4.1 Le DSLC

Le cycle de vie de la science des données (*data science life cycle*, DSLC) décrit six étapes. Illustrons-les avec la question « pourquoi les ventes de la boutique en ligne baissent-elles ? ».

1. **Formuler** un problème du domaine, poser une question précise et collecter des données. *Quelles ventes (chiffre d'affaires, nombre de commandes) ? Comparées à quelle période ? Données : commandes, clients, campagnes, trafic du site.*
2. **Nettoyer, prétraiter et explorer** les données. *Commandes en double, remboursements, effet de saisonnalité.*
3. *(Optionnel)* **Explorer la structure intrinsèque** des données. *Existe-t-il des segments de clients dont le comportement a changé ?*
4. **Modéliser** : entraîner un algorithme prédictif ou mener une analyse inférentielle. *Modèle des ventes en fonction des prix, de la publicité, de la saison.*
5. **Évaluer** les résultats (cadre PCS). *Le modèle prédit-il bien les mois récents ? Les conclusions tiennent-elles avec un autre traitement des remboursements ?*
6. **Communiquer** les conclusions et mettre à jour les connaissances du domaine. *Présentation à l'équipe marketing, avec limites et recommandations.*

Ce cycle n'est **pas linéaire** : on revient souvent en arrière (une anomalie découverte à l'étape 4 oblige à revoir le nettoyage). Tous les projets ne parcourent pas toutes les étapes. Les jugements faits pendant le nettoyage influencent les résultats ; ils doivent donc être documentés. Les experts du domaine doivent être consultés **tout au long** du projet, pas seulement au début.

Quand un résultat est **mis en production**, il est intégré dans un outil utilisé en continu : système de recommandation, score de risque de résiliation recalculé chaque semaine, tableau de bord. Il doit alors être surveillé : les données de demain peuvent différer de celles d'aujourd'hui, même si elles sont collectées de la même façon. Les confinements de 2020 ont par exemple bouleversé du jour au lendemain les habitudes d'achat, et donc la qualité de nombreux modèles de prévision.

1.4.2 CRISP-DM

Dans l'industrie, la démarche de référence pour la fouille de données (*Knowledge Discovery in Databases*, KDD) est **CRISP-DM** (*Cross-Industry Standard Process for Data Mining*, 1999). Elle comporte six phases : compréhension du métier, compréhension des données, préparation des données, modélisation, évaluation et déploiement. On y retrouve la même logique itérative que dans le DSLC.

1.4.3 Gouvernance des données

La **gouvernance des données** désigne les règles qui protègent les données et les personnes qu'elles décrivent : qui peut accéder à quoi, comment les données sont stockées et pendant combien de temps, sur quelle base légale elles sont traitées (en Europe, le RGPD). En marketing, où l'on traite des données personnelles de clients, ces questions sont centrales ; elles sont reprises au chapitre 6.

1.5 Résumé

- La science des données vise à **aider une décision réelle** ; elle combine informatique, statistique et connaissance du domaine.
- En marketing : segmentation, ciblage, attrition, recommandation, tests A/B.
- Un résultat issu de données est une **preuve**, pas une démonstration : il faut l'évaluer avec le cadre **PCS** (prédictibilité, calculabilité, stabilité).
- Se méfier des biais de l'analyste (confirmation, désirabilité) et des biais des données (survie, sélection, mesure).
- Découper les données (entraînement / validation / test) **avant** toute exploration, selon un découpage adapté (aléatoire, temporel, par groupes) ; éviter la fuite de données et le *data snooping*.
- Le cycle de vie d'un projet est **itératif**.

1.6 Exercices

Exercice 1.1 — Vrai ou faux ? Pour chaque affirmation, indiquez si elle est vraie ou fausse et justifiez brièvement votre réponse.

a. Les données sont toujours une bonne approximation de la réalité.
b. Avec de l'entraînement, il est possible d'éliminer le biais de confirmation.
c. Les résultats calculés à partir d'un jeu de données collecté aujourd'hui peuvent ne pas s'appliquer à des données qui seront collectées à l'avenir, même si les deux jeux de données sont collectés selon le même mécanisme.
d. Dès que votre analyse répond à votre question de domaine, vous avez répondu de manière concluante.
e. Il existe plusieurs façons de démontrer la prédictibilité de vos résultats.
f. Il existe de nombreuses façons de formuler un problème de science des données.
g. Les évaluations de prédictibilité, de calculabilité et de stabilité (PCS) ne sont nécessaires que lorsque vous présentez vos résultats à un public externe.
h. Une fois que vous avez avancé à une nouvelle étape du cycle de vie de la science des données (DSLC), il est acceptable de revenir à des étapes précédentes et de les mettre à jour.
i. Il n'existe qu'une seule bonne façon de mener chaque analyse de données.
j. Chaque projet de science des données doit passer par toutes les étapes du DSLC.
k. Les jugements effectués lors de l'étape de nettoyage des données peuvent influencer

les résultats ultérieurs.

l. La communication avec les experts du domaine n'est importante que pendant la phase de formulation du problème.

Éléments de réponse

a. **Faux.** Les données ne sont qu'une approximation, partielle et bruitée, de la réalité, collectée dans un contexte donné (les « trois mondes »). Leur qualité doit toujours être examinée.
b. **Faux.** On ne peut pas éliminer ce biais ; on peut seulement le limiter en vérifiant **tous** les résultats avec le même scepticisme, qu'ils confirment ou non nos attentes.
c. **Vrai.** Le monde change : comportements, prix, contexte (les confinements de 2020 ont bouleversé les habitudes d'achat). Un résultat mis en production doit être surveillé.
d. **Faux.** Une réponse n'est qu'une **preuve** en faveur d'une conclusion. Il faut encore évaluer sa prédictibilité et sa stabilité, et communiquer son incertitude.
e. **Vrai.** Vérification sur des données futures ou externes, sur un jeu de validation ou de test, avec différents découpages, par réplication de l'étude…
f. **Vrai.** Une question floue (« pourquoi les ventes baissent-elles ? ») peut se traduire en plusieurs questions précises, mesurées de différentes façons (chiffre d'affaires, nombre de commandes, clients actifs…).
g. **Faux.** Le cadre PCS s'applique **à chaque étape** du projet, d'abord pour soi-même : c'est ce qui permet de faire confiance à ses propres résultats.
h. **Vrai.** Le DSLC est **itératif** : une anomalie découverte pendant la modélisation oblige souvent à revoir le nettoyage, voire la question.
i. **Faux.** De nombreux choix raisonnables existent à chaque étape (nettoyage, méthode, paramètres). Explorer ces alternatives, c'est précisément évaluer la **stabilité**.
j. **Faux.** Certaines étapes sont optionnelles (l'exploration de la structure intrinsèque) ; un projet descriptif n'a pas forcément de modèle prédictif.
k. **Vrai.** Le traitement des valeurs manquantes ou aberrantes, par exemple, peut changer les conclusions. Ces jugements doivent être documentés et leur influence testée.
l. **Faux.** Les experts du domaine sont utiles **tout au long** du projet : pour comprendre ce que signifient les données, juger de la plausibilité des résultats et les communiquer.

Exercice 1.2 — Preuve et incertitude.

1. Expliquez pourquoi un résultat dérivé de données doit être considéré comme une **preuve** plutôt qu'une **démonstration** d'un phénomène réel.
2. Décrivez au moins deux sources d'incertitude associées à tout résultat fondé sur les données.
3. Décrivez deux techniques (quantitatives ou qualitatives) qui permettent de renforcer la crédibilité d'un résultat fondé sur les données.

Éléments de réponse

1. Un résultat est calculé dans le « monde des constructions mentales », à partir de données qui n'approximent la réalité que partiellement, et à travers des choix humains (nettoyage, méthode). Il **soutient** une conclusion sans la garantir : une démonstration supposerait d'avoir testé tous les scénarios de données et d'analyse possibles, ce qui est impossible.
2. Par exemple : l'**échantillonnage** (avec un autre échantillon, on aurait obtenu un

autre résultat), la **mesure** (erreurs de saisie, définition des variables), les **choix de nettoyage** (traitement des valeurs manquantes) et les **choix de modélisation** (méthode, paramètres).
3. **Quantitatives** : vérifier la prédictibilité sur un jeu de test ou des données externes ; tester la stabilité en perturbant les données (*bootstrap*) ou les choix d'analyse. **Qualitatives** : confronter le résultat aux experts du domaine, vérifier sa plausibilité, aller observer le terrain.

Exercice 1.3 — Les feux de brousse australiens.

1. Énumérez au moins deux raisons pour lesquelles les calculs du professeur Dickman pourraient ne pas refléter fidèlement le nombre réel d'animaux morts lors des incendies de brousse en Australie.
2. Imaginez que vous disposiez de ressources illimitées. Décrivez les données hypothétiques que vous collecteriez, et la manière dont vous les collecteriez, afin de répondre à la question du nombre d'animaux morts avec la plus grande précision possible. Soyez précis : quelles valeurs mesureriez-vous, et comment les mesureriez-vous physiquement ?

Éléments de réponse

1. Les densités d'animaux proviennent d'une étude de 2007 sur une zone limitée de Nouvelle-Galles du Sud : elles varient d'une région à l'autre et ont pu évoluer depuis. Le calcul suppose que tous les animaux présents sur les surfaces brûlées sont morts, alors que certains ont pu fuir ou survivre ; à l'inverse, des animaux ont pu mourir après les feux (faim, perte d'habitat). Enfin, le chiffre exclut grenouilles, insectes et invertébrés.
2. Par exemple : avant les feux, des inventaires de faune par espèce dans des parcelles tirées au hasard dans chaque type de végétation et chaque région (pièges photographiques, transects, captures-recaptures, relevés acoustiques pour les oiseaux) ; après les feux, les mêmes relevés sur les mêmes parcelles, brûlées ou non, pour estimer la mortalité par comparaison ; cartographie précise des surfaces brûlées par satellite ; suivi dans le temps pour mesurer la mortalité différée.

Exercice 1.4 — Prédictibilité, stabilité et pièges.

1. Quel est le rôle du jeu de données de **validation** dans l'évaluation de la prédictibilité d'un résultat fondé sur les données ?
2. Décrivez une technique permettant d'évaluer la **stabilité** d'un résultat.
3. Décrivez ce qu'est une **fuite de données** (*data leakage*) et comment l'éviter.
4. Décrivez ce qu'est le ***data snooping*** (exploration abusive des données) et comment l'éviter.

Éléments de réponse

1. Il joue le rôle de **données futures** : il n'est pas utilisé pour ajuster les modèles, ce qui permet de mesurer honnêtement leur performance et de choisir entre plusieurs modèles ou réglages. Le jeu de test sert ensuite à une évaluation finale.
2. Par exemple, le **bootstrap** : refaire l'analyse sur de nombreux échantillons tirés avec remise dans les données, et regarder si le résultat varie beaucoup. On peut aussi refaire l'analyse avec d'autres choix de nettoyage, une autre méthode ou une autre graine aléatoire.

3. De l'information sur la réponse ou sur les données de test se retrouve dans l'apprentissage (variable connue seulement après l'événement à prédire, normalisation calculée sur toutes les données…), ce qui rend les performances trop optimistes. On l'évite en découpant les données **avant toute exploration** et en n'utilisant que des variables disponibles au moment de la prédiction.
4. Tester de nombreuses hypothèses sur les mêmes données jusqu'à en trouver une « significative » par hasard. On l'évite en fixant les questions à l'avance, en gardant un jeu de test intact et en rapportant tout ce qui a été essayé.

Exercice 1.5 — Choisir un découpage. Quelle technique de séparation (par groupes, temporelle ou aléatoire) serait la plus appropriée pour créer les ensembles d'entraînement, de validation et de test dans chacun des scénarios suivants ?

a. Utiliser des données météorologiques historiques pour développer un algorithme de prévision du temps.
b. Utiliser des données provenant d'enfants de 20 écoles parisiennes pour évaluer la relation entre la taille des classes et les résultats scolaires dans d'autres écoles de Paris.
c. Utiliser les données d'élections précédentes pour développer un algorithme de prédiction des résultats d'une élection à venir.
d. Utiliser les données d'un sous-ensemble aléatoire des visiteurs d'une boutique en ligne pour identifier les caractéristiques associées à l'achat d'un produit.

Éléments de réponse

a. **Temporelle** : on prévoit le futur à partir du passé ; les années les plus récentes servent à la validation et au test.
b. **Par groupes** (écoles) : le résultat doit s'appliquer à d'autres écoles ; toutes les classes d'une même école vont dans le même ensemble.
c. **Temporelle** : les élections les plus anciennes pour l'entraînement, les plus récentes pour la validation et le test.
d. **Aléatoire** : les visiteurs tirés au hasard sont à peu près interchangeables.

Exercice 1.6 — Cycle de vie.

1. Que signifie le fait qu'un résultat ou une analyse soit « mis en production » ?
2. Décrivez brièvement chacune des étapes du cycle de vie de la science des données (DSLC).

Éléments de réponse

1. Le résultat est intégré dans un outil utilisé en continu, souvent de façon automatique (score de risque recalculé chaque semaine, système de recommandation, tableau de bord). Il doit alors être surveillé, car les données futures peuvent différer de celles qui ont servi à le construire.
2. (1) Formuler la question du domaine et collecter les données ;
   (2) nettoyer, prétraiter et explorer les données ; (3) explorer éventuellement leur structure intrinsèque ; (4) modéliser ou mener une analyse inférentielle ; (5) évaluer les résultats avec le cadre PCS ;
   (3) communiquer les conclusions et mettre à jour les connaissances du domaine. Le cycle est itératif.

Exercice 1.7 — Découpage en marketing. Pour chacun des projets suivants, proposez un découpage entraînement / validation / test et justifiez-le.

1. Prédire le chiffre d'affaires hebdomadaire d'une enseigne pour les trois prochains mois.
2. Prédire le panier moyen des clients d'un magasin qui va ouvrir à Lille, à partir des données de 40 magasins existants.
3. Prédire, parmi les visiteurs d'un site, ceux qui vont acheter, à partir d'un échantillon aléatoire de visites.

Éléments de réponse

1. Découpage **temporel** : on entraîne sur les semaines les plus anciennes et on valide et teste sur les plus récentes. Un découpage aléatoire laisserait le modèle « voir le futur ».
2. Découpage **par groupes** (magasins) : chaque magasin est entièrement dans un seul ensemble, pour mesurer la performance sur des magasins jamais vus.
3. Découpage **aléatoire** : les visites tirées au hasard sont à peu près interchangeables. Attention toutefois : si un même visiteur apparaît plusieurs fois, il vaut mieux découper par visiteur.

Exercice 1.8 — Fuite de données. Une banque veut prédire, au moment de l'ouverture d'un compte, si le client souscrira une assurance dans l'année. Le fichier contient : âge, revenu, canal d'ouverture, montant des primes payées, nombre de contacts avec un conseiller, date de souscription. Quelles variables posent problème ? Pourquoi ?

Éléments de réponse

Le **montant des primes payées** et la **date de souscription** n'existent que si le client a souscrit : ils révèlent la réponse. Le **nombre de contacts avec un conseiller** est ambigu : s'il est compté sur toute l'année, il inclut des contacts postérieurs à l'ouverture, voire liés à la souscription elle-même. Seules les informations disponibles **à l'ouverture du compte** doivent être utilisées.

Pour aller plus loin

- B. Yu et R. Barter, *Veridical Data Science*, chapitres 1 et 2 (vdsbook.com).
- D. Donoho, « 50 Years of Data Science », *Journal of Computational and Graphical Statistics*, 26(4), 2017.
- R. Kohavi, D. Tang et Y. Xu, *Trustworthy Online Controlled Experiments: A Practical Guide to A/B Testing*, Cambridge University Press, 2020.

Chapitre 2

Introduction au langage R

Objectifs du chapitre

- Prendre en main RStudio : ses panneaux, les scripts, les projets et les notebooks.
- Connaître les types de données et les structures de données de R.
- Utiliser les opérateurs et accéder aux éléments d'un objet.
- Écrire des conditions, des boucles et ses propres fonctions.
- Installer et charger des packages ; trouver de l'aide.

2.1 R : un peu d'histoire

- Années 1970 : création du langage **S** aux Bell Laboratories (John Chambers et collègues) pour manipuler des données, faire des statistiques et des graphiques.
- Années 1990 : **R** est créé à l'Université d'Auckland par Ross Ihaka et Robert Gentleman, inspiré de S et du langage Scheme. Il est distribué sous licence libre GPL et développé par la *R Core Team* et une large communauté.
- Aujourd'hui, le CRAN (*Comprehensive R Archive Network*) propose plus de 20 000 packages, et R est l'un des deux langages de référence de la science des données, avec Python.

R est un langage **interprété** : il n'y a pas d'étape de compilation, chaque instruction est exécutée dès qu'on la soumet.

2.2 Introduction à RStudio

2.2.1 R et RStudio : le moteur et le tableau de bord

R est le langage et le logiciel qui exécute les calculs. **RStudio** est un **environnement de développement intégré** (*IDE*) : une interface qui rend le travail avec R plus confortable (éditeur de code, gestion des fichiers, graphiques, aide…). On peut comparer R au moteur d'une voiture et RStudio à son tableau de bord : RStudio ne fonctionne pas sans R, mais R fonctionne sans RStudio.

Installation, dans cet ordre :

1. installer R depuis le CRAN : cran.r-project.org ;
2. installer RStudio Desktop (gratuit) depuis posit.co/download/rstudio-desktop.

En cas de problème d'installation, Posit Cloud permet d'utiliser RStudio dans un navigateur web, gratuitement dans la limite d'un quota mensuel.

2.2.2 Les quatre panneaux

Au lancement, RStudio affiche quatre panneaux. Si vous n'en voyez que trois, c'est qu'aucun fichier n'est ouvert : *File > New File > R Script* fait apparaître l'éditeur.

Panneau	Onglet	À quoi il sert
Éditeur	—	écrire, enregistrer et exécuter des scripts et des notebooks
Console	*Console*	exécuter du code ; voir les résultats, avertissements et erreurs
	Terminal	ligne de commande du système (Git, fichiers…)
Environnement	*Environment*	liste des objets en mémoire ; un clic sur un tableau l'ouvre avec `View()` ; bouton *Import Dataset*
	History	toutes les commandes exécutées
Divers	*Files*	explorateur de fichiers du projet
	Plots	graphiques produits ; bouton *Export* pour les enregistrer
	Packages	packages installés ; cocher une case équivaut à `library()`
	Help	pages d'aide, ouvertes par `?fonction` ou la touche *F1*
	Viewer	sorties web (tableaux interactifs, applications Shiny, notebooks)

La disposition se règle dans *Tools > Global Options > Pane Layout*, et l'apparence (thème sombre, taille de police) dans *Appearance*.

2.2.3 Console ou script ?

Dans la console, on tape une expression après l'invite `>` et on valide avec *Entrée* :

```
2 + 1
```

```
#> [1] 3
```

Les flèches *haut* et *bas* rappellent les commandes précédentes. Si l'invite devient `+`, R attend la fin d'une commande incomplète (une parenthèse non fermée, par exemple) : complétez-la ou appuyez sur *Échap*.

La console est pratique pour un test rapide, mais **rien n'y est conservé**. On écrit donc le code dans un **script** (*File > New File > R Script*, extension `.R`), qu'on enregistre et qu'on exécute :

Action	Windows / Linux	macOS
exécuter la ligne courante ou la sélection	*Ctrl + Entrée*	*Cmd + Entrée*
exécuter tout le script	*Ctrl + Maj + Entrée*	*Cmd + Maj + Entrée*
enregistrer	*Ctrl + S*	*Cmd + S*

Dans un script, tout ce qui suit `#` est un **commentaire**, ignoré par R :

```
# Analyse des ventes de la boutique - version du 16/10/2026
prix_ht <- 49.90
prix_ttc <- prix_ht * 1.20   # TVA à 20 %
```

2.2.4 Les projets RStudio

Un **projet** RStudio (*File > New Project > New Directory*) regroupe dans un dossier tout ce qui concerne une analyse : données, scripts, résultats. Ouvrir le fichier `.Rproj` :

- fixe le **répertoire de travail** sur le dossier du projet, ce qui permet d'utiliser des **chemins relatifs** (`"donnees/ventes.csv"`) ;
- rouvre les fichiers sur lesquels on travaillait ;
- sépare proprement les projets les uns des autres.

> **Un projet RStudio par projet d'analyse** (et donc un pour ce cours, un pour votre projet de groupe). C'est la condition pour que votre code fonctionne aussi sur l'ordinateur de vos camarades et de l'enseignant.

2.2.5 Les notebooks : R Markdown et Quarto

Un **notebook** mêle dans un même fichier du **texte** (en Markdown), du **code** et ses **résultats** (tableaux, graphiques). C'est le format idéal pour les comptes rendus de TP et le rapport de projet : le document se régénère entièrement à partir du code.

- **R Markdown** (`.Rmd`, *File > New File > R Markdown*) : le format historique ; ces notes sont écrites en R Markdown.
- **Quarto** (`.qmd`, *File > New File > Quarto Document*) : son successeur, utilisé dans les supports de TP du cours.

La syntaxe est presque identique. Le code est placé dans des **blocs** (*chunks*) :

````
## Panier moyen

```{r}
paniers <- c(42.5, 18.0, 73.9)
mean(paniers)
```
````

Action	Windows / Linux	macOS
insérer un bloc de code	*Ctrl + Alt + I*	*Cmd + Option + I*
exécuter le bloc courant	*Ctrl + Maj + Entrée*	*Cmd + Maj + Entrée*
produire le document (*Knit / Render*)	*Ctrl + Maj + K*	*Cmd + Maj + K*

2.2.6 L'onglet Environnement

L'onglet *Environment* liste les objets créés pendant la session. Pour un tableau de données, il affiche le nombre d'observations et de variables ; le triangle bleu montre sa structure, et un clic sur son nom l'ouvre dans un visualiseur de type tableur (équivalent de `View()`).

Le bouton *Import Dataset* ouvre un assistant d'import (CSV, Excel, SPSS…) qui **génère le code R** correspondant : copiez ce code dans votre script plutôt que de refaire l'import à la souris à chaque fois.

Le balai efface tous les objets. Pour repartir vraiment de zéro, préférez **redémarrer R** (*Session > Restart R*) : si votre script ne fonctionne plus après un redémarrage, c'est qu'il dépendait d'un objet créé à la main.

2.2.7 Écrire du code efficacement

- **Complétion automatique** : tapez le début d'un nom puis *Tab* ; RStudio propose les objets, les fonctions et leurs arguments.
- **Aide contextuelle** : curseur sur un nom de fonction, puis *F1*.
- **Diagnostics** : RStudio signale dans la marge les parenthèses non fermées et les erreurs de syntaxe.

Raccourci	Windows / Linux	macOS
`<-` (assignation)	*Alt + -*	*Option + -*
`|>` ou `%>%` (*pipe*)	*Ctrl + Maj + M*	*Cmd + Maj + M*
commenter / décommenter	*Ctrl + Maj + C*	*Cmd + Maj + C*
réindenter / reformater	*Ctrl + I / Ctrl + Maj + A*	*Cmd + I / Cmd + Maj + A*
insérer une section de script	*Ctrl + Maj + R*	*Cmd + Maj + R*
vider la console	*Ctrl + L*	*Ctrl + L*
redémarrer R	*Ctrl + Maj + F10*	*Cmd + Maj + 0*
liste de tous les raccourcis	*Alt + Maj + K*	*Option + Maj + K*

2.2.8 Réglages recommandés

Dans *Tools > Global Options* :

- *General > Basic* : décocher **Restore .RData into workspace at startup** et régler **Save workspace to .RData on exit** sur **Never**. Votre script doit suffire à tout recréer ; un espace de travail restauré masque les oublis.
- *Code > Editing* : cocher **Use native pipe operator, |>** pour que *Ctrl + Maj + M* insère le *pipe* natif.
- *Code > Saving* : encodage par défaut **UTF-8** (indispensable pour les accents).

Le menu *Addins* donne accès à des extensions de l'interface, comme `esquisse` pour construire des graphiques à la souris (chapitre 4).

2.3 Premiers pas : R comme calculatrice

```
2 + 1
```

```
#> [1] 3
```

```
10 / 3
```

```
#> [1] 3.333
```

```
2^10
```

```
#> [1] 1024
```

```
sqrt(16)        # racine carrée
```

```
#> [1] 4
```

```
exp(1)          # e
```

```
#> [1] 2.718
```

```
log(100, base = 10)
```

```
#> [1] 2
```

```
round(19.99 * 1.2, 2)
```

```
#> [1] 23.99
```

2.4 Variables et assignation

Le résultat d'une expression est affiché, mais pas conservé. Pour le conserver, on l'**assigne** à une variable avec `<-` :

```
x <- 2 + 1      # assignation : rien n'est affiché
x               # affichage du contenu
```

```
#> [1] 3
```

```
y <- x^2
y
```

```
#> [1] 9
```

```
(z <- x + y)    # des parenthèses autour forcent l'affichage
```

```
#> [1] 12
```

Le signe `=` fonctionne aussi, mais l'usage en R est de réserver `=` aux arguments des fonctions.

Règles de nommage : lettres, chiffres, `_` et `.` ; le nom ne commence pas par un chiffre et ne contient pas d'espace. R **distingue les majuscules et les minuscules** (`x` et `X` sont deux objets différents).

Convention du cours : tout en minuscules, mots séparés par `_` (*snake_case*), noms explicites : `taux_conversion`, `panier_moyen`.

```
panier_moyen <- 54.3
Panier_moyen    # erreur : la casse compte
```

```
#> Error: object 'Panier_moyen' not found
```

Gestion des objets en mémoire :

```
premier <- 3
second <- premier  # copie de la valeur
premier <- 4       # modifier premier ne modifie pas second
```

```
second
```

```
#> [1] 3
```

```
rm(premier)        # suppression
exists("premier")
```

```
#> [1] FALSE
```

2.5 Types de données

Tout objet R possède un **type** (ou mode), une **longueur** et un contenu. Les fonctions `typeof()`, `class()` et `length()` permettent de les connaître.

2.5.1 Les types de base

Type	Exemple	Test	Conversion
`double` (réel)	`3.14`, `2`	`is.double()`	`as.double()`
`integer` (entier)	`2L`, `1:3`	`is.integer()`	`as.integer()`
`character` (texte)	`"Paris"`	`is.character()`	`as.character()`
`logical` (booléen)	`TRUE`, `FALSE`	`is.logical()`	`as.logical()`
`complex`	`2+3i`	`is.complex()`	`as.complex()`

`double` et `integer` sont tous deux **numériques** (`is.numeric()` vaut `TRUE`).

```
typeof(2)          # un nombre écrit sans L est un réel
```

```
#> [1] "double"
```

```
typeof(2L)         # le suffixe L crée un entier
```

```
#> [1] "integer"
```

```
typeof("Bonjour")
```

```
#> [1] "character"
```

```
typeof(TRUE)
```

```
#> [1] "logical"
```

```
z <- 2 + 3i
c(Re(z), Im(z), Mod(z))
```

```
#> [1] 2.000 3.000 3.606
```

`TRUE` et `FALSE` valent 1 et 0 dans un calcul. C'est très utile pour compter, par exemple les visiteurs qui ont acheté :

```
a_achete <- c(FALSE, TRUE, FALSE, FALSE, TRUE, FALSE, FALSE, FALSE)
sum(a_achete)      # nombre d'acheteurs
```

```
#> [1] 2
```

```
mean(a_achete)      # taux de conversion
```

```
#> [1] 0.25
```

2.5.2 Conversions

```
as.numeric("3.5")
```

```
#> [1] 3.5
```

```
as.integer(3.9)       # troncature, pas arrondi
```

```
#> [1] 3
```

```
as.character(42)
```

```
#> [1] "42"
```

```
as.numeric("trois")   # conversion impossible : NA avec un avertissement
```

```
#> Warning: NAs introduced by coercion
```

```
#> [1] NA
```

2.5.3 Valeurs spéciales

Valeur	Signification	Test
`NA`	valeur manquante (*Not Available*)	`is.na()`
`NULL`	objet vide, de longueur 0	`is.null()`
`NaN`	résultat indéfini (*Not a Number*), ex. `0/0`	`is.nan()`
`Inf`, `-Inf`	infini, ex. `1/0`	`is.infinite()`

```
ages <- c(21, NA, 23)
mean(ages)                  # NA contamine le calcul
```

```
#> [1] NA
```

```
mean(ages, na.rm = TRUE)    # on ignore les valeurs manquantes
```

```
#> [1] 22
```

```
length(NULL)
```

```
#> [1] 0
```

```
0 / 0
```

```
#> [1] NaN
```

> On ne teste **jamais** une valeur manquante avec `x == NA` (le résultat est toujours `NA`). On utilise `is.na(x)`.

2.6 Structures de données

La fonction `class()` indique la structure d'un objet. Les principales structures sont résumées ci-dessous.

Structure	Dimensions	Types des éléments
vecteur	1	un seul type
facteur	1	modalités d'une variable qualitative
matrice	2	un seul type
tableau (*array*)	n	un seul type
liste	1	types quelconques
data frame	2	un type par colonne

2.6.1 Vecteurs

Le vecteur est l'objet de base de R : un nombre seul est un vecteur de longueur 1. On le crée avec `c()` (*combine*).

```
ventes <- c(12500, 14000, 9800, 16200)
length(ventes)
```

```
#> [1] 4
```

```
names(ventes) <- c("Paris", "Lyon", "Lille", "Bordeaux")
ventes
```

```
#>    Paris     Lyon    Lille Bordeaux
#>    12500    14000     9800    16200
```

Tous les éléments d'un vecteur ont **le même type**. En cas de mélange, R convertit vers le type le plus général (logique → entier → réel → texte) :

```
c("deux", 1, TRUE)
```

```
#> [1] "deux" "1"    "TRUE"
```

```
c(0, FALSE)
```

```
#> [1] 0 0
```

2.6.2 Facteurs

Un **facteur** représente une variable **qualitative** (catégorielle). Il stocke les valeurs possibles, appelées **niveaux** (*levels*).

```
canal <- factor(c("Email", "Réseaux sociaux", "Email", "Direct", "Réseaux
 ↪  sociaux"))
canal
```

```
#> [1] Email           Réseaux sociaux Email           Direct
#> [5] Réseaux sociaux
#> Levels: Direct Email Réseaux sociaux
```

```
levels(canal)                          # ordre alphabétique par défaut
```

```
#> [1] "Direct"          "Email"           "Réseaux sociaux"
```

```
table(canal)                              # effectifs par modalité
```

```
#> canal
#>          Direct           Email Réseaux sociaux
#>               1               2               2
```

```
canal <- relevel(canal, ref = "Direct")   # changer la modalité de référence
levels(canal)
```

```
#> [1] "Direct"          "Email"           "Réseaux sociaux"
```

Pour une variable **ordinale** (avec un ordre naturel), on utilise un facteur ordonné :

```
satisfaction <- factor(c("Satisfait", "Très satisfait", "Peu satisfait",
 ↪  "Satisfait"),
                       levels = c("Peu satisfait", "Satisfait", "Très
                        ↪  satisfait"),
                       ordered = TRUE)
satisfaction
```

```
#> [1] Satisfait      Très satisfait Peu satisfait  Satisfait
#> Levels: Peu satisfait < Satisfait < Très satisfait
```

```
satisfaction >= "Satisfait"
```

```
#> [1]  TRUE  TRUE FALSE  TRUE
```

2.6.3 Matrices et tableaux

Une matrice est un vecteur avec deux dimensions. Elle se remplit **colonne par colonne** par défaut.

```
m <- matrix(1:6, nrow = 3, ncol = 2)
m
```

```
#>      [,1] [,2]
#> [1,]    1    4
#> [2,]    2    5
#> [3,]    3    6
```

```
matrix(1:6, nrow = 3, byrow = TRUE)   # remplissage ligne par ligne
```

```
#>      [,1] [,2]
#> [1,]    1    2
#> [2,]    3    4
#> [3,]    5    6
```

```
dim(m)
```

```
#> [1] 3 2
```

```
t(m)                                  # transposée
```

```
#>      [,1] [,2] [,3]
#> [1,]    1    2    3
#> [2,]    4    5    6
```

```r
a <- array(1:24, dim = c(2, 4, 3))     # tableau à 3 dimensions
dim(a)
```

```
#> [1] 2 4 3
```

Opérations matricielles utiles en analyse de données (l'ACP et la régression reposent sur elles) :

Opération	Syntaxe
produit matriciel	`A %*% B`
produit terme à terme	`A * B`
transposée, inverse	`t(A)`, `solve(A)`
$A^\top B$	`crossprod(A, B)`
déterminant, diagonale	`det(A)`, `diag(A)`
juxtaposer, superposer	`cbind()`, `rbind()`

```r
A <- matrix(c(1, 3, 2, 2, 2, 1, 3, 1, 3), ncol = 3)
round(A %*% solve(A), 10)       # matrice identité
```

```
#>      [,1] [,2] [,3]
#> [1,]    1    0    0
#> [2,]    0    1    0
#> [3,]    0    0    1
```

`%/%` est la division **entière**, appliquée terme à terme ; ce n'est pas une division matricielle. Pour « diviser » par une matrice, on multiplie par son inverse : `B %*% solve(A)`.

2.6.4 Listes

Une liste peut contenir des éléments de **types et de tailles différents**, y compris d'autres listes. Beaucoup de fonctions R renvoient leurs résultats sous forme de liste (par exemple `lm()`).

```r
client <- list(id = "C0042", paniers = c(54.9, 23.5, 89.0), newsletter = TRUE)
client
```

```
#> $id
#> [1] "C0042"
#>
#> $paniers
#> [1] 54.9 23.5 89.0
#>
#> $newsletter
#> [1] TRUE
```

```r
str(client)       # structure compacte, très utile
```

```
#> List of 3
#>  $ id        : chr "C0042"
#>  $ paniers   : num [1:3] 54.9 23.5 89
#>  $ newsletter: logi TRUE
```

2.6.5 Data frames

Le **data frame** est la structure centrale de l'analyse de données : un tableau dont chaque **colonne est une variable** (un vecteur, avec son propre type) et chaque **ligne une observation**. Techniquement, c'est une liste de vecteurs de même longueur.

```
commandes <- data.frame(
  client  = c("C01", "C02", "C01", "C03", "C02"),
  montant = c(54.9, 23.5, 89.0, 42.0, 17.9),
  promo   = c(FALSE, TRUE, FALSE, FALSE, TRUE)
)
commandes
```

```
#>   client montant promo
#> 1    C01    54.9 FALSE
#> 2    C02    23.5  TRUE
#> 3    C01    89.0 FALSE
#> 4    C03    42.0 FALSE
#> 5    C02    17.9  TRUE
```

```
dim(commandes)
```

```
#> [1] 5 3
```

```
str(commandes)
```

```
#> 'data.frame':    5 obs. of  3 variables:
#>  $ client : chr  "C01" "C02" "C01" "C03" ...
#>  $ montant: num  54.9 23.5 89 42 17.9
#>  $ promo  : logi  FALSE TRUE FALSE FALSE TRUE
```

```
summary(commandes)
```

```
#>     client             montant       promo
#>  Length:5           Min.   :17.9   Mode :logical
#>  Class :character   1st Qu.:23.5   FALSE:3
#>  Mode  :character   Median :42.0   TRUE :2
#>                     Mean   :45.5
#>                     3rd Qu.:54.9
#>                     Max.   :89.0
```

Le *tidyverse* (chapitre 3) utilise une variante moderne, le **tibble**, qui s'affiche de façon plus lisible et se comporte de façon plus prévisible.

2.7 Opérateurs

2.7.1 Opérateurs arithmétiques

`+`, `-`, `*`, `/`, `^` (puissance), `%%` (reste de la division entière, ou modulo), `%/%` (quotient de la division entière).

Ils sont **vectorisés** : l'opération s'applique élément par élément, sans boucle.

```
prix <- c(19.9, 49.0, 5.5)
quantite <- c(2, 1, 10)
prix * quantite           # chiffre d'affaires par produit
```

```
#> [1] 39.8 49.0 55.0
```

```
sum(prix * quantite)
```

```
#> [1] 143.8
```

```
prix * 1.2                # le scalaire est appliqué à chaque élément
```

```
#> [1] 23.88 58.80  6.60
```

Recyclage : si deux vecteurs n'ont pas la même longueur, R répète le plus court. C'est pratique avec un scalaire, mais source d'erreurs silencieuses sinon. R n'affiche un avertissement que si la longueur du plus long n'est pas un multiple de celle du plus court.

```
c(1, 2, 3, 4) + c(10, 20)      # aucun avertissement !
```

```
#> [1] 11 22 13 24
```

2.7.2 Opérateurs de comparaison

`<`, `<=`, `>`, `>=`, `==` (égal), `!=` (différent). Le résultat est un vecteur logique.

```
paniers <- c(42.5, 18.0, 73.9, 120.0)
paniers > 50
```

```
#> [1] FALSE FALSE  TRUE  TRUE
```

```
paniers == 18
```

```
#> [1] FALSE  TRUE FALSE FALSE
```

Les nombres réels sont stockés de façon approchée. Ne comparez pas deux réels avec `==` ; utilisez `all.equal()` ou une tolérance.

```
0.9 == 1.1 - 0.2
```

```
#> [1] FALSE
```

```
isTRUE(all.equal(0.9, 1.1 - 0.2))
```

```
#> [1] TRUE
```

2.7.3 Opérateurs logiques

Opérateur	Sens	Usage
`!`	NON	vectorisé
`&`, `|`	ET, OU	**vectorisés** : filtres, sélections
`&&`, `||`	ET, OU	**un seul** `TRUE`/`FALSE` : conditions de `if`
`xor()`	OU exclusif	vectorisé
`any()`, `all()`	au moins un / tous `TRUE`	résumé d'un vecteur

```
a <- c(TRUE, TRUE, FALSE, FALSE)
b <- c(TRUE, FALSE, TRUE, FALSE)
a & b
```

```
#> [1]  TRUE FALSE FALSE FALSE
```

```
a | b
```

```
#> [1]  TRUE  TRUE  TRUE FALSE
```

```
any(a); all(a)
```

```
#> [1] TRUE
```

```
#> [1] FALSE
```

```
a && b      # erreur depuis R 4.3 : && n'accepte que des valeurs de longueur 1
```

```
#> Error in a && b: 'length = 4' in coercion to 'logical(1)'
```

`&&` et `||` n'évaluent leur second argument que si c'est nécessaire, ce qui permet d'écrire `!is.null(x) && x > 0` sans erreur quand `x` vaut `NULL`.

L'opérateur `%in%` teste l'appartenance à un ensemble :

```
c("Lyon", "Nice") %in% c("Paris", "Lyon", "Lille")
```

```
#> [1]  TRUE FALSE
```

2.8 Accéder aux éléments et les modifier

Trois opérateurs d'accès : `[ ]`, `[[ ]]` et `$`.

2.8.1 Vecteurs

On peut indexer par **position**, par **position négative** (exclusion), par **vecteur logique** ou par **nom**.

```
x <- c(4, 7, 3, 5, 0)
x[2]              # deuxième élément (les indices commencent à 1)
```

```
#> [1] 7
```

```
x[c(1, 3)]
```

```
#> [1] 4 3
```

```
x[-2]             # tout sauf le deuxième
```

```
#> [1] 4 3 5 0
```

```
x[x > 3]          # filtrage logique
```

```
#> [1] 4 7 5
```

```
which(x > 3)      # positions des éléments TRUE
```

```
#> [1] 1 2 4
```

```
which.max(x)      # position du maximum
```

```
#> [1] 2
```

```
x[x == 0] <- 10   # remplacement
x
```

```
#> [1]  4  7  3  5 10
```

```
ventes["Lyon"]
```

```
#>  Lyon
#> 14000
```

2.8.2 Matrices et data frames : `[ligne, colonne]`

Laisser un indice vide signifie « tout ».

```
m <- matrix(1:9, nrow = 3)
m[1, 2]                 # ligne 1, colonne 2
```

```
#> [1] 4
```

```
m[, 2]                  # colonne 2 (renvoyée sous forme de vecteur)
```

```
#> [1] 4 5 6
```

```
m[, 2, drop = FALSE]    # reste une matrice
```

```
#>      [,1]
#> [1,]    4
#> [2,]    5
#> [3,]    6
```

```
commandes[2, ]                          # deuxième commande
```

```
#>   client montant promo
#> 2    C02    23.5  TRUE
```

```
commandes$montant                       # une colonne, sous forme de vecteur
```

```
#> [1] 54.9 23.5 89.0 42.0 17.9
```

```
commandes[commandes$montant > 40, ]   # filtrer des lignes
```

```
#>   client montant promo
#> 1    C01    54.9 FALSE
#> 3    C01    89.0 FALSE
#> 4    C03    42.0 FALSE
```

```
commandes$montant_ht <- round(commandes$montant / 1.2, 2)   # nouvelle colonne
commandes
```

```
#>   client montant promo montant_ht
#> 1    C01    54.9 FALSE      45.75
#> 2    C02    23.5  TRUE      19.58
#> 3    C01    89.0 FALSE      74.17
#> 4    C03    42.0 FALSE      35.00
#> 5    C02    17.9  TRUE      14.92
```

2.8.3 Listes : `[ ]` contre `[[ ]]`

- `liste[1]` renvoie une **liste** contenant le premier élément ;
- `liste[[1]]` ou `liste$nom` renvoie **le contenu** de l'élément.

Image utile : si la liste est un train, `[ ]` renvoie un train plus court, `[[ ]]` renvoie le contenu d'un wagon.

```r
client[2]          # une liste
```

```
#> $paniers
#> [1] 54.9 23.5 89.0
```

```r
client[[2]]        # un vecteur
```

```
#> [1] 54.9 23.5 89.0
```

```r
client$paniers[3]  # troisième panier
```

```
#> [1] 89
```

```r
client$ville <- "Paris"   # ajout d'un élément
names(client)
```

```
#> [1] "id"         "paniers"    "newsletter" "ville"
```

2.9 Structures algorithmiques

2.9.1 Branchements conditionnels : `if`, `else`

```r
panier <- 65
if (panier >= 100) {
  print("Livraison gratuite")
} else if (panier >= 50) {
  print("Livraison à 2,90 €")
} else {
  print("Livraison à 4,90 €")
}
```

```
#> [1] "Livraison à 2,90 €"
```

La condition d'un `if` doit être **un seul** `TRUE` ou `FALSE`. Pour appliquer une condition à **chaque élément** d'un vecteur, on utilise la fonction vectorisée `ifelse()` (ou `dplyr::case_when()` au chapitre suivant) :

```r
paniers <- c(42.5, 18.0, 73.9, 120.0)
ifelse(paniers >= 50, "Gros panier", "Petit panier")
```

```
#> [1] "Petit panier" "Petit panier" "Gros panier"  "Gros panier"
```

`switch()` choisit une branche selon la valeur d'un texte :

```r
canal <- "Email"
switch(canal,
       "Email" = 0.05,            # coût par contact
       "Réseaux sociaux" = 0.80,
       "Courrier" = 1.20)
```

```
#> [1] 0.05
```

2.9.2 Boucles `for`

Une boucle `for` répète un bloc pour chaque élément d'une séquence.

```r
for (ville in c("Paris", "Lyon", "Lille")) {
  print(paste("Rapport des ventes :", ville))
}
```

```
#> [1] "Rapport des ventes : Paris"
#> [1] "Rapport des ventes : Lyon"
#> [1] "Rapport des ventes : Lille"
```

Pour parcourir les positions d'un vecteur, utilisez `seq_along()` (plutôt que `1:length(x)`, qui donne `1 0` quand `x` est vide) et **préallouez** le résultat :

```r
paniers <- c(42.5, 18.0, 73.9, 120.0)
frais_port <- numeric(length(paniers))      # préallocation
for (i in seq_along(paniers)) {
  frais_port[i] <- if (paniers[i] >= 50) 0 else 4.90
}
frais_port
```

```
#> [1] 4.9 4.9 0.0 0.0
```

2.9.3 Boucles `while` et `repeat`

`while` répète tant qu'une condition est vraie ; `break` sort d'une boucle, `next` passe directement à l'itération suivante.

```r
# Combien de mois pour atteindre 10 000 abonnés avec une croissance de 8 % par
 ↪  mois ?
abonnes <- 2500
mois <- 0
while (abonnes < 10000) {
  abonnes <- abonnes * 1.08
  mois <- mois + 1
}
mois
```

```
#> [1] 19
```

2.9.4 Boucle ou vectorisation ?

En R, la plupart des opérations sont déjà vectorisées. Un code vectorisé est plus court, plus lisible et beaucoup plus rapide.

```r
# Avec une boucle
frais_port <- numeric(length(paniers))
for (i in seq_along(paniers)) frais_port[i] <- if (paniers[i] >= 50) 0 else
 ↪  4.90
# Vectorisé : préférable
frais_port <- ifelse(paniers >= 50, 0, 4.90)
frais_port
```

```
#> [1] 4.9 4.9 0.0 0.0
```

Gardez les boucles pour les cas où chaque itération dépend de la précédente (comme l'exemple des abonnés) ou pour répéter une tâche complexe (lire plusieurs fichiers, simuler plusieurs scénarios).

2.10 Fonctions et programmation modulaire

Un programme lisible est découpé en **petites unités** qui font chacune une seule chose, ont un nom explicite et peuvent être testées séparément.

2.10.1 Écrire une fonction

```
prix_ttc <- function(prix_ht, tva = 0.20) {
  round(prix_ht * (1 + tva), 2)
}
prix_ttc(100)
```

```
#> [1] 120
```

```
prix_ttc(100, tva = 0.055)          # taux réduit
```

```
#> [1] 105.5
```

```
prix_ttc(c(10, 25, 49.9))           # vectorisée « gratuitement »
```

```
#> [1] 12.00 30.00 59.88
```

Anatomie d'une fonction :

- un **nom** (un verbe ou un nom explicite) ;
- des **arguments**, éventuellement avec des **valeurs par défaut** ;
- un **corps** ; la **dernière expression évaluée est renvoyée**. `return()` sert surtout à sortir plus tôt.

```
resumer <- function(x, arrondi = 2) {
  if (!is.numeric(x)) {
    stop("x doit être numérique")      # message d'erreur explicite
  }
  c(moyenne    = round(mean(x, na.rm = TRUE), arrondi),
    mediane    = round(median(x, na.rm = TRUE), arrondi),
    n_manquant = sum(is.na(x)))
}
resumer(c(42.5, 18.0, NA, 73.9, 120.0))
```

```
#>    moyenne    mediane n_manquant
#>       63.6       58.2        1.0
```

Portée des variables : les variables créées dans une fonction sont **locales** ; elles n'existent plus après l'appel et ne modifient pas les variables du même nom à l'extérieur. Une fonction doit recevoir ce dont elle a besoin **par ses arguments** et transmettre son résultat **par sa valeur de retour**.

2.10.2 Fonctions et procédures

On distingue parfois :

- les **fonctions** (au sens strict), qui calculent et **renvoient** une valeur sans rien modifier d'autre : `prix_ttc()`, `resumer()` ;
- les **procédures**, appelées pour leurs **effets de bord** : afficher, écrire un fichier, tracer un graphique. Elles renvoient souvent leur entrée de façon invisible avec `invisible()`, ce qui permet de les insérer dans une chaîne de *pipes*.

```
decrire_table <- function(df) {
  cat("Lignes :", nrow(df), "| Colonnes :", ncol(df), "\n")
  cat("Valeurs manquantes :", sum(is.na(df)), "\n")
  invisible(df)
}
decrire_table(commandes)
```

```
#> Lignes : 5 | Colonnes : 4
#> Valeurs manquantes : 0
```

2.10.3 Appliquer une fonction à chaque élément

Les fonctions de la famille `apply` évitent d'écrire des boucles :

```
paniers_par_canal <- list(email = c(54, 61, 48), social = c(32, 29, 41, 25),
                          direct = c(88, 95))
sapply(paniers_par_canal, mean)        # simplifie le résultat en vecteur
```

```
#>  email social direct
#>  54.33  31.75  91.50
```

```
lapply(paniers_par_canal, range)       # renvoie toujours une liste
```

```
#> $email
#> [1] 48 61
#>
#> $social
#> [1] 25 41
#>
#> $direct
#> [1] 88 95
```

```
apply(matrix(1:6, nrow = 2), 2, sum)   # sur les colonnes (1 = lignes)
```

```
#> [1]  3  7 11
```

2.10.4 Découper un programme en modules

Dès qu'un projet grandit, on range les fonctions dans des fichiers séparés, chargés avec `source()`. Une organisation simple :

```
mon_projet/
|-- mon_projet.Rproj
|-- donnees/
|   |-- brutes/           # données d'origine, jamais modifiées
|   `-- propres/          # données nettoyées par le code
|-- R/
|   |-- nettoyage.R       # fonctions de préparation des données
|   |-- indicateurs.R     # calculs (RFM, taux de conversion...)
|   `-- graphiques.R      # fonctions de visualisation
|-- analyse.qmd           # notebook principal : appelle les fonctions
`-- resultats/            # figures et tableaux produits
```

```
# En tête de analyse.qmd
source("R/nettoyage.R")
source("R/indicateurs.R")
```

Règles d'un code modulaire

- Ne jamais copier-coller un bloc plus de deux fois : en faire une fonction.
- Une fonction = une tâche, un nom explicite, des arguments, une valeur de retour.
- Pas de variables globales cachées dans les fonctions.
- Les données brutes ne sont jamais modifiées à la main : tout passe par le code.
- Commenter le **pourquoi**, pas le **comment**.

2.11 Packages et aide

2.11.1 Installer et charger un package

Un **package** (ou extension) regroupe des fonctions, de la documentation et parfois des jeux de données. Certains sont chargés au démarrage (`base`, `stats`, `graphics`, `utils`, `datasets`…). Pour les autres :

```
install.packages("questionr")   # une seule fois par ordinateur (télécharge
↪  depuis le CRAN)
library(questionr)              # à chaque session, en début de script
```

Dans RStudio, l'onglet *Packages* permet aussi d'installer (bouton *Install*) et de mettre à jour (bouton *Update*) les packages.

On peut appeler une fonction sans charger tout le package, avec `package::fonction()`, ce qui rend explicite la provenance de la fonction :

```
stats::median(c(3, 1, 2))
```

```
#> [1] 2
```

`sessionInfo()` affiche la version de R et des packages chargés : à joindre à toute demande d'aide et à votre rapport de projet.

2.11.2 Trouver de l'aide

```
?mean                    # page d'aide d'une fonction (équivalent :
↪  help("mean"), ou F1)
??regression             # recherche par mot-clé (équivalent : help.search())
example(mean)            # exécute les exemples de la page d'aide
vignette("dplyr")        # documentation longue d'un package
```

Une page d'aide se lit dans cet ordre : *Description*, *Usage* (les arguments et leurs valeurs par défaut), *Arguments*, *Value* (ce que renvoie la fonction), puis *Examples*, à exécuter.

Autres ressources : les *cheatsheets* de Posit (*Help* > *Cheat Sheets* dans RStudio), Stack Overflow, et les messages d'erreur eux-mêmes, à lire attentivement.

2.12 Résumé

- **RStudio** : éditeur, console, environnement, fichiers/graphiques/aide. Écrire dans des **scripts** ou des **notebooks**, travailler dans un **projet**, ne jamais restaurer l'espace de travail.
- `<-` pour assigner ; types `double`, `integer`, `character`, `logical` ; structures vecteur, facteur, matrice, liste, data frame.

- Accès : `[ ]`, `[[ ]]`, `$` ; filtres logiques avec `&`, `|`, `%in%`.
- `if` / `else` pour **une** condition ; `ifelse()` pour un vecteur ; `for`, `while` pour répéter ; préférer la vectorisation.
- Fonctions : un nom, des arguments, une valeur de retour ; modules rangés dans `R/` et chargés avec `source()`.
- `install.packages()` une fois, `library()` à chaque session ; `?fonction` pour l'aide.

2.13 Exercices

Exercice 2.1 — Prise en main de RStudio.

1. Créer un projet RStudio `data-science-R` et y créer un dossier `donnees`.
2. Appliquer les réglages recommandés (espace de travail, *pipe* natif).
3. Créer un script `premier-script.R` qui calcule le prix TTC de trois articles et l'exécuter ligne par ligne, puis en entier.
4. Créer un document Quarto, y insérer un bloc de code avec le raccourci clavier, puis produire le document HTML.
5. Redémarrer R et vérifier que le script s'exécute toujours.

Exercice 2.2 — Types et structures.

1. Que permet une liste par rapport à un vecteur ?
2. Que renvoient `c("deux", 1, TRUE)`, `c(0, FALSE)` et `c(0, "FALSE")` ? Pourquoi ?
3. Créer un facteur de longueur 5 dont les modalités sont `"Google"`, `"Instagram"` et `"TikTok"`, chacune présente au moins une fois, avec `"Google"` comme modalité de référence.
4. Créer la matrice $\begin{pmatrix} 1 & 3 \\ 2 & 4 \end{pmatrix}$, puis la transformer en data frame.

Solution

```
# 1. Une liste peut contenir des éléments de types et de longueurs
 ↪  différents.
# 2. Conversion vers le type le plus général :
c("deux", 1, TRUE)   # tout devient du texte
```

```
#> [1] "deux" "1"    "TRUE"
```

```
c(0, FALSE)          # FALSE devient 0
```

```
#> [1] 0 0
```

```
c(0, "FALSE")        # tout devient du texte
```

```
#> [1] "0"     "FALSE"
```

```
# 3.
source_trafic <- factor(c("TikTok", "Google", "Instagram", "Google",
 ↪  "TikTok"))
source_trafic <- relevel(source_trafic, ref = "Google")
levels(source_trafic)
```

```
#> [1] "Google"    "Instagram" "TikTok"
```

```r
# 4.
m <- matrix(1:4, nrow = 2)
as.data.frame(m)
```

```
#>   V1 V2
#> 1  1  3
#> 2  2  4
```

Exercice 2.3 — Vecteurs et indexation. Une campagne a généré les nombres de clics journaliers `clics <- c(120, 340, 95, 410, 220, 180, 60)` (du lundi au dimanche).

1. Nommer les éléments avec les jours de la semaine.
2. Calculer la part de chaque jour dans le total des clics.
3. Extraire les jours avec plus de 200 clics.
4. Quel jour a eu le plus de clics ? le moins ?
5. Normaliser les clics entre 0 et 1 : $(x - \min x)/(\max x - \min x)$.

Solution

```r
clics <- c(120, 340, 95, 410, 220, 180, 60)
names(clics) <- c("lun", "mar", "mer", "jeu", "ven", "sam", "dim")
round(clics / sum(clics), 3)
```

```
#>   lun   mar   mer   jeu   ven   sam   dim
#> 0.084 0.239 0.067 0.288 0.154 0.126 0.042
```

```r
clics[clics > 200]
```

```
#> mar jeu ven
#> 340 410 220
```

```r
names(which.max(clics)); names(which.min(clics))
```

```
#> [1] "jeu"
#> [1] "dim"
```

```r
(clics - min(clics)) / (max(clics) - min(clics))
```

```
#>    lun    mar    mer    jeu    ven    sam    dim
#> 0.1714 0.8000 0.1000 1.0000 0.4571 0.3429 0.0000
```

Exercice 2.4 — Accès aux éléments.

1. Extraire le troisième élément du vecteur `x <- c(3, 5, 6, 9)`.
2. Extraire les éléments de `x` supérieurs à 5.
3. Extraire les éléments de `x` qui ne sont pas dans l'intervalle $[4, 6]$.
4. Déterminer le maximum de `x` et donner sa position.
5. Remplacer ce maximum par le minimum.
6. Remplacer par `NA` les éléments négatifs de la matrice $M = \begin{pmatrix} 1 & -2 & 3 \\ -4 & 5 & -6 \end{pmatrix}$, puis extraire la deuxième colonne de la matrice obtenue.
7. Soit `L <- list(nombre = rnorm(10), lettre = LETTERS[1:26], mois = month.abb)`. Extraire le troisième élément de `L`, d'abord par sa position, puis par son nom ; extraire tous les éléments de `L` sauf le deuxième.
8. Toujours par le nom, ajouter 10 aux valeurs du premier élément de `L` (le résultat doit modifier `L`).

9. Soit `df <- data.frame(x = 1:10, y = letters[1:10], z = rev(letters[1:10]))`. Afficher les valeurs de la variable `y`, puis uniquement les colonnes `x` et `z`.

Solution

```
x <- c(3, 5, 6, 9)
x[3]
```

```
#> [1] 6
```

```
x[x > 5]
```

```
#> [1] 6 9
```

```
x[x < 4 | x > 6]
```

```
#> [1] 3 9
```

```
max(x); which.max(x)
```

```
#> [1] 9
#> [1] 4
```

```
x[which.max(x)] <- min(x)
x
```

```
#> [1] 3 5 6 3
```

```
M <- matrix(c(1, -4, -2, 5, 3, -6), nrow = 2)
M[M < 0] <- NA
M
```

```
#>      [,1] [,2] [,3]
#> [1,]    1   NA    3
#> [2,]   NA    5   NA
```

```
M[, 2]
```

```
#> [1] NA  5
```

```
set.seed(1)
L <- list(nombre = rnorm(10), lettre = LETTERS[1:26], mois = month.abb)
L[[3]]
```

```
#>  [1] "Jan" "Feb" "Mar" "Apr" "May" "Jun" "Jul" "Aug" "Sep" "Oct" "Nov"
#> [12] "Dec"
```

```
L$mois
```

```
#>  [1] "Jan" "Feb" "Mar" "Apr" "May" "Jun" "Jul" "Aug" "Sep" "Oct" "Nov"
#> [12] "Dec"
```

```
L[-2]
```

```
#> $nombre
#>  [1] -0.6265  0.1836 -0.8356  1.5953  0.3295 -0.8205  0.4874  0.7383
#>  [9]  0.5758 -0.3054
#>
#> $mois
#>  [1] "Jan" "Feb" "Mar" "Apr" "May" "Jun" "Jul" "Aug" "Sep" "Oct" "Nov"
#> [12] "Dec"
```

```r
L$nombre <- L$nombre + 10

df <- data.frame(x = 1:10, y = letters[1:10], z = rev(letters[1:10]))
df$y
```

```
#>  [1] "a" "b" "c" "d" "e" "f" "g" "h" "i" "j"
```

```r
df[, c("x", "z")]
```

```
#>     x z
#> 1   1 j
#> 2   2 i
#> 3   3 h
#> 4   4 g
#> 5   5 f
#> 6   6 e
#> 7   7 d
#> 8   8 c
#> 9   9 b
#> 10 10 a
```

Exercice 2.5 — Conditions et boucles.

1. Écrire une boucle qui affiche les nombres de 1 à 20 en remplaçant les multiples de 3 par `"Fizz"`, ceux de 5 par `"Buzz"` et ceux de 15 par `"FizzBuzz"`.
2. Réécrire la même chose **sans boucle**, à l'aide de `ifelse()`.

Solution

```r
for (i in 1:20) {
  if (i %% 15 == 0) {
    print("FizzBuzz")
  } else if (i %% 3 == 0) {
    print("Fizz")
  } else if (i %% 5 == 0) {
    print("Buzz")
  } else {
    print(i)
  }
}
```

```
#> [1] 1
#> [1] 2
#> [1] "Fizz"
#> [1] 4
#> [1] "Buzz"
#> [1] "Fizz"
#> [1] 7
#> [1] 8
#> [1] "Fizz"
#> [1] "Buzz"
#> [1] 11
#> [1] "Fizz"
#> [1] 13
#> [1] 14
#> [1] "FizzBuzz"
```

```
#> [1] 16
#> [1] 17
#> [1] "Fizz"
#> [1] 19
#> [1] "Buzz"
```

```
i <- 1:20
ifelse(i %% 15 == 0, "FizzBuzz",
       ifelse(i %% 3 == 0, "Fizz",
              ifelse(i %% 5 == 0, "Buzz", i)))
```

```
#>  [1] "1"        "2"        "Fizz"     "4"        "Buzz"     "Fizz"
#>  [7] "7"        "8"        "Fizz"     "Buzz"     "11"       "Fizz"
#> [13] "13"       "14"       "FizzBuzz" "16"       "17"       "Fizz"
#> [19] "19"       "Buzz"
```

Exercice 2.6 — Fonctions.

1. Écrire une fonction `taux_conversion(visiteurs, acheteurs)` qui renvoie le taux de conversion en pourcentage, arrondi à une décimale, et qui produit une erreur explicite si `acheteurs > visiteurs`.
2. L'appliquer aux campagnes : visiteurs `c(1200, 800, 1500)`, acheteurs `c(36, 40, 30)`.

Solution

```
taux_conversion <- function(visiteurs, acheteurs) {
  if (any(acheteurs > visiteurs)) {
    stop("Il ne peut pas y avoir plus d'acheteurs que de visiteurs")
  }
  round(100 * acheteurs / visiteurs, 1)
}
taux_conversion(c(1200, 800, 1500), c(36, 40, 30))
```

```
#> [1] 3 5 2
```

Exercice 2.7 — Moindres carrés « à la main ». On considère le modèle $\text{mpg} = \beta_0 + \beta_1\,\text{hp} + \beta_2\,\text{wt} + \varepsilon$ sur le jeu `mtcars`. L'estimateur des moindres carrés ordinaires est $\hat{\beta} = (X^\top X)^{-1} X^\top y$.

1. Créer le vecteur `y` (consommation `mpg`) et la matrice `X` à trois colonnes : une colonne de 1, `hp` et `wt`.
2. Calculer $\hat{\beta}$ par calcul matriciel et comparer avec `lm(mpg ~ hp + wt, data = mtcars)`.

Solution

```
y <- mtcars$mpg
X <- cbind(constante = 1, hp = mtcars$hp, wt = mtcars$wt)
solve(t(X) %*% X) %*% t(X) %*% y
```

```
#>                [,1]
#> constante 37.22727
#> hp        -0.03177
#> wt        -3.87783
```

```
coef(lm(mpg ~ hp + wt, data = mtcars))
```

```
#> (Intercept)          hp          wt
#>    37.22727    -0.03177    -3.87783
```

Pour aller plus loin

- J. Barnier, *Introduction à R et au tidyverse*, chapitres « Prise en main » et « Premier travail avec des données » (juba.github.io/tidyverse).
- H. Wickham et al., *R for Data Science*, 2e éd., chapitres « Workflow » (r4ds.hadley.nz).
- Documentation de RStudio : docs.posit.co/ide/user.

Chapitre 3

Manipulation de données avec R

Objectifs du chapitre

- Importer et exporter des données ; utiliser les données disponibles dans les packages.
- Décrire la structure d'un tableau de données et accéder à ses variables.
- Décrire une variable (analyse univariée) puis la relation entre deux variables (analyse bivariée), avec les indicateurs et tests adaptés.
- Filtrer, sélectionner, transformer, agréger et joindre des tables avec `dplyr`.
- Mettre des données au format *tidy* ; traiter les valeurs manquantes ; générer des données.

```
library(tidyverse)      # dplyr, tidyr, readr, ggplot2, tibble...
library(questionr)      # jeux de données hdv2003 et rp2018, fonctions freq(),
 ↪  lprop()...
library(nycflights13)   # vols au départ de New York en 2013
library(ISLR)           # jeu de données Carseats (ventes de sièges auto)
```

3.1 Les données de ce chapitre

3.1.1 La boutique en ligne (données simulées)

Ces données simulent l'activité d'un site de vente en ligne français en 2025. Elles se composent de quatre tables, au format CSV, reliées par des **clés** : `client_id` relie les commandes et la campagne aux clients, `categorie` relie les commandes à la table des catégories.

`boutique_clients.csv` — 2 000 lignes, une par client inscrit sur le site.

Variable	Type	Description
`client_id`	texte	identifiant du client (`C0001` à `C2000`)
`age`	numérique	âge en années ; non renseigné (`NA`) pour 60 clients
`region`	texte	région de résidence (7 régions et « Autres régions »)
`canal_acquisition`	texte	canal par lequel le client est arrivé : `Direct`, `Email`, `Moteur de recherche`, `Réseaux sociaux`, `Affiliation` (sites partenaires rémunérés à la vente)
`date_inscription`	date	date de création du compte (2021 à 2025)

Variable	Type	Description
`newsletter`	texte	abonnement à la newsletter : `Oui` / `Non`

`boutique_commandes.csv` — 9 508 lignes, une par commande passée en 2025. Certains clients (259) n'ont passé aucune commande en 2025.

Variable	Type	Description
`commande_id`	texte	identifiant de la commande (`K00001…`)
`client_id`	texte	client qui a passé la commande (clé vers `clients`)
`date`	date	date de la commande (du 01/01/2025 au 31/12/2025)
`categorie`	texte	catégorie de produits : `Mode`, `Maison`, `Beauté`, `High-tech`, `Sport`
`nb_articles`	entier	nombre d'articles dans la commande
`promo`	logique	`TRUE` si un code promo (20 % de remise) a été utilisé
`montant`	numérique	montant payé, en euros TTC, après remise éventuelle

`boutique_categories.csv` — 6 lignes, table de référence des catégories.

Variable	Type	Description
`categorie`	texte	nom de la catégorie (clé) ; `Jardin` n'a aucune commande
`taux_marge`	numérique	marge brute, en proportion du montant hors taxes
`univers`	texte	regroupement de catégories (`Personne`, `Habitat`, `Technologie`, `Loisirs`)

`boutique_campagne.csv` — 944 lignes, une par abonné à la newsletter ayant reçu l'emailing de novembre 2025 (test A/B).

Variable	Type	Description
`client_id`	texte	client destinataire (clé vers `clients`)
`version`	texte	version reçue, tirée au hasard : `A` (objet habituel) ou `B` (objet personnalisé avec le prénom)
`ouvert`	logique	l'email a été ouvert
`clic`	logique	le client a cliqué sur un lien de l'email (implique `ouvert`)
`achat`	logique	le client a acheté après avoir cliqué (implique `clic`)

Comment les données ont été simulées. Chaque client appartient à l'un de quatre segments « cachés » (non fournis dans les fichiers) : clients fidèles, chasseurs de promotions, clients occasionnels et nouveaux clients. Le segment détermine l'âge, le canal d'acquisition, l'abonnement à

la newsletter, le nombre de commandes (loi de Poisson), leur montant (loi log-normale), le recours aux promotions et les catégories achetées. Les dates de commande suivent une saisonnalité, avec un pic en novembre et décembre. Retrouver ces segments est l'objet du chapitre 6.

3.2 Importer et exporter des données

3.2.1 Le répertoire de travail

R lit et écrit les fichiers **relativement au répertoire de travail** (`getwd()` pour l'afficher). Avec un **projet RStudio**, c'est le dossier du projet ; on utilise alors des **chemins relatifs**.

3.2.2 Lire un fichier CSV

Le format CSV (*comma-separated values*) est le format d'échange le plus courant. Le package `readr` fournit `read_csv()`, qui renvoie un **tibble** et indique le type deviné pour chaque colonne :

```
clients   <- read_csv("../datasets/boutique_clients.csv")
commandes <- read_csv("../datasets/boutique_commandes.csv")
campagne  <- read_csv("../datasets/boutique_campagne.csv")
categories <- read_csv("../datasets/boutique_categories.csv")
```

`read_csv()` a reconnu automatiquement les dates (`date`, `date_inscription`) et les booléens (`promo`). Vérifiez toujours ces types : un montant lu comme du texte, par exemple à cause d'une virgule décimale, empêche tout calcul.

`readr`	R de base	Séparateur de champs	Séparateur décimal	Usage
`read_csv()`	`read.csv()`	`,`	`.`	CSV « anglo-saxon »
`read_csv2()`	`read.csv2()`	`;`	`,`	CSV produit par un Excel français
`read_tsv()`	`read.delim()`	tabulation	`.`	fichiers `.tsv`
`read_delim(..., delim = ...)`	`read.table(sep = ...)`	au choix	au choix	autres cas

La fonction de base `read.table()` lit aussi les fichiers texte à colonnes séparées par des espaces, comme `phbirths.dat` (poids de naissance de 1 115 bébés à Philadelphie) :

```
phbirths <- read.table("../datasets/phbirths.dat")
head(phbirths, 3)
```

```
#>   black educ smoke gestate grams
#> 1 FALSE    0  TRUE      40  2898
#> 2  TRUE    0  TRUE      26   994
#> 3 FALSE    2 FALSE      38  3977
```

3.2.3 Autres formats

Format	Package	Fonction
Excel `.xls`, `.xlsx`	`readxl`	`read_excel(chemin, sheet = 1)`
SAS, SPSS, Stata	`haven`	`read_sas()`, `read_sav()`, `read_dta()`
JSON (API web, outils marketing)	`jsonlite`	`fromJSON()`
Bases de données SQL	`DBI`	`dbGetQuery()`
Format R natif	base	`readRDS()`, `load()`

```
library(readxl)
chemin <- readxl_example("datasets.xlsx")    # fichier d'exemple fourni par
 ↪  readxl
excel_sheets(chemin)                         # noms des feuilles
```

```
#> [1] "mtcars"   "chickwts" "quakes"
```

```
read_excel(chemin, sheet = "mtcars", n_max = 3)
```

```
#> # A tibble: 3 x 11
#>     mpg   cyl  disp    hp  drat    wt  qsec    vs    am  gear  carb
#>   <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
#> 1  21       6   160   110  3.9   2.62  16.5     0     1     4     4
#> 2  21       6   160   110  3.9   2.88  17.0     0     1     4     4
#> 3  22.8     4   108    93  3.85  2.32  18.6     1     1     4     1
```

3.2.4 Écrire des données

```
write_csv(clients, "clients.csv")                # CSV standard
write_csv2(clients, "clients_excel_fr.csv")      # ; et virgule décimale, pour
 ↪  Excel
writexl::write_xlsx(clients, "clients.xlsx")     # Excel
saveRDS(clients, "clients.rds")                  # format R : conserve
 ↪  exactement les types
clients <- readRDS("clients.rds")
```

3.2.5 Données intégrées aux packages

```
data(hdv2003)        # rend le jeu de données disponible (onglet Environment)
d <- hdv2003         # copie sous un nom plus court
```

`data()` liste les jeux de données disponibles ; `?hdv2003` affiche leur documentation.

3.3 Structure d'un tableau de données

```
dim(clients)          # lignes, colonnes
```

```
#> [1] 2000    6
```

```
names(clients)        # noms des variables
```

```
#> [1] "client_id"         "age"              "region"
#> [4] "canal_acquisition" "date_inscription"  "newsletter"
```

```
glimpse(clients)      # structure (version dplyr de str())
```

```
#> Rows: 2,000
#> Columns: 6
#> $ client_id         <chr> "C0001", "C0002", "C0003", "C0004", "C0005",~
#> $ age               <dbl> 62, 39, 53, 38, 41, 35, 37, 29, 36, 35, 49, ~
#> $ region            <chr> "Île-de-France", "Occitanie", "Hauts-de-Fran~
#> $ canal_acquisition <chr> "Réseaux sociaux", "Email", "Direct", "Résea~
#> $ date_inscription  <date> 2023-10-25, 2022-01-29, 2023-05-04, 2023-06~
#> $ newsletter        <chr> "Oui", "Oui", "Non", "Non", "Oui", "Non", "O~
```

`View(clients)`, ou un clic sur le tableau dans l'onglet *Environment*, ouvre le tableau dans un visualiseur de type tableur.

On accède à une variable avec `$`, et on crée une variable en lui assignant une valeur :

```
head(commandes$montant)
```

```
#> [1]  22.09  54.99 129.26 177.88  68.39  52.00
```

```
commandes$montant_ht <- round(commandes$montant / 1.2, 2)
head(commandes[, c("montant", "montant_ht")], 3)
```

```
#> # A tibble: 3 x 2
#>   montant montant_ht
#>     <dbl>      <dbl>
#> 1    22.1       18.4
#> 2    55.0       45.8
#> 3   129.       108.
```

3.4 Analyse univariée

Les indicateurs et graphiques dépendent du type de la variable : **quantitative** (un nombre pouvant prendre de nombreuses valeurs : âge, montant) ou **qualitative** (un nombre limité de modalités : canal, région).

3.4.1 Variable quantitative

Indicateurs de position (centralité)

```
range(commandes$montant)
```

```
#> [1]   6.99 435.71
```

```
mean(commandes$montant)
```

```
#> [1] 64.55
```

```
median(commandes$montant)
```

```
#> [1] 54.43
```

Le montant moyen d'une commande (64.55 €) est nettement supérieur au montant médian (54.43 €) : quelques grosses commandes « tirent » la moyenne vers le haut. La médiane est plus **robuste** aux valeurs extrêmes. C'est fréquent pour les montants, les revenus ou le nombre de visites : regardez toujours les deux.

Indicateurs de dispersion

```
sd(commandes$montant)                                    # écart-type
```

```
#> [1] 41.63
```

```
quantile(commandes$montant, c(0.25, 0.5, 0.75))  # quartiles
```

```
#>   25%   50%   75%
#> 34.02 54.43 83.84
```

```
quantile(commandes$montant, seq(0.1, 0.9, 0.1))  # déciles
```

```
#>    10%    20%    30%    40%    50%    60%    70%    80%    90%
#>  23.16  30.35  37.84  45.69  54.43  64.40  76.40  93.39 119.44
```

```
IQR(commandes$montant)                                   # écart interquartile
```

```
#> [1] 49.82
```

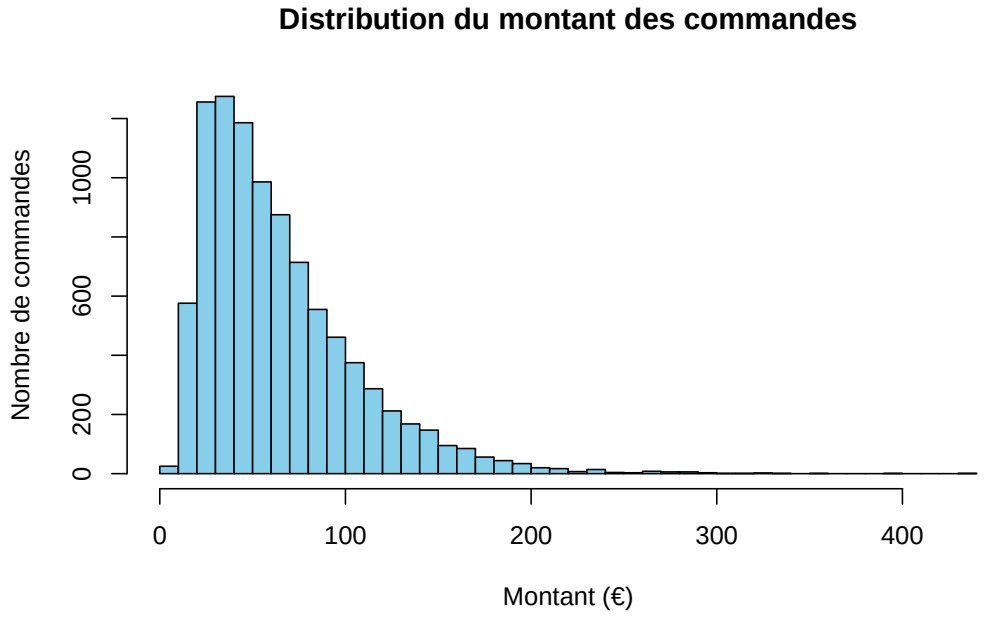
```
summary(commandes$montant)
```

```
#>    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
#>    6.99   34.02   54.43   64.55   83.84  435.71
```

Les déciles sont très utilisés en marketing : « les 10 % de commandes les plus élevées dépassent 119 € ».

Représentation graphique

```
hist(commandes$montant, breaks = 40, col = "skyblue",
     main = "Distribution du montant des commandes",
     xlab = "Montant (€)", ylab = "Nombre de commandes")
```



La distribution est **asymétrique à droite** : beaucoup de petites et moyennes commandes, peu de très grosses.

3.4.2 Variable qualitative

Le **tri à plat** compte les observations de chaque modalité :

```
table(clients$canal_acquisition)
```

```
#>
#>         Affiliation              Direct               Email
#>                 271                 317                 250
#> Moteur de recherche     Réseaux sociaux
#>                 586                 576
```

```
sort(table(clients$canal_acquisition), decreasing = TRUE)
```

```
#>
#> Moteur de recherche     Réseaux sociaux              Direct
#>                 586                 576                 317
#>         Affiliation               Email
#>                 271                 250
```

La fonction `freq()` de `questionr` ajoute les pourcentages et les valeurs manquantes :

```
freq(clients$canal_acquisition, sort = "dec", total = TRUE)
```

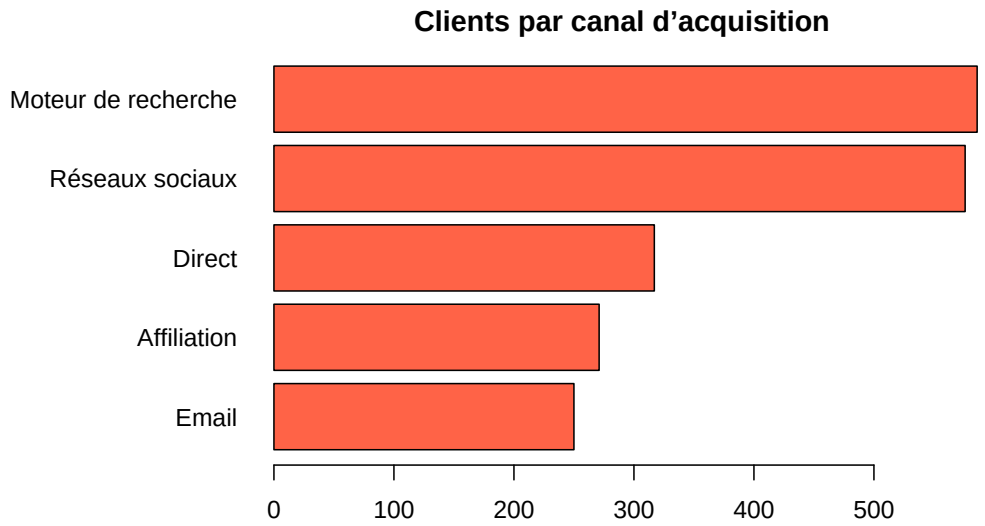
```
#>                        n     %  val%
#> Moteur de recherche  586  29.3  29.3
#> Réseaux sociaux      576  28.8  28.8
#> Direct               317  15.8  15.8
#> Affiliation          271  13.6  13.6
#> Email                250  12.5  12.5
#> Total               2000 100.0 100.0
```

```
freq(d$qualif)        # avec des valeurs manquantes (enquête hdv2003)
```

```
#>                           n    % val%
#> Ouvrier specialise      203 10.2 12.3
#> Ouvrier qualifie        292 14.6 17.7
#> Technicien               86  4.3  5.2
#> Profession intermediaire 160  8.0  9.7
#> Cadre                    260 13.0 15.7
#> Employe                  594 29.7 35.9
#> Autre                    58  2.9  3.5
#> NA                       347 17.3   NA
```

> `table()` n'affiche pas les valeurs manquantes par défaut. Utilisez `table(x, useNA = "ifany")` ou `freq()`.

```
par(mar = c(4, 10, 2, 1))                # marge gauche plus large pour les
 ↪  étiquettes
barplot(sort(table(clients$canal_acquisition)), horiz = TRUE, las = 1,
        col = "tomato", main = "Clients par canal d'acquisition")
```




```
par(mar = c(5, 4, 4, 2) + 0.1)            # marges par défaut
```

Le diagramme de Cleveland, `dotchart(sort(table(x)))`, est une alternative plus sobre au diagramme en barres.

3.5 Analyse bivariée

Étudier la relation entre deux variables : sont-elles liées ? Les outils dépendent des types des deux variables. Ils mettent en évidence des **associations**, pas des relations de cause à effet.

Variables	Tableau / indicateur	Graphique	Test
2 qualitatives	tableau croisé, pourcentages ligne ou colonne	`mosaicplot()`, barres	χ^2 (`chisq.test()`)
quantitative × qualitative	moyennes ou médianes par groupe	boîtes à moustaches	t de Student (`t.test()`), Wilcoxon (`wilcox.test()`)
2 quantitatives	corrélation (Pearson, Spearman)	nuage de points	test de corrélation (`cor.test()`), régression

3.5.1 Deux variables qualitatives : un test A/B

En novembre 2025, la boutique a envoyé une campagne d'emailing à ses abonnés. Chaque abonné a reçu, **au hasard**, la version A (objet habituel) ou la version B (objet personnalisé avec le prénom). C'est un **test A/B** : comme l'attribution est aléatoire, une différence de comportement entre les deux groupes peut être attribuée à la version de l'email.

```
tab_ouv <- table(campagne$version, campagne$ouvert)
tab_ouv
```

```
#>
#>     FALSE TRUE
#>   A   370   95
```

```
#>   B   347  132
```

```
lprop(tab_ouv)      # pourcentages en ligne (questionr)
```

```
#>
#>       FALSE TRUE  Total
#>   A    79.6  20.4 100.0
#>   B    72.4  27.6 100.0
#>   All  76.0  24.0 100.0
```

> **Pourcentages ligne ou colonne ?** On calcule les pourcentages **dans les modalités de la variable explicative** (indépendante). Ici, la version de l'email *explique* l'ouverture, et elle est en ligne : on lit donc les pourcentages en ligne pour comparer les taux d'ouverture de A et B.

Le taux d'ouverture est plus élevé pour B. Cette différence pourrait-elle être due au hasard du tirage ? Le **test du χ^2 d'indépendance** répond à cette question. Son hypothèse nulle est que les deux variables sont indépendantes (ici : que la version n'a aucun effet sur l'ouverture).

```
chisq.test(tab_ouv)
```

```
#>
#>  Pearson's Chi-squared test with Yates' continuity correction
#>
#> data:  tab_ouv
#> X-squared = 6.2, df = 1, p-value = 0.01
```

La p-valeur est inférieure à 0,05 : on rejette l'hypothèse d'indépendance au seuil de 5 %. L'objet personnalisé augmente le taux d'ouverture.

Et les achats ?

```
tab_achat <- table(campagne$version, campagne$achat)
lprop(tab_achat)
```

```
#>
#>       FALSE TRUE  Total
#>   A    98.5   1.5 100.0
#>   B    97.3   2.7 100.0
#>   All  97.9   2.1 100.0
```

```
chisq.test(tab_achat)
```

```
#>
#>  Pearson's Chi-squared test with Yates' continuity correction
#>
#> data:  tab_achat
#> X-squared = 1.1, df = 1, p-value = 0.3
```

Le taux d'achat est plus élevé avec B, mais la différence **n'est pas significative**. Cela ne prouve pas que B est sans effet sur les ventes : les achats sont rares (quelques dizaines), et le test manque de **puissance**. Pour conclure, il faudrait un échantillon beaucoup plus grand. « Pas de différence significative » ne veut pas dire « pas de différence ».

Résidus et graphique. Sur un tableau plus grand, les résidus du test indiquent les cases qui s'écartent de l'indépendance (au-delà de 2 en valeur absolue) :

```
tab_canal <- table(clients$canal_acquisition, clients$newsletter)
lprop(tab_canal)
```

```
#>
#>                        Non   Oui   Total
#>   Affiliation          47.6  52.4 100.0
#>   Direct               52.7  47.3 100.0
#>   Email                47.2  52.8 100.0
#>   Moteur de recherche  56.7  43.3 100.0
#>   Réseaux sociaux      53.8  46.2 100.0
#>   All                  52.8  47.2 100.0
```

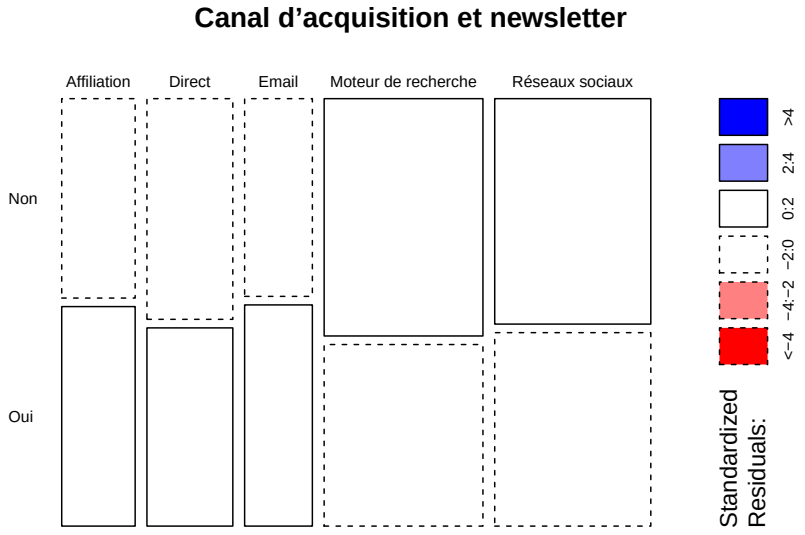
```
chisq.test(tab_canal)
```

```
#>
#>  Pearson's Chi-squared test
#>
#> data:  tab_canal
#> X-squared = 9.8, df = 4, p-value = 0.04
```

```
chisq.residuals(tab_canal)
```

```
#>
#>                         Non   Oui
#>   Affiliation         -1.18  1.25
#>   Direct              -0.03  0.03
#>   Email               -1.22  1.29
#>   Moteur de recherche  1.28 -1.36
#>   Réseaux sociaux      0.34 -0.36
```

```
mosaicplot(tab_canal, shade = TRUE, las = 1, main = "Canal d'acquisition et
 ↪  newsletter")
```



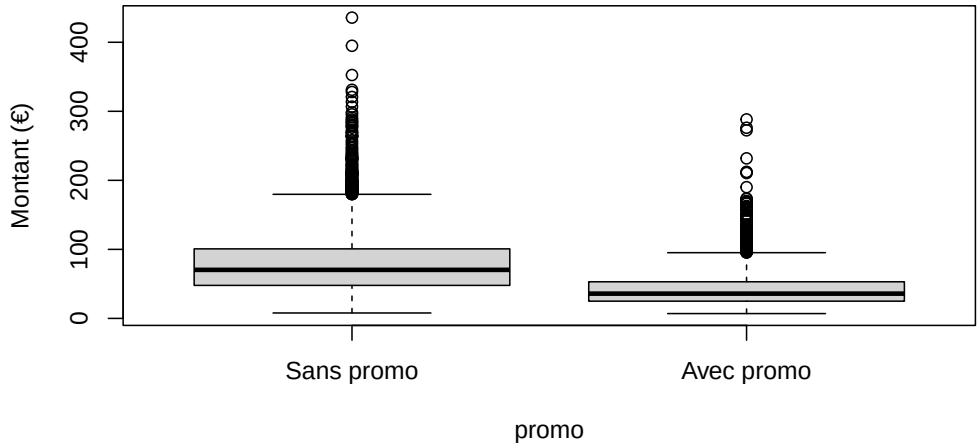
Dans un `mosaicplot`, la largeur des colonnes représente le poids de chaque canal et la hauteur des rectangles la répartition des abonnés dans chaque canal. Avec `shade = TRUE`, les cases dont le résidu dépasse 2 en valeur absolue sont colorées (bleu : sur-représentation, rouge : sous-représentation) ; un contour en pointillés signale un résidu négatif.

Ici, aucune case n'est colorée : la p-valeur globale est juste sous 5 %, mais aucun résidu ne dépasse 1,4 en valeur absolue. La liaison est donc **faible** : les clients venus par moteur de recherche sont un peu moins souvent abonnés, ceux venus par email ou affiliation un peu plus, sans écart marqué. Avec de grands effectifs, des écarts minimes deviennent significatifs ; il faut toujours regarder **l'ampleur** des écarts, pas seulement la p-valeur.

3.5.2 Une variable quantitative et une qualitative

Le montant d'une commande dépend-il de l'utilisation d'un code promo ?

```
boxplot(montant ~ promo, data = commandes,
        names = c("Sans promo", "Avec promo"), ylab = "Montant (€)")
```




```
tapply(commandes$montant, commandes$promo, mean)
```

```
#> FALSE  TRUE
#> 79.19 42.86
```

```
tapply(commandes$montant, commandes$promo, median)
```

```
#> FALSE  TRUE
#> 70.31 35.88
```

La formule `montant ~ promo` se lit « montant en fonction de promo ». Les commandes avec promotion sont nettement plus faibles. Le **test *t* de Student** compare les moyennes de deux groupes ; le **test de Wilcoxon** (non paramétrique) compare leurs distributions sans hypothèse de normalité :

```
t.test(montant ~ promo, data = commandes)
```

```
#>
#>  Welch Two Sample t-test
#>
#> data:  montant by promo
#> t = 50, df = 9432, p-value <2e-16
#> alternative hypothesis: true difference in means between group FALSE and group TRUE
 ↪  is not equal to 0
#> 95 percent confidence interval:
#>  34.92 37.74
#> sample estimates:
#> mean in group FALSE  mean in group TRUE
#>               79.19                42.86
```

```
wilcox.test(montant ~ promo, data = commandes)
```

```
#>
#>  Wilcoxon rank sum test with continuity correction
#>
#> data:  montant by promo
#> W = 1.7e+07, p-value <2e-16
#> alternative hypothesis: true location shift is not equal to 0
```

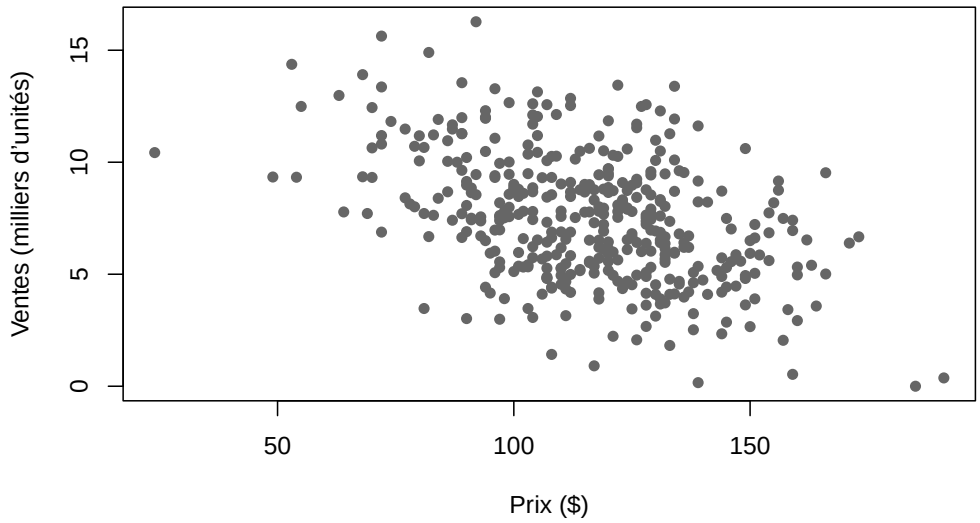
Interpréter avec prudence. La remise est de 20 %, mais les commandes en promotion sont plus faibles de près de moitié. Une grande partie de l'écart vient de **qui** utilise les promotions : les clients « chasseurs de promos » ont de toute façon de petits paniers. C'est un **facteur de confusion** : conclure que « les promotions divisent le panier par deux » serait faux.
De plus, les tests supposent des observations **indépendantes** ; or plusieurs commandes viennent du même client. Une analyse rigoureuse raisonnerait au niveau des clients (voir la section `dplyr` ci-dessous).
Enfin, le test t suppose la normalité des données **dans chaque groupe** ; avec de grands effectifs (plusieurs centaines par groupe), il reste fiable même si elle n'est pas vérifiée. Le test de normalité de Shapiro-Wilk (`shapiro.test()`) rejette presque toujours la normalité sur de grands échantillons : un histogramme est plus informatif.

3.5.3 Deux variables quantitatives

Le jeu `Carseats` (package `ISLR`) décrit les ventes d'un siège auto pour enfants dans 400 magasins. Ce sont des données **simulées** par les auteurs du livre *An Introduction to Statistical Learning*, mais réalistes : elles sont classiquement utilisées pour étudier l'effet du prix et de la publicité sur les ventes. Elles sont réutilisées au chapitre 4.

Variable	Description
`Sales`	ventes unitaires du magasin, en milliers
`Price`	prix pratiqué par le magasin pour ce siège ($)
`CompPrice`	prix pratiqué par le principal concurrent sur le même emplacement ($)
`Advertising`	budget publicitaire local de l'entreprise, en milliers de dollars
`ShelveLoc`	qualité de l'emplacement en rayon : `Bad`, `Medium`, `Good`
`Income`	revenu moyen de la zone de chalandise, en milliers de dollars
`Population`	population de la zone, en milliers
`Age`	âge moyen de la population locale
`Education`	niveau d'études moyen dans la zone
`Urban`	magasin en zone urbaine : `Yes` / `No`
`US`	magasin situé aux États-Unis : `Yes` / `No`

```
plot(Sales ~ Price, data = Carseats, pch = 16, col = "grey40",
     xlab = "Prix ($)", ylab = "Ventes (milliers d'unités)")
```



Le **coefficient de corrélation de Pearson** mesure l'intensité d'une liaison **linéaire**, entre -1 et 1 ; celui de **Spearman**, calculé sur les rangs, mesure une liaison **monotone** et résiste mieux aux valeurs extrêmes.

```
cor(Carseats$Price, Carseats$Sales)
```

```
#> [1] -0.445
```

```
cor(Carseats$Price, Carseats$Sales, method = "spearman")
```

```
#> [1] -0.4076
```

```
cor.test(Carseats$Price, Carseats$Sales)
```

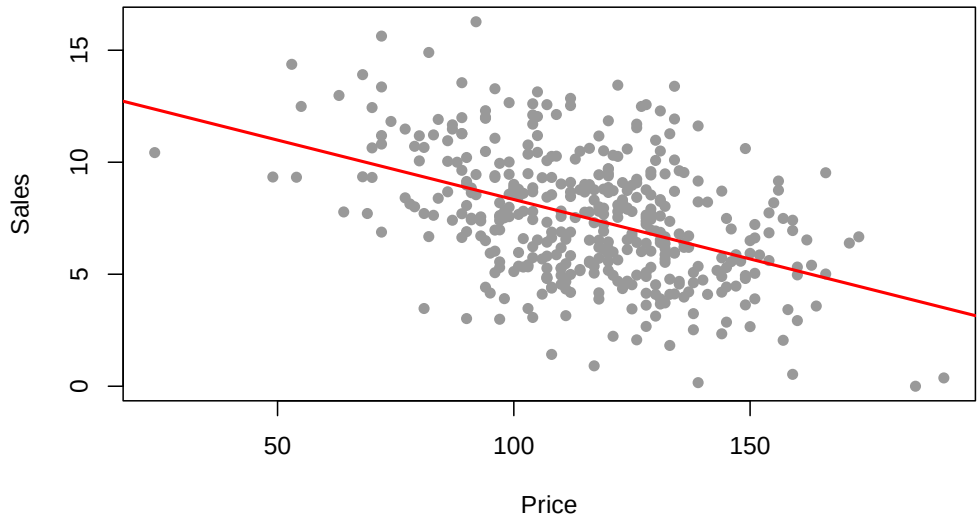
```
#>
#>  Pearson's product-moment correlation
#>
#> data:  Carseats$Price and Carseats$Sales
#> t = -9.9, df = 398, p-value <2e-16
#> alternative hypothesis: true correlation is not equal to 0
#> 95 percent confidence interval:
#>  -0.5203 -0.3627
#> sample estimates:
#>    cor
#> -0.445
```

La liaison est négative et modérée : les magasins qui vendent plus cher vendent moins, avec beaucoup de variabilité. La **droite de régression** résume cette tendance (détaillée au chapitre 4) :

```
reg <- lm(Sales ~ Price, data = Carseats)
coef(reg)
```

```
#> (Intercept)       Price
#>    13.64192    -0.05307
```

```
plot(Sales ~ Price, data = Carseats, pch = 16, col = "grey60")
abline(reg, col = "red", lwd = 2)
```



Chaque dollar supplémentaire est associé, en moyenne, à 53 sièges vendus en moins par magasin.

> Une corrélation proche de 0 signifie l'absence de liaison **linéaire**, pas l'absence de liaison : une relation en U a une corrélation nulle. Et une forte corrélation n'implique pas une causalité. **Regardez toujours le nuage de points.**

3.6 Manipuler des données avec `dplyr`

`dplyr` propose un ensemble cohérent de fonctions (« verbes ») qui prennent toutes **un data frame en premier argument** et **renvoient un data frame**. Les noms de colonnes s'écrivent directement, sans guillemets ni `$`.

Verbe	Rôle	Équivalent base R
`filter()`	garder des lignes selon une condition	`df[cond, ]`, `subset()`
`slice()`	garder des lignes selon leur position	`df[1:5, ]`
`arrange()`	trier les lignes	`df[order(...), ]`
`select()`, `rename()`, `relocate()`	choisir, renommer, réordonner des colonnes	`df[, c(...)]`
`mutate()`	créer ou modifier des colonnes	`df$x <- ...`
`summarise()`	résumer en une ligne	`aggregate()`
`group_by()`	définir des groupes	argument `by` d'`aggregate()`

3.6.1 Filtrer les lignes

```
filter(commandes, montant > 300)
```

```
#> # A tibble: 8 x 8
#>   commande_id client_id date       categorie nb_articles promo montant
#>   <chr>       <chr>     <date>     <chr>           <dbl> <lgl>   <dbl>
#> 1 K01194      C0998     2025-02-27 Maison              2 FALSE    331.
#> 2 K01602      C0155     2025-03-18 Maison              1 FALSE    353.
#> 3 K02251      C1415     2025-04-19 Maison              2 FALSE    321.
#> 4 K03091      C1338     2025-05-25 Mode                2 FALSE    307.
#> 5 K03481      C1762     2025-06-10 High-tech           2 FALSE    314.
#> 6 K05848      C1418     2025-09-27 High-tech           1 FALSE    395.
#> 7 K07149      C0946     2025-11-10 Maison              1 FALSE    436.
#> 8 K09359      C0247     2025-12-29 Mode                2 FALSE    328.
#> # i 1 more variable: montant_ht <dbl>
```

```
filter(commandes, categorie %in% c("Mode", "Beauté"), promo)   # la virgule
 ↪  signifie ET
```

```
#> # A tibble: 2,229 x 8
#>    commande_id client_id date       categorie nb_articles promo montant
#>    <chr>       <chr>     <date>     <chr>           <dbl> <lgl>   <dbl>
#>  1 K00001      C0103     2025-01-01 Mode                3 TRUE     22.1
#>  2 K00007      C0648     2025-01-01 Beauté              3 TRUE     43.8
#>  3 K00020      C1855     2025-01-01 Beauté              1 TRUE     38.4
#>  4 K00023      C0222     2025-01-02 Mode                3 TRUE     20
#>  5 K00027      C0622     2025-01-02 Mode                3 TRUE     36.0
#>  6 K00043      C1837     2025-01-02 Beauté              3 TRUE     48.6
#>  7 K00047      C0298     2025-01-03 Mode                2 TRUE     25.7
#>  8 K00050      C0517     2025-01-03 Mode                5 TRUE     17.0
#>  9 K00057      C0807     2025-01-03 Mode                6 TRUE     40.9
#> 10 K00074      C0686     2025-01-04 Beauté              3 TRUE     39.2
#> # i 2,219 more rows
#> # i 1 more variable: montant_ht <dbl>
```

```
filter(clients, is.na(age))                                    # âge non
 ↪  renseigné
```

```
#> # A tibble: 60 x 6
#>    client_id   age region  canal_acquisition date_inscription newsletter
#>    <chr>     <dbl> <chr>   <chr>             <date>           <chr>
#>  1 C0016        NA Autres~ Moteur de recher~ 2021-07-15       Non
#>  2 C0037        NA Autres~ Moteur de recher~ 2024-07-15       Non
#>  3 C0130        NA Île-de~ Email             2024-10-23       Oui
#>  4 C0156        NA Île-de~ Direct            2022-09-08       Oui
#>  5 C0196        NA Île-de~ Affiliation       2023-10-24       Oui
#>  6 C0387        NA Grand ~ Affiliation       2021-12-13       Oui
#>  7 C0404        NA Autres~ Direct            2023-04-28       Non
#>  8 C0416        NA Auverg~ Moteur de recher~ 2024-07-27       Non
#>  9 C0419        NA Autres~ Réseaux sociaux   2025-07-13       Non
#> 10 C0423        NA Autres~ Direct            2021-09-01       Oui
#> # i 50 more rows
```

`slice()` et ses variantes sélectionnent des lignes par position :

```
slice(commandes, 1:3)
```

```
#> # A tibble: 3 x 8
#>   commande_id client_id date       categorie nb_articles promo montant
#>   <chr>       <chr>     <date>     <chr>           <dbl> <lgl>   <dbl>
#> 1 K00001      C0103     2025-01-01 Mode                3 TRUE     22.1
#> 2 K00002      C0187     2025-01-01 Maison              1 FALSE    55.0
#> 3 K00003      C0259     2025-01-01 High-tech           1 FALSE   129.
```

```
#> # i 1 more variable: montant_ht <dbl>
```

```
slice_max(commandes, montant, n = 3)    # les 3 plus grosses commandes
```

```
#> # A tibble: 3 x 8
#>   commande_id client_id date       categorie nb_articles promo montant
#>   <chr>       <chr>     <date>     <chr>           <dbl> <lgl>   <dbl>
#> 1 K07149      C0946     2025-11-10 Maison              1 FALSE    436.
#> 2 K05848      C1418     2025-09-27 High-tech           1 FALSE    395.
#> 3 K01602      C0155     2025-03-18 Maison              1 FALSE    353.
#> # i 1 more variable: montant_ht <dbl>
```

```
slice_sample(clients, n = 3)            # 3 clients au hasard (remplace
 ↪  sample_n())
```

```
#> # A tibble: 3 x 6
#>   client_id   age region   canal_acquisition date_inscription newsletter
#>   <chr>     <dbl> <chr>    <chr>             <date>           <chr>
#> 1 C1865        18 Île-de-~ Réseaux sociaux   2025-10-18       Oui
#> 2 C1615        29 Nouvell~ Réseaux sociaux   2023-08-29       Oui
#> 3 C1749        46 Île-de-~ Direct            2021-09-20       Oui
```

> `filter()` élimine les lignes pour lesquelles la condition vaut `NA`. Pour garder ou repérer les valeurs manquantes, utilisez `is.na()` explicitement.

3.6.2 Trier

```
arrange(commandes, desc(montant))
```

```
#> # A tibble: 9,508 x 8
#>    commande_id client_id date       categorie nb_articles promo montant
#>    <chr>       <chr>     <date>     <chr>           <dbl> <lgl>   <dbl>
#>  1 K07149      C0946     2025-11-10 Maison              1 FALSE    436.
#>  2 K05848      C1418     2025-09-27 High-tech           1 FALSE    395.
#>  3 K01602      C0155     2025-03-18 Maison              1 FALSE    353.
#>  4 K01194      C0998     2025-02-27 Maison              2 FALSE    331.
#>  5 K09359      C0247     2025-12-29 Mode                2 FALSE    328.
#>  6 K02251      C1415     2025-04-19 Maison              2 FALSE    321.
#>  7 K03481      C1762     2025-06-10 High-tech           2 FALSE    314.
#>  8 K03091      C1338     2025-05-25 Mode                2 FALSE    307.
#>  9 K06873      C1428     2025-11-03 Sport               5 FALSE    298.
#> 10 K04004      C0332     2025-07-02 Maison              1 FALSE    296.
#> # i 9,498 more rows
#> # i 1 more variable: montant_ht <dbl>
```

```
arrange(clients, region, desc(date_inscription))
```

```
#> # A tibble: 2,000 x 6
#>    client_id   age region  canal_acquisition date_inscription newsletter
#>    <chr>     <dbl> <chr>   <chr>             <date>           <chr>
#>  1 C0258        25 Autres~ Réseaux sociaux   2025-12-24       Non
#>  2 C1029        33 Autres~ Affiliation       2025-12-22       Non
#>  3 C1980        22 Autres~ Moteur de recher~ 2025-12-21       Oui
#>  4 C1876        25 Autres~ Réseaux sociaux   2025-12-10       Non
#>  5 C0479        42 Autres~ Réseaux sociaux   2025-12-09       Oui
#>  6 C1355        47 Autres~ Réseaux sociaux   2025-12-04       Oui
#>  7 C1781        39 Autres~ Moteur de recher~ 2025-11-15       Oui
#>  8 C1149        23 Autres~ Réseaux sociaux   2025-11-11       Non
```

```
#>  9 C0772        18 Autres~ Réseaux sociaux   2025-11-07       Non
#> 10 C0178        36 Autres~ Réseaux sociaux   2025-11-02       Oui
#> # i 1,990 more rows
```

3.6.3 Sélectionner, renommer et réordonner des colonnes

```
select(commandes, client_id, date, montant)
```

```
#> # A tibble: 9,508 x 3
#>    client_id date       montant
#>    <chr>     <date>       <dbl>
#>  1 C0103     2025-01-01    22.1
#>  2 C0187     2025-01-01    55.0
#>  3 C0259     2025-01-01   129.
#>  4 C0282     2025-01-01   178.
#>  5 C0354     2025-01-01    68.4
#>  6 C0640     2025-01-01    52
#>  7 C0648     2025-01-01    43.8
#>  8 C0650     2025-01-01    73.4
#>  9 C0798     2025-01-01    19.4
#> 10 C0962     2025-01-01    90.5
#> # i 9,498 more rows
```

```
select(clients, -age, -region)                    # tout sauf...
```

```
#> # A tibble: 2,000 x 4
#>    client_id canal_acquisition   date_inscription newsletter
#>    <chr>     <chr>               <date>           <chr>
#>  1 C0001     Réseaux sociaux     2023-10-25       Oui
#>  2 C0002     Email               2022-01-29       Oui
#>  3 C0003     Direct              2023-05-04       Non
#>  4 C0004     Réseaux sociaux     2023-06-12       Non
#>  5 C0005     Réseaux sociaux     2023-12-27       Oui
#>  6 C0006     Moteur de recherche 2022-11-30       Non
#>  7 C0007     Email               2022-02-25       Oui
#>  8 C0008     Affiliation         2025-07-30       Non
#>  9 C0009     Moteur de recherche 2022-06-30       Non
#> 10 C0010     Affiliation         2024-05-07       Non
#> # i 1,990 more rows
```

```
select(commandes, starts_with("montant"))
```

```
#> # A tibble: 9,508 x 2
#>    montant montant_ht
#>      <dbl>      <dbl>
#>  1    22.1       18.4
#>  2    55.0       45.8
#>  3   129.       108.
#>  4   178.       148.
#>  5    68.4       57.0
#>  6    52         43.3
#>  7    43.8       36.5
#>  8    73.4       61.2
#>  9    19.4       16.1
#> 10    90.5       75.4
#> # i 9,498 more rows
```

```
select(commandes, where(is.numeric))
```

```
#> # A tibble: 9,508 x 3
#>    nb_articles montant montant_ht
#>          <dbl>   <dbl>      <dbl>
#>  1           3    22.1       18.4
#>  2           1    55.0       45.8
#>  3           1   129.       108.
#>  4           3   178.       148.
#>  5           2    68.4       57.0
#>  6           1    52         43.3
#>  7           3    43.8       36.5
#>  8           1    73.4       61.2
#>  9           4    19.4       16.1
#> 10           2    90.5       75.4
#> # i 9,498 more rows
```

```
rename(clients, canal = canal_acquisition)          # nouveau_nom = ancien_nom
```

```
#> # A tibble: 2,000 x 6
#>    client_id   age region             canal  date_inscription newsletter
#>    <chr>     <dbl> <chr>              <chr>  <date>           <chr>
#>  1 C0001        62 Île-de-France      Résea~ 2023-10-25       Oui
#>  2 C0002        39 Occitanie          Email  2022-01-29       Oui
#>  3 C0003        53 Hauts-de-France    Direct 2023-05-04       Non
#>  4 C0004        38 Hauts-de-France    Résea~ 2023-06-12       Non
#>  5 C0005        41 Grand Est          Résea~ 2023-12-27       Oui
#>  6 C0006        35 Autres régions     Moteu~ 2022-11-30       Non
#>  7 C0007        37 Nouvelle-Aquitaine Email  2022-02-25       Oui
#>  8 C0008        29 Autres régions     Affil~ 2025-07-30       Non
#>  9 C0009        36 Île-de-France      Moteu~ 2022-06-30       Non
#> 10 C0010        35 Île-de-France      Affil~ 2024-05-07       Non
#> # i 1,990 more rows
```

```
relocate(commandes, montant, .after = client_id)
```

```
#> # A tibble: 9,508 x 8
#>    commande_id client_id montant date       categorie nb_articles promo
#>    <chr>       <chr>       <dbl> <date>     <chr>           <dbl> <lgl>
#>  1 K00001      C0103        22.1 2025-01-01 Mode                3 TRUE
#>  2 K00002      C0187        55.0 2025-01-01 Maison              1 FALSE
#>  3 K00003      C0259       129.  2025-01-01 High-tech           1 FALSE
#>  4 K00004      C0282       178.  2025-01-01 High-tech           3 FALSE
#>  5 K00005      C0354        68.4 2025-01-01 Beauté              2 FALSE
#>  6 K00006      C0640        52   2025-01-01 High-tech           1 FALSE
#>  7 K00007      C0648        43.8 2025-01-01 Beauté              3 TRUE
#>  8 K00008      C0650        73.4 2025-01-01 Mode                1 FALSE
#>  9 K00009      C0798        19.4 2025-01-01 High-tech           4 TRUE
#> 10 K00010      C0962        90.5 2025-01-01 High-tech           2 FALSE
#> # i 9,498 more rows
#> # i 1 more variable: montant_ht <dbl>
```

Fonctions d'aide à la sélection : `starts_with()`, `ends_with()`, `contains()`, `where()`, `all_of()` / `any_of()` (noms donnés dans un vecteur de texte), `everything()`.

3.6.4 Créer des variables

```
commandes <- mutate(commandes,
                    mois = month(date),                          # package
                     ↪  lubridate (tidyverse)
```

```
                    jour_semaine = wday(date, label = TRUE, week_start = 1),
                    prix_moyen_article = montant / nb_articles)
select(commandes, date, mois, jour_semaine, prix_moyen_article)
```

```
#> # A tibble: 9,508 x 4
#>    date        mois jour_semaine prix_moyen_article
#>    <date>     <dbl> <ord>                     <dbl>
#>  1 2025-01-01     1 Wed                        7.36
#>  2 2025-01-01     1 Wed                       55.0
#>  3 2025-01-01     1 Wed                      129.
#>  4 2025-01-01     1 Wed                       59.3
#>  5 2025-01-01     1 Wed                       34.2
#>  6 2025-01-01     1 Wed                       52
#>  7 2025-01-01     1 Wed                       14.6
#>  8 2025-01-01     1 Wed                       73.4
#>  9 2025-01-01     1 Wed                        4.84
#> 10 2025-01-01     1 Wed                       45.2
#> # i 9,498 more rows
```

`case_when()` crée une variable selon plusieurs conditions, testées dans l'ordre :

```
commandes <- mutate(commandes,
                    taille_panier = case_when(
                      montant < 30  ~ "Petit",
                      montant < 100 ~ "Moyen",
                      .default = "Grand"
                    ))
count(commandes, taille_panier)
```

```
#> # A tibble: 3 x 2
#>   taille_panier     n
#>   <chr>         <int>
#> 1 Grand          1599
#> 2 Moyen          6054
#> 3 Petit          1855
```

3.6.5 Enchaîner les opérations : le *pipe*

Le *pipe* `|>` (natif depuis R 4.1) ou `%>%` (package `magrittr`) passe le résultat de gauche comme **premier argument** de la fonction de droite : `x |> f(y)` équivaut à `f(x, y)`. On lit la chaîne de haut en bas : « prendre les commandes, *puis* filtrer, *puis* sélectionner, *puis* trier ».

```
# Sans pipe : se lit de l'intérieur vers l'extérieur
arrange(select(filter(commandes, categorie == "High-tech"), client_id,
 ↪  montant), desc(montant))
```

```
#> # A tibble: 2,112 x 2
#>    client_id montant
#>    <chr>       <dbl>
#>  1 C1418        395.
#>  2 C1762        314.
#>  3 C0336        283.
#>  4 C0961        272.
#>  5 C1105        270.
#>  6 C0696        269.
#>  7 C1135        268.
#>  8 C1247        264.
```

```
#>  9 C1860        264.
#> 10 C1330        258.
#> # i 2,102 more rows
```

```
# Avec pipe : se lit de haut en bas
commandes |>
  filter(categorie == "High-tech") |>
  select(client_id, montant) |>
  arrange(desc(montant))
```

```
#> # A tibble: 2,112 x 2
#>    client_id montant
#>    <chr>       <dbl>
#>  1 C1418        395.
#>  2 C1762        314.
#>  3 C0336        283.
#>  4 C0961        272.
#>  5 C1105        270.
#>  6 C0696        269.
#>  7 C1135        268.
#>  8 C1247        264.
#>  9 C1860        264.
#> 10 C1330        258.
#> # i 2,102 more rows
```

3.6.6 Résumer par groupe

`summarise()` réduit une table à une ligne de statistiques ; combinée à `group_by()`, elle les calcule **pour chaque groupe** (*split-apply-combine*).

```
commandes |>
  summarise(chiffre_affaires = sum(montant),
            nb_commandes = n(),
            panier_moyen = mean(montant),
            nb_clients = n_distinct(client_id))
```

```
#> # A tibble: 1 x 4
#>   chiffre_affaires nb_commandes panier_moyen nb_clients
#>              <dbl>        <int>        <dbl>      <int>
#> 1          613761.         9508         64.6       1741
```

```
commandes |>
  group_by(categorie) |>
  summarise(chiffre_affaires = sum(montant),
            panier_moyen = mean(montant),
            part_promo = mean(promo)) |>
  arrange(desc(chiffre_affaires))
```

```
#> # A tibble: 5 x 4
#>   categorie chiffre_affaires panier_moyen part_promo
#>   <chr>                <dbl>        <dbl>      <dbl>
#> 1 High-tech          165067.         78.2      0.256
#> 2 Maison             153180.         75.8      0.307
#> 3 Mode               135095.         54.2      0.514
#> 4 Beauté              91593.         51.6      0.534
#> 5 Sport               68827.         62.2      0.399
```

`count()` est un raccourci pour `group_by()` + `summarise(n = n())` :

```
commandes |> count(mois)
```

```
#> # A tibble: 12 x 2
#>     mois     n
#>    <dbl> <int>
#>  1     1   716
#>  2     2   504
#>  3     3   638
#>  4     4   631
#>  5     5   748
#>  6     6   744
#>  7     7   630
#>  8     8   592
#>  9     9   727
#> 10    10   845
#> 11    11  1221
#> 12    12  1512
```

`group_by()` agit aussi sur `mutate()`, `filter()` et `slice_*()`. Pour garder **la plus grosse commande de chaque catégorie**, ou comparer chaque commande à la moyenne de son client :

```
commandes |>
  group_by(categorie) |>
  slice_max(montant, n = 1) |>
  ungroup() |>
  select(categorie, client_id, montant)
```

```
#> # A tibble: 5 x 3
#>   categorie client_id montant
#>   <chr>     <chr>       <dbl>
#> 1 Beauté    C0582        288.
#> 2 High-tech C1418        395.
#> 3 Maison    C0946        436.
#> 4 Mode      C0247        328.
#> 5 Sport     C1428        298.
```

```
commandes |>
  group_by(client_id) |>
  mutate(ecart_a_sa_moyenne = montant - mean(montant)) |>
  ungroup() |>
  select(client_id, montant, ecart_a_sa_moyenne)
```

```
#> # A tibble: 9,508 x 3
#>    client_id montant ecart_a_sa_moyenne
#>    <chr>       <dbl>              <dbl>
#>  1 C0103        22.1              -1.60
#>  2 C0187        55.0             -16.4
#>  3 C0259       129.               29.3
#>  4 C0282       178.               69.4
#>  5 C0354        68.4               0
#>  6 C0640        52                13.7
#>  7 C0648        43.8              -2.83
#>  8 C0650        73.4              23.2
#>  9 C0798        19.4             -19.6
#> 10 C0962        90.5             -18.8
#> # i 9,498 more rows
```

- Après un `group_by()`, la table reste groupée : terminez par `ungroup()`.
- `arrange()` ignore les groupes, sauf avec `.by_group = TRUE`.
- Dans `summarise()` et `mutate()`, une expression voit les colonnes **créées juste avant**. Écrire `summarise(montant = sum(montant), moyen = mean(montant))` calcule la moyenne… de la somme. Donnez un **nouveau nom** aux résultats.

3.6.7 Exemple marketing : les indicateurs RFM

L'analyse **RFM** décrit chaque client par trois indicateurs :

- **Récence** : nombre de jours depuis sa dernière commande ;
- **Fréquence** : nombre de commandes sur la période ;
- **Montant** : chiffre d'affaires total.

Un client récent, fréquent et dépensier est précieux ; un client ancien et peu actif est « en train de partir ». On calcule ces indicateurs à partir de la table des commandes, au 1er janvier 2026 :

```
date_reference <- as.Date("2026-01-01")

rfm <- commandes |>
  group_by(client_id) |>
  summarise(recence    = as.numeric(date_reference - max(date)),
            frequence = n(),
            montant_total = sum(montant)) |>
  ungroup()
rfm
```

```
#> # A tibble: 1,741 x 4
#>    client_id recence frequence montant_total
#>    <chr>       <dbl>     <int>         <dbl>
#>  1 C0001           7         8         808.
#>  2 C0002          16         8         227.
#>  3 C0003          78         2         146.
#>  4 C0004          38         5         203.
#>  5 C0005           3         7         298.
#>  6 C0006         276         1         114.
#>  7 C0007          15        14         410.
#>  8 C0008           1         3         113.
#>  9 C0009          44         1          67.4
#> 10 C0010          14        11         322.
#> # i 1,731 more rows
```

```
summary(rfm[, -1])
```

```
#>     recence        frequence     montant_total
#>  Min.   :  1.0   Min.   : 1.00   Min.   :  16.1
#>  1st Qu.: 12.0   1st Qu.: 2.00   1st Qu.: 104.2
#>  Median : 30.0   Median : 4.00   Median : 191.4
#>  Mean   : 68.8   Mean   : 5.46   Mean   : 352.5
#>  3rd Qu.: 86.0   3rd Qu.: 8.00   3rd Qu.: 355.2
#>  Max.   :365.0   Max.   :19.00   Max.   :1862.1
```

On peut ensuite répartir les clients en groupes de valeur, par exemple en quartiles de montant avec `ntile()` :

```
rfm |>
  mutate(quartile_montant = ntile(montant_total, 4)) |>
```

```
  group_by(quartile_montant) |>
  summarise(nb_clients = n(),
            ca_total = sum(montant_total)) |>
  mutate(part_ca = round(100 * ca_total / sum(ca_total), 1))
```

```
#> # A tibble: 4 x 4
#>   quartile_montant nb_clients ca_total part_ca
#>              <int>      <int>    <dbl>   <dbl>
#> 1                1        436   28000.     4.6
#> 2                2        435   63147.    10.3
#> 3                3        435  111592.    18.2
#> 4                4        435  411023.    67
```

Le quart des clients qui dépensent le plus réalise environ les deux tiers du chiffre d'affaires : une illustration de la loi de Pareto, très fréquente en commerce.

3.6.8 Décaler des valeurs : `lag()` et `lead()`

`lag()` renvoie la valeur **précédente**, `lead()` la valeur **suivante** :

```
lag(c(5, 3, 8))
```

```
#> [1] NA  5  3
```

```
lead(c(5, 3, 8))
```

```
#> [1]  3  8 NA
```

Combinées à `group_by()` et `arrange()`, elles calculent par exemple le délai entre deux commandes successives d'un même client :

```
delais <- commandes |>
  arrange(client_id, date) |>
  group_by(client_id) |>
  mutate(jours_depuis_precedente = as.numeric(date - lag(date))) |>
  ungroup()
delais |> select(client_id, date, jours_depuis_precedente) |> head(6)
```

```
#> # A tibble: 6 x 3
#>   client_id date       jours_depuis_precedente
#>   <chr>     <date>                       <dbl>
#> 1 C0001     2025-04-14                      NA
#> 2 C0001     2025-05-08                      24
#> 3 C0001     2025-06-22                      45
#> 4 C0001     2025-09-23                      93
#> 5 C0001     2025-09-25                       2
#> 6 C0001     2025-10-08                      13
```

```
median(delais$jours_depuis_precedente, na.rm = TRUE)
```

```
#> [1] 27
```

La première commande de chaque client n'a pas de précédente : `NA`.

3.6.9 Concaténer des tables

`bind_rows()` empile des tables (mêmes colonnes) ; `bind_cols()` les juxtapose **par position**, ce qui suppose des lignes dans le même ordre.

```
commandes_s1 <- filter(commandes, mois <= 6)
commandes_s2 <- filter(commandes, mois > 6)
bind_rows(S1 = commandes_s1, S2 = commandes_s2, .id = "semestre") |>
  count(semestre)
```

```
#> # A tibble: 2 x 2
#>   semestre     n
#>   <chr>    <int>
#> 1 S1        3981
#> 2 S2        5527
```

3.6.10 Joindre des tables

Les informations sont souvent réparties dans plusieurs tables reliées par une **clé** : `client_id` relie commandes et clients, `categorie` relie commandes et catégories.

Jointure	Lignes conservées
`left_join(x, y)`	toutes celles de `x` (la plus utilisée)
`right_join(x, y)`	toutes celles de `y`
`inner_join(x, y)`	celles présentes dans `x` **et** `y`
`full_join(x, y)`	celles présentes dans `x` **ou** `y`
`semi_join(x, y)`	celles de `x` qui ont une correspondance dans `y` (sans ajouter de colonnes)
`anti_join(x, y)`	celles de `x` **sans** correspondance dans `y`

Ajouter des informations : la marge de chaque catégorie, puis la marge réalisée par catégorie.

```
commandes |>
  left_join(categories, by = "categorie") |>
  mutate(marge = montant / 1.2 * taux_marge) |>
  group_by(categorie, univers) |>
  summarise(ca = sum(montant), marge = sum(marge), .groups = "drop") |>
  arrange(desc(marge))
```

```
#> # A tibble: 5 x 4
#>   categorie univers          ca  marge
#>   <chr>     <chr>         <dbl>  <dbl>
#> 1 Mode      Personne    135095. 61919.
#> 2 Maison    Habitat     153180. 51060.
#> 3 Beauté    Personne     91593. 45797.
#> 4 High-tech Technologie 165067. 24760.
#> 5 Sport     Loisirs      68827. 20074.
```

La catégorie High-tech fait un gros chiffre d'affaires mais une marge faible.

Compter les commandes par catégorie, y compris celles qui n'en ont aucune :

```
categories |>
  left_join(count(commandes, categorie), by = "categorie")
```

```
#> # A tibble: 6 x 4
#>   categorie taux_marge univers         n
#>   <chr>          <dbl> <chr>       <int>
#> 1 Mode            0.55 Personne     2493
#> 2 Maison          0.4  Habitat      2020
```

```
#> 3 Beauté          0.6  Personne     1776
#> 4 High-tech       0.18 Technologie  2112
#> 5 Sport           0.35 Loisirs      1107
#> 6 Jardin          0.45 Habitat        NA
```

« Jardin » n'a aucune commande : `n` vaut `NA` (on peut le remplacer par 0 avec `replace_na()`). Avec `inner_join()`, la ligne aurait disparu.

Trouver les clients inactifs (inscrits mais sans commande en 2025), par exemple pour une campagne de réactivation :

```
inactifs <- clients |> anti_join(commandes, by = "client_id")
nrow(inactifs)
```

```
#> [1] 259
```

```
freq(inactifs$canal_acquisition, sort = "dec")
```

```
#>                       n    % val%
#> Moteur de recherche 96 37.1 37.1
#> Réseaux sociaux     73 28.2 28.2
#> Direct              45 17.4 17.4
#> Email               23  8.9  8.9
#> Affiliation         22  8.5  8.5
```

Clés de noms différents : `by = c("nom_dans_x" = "nom_dans_y")`, par exemple avec `nycflights13` :

```
flights |>
  count(dest) |>
  left_join(airports, by = c("dest" = "faa")) |>
  select(dest, name, n) |>
  arrange(desc(n)) |>
  head(5)
```

```
#> # A tibble: 5 x 3
#>   dest  name                                n
#>   <chr> <chr>                           <int>
#> 1 ORD   Chicago Ohare Intl              17283
#> 2 ATL   Hartsfield Jackson Atlanta Intl  17215
#> 3 LAX   Los Angeles Intl                16174
#> 4 BOS   General Edward Lawrence Logan Intl 15508
#> 5 MCO   Orlando Intl                    14082
```

> Vérifiez toujours le nombre de lignes avant et après une jointure. Si la clé n'est pas unique dans `y`, une `left_join()` **duplique** des lignes de `x` (et `dplyr` affiche un avertissement).

3.7 Données rangées (*tidy data*)

Un jeu de données est **rangé** (*tidy*) lorsque :

1. chaque **variable** est une colonne ;
2. chaque **observation** est une ligne ;
3. chaque **valeur** est dans une cellule.

`dplyr` et `ggplot2` sont conçus pour ce format. Les tableaux de bord et les exports d'outils marketing sont pourtant souvent au format **large** : une colonne par mois, par exemple.

```
ca_large <- commandes |>
  group_by(categorie, mois) |>
  summarise(ca = round(sum(montant)), .groups = "drop") |>
  pivot_wider(names_from = mois, values_from = ca, names_prefix = "m")
ca_large
```

```
#> # A tibble: 5 x 13
#>   categorie    m1    m2    m3    m4    m5    m6    m7    m8    m9   m10
#>   <chr>     <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
#> 1 Beauté     7049  5492  6466  6041  5632  7593  6436  6835  6262  8553
#> 2 High-tech 12602  9774 11329  9884 12537 13962 11700  8473 11682 13780
#> 3 Maison    12822  8858 10069 10336 11386 12401  9988  9358 12927 12459
#> 4 Mode       9002  6679  8030  9199 12018  8977  9579  7394 11167 12325
#> 5 Sport      4715  3934  4922  4058  5507  5660  4132  4657  4756  6187
#> # i 2 more variables: m11 <dbl>, m12 <dbl>
```

Ce format est lisible pour un humain, mais « mois » n'y est pas une variable. `pivot_longer()` revient au format **long** :

```
ca_long <- ca_large |>
  pivot_longer(cols = starts_with("m"), names_to = "mois",
               names_prefix = "m", values_to = "ca") |>
  mutate(mois = as.integer(mois))
ca_long
```

```
#> # A tibble: 60 x 3
#>    categorie  mois    ca
#>    <chr>     <int> <dbl>
#>  1 Beauté        1  7049
#>  2 Beauté        2  5492
#>  3 Beauté        3  6466
#>  4 Beauté        4  6041
#>  5 Beauté        5  5632
#>  6 Beauté        6  7593
#>  7 Beauté        7  6436
#>  8 Beauté        8  6835
#>  9 Beauté        9  6262
#> 10 Beauté       10  8553
#> # i 50 more rows
```

`pivot_longer()` et `pivot_wider()` remplacent `melt()` / `dcast()` (package `reshape2`) et `stack()` / `unstack()` (R de base), que l'on trouve dans du code plus ancien. Autres fonctions utiles de `tidyr` : `separate_wider_delim()` (découper une colonne), `unite()` (coller des colonnes), `drop_na()`, `replace_na()`.

3.8 Valeurs manquantes

```
colSums(is.na(clients))
```

```
#>         client_id               age            region canal_acquisition
#>                 0                60                 0                 0
#>  date_inscription        newsletter
#>                 0                 0
```

```
clients |> summarise(across(everything(), \(x) sum(is.na(x))))
```

```
#> # A tibble: 1 x 6
#>   client_id   age region canal_acquisition date_inscription newsletter
#>       <int> <int>  <int>             <int>            <int>      <int>
#> 1         0    60      0                 0                0          0
```

```
mean(clients$age)                    # NA
```

```
#> [1] NA
```

```
mean(clients$age, na.rm = TRUE)
```

```
#> [1] 36.93
```

Que faire des valeurs manquantes ? Il n'y a pas de réponse unique : les supprimer, les remplacer (par la médiane, une valeur prédite, ...), ou les garder comme une modalité à part (« âge non renseigné »). Le choix dépend du **mécanisme** qui les a produites : si les clients les plus jeunes renseignent moins souvent leur âge, supprimer ces lignes biaise l'analyse. Documentez votre choix et, dans l'esprit du cadre PCS, vérifiez que vos conclusions n'en dépendent pas trop.

3.9 Générer des données

3.9.1 Séquences régulières

```
1:10
```

```
#>  [1]  1  2  3  4  5  6  7  8  9 10
```

```
seq(0, 1, by = 0.25)
```

```
#> [1] 0.00 0.25 0.50 0.75 1.00
```

```
seq(1, 3, length.out = 5)
```

```
#> [1] 1.0 1.5 2.0 2.5 3.0
```

```
seq(as.Date("2026-01-01"), by = "month", length.out = 4)
```

```
#> [1] "2026-01-01" "2026-02-01" "2026-03-01" "2026-04-01"
```

```
rep(c("A", "B"), times = 3)
```

```
#> [1] "A" "B" "A" "B" "A" "B"
```

```
rep(c("A", "B"), each = 2)
```

```
#> [1] "A" "A" "B" "B"
```

```
expand.grid(version = c("A", "B"), segment = c("nouveaux", "fidèles"))
```

```
#>   version  segment
#> 1       A nouveaux
#> 2       B nouveaux
#> 3       A  fidèles
#> 4       B  fidèles
```

3.9.2 Tirages aléatoires

Pour chaque loi de probabilité, R fournit quatre fonctions, dont le nom commence par :

Préfixe	Rôle	Exemple (loi normale)
`r`	tirage aléatoire	`rnorm(10, mean = 0, sd = 1)`
`d`	densité (ou probabilité ponctuelle)	`dnorm(0)`
`p`	fonction de répartition $P(X \leq q)$	`pnorm(1.96)`
`q`	quantile (inverse de `p`)	`qnorm(0.975)`

Lois courantes : `norm`, `unif`, `binom`, `pois`, `exp`, `lnorm`, `t`, `chisq`, `f`.

```
set.seed(42)                          # rend les tirages reproductibles
rbinom(5, size = 1000, prob = 0.03)   # acheteurs parmi 1 000 visiteurs,
 ↪  conversion de 3 %
```

```
#> [1] 38 30 36 27 46
```

```
rpois(5, lambda = 4)                  # nombre de commandes par client
```

```
#> [1] 9 7 1 4 3
```

```
rlnorm(3, meanlog = log(50), sdlog = 0.5)   # montants (loi log-normale,
 ↪  asymétrique)
```

```
#> [1] 96.50 81.54 77.70
```

```
sample(c("A", "B"), 10, replace = TRUE)     # attribution aléatoire d'un test
 ↪  A/B
```

```
#>  [1] "B" "A" "B" "A" "B" "B" "B" "A" "A" "B"
```

Simuler des données permet de tester une méthode sur des données dont on **connaît la vérité**.Par exemple, on a utilisé ces fonctions pour créer la boutique en ligne : - nombre de commandes selon une loi de Poisson, - montants selon une loi log-normale, - ouvertures d'emails selon une loi de Bernoulli. On a simulé aussi des segments de clients « cachés » que l'on cherchera à retrouver au chapitre 6.

Exemple : quelle taille d'échantillon pour un test A/B ? Si le taux d'achat réel est de 1,5 % avec A et de 2,5 % avec B, combien de fois sur 100 un test avec 500 abonnés par version détecte-t-il la différence ?

```
set.seed(1)
detecte <- replicate(1000, {
  achats_a <- rbinom(1, 500, 0.015)
  achats_b <- rbinom(1, 500, 0.025)
  prop.test(c(achats_a, achats_b), c(500, 500))$p.value < 0.05
})
mean(detecte)
```

```
#> [1] 0.14
```

Le test ne détecte la différence que dans une minorité des cas : c'est ce qu'on appelle un manque de **puissance**, et cela explique le résultat non significatif sur les achats de la campagne.

3.10 Résumé

- Lire : `read_csv()`, `read_csv2()` pour les CSV français, `read_excel()` ; écrire : `write_csv()`, `saveRDS()` ; chemins **relatifs**.
- Structure : `dim()`, `names()`, `glimpse()`, `View()` ; variables avec `$`.
- Univarié : moyenne **et** médiane, écart-type, quantiles, histogramme ; `table()`, `freq()`, diagramme en barres.
- Bivarié : tableau croisé + pourcentages sur la variable explicative + χ^2 ; boîtes à moustaches + t / Wilcoxon ; nuage de points + corrélation. Significatif $\neq$ important ; non significatif $\neq$ absence d'effet ; association $\neq$ causalité.
- `dplyr` : `filter`, `arrange`, `select`, `mutate`, `summarise`, `group_by`, `count`, `lag`, `*_join`, enchaînés avec `|>` ou `%>%`.
- *Tidy data* : une variable par colonne, une observation par ligne ; `pivot_longer()` / `pivot_wider()`.
- Valeurs manquantes : `is.na()`, `na.rm = TRUE` ; leur traitement est un **choix** à documenter.
- Générer : `seq()`, `rep()`, `r*/d*/p*/q*`, `sample()` ; toujours `set.seed()`.

3.11 Exercices

Exercice 3.1 — Analyse univariée (`hdv2003`). Le jeu `hdv2003` (package `questionr`) est un extrait de l'enquête *Histoire de vie* de l'INSEE (2003) : 2 000 personnes et 20 variables (âge, sexe, niveau d'études, catégorie socio-professionnelle, loisirs, heures passées devant la télévision…).

1. Créer un nouveau script qui charge l'extension `questionr`, charge le jeu de données `hdv2003`, le copie dans un objet `df`, puis affiche les dimensions et la liste des variables de `df`.
2. On étudie le temps passé quotidiennement devant la télévision (variable `heures.tv`). Afficher la valeur minimale, la valeur maximale, la moyenne, la médiane et l'écart-type, puis représenter la distribution par un histogramme en 10 classes.
3. On s'intéresse à l'importance accordée au travail (variable `trav.imp`). Faire le tri à plat des effectifs avec `table()`, puis un tri à plat avec effectifs et pourcentages. Y a-t-il des valeurs manquantes ?
4. Représenter les effectifs des modalités de `trav.imp` par un diagramme en barres trié, de couleur `"tomato"`. La commande `colors()` liste les noms de couleurs disponibles dans R.

Solution

```
library(questionr)
data(hdv2003)
df <- hdv2003
dim(df)
```

```
#> [1] 2000   20
```

```
names(df)
```

```
#>  [1] "id"           "age"          "sexe"         "nivetud"
#>  [5] "poids"        "occup"        "qualif"       "freres.soeurs"
#>  [9] "clso"         "relig"        "trav.imp"     "trav.satisf"
#> [13] "hard.rock"    "lecture.bd"   "peche.chasse" "cuisine"
#> [17] "bricol"       "cinema"       "sport"        "heures.tv"
```

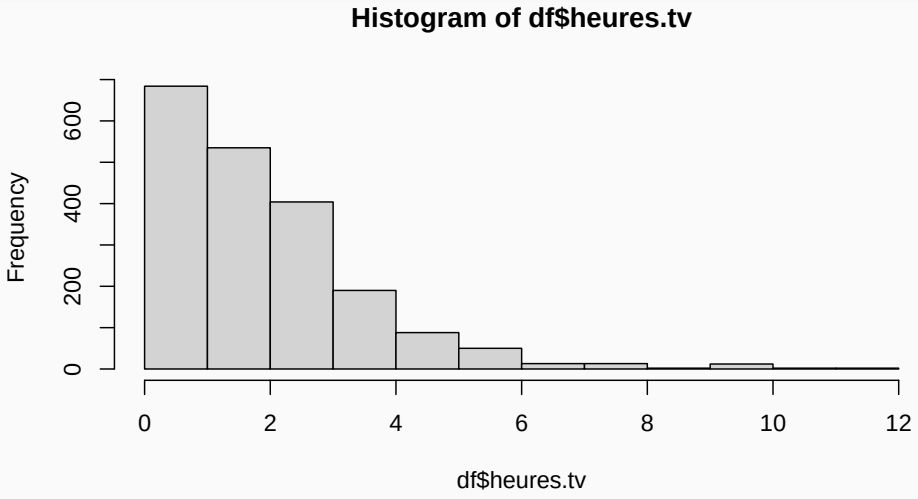
```
summary(df$heures.tv)
```

```
#>    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.    NA's
#>    0.00    1.00    2.00    2.25    3.00   12.00       5
```

```
sd(df$heures.tv, na.rm = TRUE)
```

```
#> [1] 1.776
```

```
hist(df$heures.tv, breaks = 10)
```



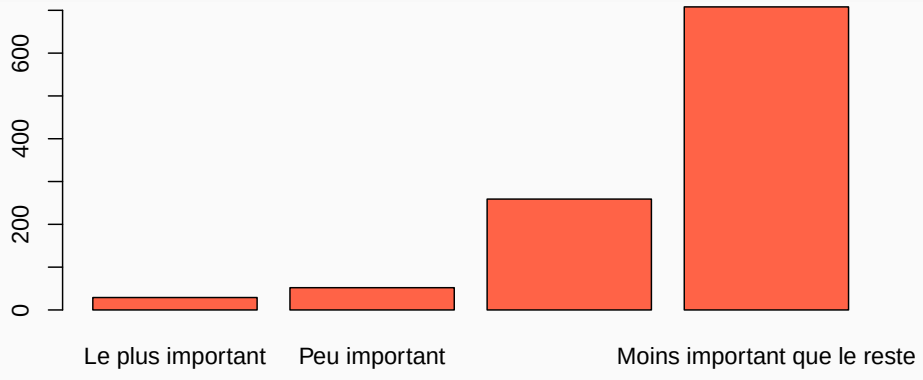

```
table(df$trav.imp)
```

```
#>
#>            Le plus important Aussi important que le reste
#>                           29                          259
#> Moins important que le reste                Peu important
#>                          708                           52
```

```
freq(df$trav.imp)             # de nombreuses valeurs manquantes : les
 ↪  personnes sans emploi
```

```
#>                                n    % val%
#> Le plus important             29  1.5  2.8
#> Aussi important que le reste 259 13.0 24.7
#> Moins important que le reste 708 35.4 67.6
#> Peu important                 52  2.6  5.0
#> NA                           952 47.6   NA
```

```
tab <- sort(table(df$trav.imp))
barplot(tab, col = "tomato")
```



Exercice 3.2 — Analyse bivariée (`hdv2003` et `rp2018`). Le jeu `rp2018` (package `questionr`) contient des résultats du recensement de 2018 pour les 5 417 communes françaises de plus de 2 000 habitants : département, population, part de cadres, d'ouvriers, de diplômés, de propriétaires, taux de chômage…

1. Dans `hdv2003`, faire le tableau croisé entre la catégorie socio-professionnelle (`qualif`) et le fait de croire ou non en l'existence des classes sociales (`clso`). Identifier la variable indépendante et la variable dépendante, calculer les pourcentages ligne ou colonne et interpréter.
2. Faire un test du χ^2. Peut-on rejeter l'hypothèse d'indépendance ? Représenter le tableau par un `mosaicplot` coloré selon les résidus.
3. Faire la boîte à moustaches du nombre d'heures passées devant la télévision (`heures.tv`) selon le statut d'occupation (`occup`), puis calculer la durée moyenne par statut avec `tapply()`.
4. Dans `rp2018`, représenter le nuage de points croisant le pourcentage de personnes sans diplôme (`dipl_aucun`) et le pourcentage de propriétaires (`proprio`), et calculer le coefficient de corrélation linéaire.

Solution

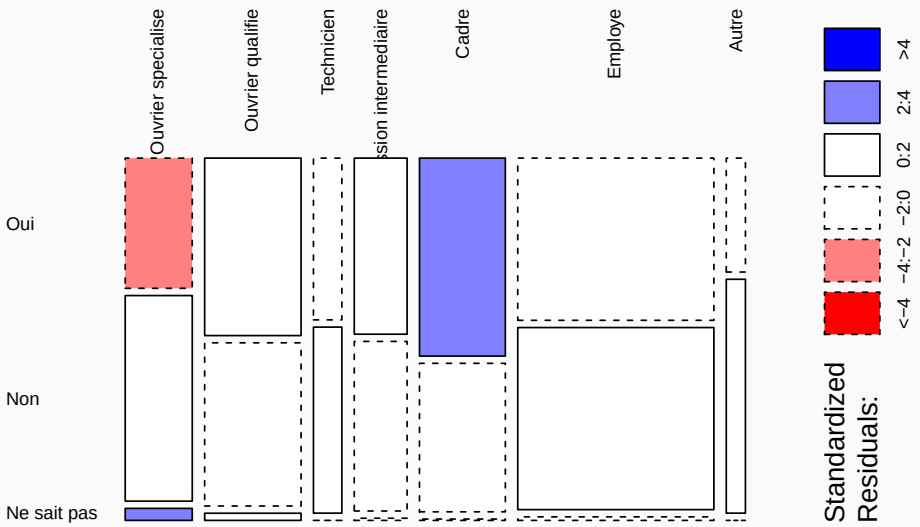
```
tab <- table(hdv2003$qualif, hdv2003$clso)
lprop(tab)          # qualif est la variable indépendante, en ligne
```

```
#>
#>                            Oui   Non   Ne sait pas Total
#>   Ouvrier specialise        37.4  59.1   3.4        100.0
#>   Ouvrier qualifie          51.0  46.9   2.1        100.0
#>   Technicien                46.5  53.5   0.0        100.0
#>   Profession intermediaire  50.6  48.8   0.6        100.0
#>   Cadre                     56.9  42.7   0.4        100.0
#>   Employe                   46.6  52.4   1.0        100.0
#>   Autre                     32.8  67.2   0.0        100.0
#>   All                       47.8  50.9   1.3        100.0
```

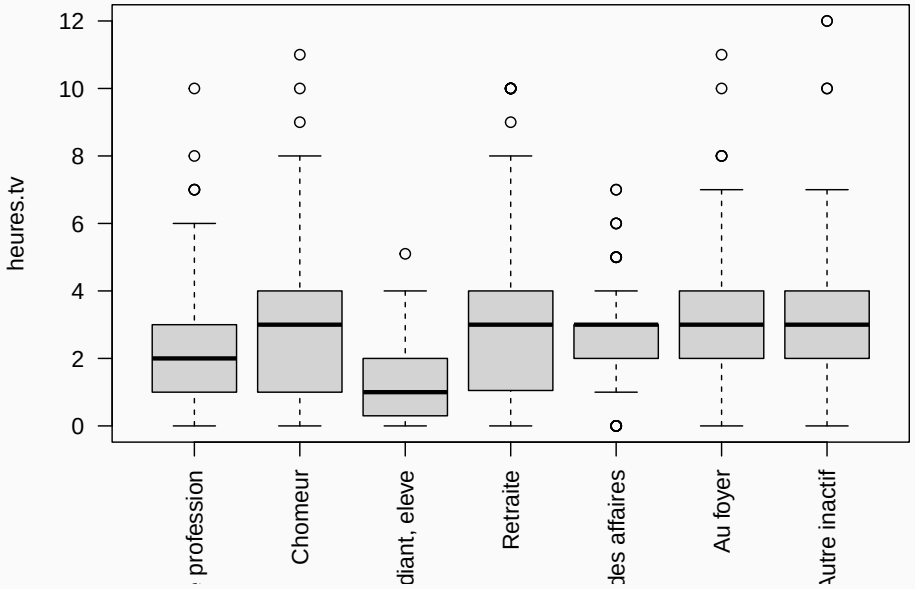
```
chisq.test(tab)     # p-valeur très faible : on rejette l'indépendance
```

```
#> Warning in chisq.test(tab): Chi-squared approximation may be incorrect
#>
#>  Pearson's Chi-squared test
#>
#> data:  tab
#> X-squared = 37, df = 12, p-value = 2e-04
```

```
mosaicplot(tab, shade = TRUE, las = 2, main = "")
```



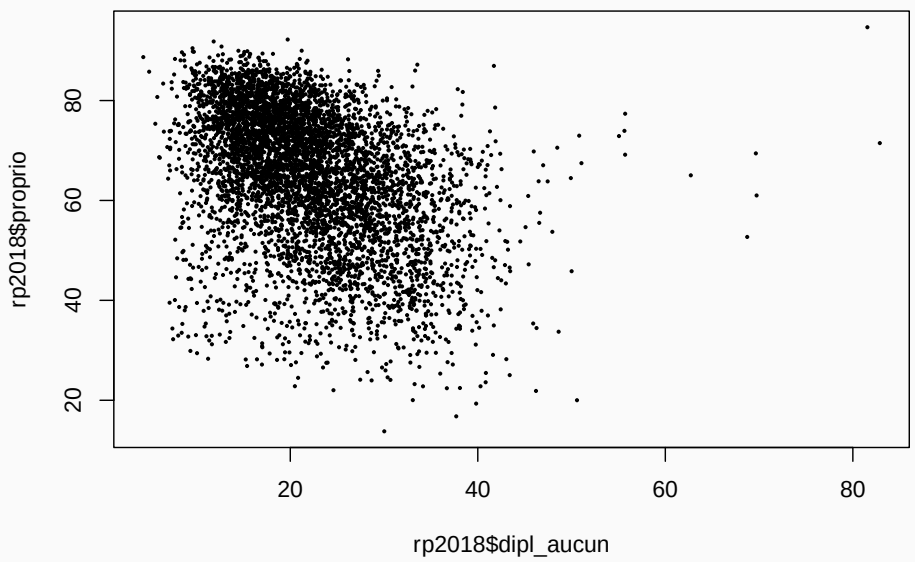

```
boxplot(heures.tv ~ occup, data = hdv2003, las = 2, xlab = "")
```




```
tapply(hdv2003$heures.tv, hdv2003$occup, mean, na.rm = TRUE)
```

```
#> Exerce une profession              Chomeur       Etudiant, eleve
#>                 1.822                2.875                 1.330
#>               Retraite   Retire des affaires             Au foyer
#>                 2.850                2.625                 2.822
#>         Autre inactif
#>                 3.265
```

```
data(rp2018)
plot(rp2018$dipl_aucun, rp2018$proprio, pch = 16, cex = 0.4)
```




```
cor(rp2018$dipl_aucun, rp2018$proprio)
```

```
#> [1] -0.4064
```

La croyance en l'existence des classes sociales varie selon la catégorie socio-professionnelle ; la relation entre la part de non-diplômés et la part de propriétaires est faible et négative.

Les exercices 3.3 à 3.6 portent sur le jeu `nycflights13` : les 336 776 vols au départ des trois aéroports de New York en 2013, répartis en plusieurs tables. `flights` décrit les vols (date, horaires, retards au départ `dep_delay` et à l'arrivée `arr_delay` en minutes, compagnie `carrier`, aéroports d'origine `origin` et de destination `dest`, durée de vol `air_time`, distance `distance` en miles…) ; `airports` décrit les aéroports (code `faa`, nom `name`, latitude `lat`, longitude `lon`, altitude `alt` en pieds, fuseau horaire `tz` et `tzone`) ; `airlines` donne le nom complet de chaque compagnie (`carrier`, `name`) ; `planes` décrit les avions (numéro de queue `tailnum`, année de fabrication `year`, constructeur…).

Exercice 3.3 — Sélectionner, trier, créer des variables.

1. Sélectionner la dixième ligne de la table `airports`.
2. Sélectionner les 5 premières lignes de la table `airlines`.
3. Sélectionner l'aéroport avec l'altitude la plus basse.
4. Sélectionner les vols du mois de juillet.
5. Sélectionner les vols avec un retard à l'arrivée compris entre 5 et 15 minutes.
6. Sélectionner les vols des compagnies Delta, United et American (`DL`, `UA`, `AA`).
7. Trier la table `flights` par retard au départ décroissant.
8. Sélectionner les colonnes `name`, `lat` et `lon` de la table `airports`.
9. Sélectionner toutes les colonnes de la table `airports` sauf `tz` et `tzone`.
10. Sélectionner toutes les colonnes de la table `flights` dont les noms se terminent par « delay ».
11. Dans la table `airports`, renommer la colonne `alt` en `altitude` et la colonne `tzone` en `fuseau_horaire`.
12. La colonne `alt` contient l'altitude en pieds. Créer une variable `alt_m` contenant l'altitude en mètres (on divise par 3,2808), puis ne garder que les colonnes `alt` et `alt_m`.
13. Réécrire le code de la question précédente avec le *pipe* `%>%` ou `|>`.
14. Sélectionner les vols à destination de San Francisco (`SFO`) et les trier par retard au départ décroissant.

15. Sélectionner les vols de septembre et d'octobre, conserver les colonnes `dest` et `dep_delay`, créer une variable `retard_h` contenant le retard au départ en heures, et garder uniquement les 5 lignes avec les plus grands retards.

Solution

```
slice(airports, 10)                                          # 1
```

```
#> # A tibble: 1 x 8
#>   faa   name                    lat   lon   alt    tz dst   tzone
#>   <chr> <chr>                 <dbl> <dbl> <dbl> <dbl> <chr> <chr>
#> 1 0S9   Jefferson County Intl  48.1 -123.   108    -8 A     America/Los~
```

```
slice(airlines, 1:5)                                         # 2
```

```
#> # A tibble: 5 x 2
#>   carrier name
#>   <chr>   <chr>
#> 1 9E      Endeavor Air Inc.
#> 2 AA      American Airlines Inc.
#> 3 AS      Alaska Airlines Inc.
#> 4 B6      JetBlue Airways
#> 5 DL      Delta Air Lines Inc.
```

```
slice_min(airports, alt, n = 1)                              # 3
```

```
#> # A tibble: 1 x 8
#>   faa   name          lat   lon   alt    tz dst   tzone
#>   <chr> <chr>       <dbl> <dbl> <dbl> <dbl> <chr> <chr>
#> 1 IPL   Imperial Co  32.8 -116.   -54    -8 A     America/Los_Angeles
```

```
filter(flights, month == 7)                                  # 4
```

```
#> # A tibble: 29,425 x 19
#>     year month   day dep_time sched_dep_time dep_delay arr_time
#>    <int> <int> <int>    <int>          <int>     <dbl>    <int>
#>  1  2013     7     1        1           2029       212      236
#>  2  2013     7     1        2           2359         3      344
#>  3  2013     7     1       29           2245       104      151
#>  4  2013     7     1       43           2130       193      322
#>  5  2013     7     1       44           2150       174      300
#>  6  2013     7     1       46           2051       235      304
#>  7  2013     7     1       48           2001       287      308
#>  8  2013     7     1       58           2155       183      335
#>  9  2013     7     1      100           2146       194      327
#> 10  2013     7     1      100           2245       135      337
#> # i 29,415 more rows
#> # i 12 more variables: sched_arr_time <int>, arr_delay <dbl>,
#> #   carrier <chr>, flight <int>, tailnum <chr>, origin <chr>,
#> #   dest <chr>, air_time <dbl>, distance <dbl>, hour <dbl>,
#> #   minute <dbl>, time_hour <dttm>
```

```
filter(flights, arr_delay >= 5, arr_delay <= 15)             # 5
```

```
#> # A tibble: 36,392 x 19
#>     year month   day dep_time sched_dep_time dep_delay arr_time
#>    <int> <int> <int>    <int>          <int>     <dbl>    <int>
#>  1  2013     1     1      517            515         2      830
#>  2  2013     1     1      554            558        -4      740
#>  3  2013     1     1      558            600        -2      753
#>  4  2013     1     1      558            600        -2      924
```

```
#>  5  2013     1     1      600            600         0      837
#>  6  2013     1     1      611            600        11      945
#>  7  2013     1     1      623            610        13      920
#>  8  2013     1     1      624            630        -6      840
#>  9  2013     1     1      629            630        -1      824
#> 10  2013     1     1      632            608        24      740
#> # i 36,382 more rows
#> # i 12 more variables: sched_arr_time <int>, arr_delay <dbl>,
#> #   carrier <chr>, flight <int>, tailnum <chr>, origin <chr>,
#> #   dest <chr>, air_time <dbl>, distance <dbl>, hour <dbl>,
#> #   minute <dbl>, time_hour <dttm>
```

```
filter(flights, carrier %in% c("DL", "UA", "AA"))          # 6
```

```
#> # A tibble: 139,504 x 19
#>     year month   day dep_time sched_dep_time dep_delay arr_time
#>    <int> <int> <int>    <int>          <int>     <dbl>    <int>
#>  1  2013     1     1      517            515         2      830
#>  2  2013     1     1      533            529         4      850
#>  3  2013     1     1      542            540         2      923
#>  4  2013     1     1      554            600        -6      812
#>  5  2013     1     1      554            558        -4      740
#>  6  2013     1     1      558            600        -2      753
#>  7  2013     1     1      558            600        -2      924
#>  8  2013     1     1      558            600        -2      923
#>  9  2013     1     1      559            600        -1      941
#> 10  2013     1     1      559            600        -1      854
#> # i 139,494 more rows
#> # i 12 more variables: sched_arr_time <int>, arr_delay <dbl>,
#> #   carrier <chr>, flight <int>, tailnum <chr>, origin <chr>,
#> #   dest <chr>, air_time <dbl>, distance <dbl>, hour <dbl>,
#> #   minute <dbl>, time_hour <dttm>
```

```
arrange(flights, desc(dep_delay))                          # 7
```

```
#> # A tibble: 336,776 x 19
#>     year month   day dep_time sched_dep_time dep_delay arr_time
#>    <int> <int> <int>    <int>          <int>     <dbl>    <int>
#>  1  2013     1     9      641            900      1301     1242
#>  2  2013     6    15     1432           1935      1137     1607
#>  3  2013     1    10     1121           1635      1126     1239
#>  4  2013     9    20     1139           1845      1014     1457
#>  5  2013     7    22      845           1600      1005     1044
#>  6  2013     4    10     1100           1900       960     1342
#>  7  2013     3    17     2321            810       911      135
#>  8  2013     6    27      959           1900       899     1236
#>  9  2013     7    22     2257            759       898      121
#> 10  2013    12     5      756           1700       896     1058
#> # i 336,766 more rows
#> # i 12 more variables: sched_arr_time <int>, arr_delay <dbl>,
#> #   carrier <chr>, flight <int>, tailnum <chr>, origin <chr>,
#> #   dest <chr>, air_time <dbl>, distance <dbl>, hour <dbl>,
#> #   minute <dbl>, time_hour <dttm>
```

```
select(airports, name, lat, lon)                           # 8
```

```
#> # A tibble: 1,458 x 3
#>    name                          lat    lon
#>    <chr>                       <dbl>  <dbl>
#>  1 Lansdowne Airport            41.1  -80.6
```

```
#>  2 Moton Field Municipal Airport   32.5  -85.7
#>  3 Schaumburg Regional             42.0  -88.1
#>  4 Randall Airport                 41.4  -74.4
#>  5 Jekyll Island Airport           31.1  -81.4
#>  6 Elizabethton Municipal Airport  36.4  -82.2
#>  7 Williams County Airport         41.5  -84.5
#>  8 Finger Lakes Regional Airport   42.9  -76.8
#>  9 Shoestring Aviation Airfield    39.8  -76.6
#> 10 Jefferson County Intl           48.1 -123.
#> # i 1,448 more rows
```

```
select(airports, -tz, -tzone)                                  # 9
```

```
#> # A tibble: 1,458 x 6
#>    faa   name                           lat    lon   alt dst
#>    <chr> <chr>                        <dbl>  <dbl> <dbl> <chr>
#>  1 04G   Lansdowne Airport             41.1  -80.6  1044 A
#>  2 06A   Moton Field Municipal Airport 32.5  -85.7   264 A
#>  3 06C   Schaumburg Regional           42.0  -88.1   801 A
#>  4 06N   Randall Airport               41.4  -74.4   523 A
#>  5 09J   Jekyll Island Airport         31.1  -81.4    11 A
#>  6 0A9   Elizabethton Municipal Airport 36.4  -82.2  1593 A
#>  7 0G6   Williams County Airport       41.5  -84.5   730 A
#>  8 0G7   Finger Lakes Regional Airport 42.9  -76.8   492 A
#>  9 0P2   Shoestring Aviation Airfield  39.8  -76.6  1000 U
#> 10 0S9   Jefferson County Intl         48.1 -123.    108 A
#> # i 1,448 more rows
```

```
select(flights, ends_with("delay"))                            # 10
```

```
#> # A tibble: 336,776 x 2
#>    dep_delay arr_delay
#>        <dbl>     <dbl>
#>  1         2        11
#>  2         4        20
#>  3         2        33
#>  4        -1       -18
#>  5        -6       -25
#>  6        -4        12
#>  7        -5        19
#>  8        -3       -14
#>  9        -3        -8
#> 10        -2         8
#> # i 336,766 more rows
```

```
rename(airports, altitude = alt, fuseau_horaire = tzone)    # 11
```

```
#> # A tibble: 1,458 x 8
#>    faa   name            lat    lon altitude    tz dst   fuseau_horaire
#>    <chr> <chr>         <dbl>  <dbl>    <dbl> <dbl> <chr> <chr>
#>  1 04G   Lansdowne Air~  41.1  -80.6     1044    -5 A     America/New_Y~
#>  2 06A   Moton Field M~  32.5  -85.7      264    -6 A     America/Chica~
#>  3 06C   Schaumburg Re~  42.0  -88.1      801    -6 A     America/Chica~
#>  4 06N   Randall Airpo~  41.4  -74.4      523    -5 A     America/New_Y~
#>  5 09J   Jekyll Island~  31.1  -81.4       11    -5 A     America/New_Y~
#>  6 0A9   Elizabethton ~  36.4  -82.2     1593    -5 A     America/New_Y~
#>  7 0G6   Williams Coun~  41.5  -84.5      730    -5 A     America/New_Y~
#>  8 0G7   Finger Lakes ~  42.9  -76.8      492    -5 A     America/New_Y~
#>  9 0P2   Shoestring Av~  39.8  -76.6     1000    -5 U     America/New_Y~
#> 10 0S9   Jefferson Cou~  48.1 -123.       108    -8 A     America/Los_A~
```

```
#> # i 1,448 more rows
```

```
select(mutate(airports, alt_m = alt / 3.2808), alt, alt_m) # 12
```

```
#> # A tibble: 1,458 x 2
#>      alt  alt_m
#>    <dbl>  <dbl>
#>  1  1044 318.
#>  2   264  80.5
#>  3   801 244.
#>  4   523 159.
#>  5    11   3.35
#>  6  1593 486.
#>  7   730 223.
#>  8   492 150.
#>  9  1000 305.
#> 10   108  32.9
#> # i 1,448 more rows
```

```
airports |>                                                    # 13
  mutate(alt_m = alt / 3.2808) |>
  select(alt, alt_m)
```

```
#> # A tibble: 1,458 x 2
#>      alt  alt_m
#>    <dbl>  <dbl>
#>  1  1044 318.
#>  2   264  80.5
#>  3   801 244.
#>  4   523 159.
#>  5    11   3.35
#>  6  1593 486.
#>  7   730 223.
#>  8   492 150.
#>  9  1000 305.
#> 10   108  32.9
#> # i 1,448 more rows
```

```
flights |>                                                     # 14
  filter(dest == "SFO") |>
  arrange(desc(dep_delay))
```

```
#> # A tibble: 13,331 x 19
#>     year month   day dep_time sched_dep_time dep_delay arr_time
#>    <int> <int> <int>    <int>          <int>     <dbl>    <int>
#>  1  2013     9    20     1139           1845      1014     1457
#>  2  2013     7     7     2123           1030       653       17
#>  3  2013     7     7     2059           1030       629      106
#>  4  2013     7     6      149           1600       589      456
#>  5  2013     7    10      133           1800       453      455
#>  6  2013     7    10     2342           1630       432      312
#>  7  2013     7     7     2204           1525       399      107
#>  8  2013     7     7     2306           1630       396      250
#>  9  2013     6    23     1833           1200       393       NA
#> 10  2013     7    10     2232           1609       383      138
#> # i 13,321 more rows
#> # i 12 more variables: sched_arr_time <int>, arr_delay <dbl>,
#> #   carrier <chr>, flight <int>, tailnum <chr>, origin <chr>,
#> #   dest <chr>, air_time <dbl>, distance <dbl>, hour <dbl>,
```

```
#> #   minute <dbl>, time_hour <dttm>
```

```
flights |>                                                    # 15
  filter(month %in% c(9, 10)) |>
  select(dest, dep_delay) |>
  mutate(retard_h = dep_delay / 60) |>
  slice_max(dep_delay, n = 5)
```

```
#> # A tibble: 5 x 3
#>   dest  dep_delay retard_h
#>   <chr>     <dbl>    <dbl>
#> 1 SFO        1014    16.9
#> 2 ATL         702    11.7
#> 3 DTW         696    11.6
#> 4 ATL         602    10.0
#> 5 MSP         593     9.88
```

Exercice 3.4 — Opérations groupées.

16. Afficher le nombre de vols par mois.
17. Trier la table obtenue selon le nombre de vols croissant.
18. Calculer la distance moyenne des vols selon l'aéroport de départ.
19. Calculer le nombre de vols à destination de Los Angeles (`LAX`) pour chaque mois de l'année.
20. Calculer le nombre de vols selon le mois et la destination.
21. Pour chaque mois, ne garder que la destination avec le nombre maximal de vols.
22. Calculer la part (en %) des vols annuels réalisée chaque mois.
23. Pour chaque aéroport de départ, trouver la destination dont la durée moyenne de vol est la plus longue.

Solution

```
flights |> count(month)                                       # 16
```

```
#> # A tibble: 12 x 2
#>    month     n
#>    <int> <int>
#>  1     1 27004
#>  2     2 24951
#>  3     3 28834
#>  4     4 28330
#>  5     5 28796
#>  6     6 28243
#>  7     7 29425
#>  8     8 29327
#>  9     9 27574
#> 10    10 28889
#> 11    11 27268
#> 12    12 28135
```

```
flights |> count(month) |> arrange(n)                         # 17
```

```
#> # A tibble: 12 x 2
#>    month     n
#>    <int> <int>
#>  1     2 24951
#>  2     1 27004
```

```
#>  3    11 27268
#>  4     9 27574
#>  5    12 28135
#>  6     6 28243
#>  7     4 28330
#>  8     5 28796
#>  9     3 28834
#> 10    10 28889
#> 11     8 29327
#> 12     7 29425
```

```
flights |>                                                    # 18
  group_by(origin) |>
  summarise(distance_moyenne = mean(distance))
```

```
#> # A tibble: 3 x 2
#>   origin distance_moyenne
#>   <chr>             <dbl>
#> 1 EWR               1057.
#> 2 JFK               1266.
#> 3 LGA                780.
```

```
flights |> filter(dest == "LAX") |> count(month)              # 19
```

```
#> # A tibble: 12 x 2
#>    month     n
#>    <int> <int>
#>  1     1  1159
#>  2     2  1030
#>  3     3  1178
#>  4     4  1382
#>  5     5  1453
#>  6     6  1430
#>  7     7  1500
#>  8     8  1505
#>  9     9  1384
#> 10    10  1409
#> 11    11  1336
#> 12    12  1408
```

```
flights |> count(month, dest)                                 # 20
```

```
#> # A tibble: 1,113 x 3
#>    month dest      n
#>    <int> <chr> <int>
#>  1     1 ALB      64
#>  2     1 ATL    1396
#>  3     1 AUS     169
#>  4     1 AVL       2
#>  5     1 BDL      37
#>  6     1 BHM      25
#>  7     1 BNA     399
#>  8     1 BOS    1245
#>  9     1 BQN      93
#> 10     1 BTV     223
#> # i 1,103 more rows
```

```
flights |>                                                    # 21
  count(month, dest) |>
  group_by(month) |>
  slice_max(n, n = 1) |>
  ungroup()
```

```
#> # A tibble: 12 x 3
#>    month dest      n
#>    <int> <chr> <int>
#>  1     1 ATL    1396
#>  2     2 ATL    1267
#>  3     3 ATL    1448
#>  4     4 ATL    1490
#>  5     5 ORD    1582
#>  6     6 ORD    1547
#>  7     7 ORD    1573
#>  8     8 ORD    1604
#>  9     9 ORD    1582
#> 10    10 ORD    1604
#> 11    11 ATL    1384
#> 12    12 ATL    1463
```

```
flights |>                                                    # 22
  count(month) |>
  mutate(part = round(100 * n / sum(n), 1))
```

```
#> # A tibble: 12 x 3
#>    month     n  part
#>    <int> <int> <dbl>
#>  1     1 27004   8
#>  2     2 24951   7.4
#>  3     3 28834   8.6
#>  4     4 28330   8.4
#>  5     5 28796   8.6
#>  6     6 28243   8.4
#>  7     7 29425   8.7
#>  8     8 29327   8.7
#>  9     9 27574   8.2
#> 10    10 28889   8.6
#> 11    11 27268   8.1
#> 12    12 28135   8.4
```

```
flights |>                                                    # 23
  group_by(origin, dest) |>
  summarise(duree_moyenne = mean(air_time, na.rm = TRUE), .groups =
   ↪  "drop") |>
  group_by(origin) |>
  slice_max(duree_moyenne, n = 1) |>
  ungroup()
```

```
#> # A tibble: 3 x 3
#>   origin dest  duree_moyenne
#>   <chr>  <chr>         <dbl>
#> 1 EWR    HNL            612.
#> 2 JFK    HNL            623.
#> 3 LGA    DEN            228.
```

Exercice 3.5 — Jointures.

24. Faire la jointure de la table `airlines` sur la table `flights` à l'aide de `left_join()`.
25. Calculer le retard moyen au départ pour chaque compagnie (avec son nom complet) et trier.
26. Faire la jointure de la table `airports` sur la table `flights` en utilisant comme clé le code de l'aéroport de destination.
27. Pour chaque mois, afficher le nom de l'aéroport de destination ayant eu le plus petit nombre de vols.
28. Créer une table contenant uniquement les noms de l'aéroport de départ et de l'aéroport d'arrivée de chaque vol.
29. Calculer le nombre de vols selon l'aéroport de destination, puis fusionner avec `airports` via `left_join()`. Stocker le résultat dans `flights_dest`.

Solution

```
left_join(flights, airlines, by = "carrier")                # 24
```

```
#> # A tibble: 336,776 x 20
#>     year month   day dep_time sched_dep_time dep_delay arr_time
#>    <int> <int> <int>    <int>          <int>     <dbl>    <int>
#>  1  2013     1     1      517            515         2      830
#>  2  2013     1     1      533            529         4      850
#>  3  2013     1     1      542            540         2      923
#>  4  2013     1     1      544            545        -1     1004
#>  5  2013     1     1      554            600        -6      812
#>  6  2013     1     1      554            558        -4      740
#>  7  2013     1     1      555            600        -5      913
#>  8  2013     1     1      557            600        -3      709
#>  9  2013     1     1      557            600        -3      838
#> 10  2013     1     1      558            600        -2      753
#> # i 336,766 more rows
#> # i 13 more variables: sched_arr_time <int>, arr_delay <dbl>,
#> #   carrier <chr>, flight <int>, tailnum <chr>, origin <chr>,
#> #   dest <chr>, air_time <dbl>, distance <dbl>, hour <dbl>,
#> #   minute <dbl>, time_hour <dttm>, name <chr>
```

```
flights |>                                                  # 25
  group_by(carrier) |>
  summarise(retard_moyen = mean(dep_delay, na.rm = TRUE)) |>
  left_join(airlines, by = "carrier") |>
  arrange(desc(retard_moyen))
```

```
#> # A tibble: 16 x 3
#>    carrier retard_moyen name
#>    <chr>          <dbl> <chr>
#>  1 F9              20.2  Frontier Airlines Inc.
#>  2 EV              20.0  ExpressJet Airlines Inc.
#>  3 YV              19.0  Mesa Airlines Inc.
#>  4 FL              18.7  AirTran Airways Corporation
#>  5 WN              17.7  Southwest Airlines Co.
#>  6 9E              16.7  Endeavor Air Inc.
#>  7 B6              13.0  JetBlue Airways
#>  8 VX              12.9  Virgin America
#>  9 OO              12.6  SkyWest Airlines Inc.
#> 10 UA              12.1  United Air Lines Inc.
#> 11 MQ              10.6  Envoy Air
```

```
#> 12 DL             9.26 Delta Air Lines Inc.
#> 13 AA             8.59 American Airlines Inc.
#> 14 AS             5.80 Alaska Airlines Inc.
#> 15 HA             4.90 Hawaiian Airlines Inc.
#> 16 US             3.78 US Airways Inc.
```

```
left_join(flights, airports, by = c("dest" = "faa"))       # 26
```

```
#> # A tibble: 336,776 x 26
#>     year month   day dep_time sched_dep_time dep_delay arr_time
#>    <int> <int> <int>    <int>          <int>     <dbl>    <int>
#>  1  2013     1     1      517            515         2      830
#>  2  2013     1     1      533            529         4      850
#>  3  2013     1     1      542            540         2      923
#>  4  2013     1     1      544            545        -1     1004
#>  5  2013     1     1      554            600        -6      812
#>  6  2013     1     1      554            558        -4      740
#>  7  2013     1     1      555            600        -5      913
#>  8  2013     1     1      557            600        -3      709
#>  9  2013     1     1      557            600        -3      838
#> 10  2013     1     1      558            600        -2      753
#> # i 336,766 more rows
#> # i 19 more variables: sched_arr_time <int>, arr_delay <dbl>,
#> #   carrier <chr>, flight <int>, tailnum <chr>, origin <chr>,
#> #   dest <chr>, air_time <dbl>, distance <dbl>, hour <dbl>,
#> #   minute <dbl>, time_hour <dttm>, name <chr>, lat <dbl>, lon <dbl>,
#> #   alt <dbl>, tz <dbl>, dst <chr>, tzone <chr>
```

```
flights |>                                                  # 27
  count(month, dest) |>
  group_by(month) |>
  slice_min(n, n = 1) |>
  ungroup() |>
  left_join(airports, by = c("dest" = "faa")) |>
  select(month, dest, name, n)
```

```
#> # A tibble: 14 x 4
#>    month dest  name                    n
#>    <int> <chr> <chr>               <int>
#>  1     1 EYW   Key West Intl           1
#>  2     2 JAC   Jackson Hole Airport    3
#>  3     3 BGR   Bangor Intl             2
#>  4     4 EYW   Key West Intl           1
#>  5     4 MYR   Myrtle Beach Intl       1
#>  6     5 CAE   Columbia Metropolitan   9
#>  7     6 MYR   Myrtle Beach Intl       1
#>  8     7 LGA   La Guardia              1
#>  9     8 SBN   South Bend Rgnl         1
#> 10     9 SBN   South Bend Rgnl         5
#> 11    10 ALB   Albany Intl             1
#> 12    10 SBN   South Bend Rgnl         1
#> 13    11 LEX   Blue Grass              1
#> 14    12 SBN   South Bend Rgnl         1
```

```
noms <- select(airports, faa, name)                          # 28
flights |>
  left_join(noms, by = c("origin" = "faa")) |>
  left_join(noms, by = c("dest" = "faa"), suffix = c("_depart",
   ↪  "_arrivee")) |>
  select(name_depart, name_arrivee)
```

```
#> # A tibble: 336,776 x 2
#>    name_depart         name_arrivee
#>    <chr>               <chr>
#>  1 Newark Liberty Intl George Bush Intercontinental
#>  2 La Guardia          George Bush Intercontinental
#>  3 John F Kennedy Intl Miami Intl
#>  4 John F Kennedy Intl <NA>
#>  5 La Guardia          Hartsfield Jackson Atlanta Intl
#>  6 Newark Liberty Intl Chicago Ohare Intl
#>  7 Newark Liberty Intl Fort Lauderdale Hollywood Intl
#>  8 La Guardia          Washington Dulles Intl
#>  9 John F Kennedy Intl Orlando Intl
#> 10 La Guardia          Chicago Ohare Intl
#> # i 336,766 more rows
```

```
flights_dest <- flights |>                                   # 29
  count(dest) |>
  left_join(airports, by = c("dest" = "faa"))
head(flights_dest)
```

```
#> # A tibble: 6 x 9
#>   dest      n name                  lat    lon   alt    tz dst   tzone
#>   <chr> <int> <chr>                <dbl>  <dbl> <dbl> <dbl> <chr> <chr>
#> 1 ABQ     254 Albuquerque Internat~  35.0 -107.   5355    -7 A     Amer~
#> 2 ACK     265 Nantucket Mem          41.3  -70.1    48    -5 A     Amer~
#> 3 ALB     439 Albany Intl            42.7  -73.8   285    -5 A     Amer~
#> 4 ANC       8 Ted Stevens Anchorag~  61.2 -150.    152    -9 A     Amer~
#> 5 ATL   17215 Hartsfield Jackson A~  33.6  -84.4  1026    -5 A     Amer~
#> 6 AUS    2439 Austin Bergstrom Intl  30.2  -97.7   542    -6 A     Amer~
```

Aux questions 27 et 29, certaines destinations (aéroports de Porto Rico et des îles Vierges, par exemple) sont absentes de la table `airports` : leur nom vaut `NA`.

Exercice 3.6 — Les avions. À l'aide des tables `flights` et `planes` :

1. Sélectionner uniquement les vols dont le numéro de queue de l'avion (`tailnum`) est renseigné.
2. Calculer le nombre d'avions distincts dans la table `flights`.
3. Calculer le nombre d'avions de la table `planes` dont l'année de fabrication (`year`) est renseignée.

Solution

```
flights |> filter(!is.na(tailnum)) |> select(tailnum)
```

```
#> # A tibble: 334,264 x 1
#>    tailnum
#>    <chr>
#>  1 N14228
#>  2 N24211
```

```
#>  3 N619AA
#>  4 N804JB
#>  5 N668DN
#>  6 N39463
#>  7 N516JB
#>  8 N829AS
#>  9 N593JB
#> 10 N3ALAA
#> # i 334,254 more rows
```

```
flights |> summarise(nb_avions = n_distinct(tailnum))
```

```
#> # A tibble: 1 x 1
#>   nb_avions
#>       <int>
#> 1      4044
```

```
planes |> summarise(annee_renseignee = sum(!is.na(year)))
```

```
#> # A tibble: 1 x 1
#>   annee_renseignee
#>              <int>
#> 1             3252
```

Exercice 3.7 — Analyse des ventes de la boutique. À partir des tables `commandes`, `clients` et `categories` :

1. Calculer le chiffre d'affaires et le nombre de commandes par mois. Quel est le mois le plus fort ? Pourquoi, à votre avis ?
2. Calculer le panier moyen par canal d'acquisition du client (il faut une jointure).
3. Pour chaque région, calculer le nombre de clients actifs, le chiffre d'affaires et le chiffre d'affaires par client actif. Trier par ce dernier.
4. Quelle part des commandes du week-end (samedi, dimanche) utilise un code promo, comparée à la semaine ?

Solution

```
# 1.
commandes |>
  group_by(mois) |>
  summarise(ca = sum(montant), nb = n()) |>
  arrange(desc(ca)) |>
  head(3)
```

```
#> # A tibble: 3 x 3
#>    mois     ca    nb
#>   <dbl>  <dbl> <int>
#> 1    12 99169.  1512
#> 2    11 79009.  1221
#> 3    10 53304.   845
```

```r
# Décembre puis novembre : achats de Noël et Black Friday.

# 2.
commandes |>
  left_join(clients, by = "client_id") |>
  group_by(canal_acquisition) |>
  summarise(panier_moyen = mean(montant), nb_commandes = n()) |>
  arrange(desc(panier_moyen))
```

```
#> # A tibble: 5 x 3
#>   canal_acquisition   panier_moyen nb_commandes
#>   <chr>                      <dbl>        <int>
#> 1 Direct                      83.2         1855
#> 2 Email                       78.0         1689
#> 3 Moteur de recherche         63.0         2192
#> 4 Réseaux sociaux             52.8         2562
#> 5 Affiliation                 44.8         1210
```

```r
# 3.
commandes |>
  left_join(clients, by = "client_id") |>
  group_by(region) |>
  summarise(clients_actifs = n_distinct(client_id),
            ca = sum(montant)) |>
  mutate(ca_par_client = ca / clients_actifs) |>
  arrange(desc(ca_par_client))
```

```
#> # A tibble: 8 x 4
#>   region                    clients_actifs      ca ca_par_client
#>   <chr>                              <int>   <dbl>         <dbl>
#> 1 Auvergne-Rhône-Alpes                 214  79200.          370.
#> 2 Hauts-de-France                      166  61348.          370.
#> 3 Autres régions                       323 114934.          356.
#> 4 Nouvelle-Aquitaine                   160  56885.          356.
#> 5 Provence-Alpes-Côte d'Azur           159  55401.          348.
#> 6 Occitanie                            127  44057.          347.
#> 7 Île-de-France                        450 155115.          345.
#> 8 Grand Est                            142  46823.          330.
```

```r
# 4.
commandes |>
  mutate(week_end = wday(date, week_start = 1) >= 6) |>   # 6 = samedi, 7
    ↪  = dimanche
  group_by(week_end) |>
  summarise(part_promo = mean(promo))
```

```
#> # A tibble: 2 x 2
#>   week_end part_promo
#>   <lgl>         <dbl>
#> 1 FALSE         0.405
#> 2 TRUE          0.398
```

Exercice 3.8 — Test A/B. À partir de la table `campagne` :

1. Calculer, pour chaque version, le taux d'ouverture, le taux de clic et le taux d'achat.
2. Parmi les emails **ouverts**, le taux de clic diffère-t-il entre A et B ? Faire le test

approprié.
3. Rédiger en trois phrases une recommandation pour l'équipe marketing.

Solution

```
campagne |>
  group_by(version) |>
  summarise(n = n(), ouverture = mean(ouvert), clic = mean(clic), achat =
   ↪  mean(achat))
```

```
#> # A tibble: 2 x 5
#>   version     n ouverture   clic  achat
#>   <chr>   <int>     <dbl>  <dbl>  <dbl>
#> 1 A         465     0.204 0.0473 0.0151
#> 2 B         479     0.276 0.0626 0.0271
```

```
ouverts <- filter(campagne, ouvert)
lprop(table(ouverts$version, ouverts$clic))
```

```
#>
#>       FALSE TRUE  Total
#>   A    76.8  23.2 100.0
#>   B    77.3  22.7 100.0
#>   All  77.1  22.9 100.0
```

```
chisq.test(table(ouverts$version, ouverts$clic))
```

```
#>
#>  Pearson's Chi-squared test with Yates' continuity correction
#>
#> data:  table(ouverts$version, ouverts$clic)
#> X-squared = 3e-31, df = 1, p-value = 1
```

L'objet personnalisé (B) augmente significativement le taux d'ouverture. Une fois l'email ouvert, le taux de clic est comparable entre les deux versions : l'effet de B porte sur l'objet, pas sur le contenu. L'effet sur les achats va dans le même sens mais n'est pas démontré avec cet échantillon ; on peut généraliser l'objet personnalisé et continuer à mesurer l'effet sur les ventes sur plusieurs campagnes.

Exercice 3.9 — Clients à réactiver. Construire la table des clients dont la dernière commande date de plus de 180 jours au 1er janvier 2026, avec leur canal d'acquisition et leur statut newsletter. Combien peuvent être contactés par email ? Exporter cette table en CSV.

Solution

```
a_reactiver <- rfm |>
  filter(recence > 180) |>
  left_join(clients, by = "client_id") |>
  select(client_id, recence, frequence, montant_total, canal_acquisition,
   ↪  newsletter)
nrow(a_reactiver)
```

```
#> [1] 236
```

```
count(a_reactiver, newsletter)
```

```
#> # A tibble: 2 x 2
```

```
#>   newsletter     n
#>   <chr>      <int>
#> 1 Non          166
#> 2 Oui           70
```

```
# write_csv(a_reactiver, "clients_a_reactiver.csv")
```

Pour aller plus loin

- J. Barnier, *Introduction à R et au tidyverse*, chapitres « Analyse univariée », « Analyse bivariée », « Manipuler les données avec dplyr » (juba.github.io/tidyverse).
- H. Wickham et al., *R for Data Science*, 2e éd., chapitres « Data transformation », « Data tidying », « Joins ».

Chapitre 4

Représentations graphiques et régression

Objectifs du chapitre

- Connaître les principes d'un bon graphique et choisir un graphique adapté aux données.
- Produire des graphiques avec R de base et, surtout, avec `ggplot2` : *geoms*, mappages, positions, *facets*, *scales*, thèmes.
- Ajuster, interpréter et critiquer une régression linéaire simple et multiple, puis une régression logistique.

```
library(tidyverse)
library(questionr)   # jeu de données rp2018 (exercices)
library(ISLR)        # jeux de données Carseats et OJ
source("R/boutique.R")
commandes <- lire_boutique("commandes")
clients   <- lire_boutique("clients")
```

4.1 Pourquoi visualiser ?

La visualisation est un élément essentiel de l'analyse : elle révèle des structures que les résumés numériques cachent. Le **quartet d'Anscombe** (jeu `anscombe`) contient quatre séries (x, y) qui ont presque exactement la même moyenne, la même variance, la même corrélation et la même droite de régression :

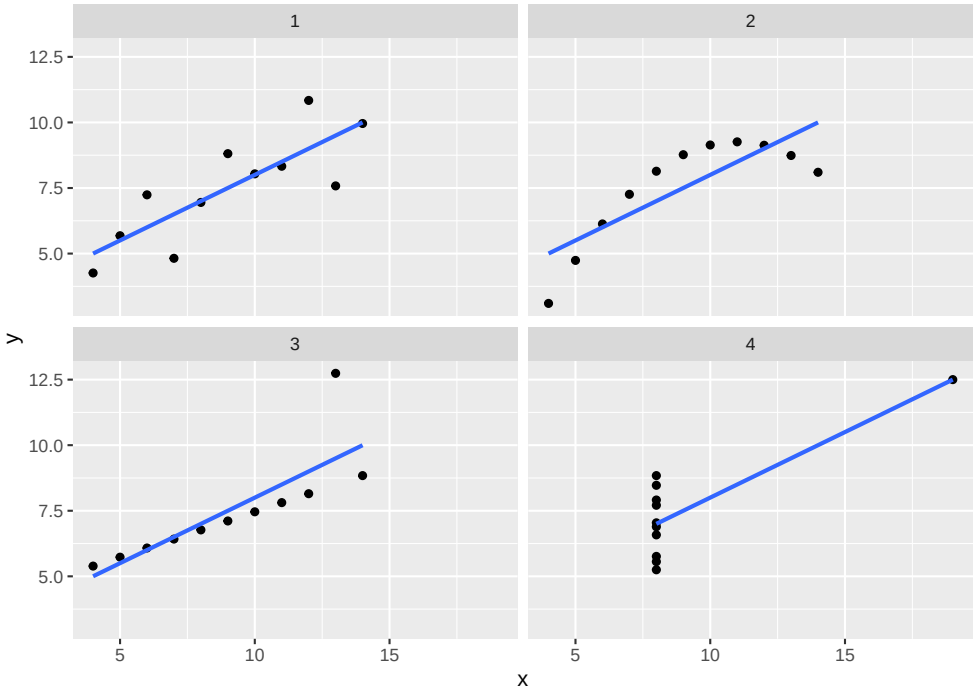
```
anscombe_long <- tibble(
  serie = rep(1:4, each = 11),
  x = c(anscombe$x1, anscombe$x2, anscombe$x3, anscombe$x4),
  y = c(anscombe$y1, anscombe$y2, anscombe$y3, anscombe$y4)
)
anscombe_long |>
  group_by(serie) |>
  summarise(moy_x = mean(x), moy_y = mean(y), var_y = var(y), correlation =
   ↪  cor(x, y))
```

```
#> # A tibble: 4 x 5
#>   serie moy_x moy_y var_y correlation
#>   <int> <dbl> <dbl> <dbl>       <dbl>
#> 1     1     9  7.50  4.13       0.816
```

```
#> 2     2     9  7.50  4.13       0.816
#> 3     3     9  7.5   4.12       0.816
#> 4     4     9  7.50  4.12       0.817
```

Pourtant, leurs graphiques n'ont rien à voir :

```
ggplot(anscombe_long, aes(x, y)) +
  geom_point() +
  geom_smooth(method = "lm", se = FALSE, formula = y ~ x) +
  facet_wrap(vars(serie))
```



Toujours regarder les données avant (et après) de les modéliser. Explorer graphiquement est un processus **itératif** : on commence simple, puis on affine en regardant les données sous différents angles.

4.1.1 Principes d'un bon graphique

Edward Tufte (*The Visual Display of Quantitative Information*) propose quelques principes :

- montrer les données avant tout ;
- ne pas les déformer (axes tronqués trompeurs, effets 3D) ;
- encourager l'œil à **comparer** ;
- montrer plusieurs niveaux de détail, de la vue d'ensemble à la structure fine.

Un graphique se décompose en quatre éléments : les **indices visuels** (position, longueur, couleur, taille…), le **système de coordonnées** (cartésien, polaire, géographique), les **échelles** (linéaire, logarithmique, catégorielle, temporelle) et le **contexte** (titre, étiquettes d'axes avec unités, source, lignes de référence).

L'œil compare très bien des **positions** et des **longueurs** sur une échelle commune, beaucoup moins bien des angles, des surfaces ou des couleurs. C'est pourquoi un diagramme en barres est presque toujours plus lisible qu'un diagramme circulaire (« camembert »).

4.1.2 Choisir un graphique

Question	Variables	Graphique	ggplot2
Distribution	1 quantitative	histogramme, densité, boîte à moustaches	`geom_histogram()`, `geom_density()`
Répartition	1 qualitative	diagramme en barres	`geom_bar()`, `geom_col()`
Relation	2 quantitatives	nuage de points	`geom_point()`
Comparaison	quantitative × qualitative	boîtes à moustaches, violons	`geom_boxplot()`, `geom_violin()`
Association	2 qualitatives	barres empilées ou juxtaposées	`geom_bar(position = ...)`
Évolution	quantitative × temps	courbe	`geom_line()`

4.2 Graphiques avec R de base

Les fonctions graphiques de base, vues au chapitre précédent, sont rapides pour explorer :

```
par(mfrow = c(2, 2))   # grille 2 x 2

hist(Carseats$Sales, breaks = 20, col = "lightblue",
     main = "Ventes par magasin", xlab = "Ventes (milliers d'unités)")
boxplot(Sales ~ ShelveLoc, data = Carseats,
        main = "Ventes selon l'emplacement", xlab = "Emplacement en rayon",
        ↪  ylab = "Ventes")
plot(Sales ~ Price, data = Carseats, pch = 16, cex = 0.6,
     col = ifelse(Carseats$US == "Yes", "red", "grey40"),
     main = "Ventes et prix", xlab = "Prix ($)", ylab = "Ventes")
legend("topright", legend = c("États-Unis", "Hors États-Unis"),
       col = c("red", "grey40"), pch = 16, cex = 0.8)
barplot(table(Carseats$ShelveLoc), main = "Emplacements", col = "tomato")
```




```
par(mfrow = c(1, 1))    # rétablir la grille par défaut
```

Paramètre	Rôle	Exemple
`main`, `xlab`, `ylab`	titres	`main = "Titre"`
`col`	couleur	`col = "steelblue"`
`pch`	symbole des points	`pch = 16` (disque plein)
`cex`	taille relative	`cex = 0.5`
`lty`, `lwd`	type et épaisseur de ligne	`lty = 2` (tirets)
`xlim`, `ylim`	bornes des axes	`xlim = c(0, 10)`

Les fonctions `abline()`, `lines()`, `points()`, `text()` et `legend()` **ajoutent** des éléments au graphique courant. Pour enregistrer : `png("fichier.png")`, puis le graphique, puis `dev.off()`, ou le bouton *Export* de l'onglet *Plots*.

4.3 Graphiques avec `ggplot2`

`ggplot2` repose sur une **grammaire des graphiques** : un graphique est construit en **superposant des couches** avec l'opérateur `+`. Une **esthétique** (*aesthetic*) est une correspondance explicite entre une **variable** et un **indice visuel**.

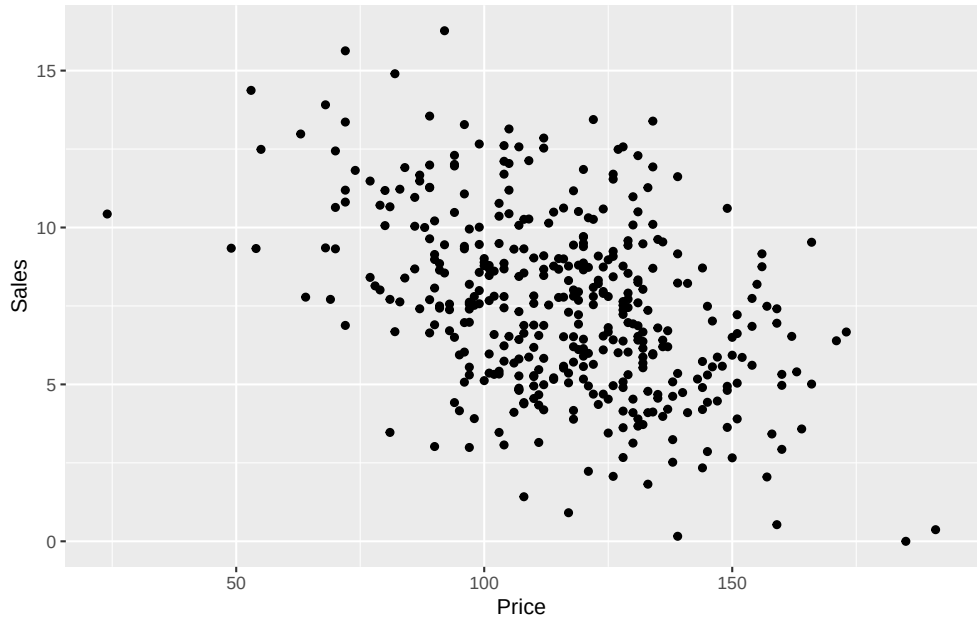
Composant	Rôle	Exemple
données	un tableau (format *tidy*)	`ggplot(Carseats)`
mappages (`aes()`)	lien entre variables et attributs graphiques	`aes(x = Price, y = Sales, color = ShelveLoc)`

Composant	Rôle	Exemple
geoms	forme géométrique	`geom_point()`, `geom_boxplot()`
facets	petits multiples	`facet_wrap(vars(ShelveLoc))`
scales	réglage des axes, couleurs, tailles	`scale_y_log10()`
thème et titres	apparence générale	`labs()`, `theme_minimal()`

4.3.1 Initialisation et premier *geom*

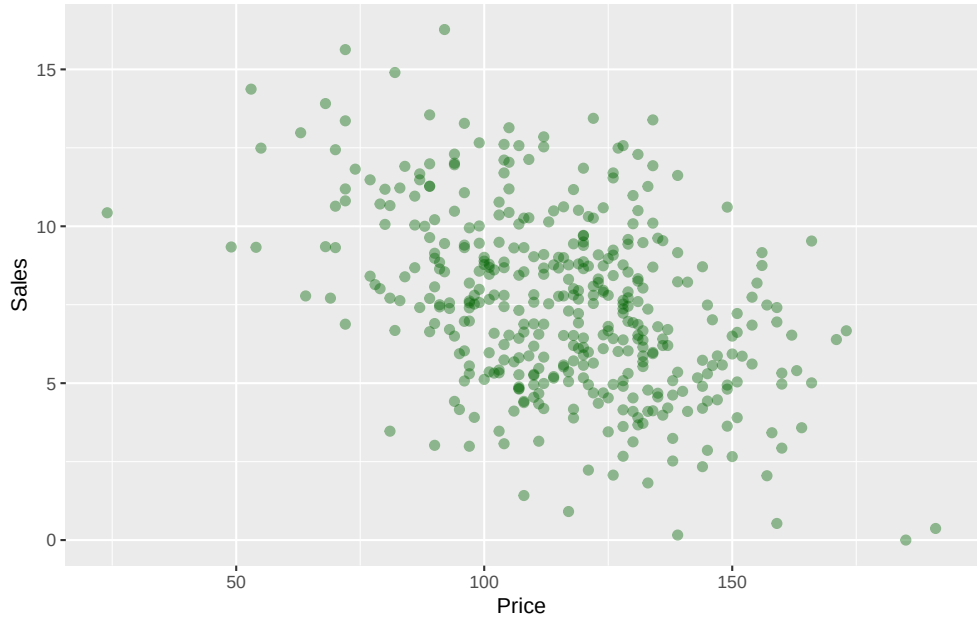
Un graphique s'initialise avec `ggplot()` et un tableau de données. On ajoute ensuite un *geom*, auquel on indique les variables à représenter via `aes()`. Les variables s'écrivent sans le nom du tableau (`Price` et non `Carseats$Price`).

```
ggplot(Carseats) +
  geom_point(aes(x = Price, y = Sales))
```



Les attributs **constants** se donnent dans le *geom*, **hors** de `aes()` :

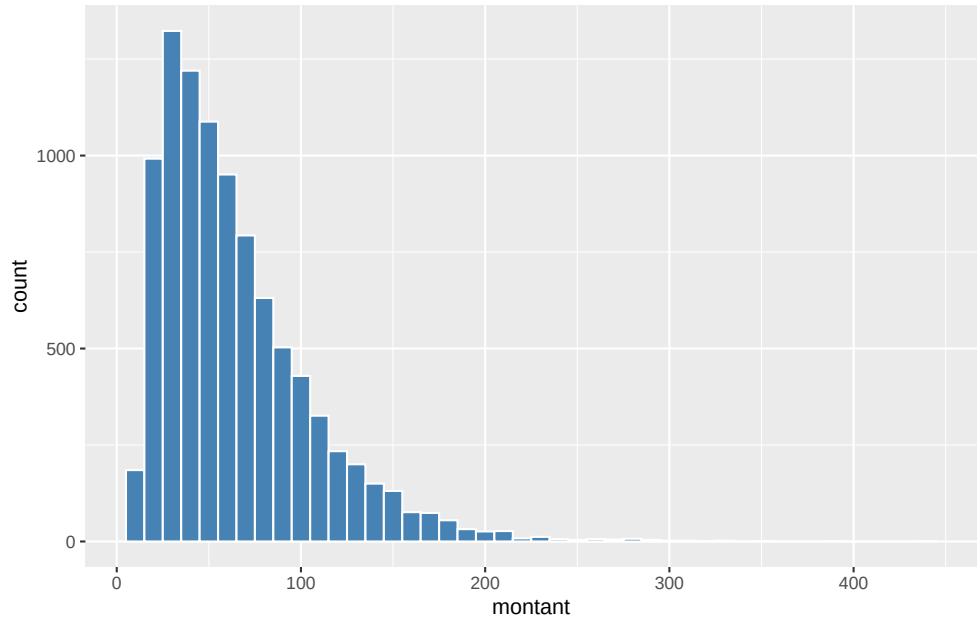
```
ggplot(Carseats) +
  geom_point(aes(x = Price, y = Sales), color = "darkgreen", size = 2, alpha =
  ↪ 0.4)
```



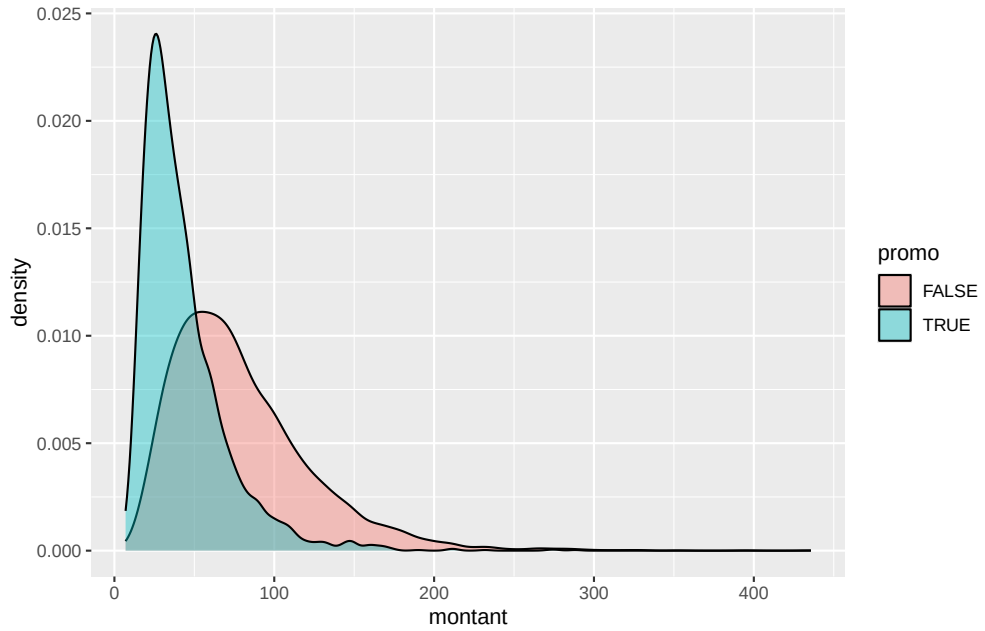
4.3.2 Les principaux *geoms*

Histogramme et densité — la distribution des montants de commande :

```
ggplot(commandes) +
  geom_histogram(aes(x = montant), binwidth = 10, fill = "steelblue", color =
   ↪  "white")
```




```
ggplot(commandes) +
  geom_density(aes(x = montant, fill = promo), alpha = 0.4)
```

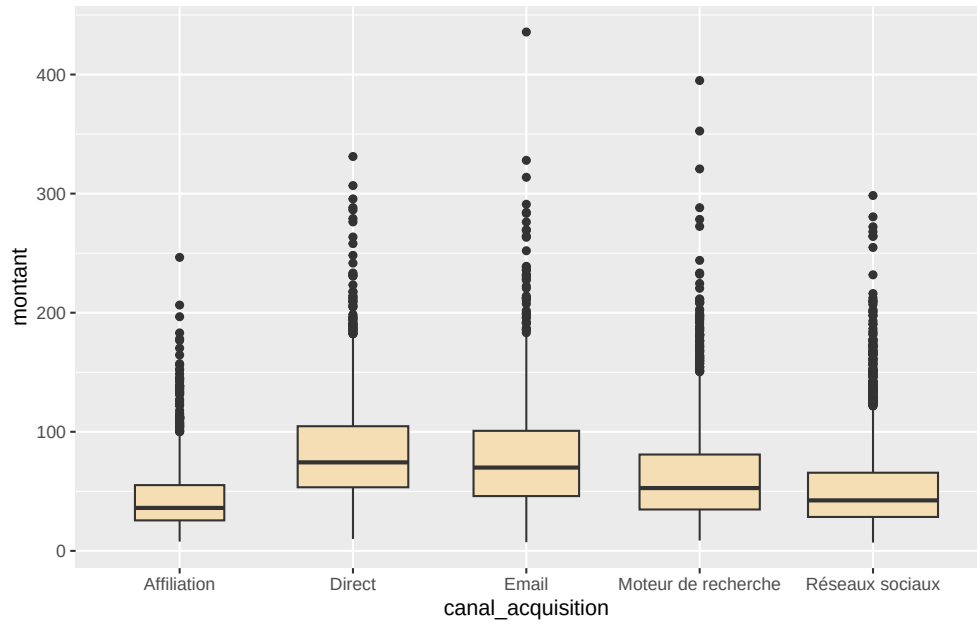


L'argument `binwidth` (ou `bins`) d'un histogramme et `bw` d'une densité règlent le niveau de détail : à tester, car ils peuvent masquer ou inventer des structures.

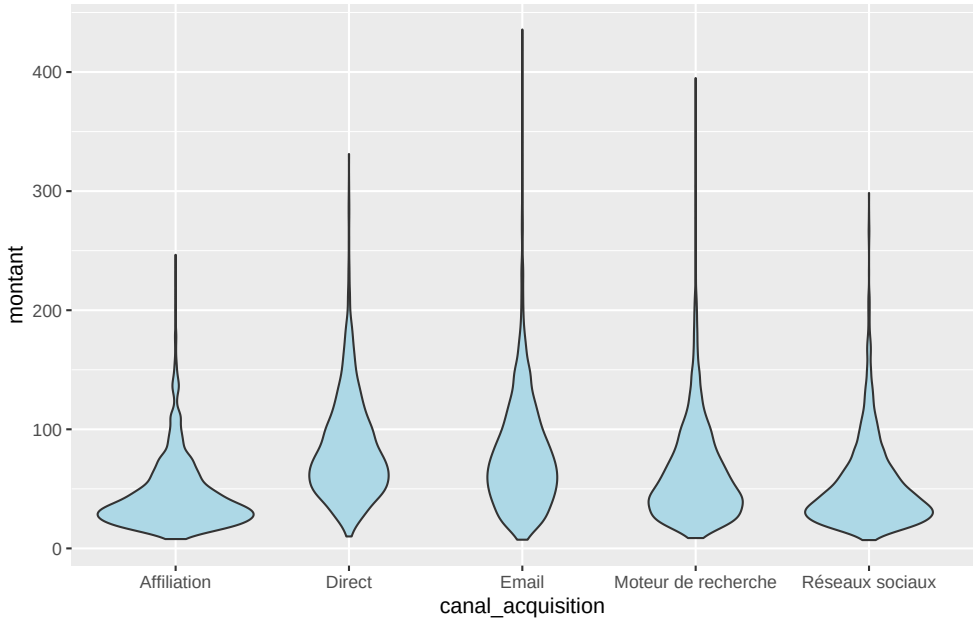
Boîtes à moustaches et violons — le panier selon le canal d'acquisition :

```
commandes_clients <- left_join(commandes, clients, by = "client_id")

ggplot(commandes_clients) +
  geom_boxplot(aes(x = canal_acquisition, y = montant), fill = "wheat",
   ↪  varwidth = TRUE)
```



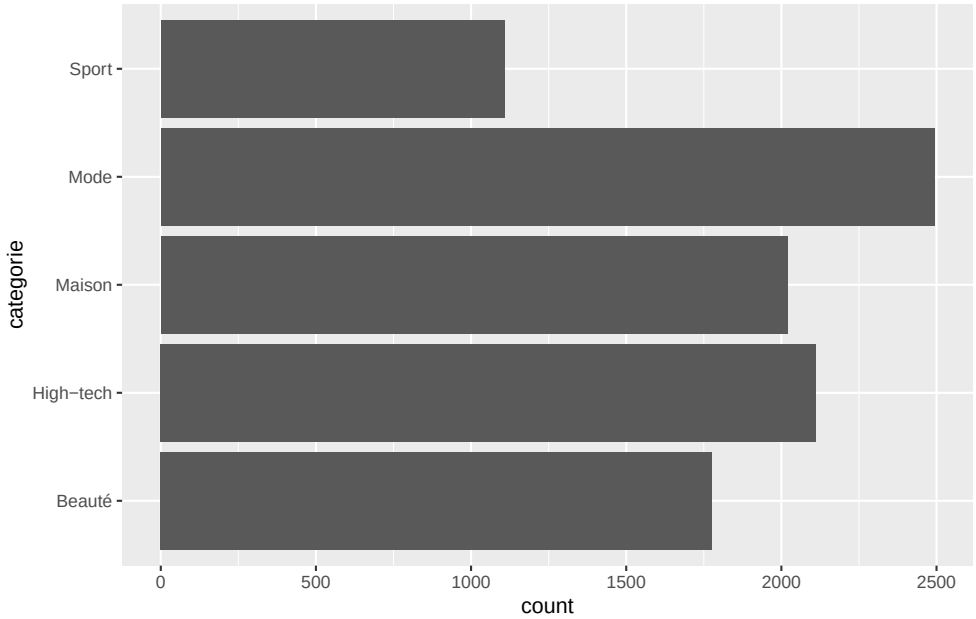

```
ggplot(commandes_clients) +
  geom_violin(aes(x = canal_acquisition, y = montant), fill = "lightblue")
```



`varwidth = TRUE` rend la largeur des boîtes proportionnelle aux effectifs ; le violon montre la forme complète de la distribution.

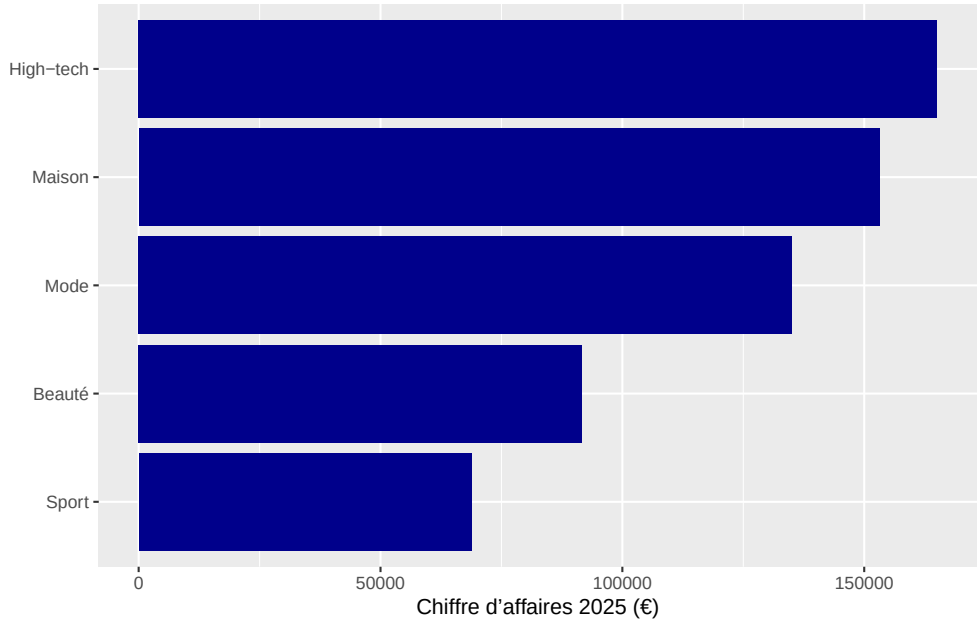
Barres : `geom_bar()` et `geom_col()` — `geom_bar()` **compte** les lignes de chaque modalité ; `geom_col()` représente des valeurs **déjà calculées**. Passer la variable en `y` donne des barres horizontales, plus lisibles avec des étiquettes longues (on peut aussi ajouter `coord_flip()`).

```
ggplot(commandes) +
  geom_bar(aes(y = categorie))
```




```
ca_categorie <- commandes |>
  group_by(categorie) |>
  summarise(ca = sum(montant))

ggplot(ca_categorie) +
  geom_col(aes(x = ca, y = reorder(categorie, ca)), fill = "darkblue") +
  labs(x = "Chiffre d'affaires 2025 (€)", y = NULL)
```

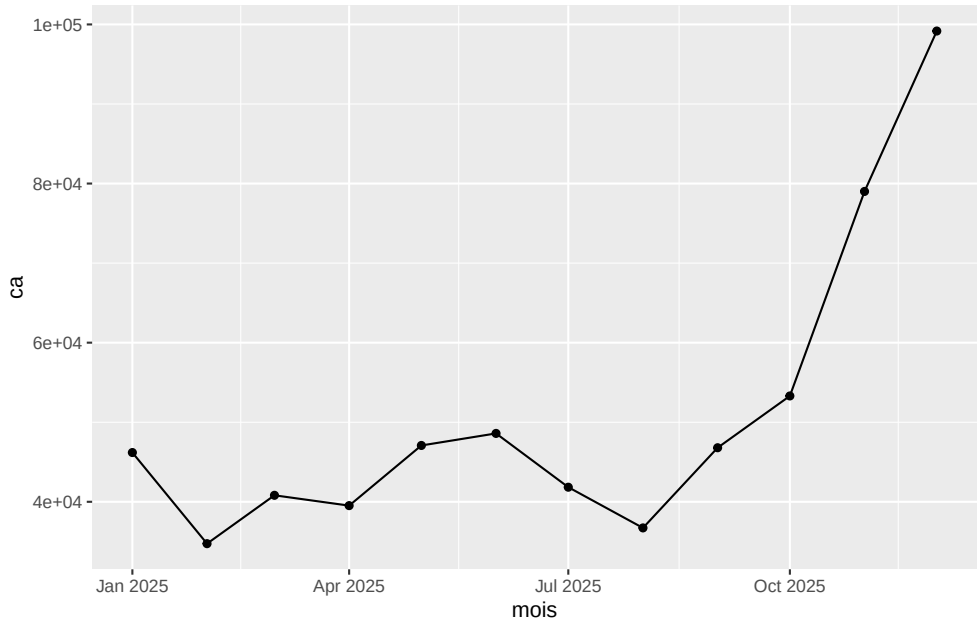


`reorder(categorie, ca)` trie les barres selon le chiffre d'affaires : un tri rend la lecture beaucoup plus rapide.

Courbes — l'évolution du chiffre d'affaires mensuel :

```
ca_mensuel <- commandes |>
  mutate(mois = floor_date(date, "month")) |>
  group_by(mois) |>
  summarise(ca = sum(montant))

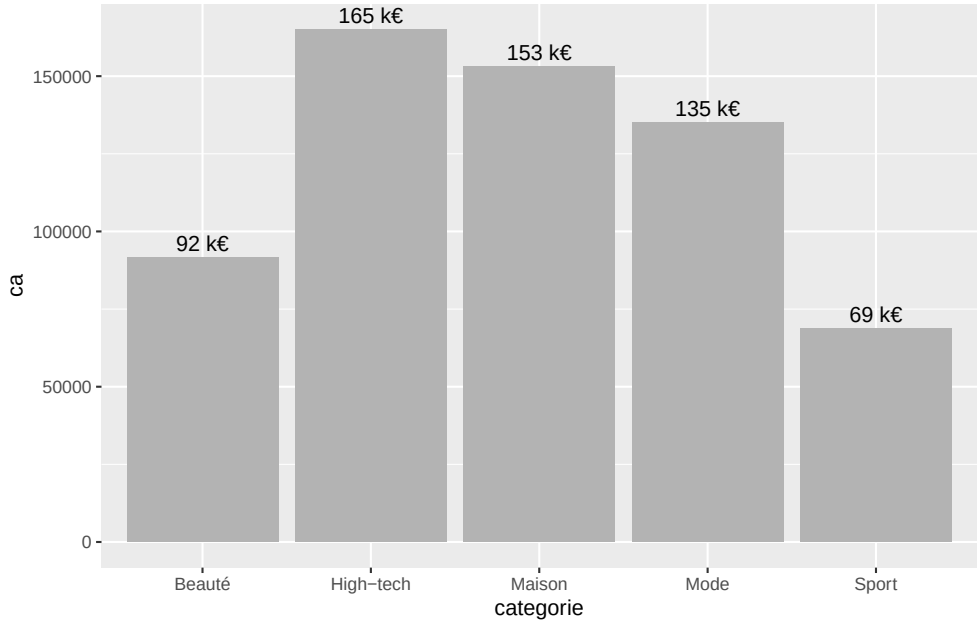
ggplot(ca_mensuel, aes(x = mois, y = ca)) +
  geom_line() +
  geom_point()
```



La saisonnalité est nette : les ventes s'envolent en novembre (*Black Friday*) et en décembre (Noël).

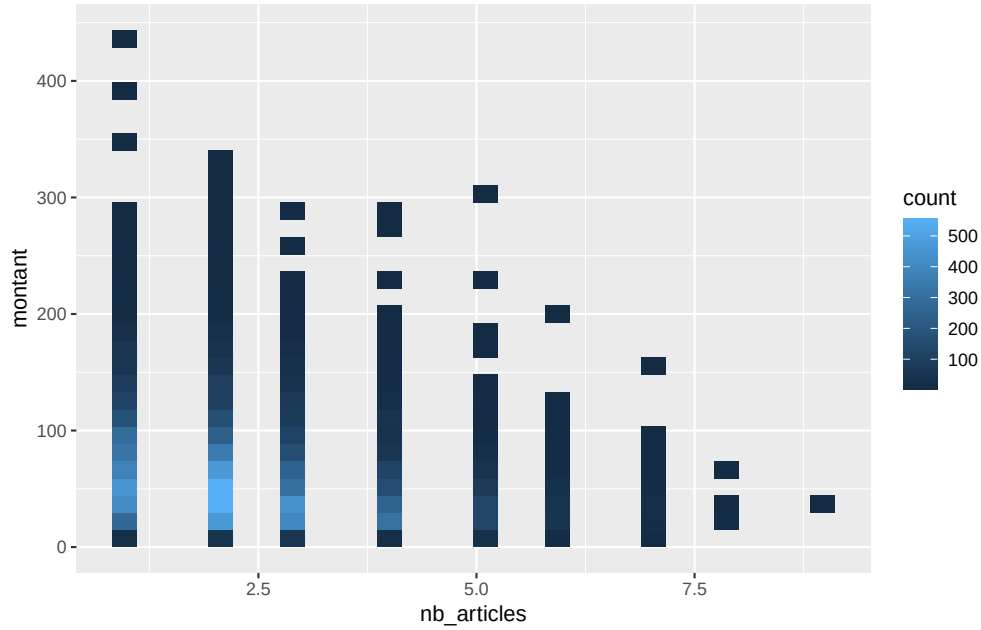
Texte et étiquettes — `geom_text()` et `geom_label()` demandent `x`, `y` et `label` :

```
ggplot(ca_categorie, aes(x = categorie, y = ca)) +
  geom_col(fill = "grey70") +
  geom_text(aes(label = paste(round(ca / 1000), "k€")), vjust = -0.4)
```



Nombreux points — quand les points se superposent, `geom_bin2d()` (ou `geom_hex()`, qui nécessite le package `hexbin`) colore une grille selon le nombre de points :

```
ggplot(commandes) +
  geom_bin2d(aes(x = nb_articles, y = montant), bins = 30)
```



4.3.3 Mappages

Un **mappage** relie un attribut graphique (position, couleur, taille, transparence, forme) à une **variable**. On peut en combiner plusieurs :

```
ggplot(Carseats) +
  geom_point(aes(x = Price, y = Sales, color = ShelveLoc, size = Advertising),
   ↪  alpha = 0.6)
```



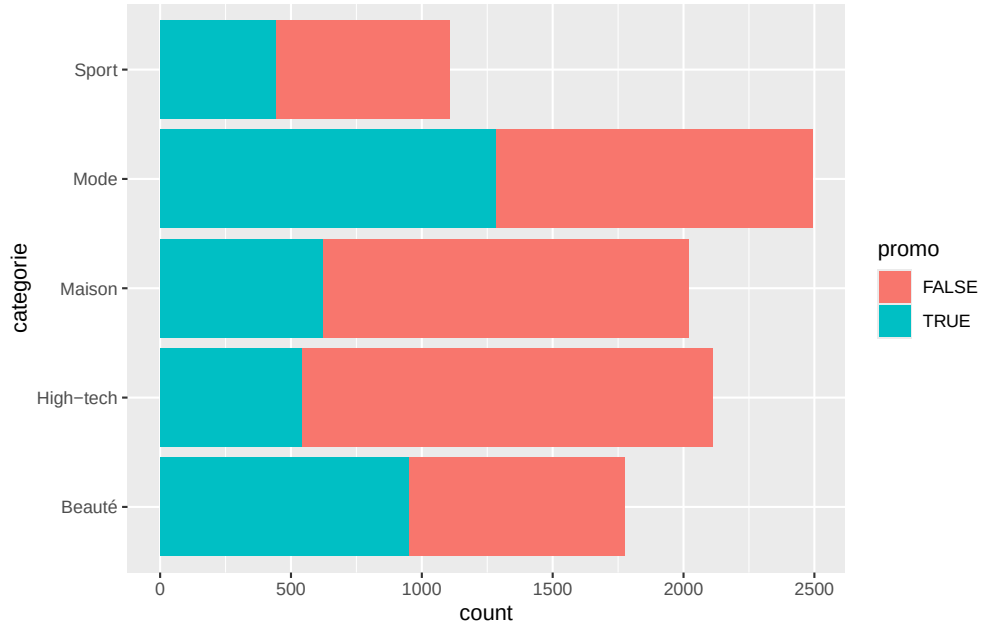
Ce graphique montre quatre variables : à prix égal, un magasin où le produit est **bien placé en rayon** (`Good`) vend nettement plus.

> **`aes()` ou pas `aes()` ?** Si l'attribut dépend d'une **variable**, c'est un mappage : il va **dans** `aes()`. S'il est **constant**, il va **hors** de `aes()`. Écrire `aes(color = "blue")` crée une légende « blue » et une couleur imprévue.

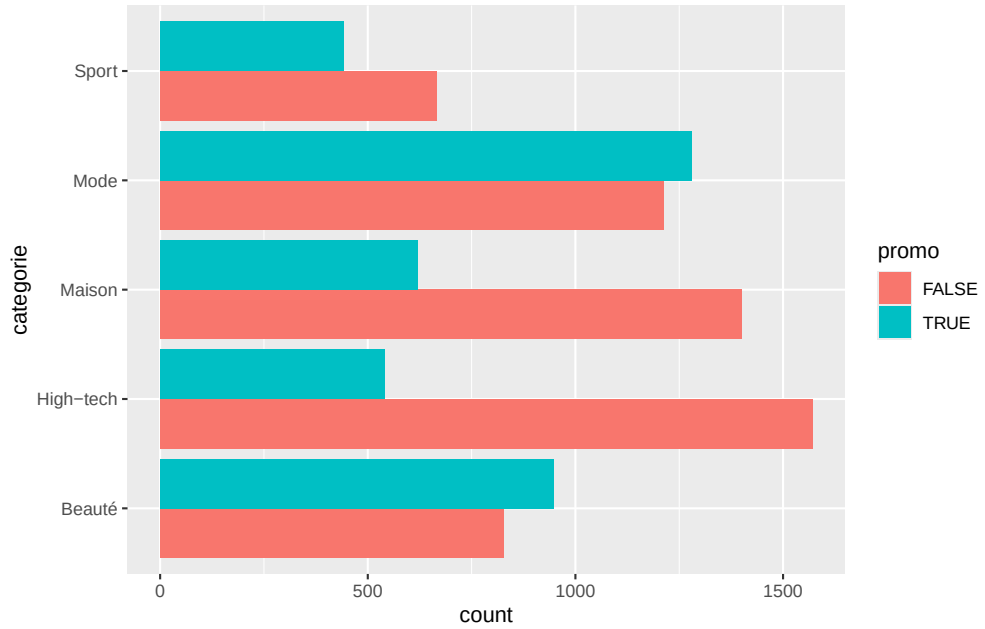
4.3.4 Positions des barres

Avec un mappage `fill`, l'argument `position` de `geom_bar()` contrôle l'agencement des barres :

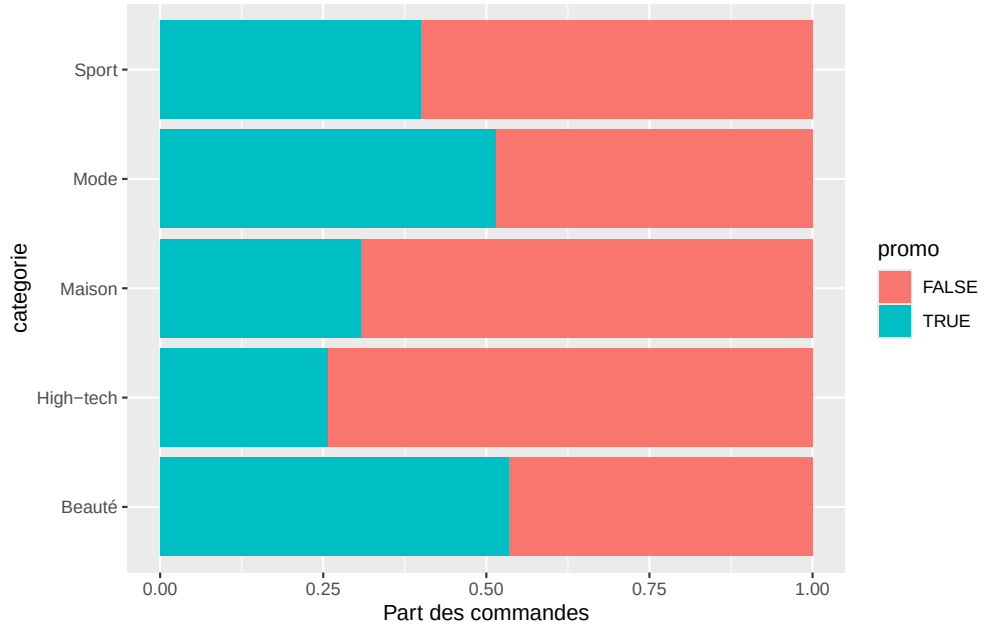
```
ggplot(commandes) +
  geom_bar(aes(y = categorie, fill = promo))                    # empilées
   ↪  (défaut)
```




```
ggplot(commandes) +
  geom_bar(aes(y = categorie, fill = promo), position = "dodge")  # côte à côte
```




```
ggplot(commandes) +
  geom_bar(aes(y = categorie, fill = promo), position = "fill") + # proportions
  labs(x = "Part des commandes")
```

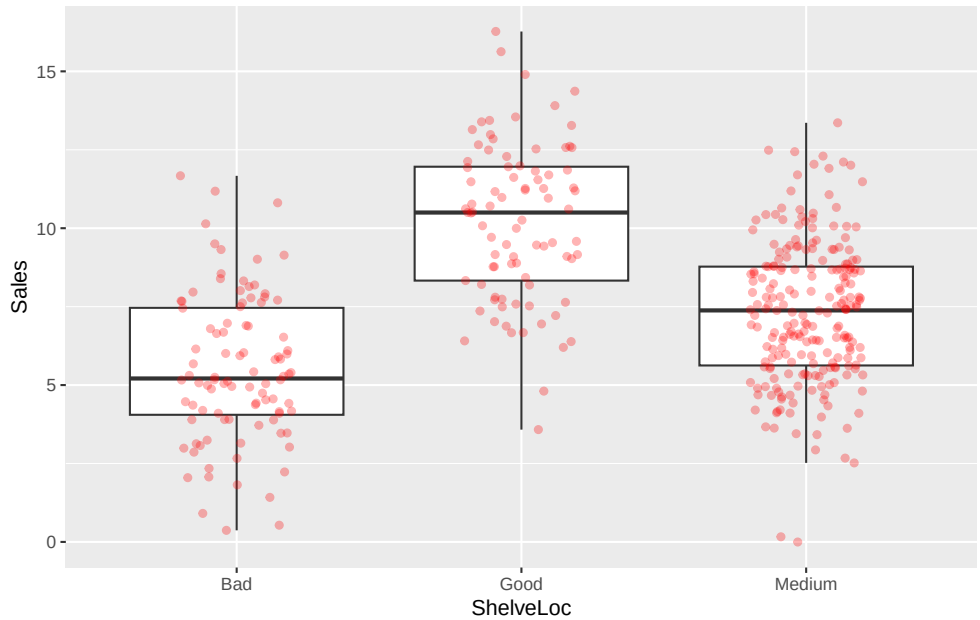


Les promotions sont surtout utilisées en Mode et en Beauté. Pour comparer des **proportions** entre groupes de tailles différentes, `position = "fill"` est le bon choix.

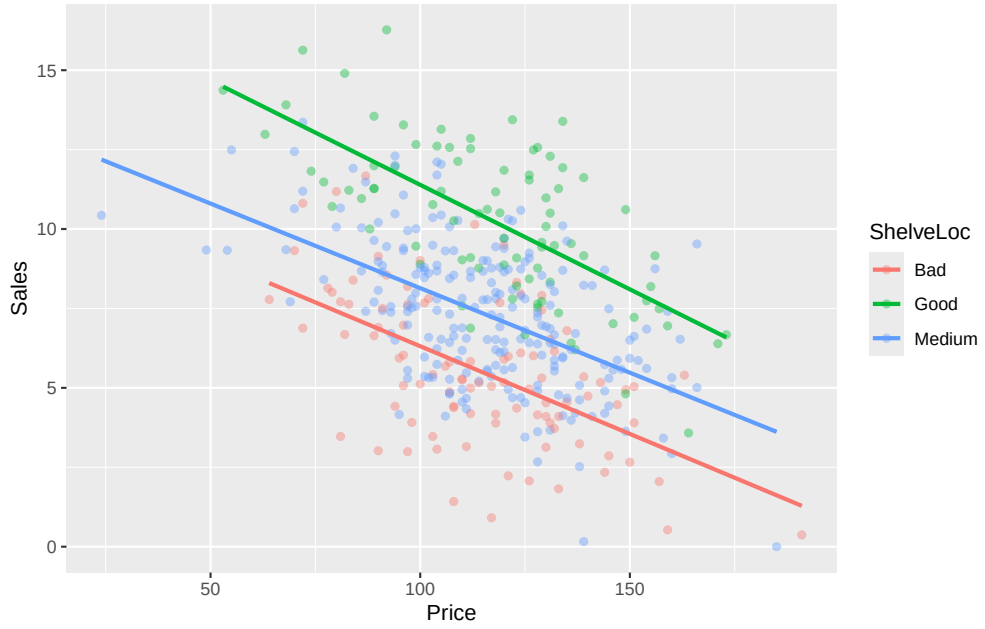
4.3.5 Superposer plusieurs *geoms*

On ajoute les couches les unes après les autres. Les mappages déclarés dans `ggplot()` sont **hérités** par tous les *geoms* :

```
ggplot(Carseats, aes(x = ShelveLoc, y = Sales)) +
  geom_boxplot(outlier.shape = NA) +
  geom_jitter(width = 0.2, alpha = 0.3, color = "red")
```




```
ggplot(Carseats, aes(x = Price, y = Sales, color = ShelveLoc)) +
  geom_point(alpha = 0.4) +
  geom_smooth(method = "lm", formula = y ~ x, se = FALSE)
```

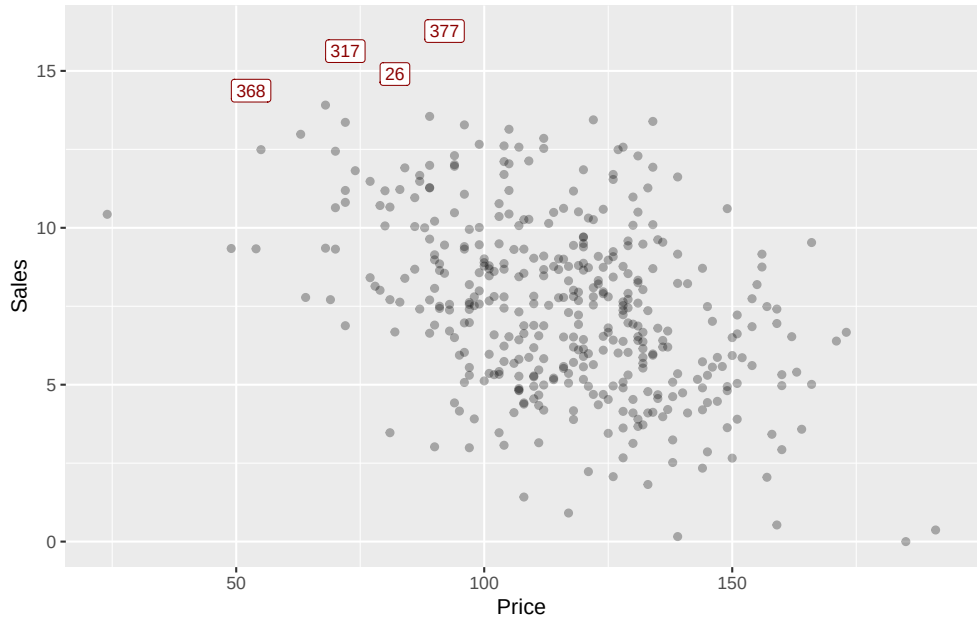


Les trois droites sont presque parallèles : l'effet du prix est à peu près le même quel que soit l'emplacement. Nous le vérifierons avec un modèle.

Chaque *geom* peut aussi avoir ses **propres données**, par exemple pour étiqueter seulement quelques points :

```
meilleurs <- Carseats |>
  mutate(magasin = row_number()) |>
  filter(Sales > 14)

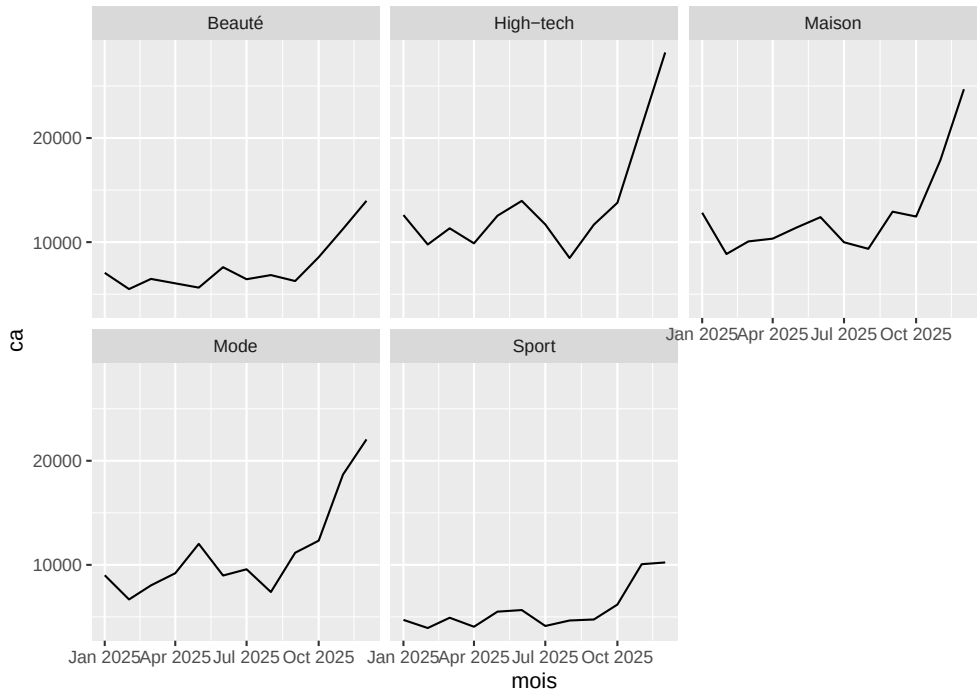
ggplot(Carseats, aes(x = Price, y = Sales)) +
  geom_point(alpha = 0.3) +
  geom_label(data = meilleurs, aes(label = magasin), color = "darkred", size =
   ↪  3)
```



4.3.6 *Faceting*

Le *faceting* répète le même graphique pour chaque modalité d'une variable qualitative, **avec les mêmes échelles**, ce qui permet la comparaison directe.

```
commandes |>
  mutate(mois = floor_date(date, "month")) |>
  group_by(categorie, mois) |>
  summarise(ca = sum(montant), .groups = "drop") |>
  ggplot(aes(x = mois, y = ca)) +
  geom_line() +
  facet_wrap(vars(categorie))
```



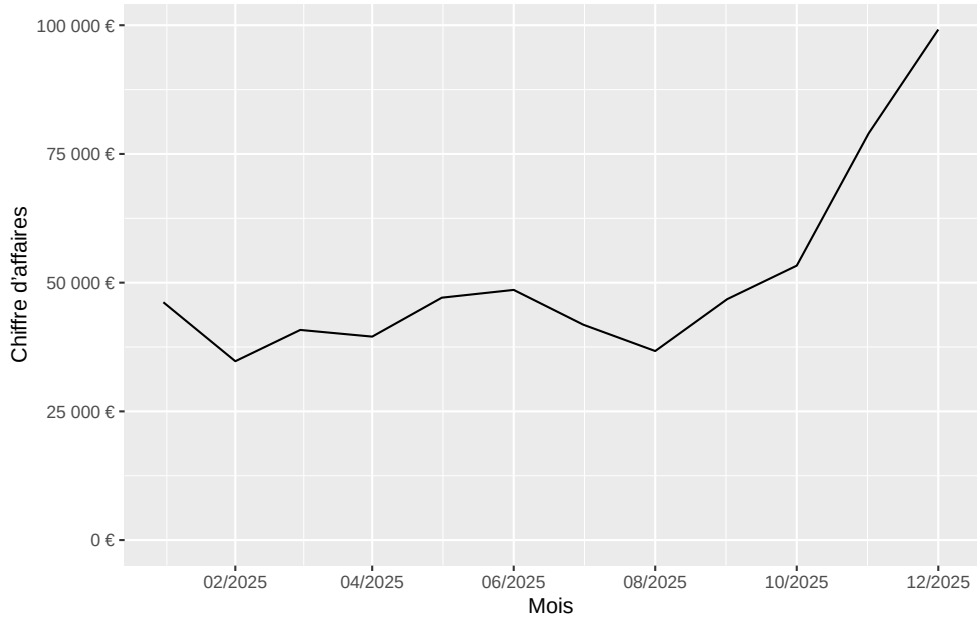
`facet_grid(rows = vars(a), cols = vars(b))` dispose les graphiques selon une grille croisant deux variables.

4.3.7 *Scales*

Les *scales* contrôlent la façon dont les valeurs sont traduites en attributs graphiques, et la légende ou l'axe correspondant. Elles prennent la forme `scale_<attribut>_<type>()`.

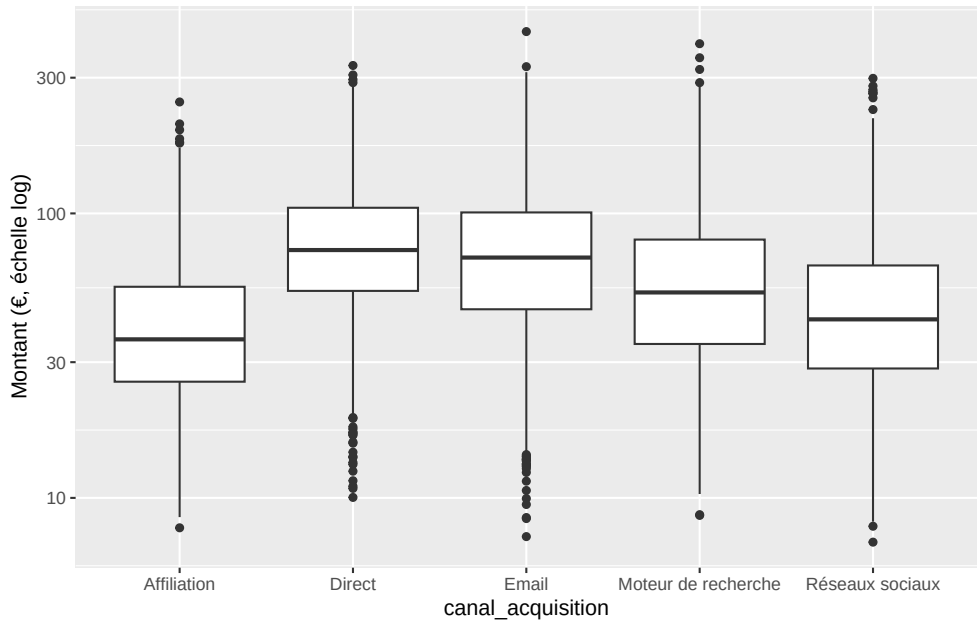
Axes — titres, limites, format des nombres, échelle logarithmique :

```
ggplot(ca_mensuel, aes(x = mois, y = ca)) +
  geom_line() +
  scale_x_date("Mois", date_labels = "%m/%Y", date_breaks = "2 months") +
  scale_y_continuous("Chiffre d'affaires", limits = c(0, NA),
                     labels = scales::label_number(suffix = " €", big.mark = "
                      ↪  "))
```



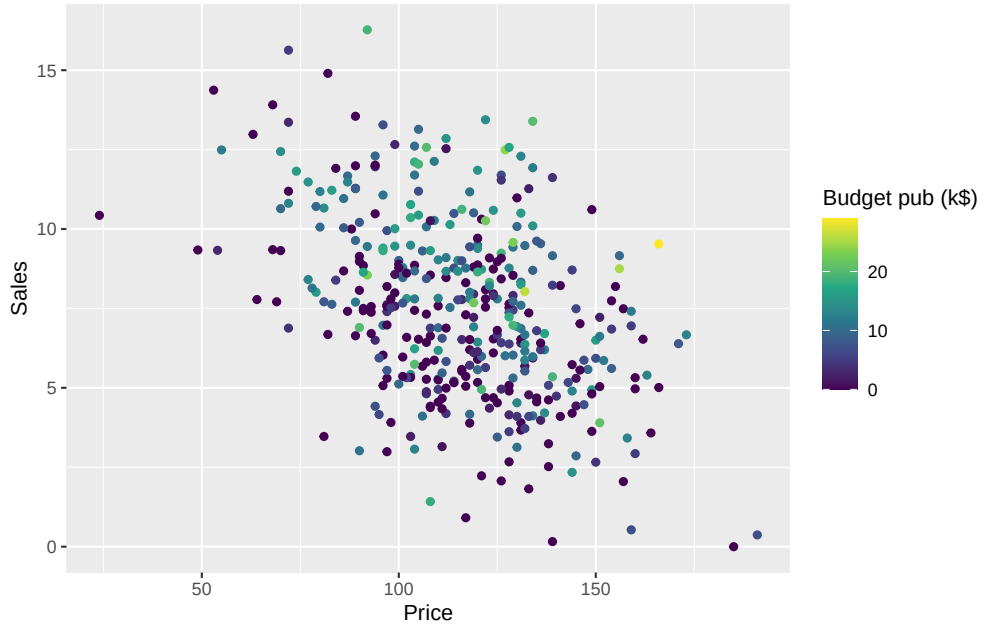
Faire partir l'axe des ordonnées de 0 évite d'exagérer visuellement les variations, ce qui est essentiel pour des barres. Les montants, très asymétriques, se lisent mieux en **échelle logarithmique** :

```
ggplot(commandes_clients) +
  geom_boxplot(aes(x = canal_acquisition, y = montant)) +
  scale_y_log10("Montant (€, échelle log)")
```

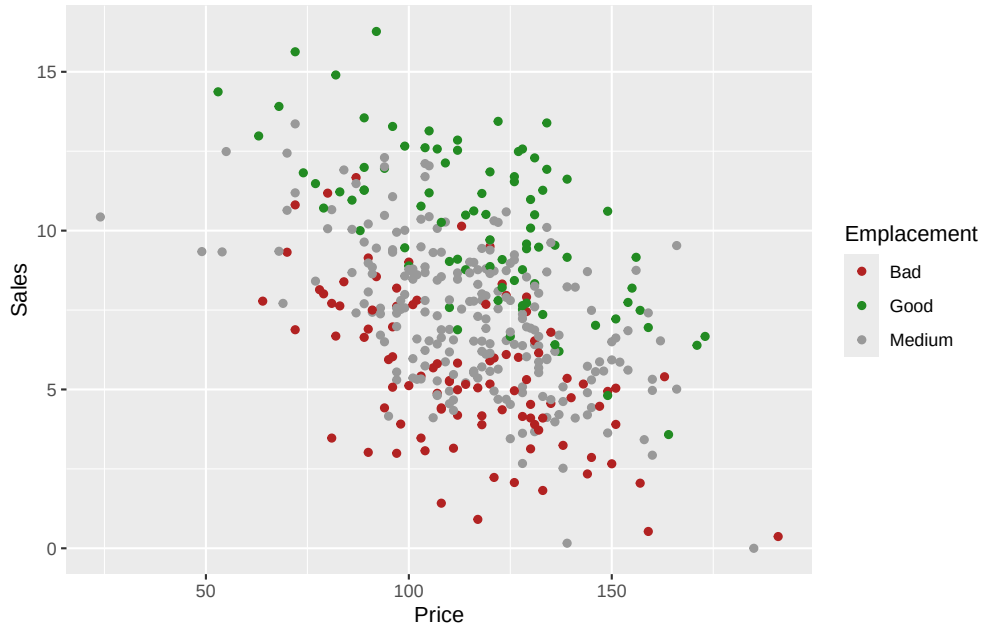


Couleurs — pour une variable quantitative, `scale_color_gradient()`, `scale_color_viridis_c()` ou `scale_color_distiller()` ; pour une variable qualitative, `scale_color_brewer()`, `scale_color_viridis_d()` ou `scale_color_manual()` :

```
ggplot(Carseats) +
  geom_point(aes(x = Price, y = Sales, color = Advertising)) +
  scale_color_viridis_c("Budget pub (k$)")
```




```
ggplot(Carseats) +
  geom_point(aes(x = Price, y = Sales, color = ShelveLoc)) +
  scale_color_manual("Emplacement",
                     values = c(Bad = "firebrick", Medium = "grey60", Good =
                      ↪  "forestgreen"))
```



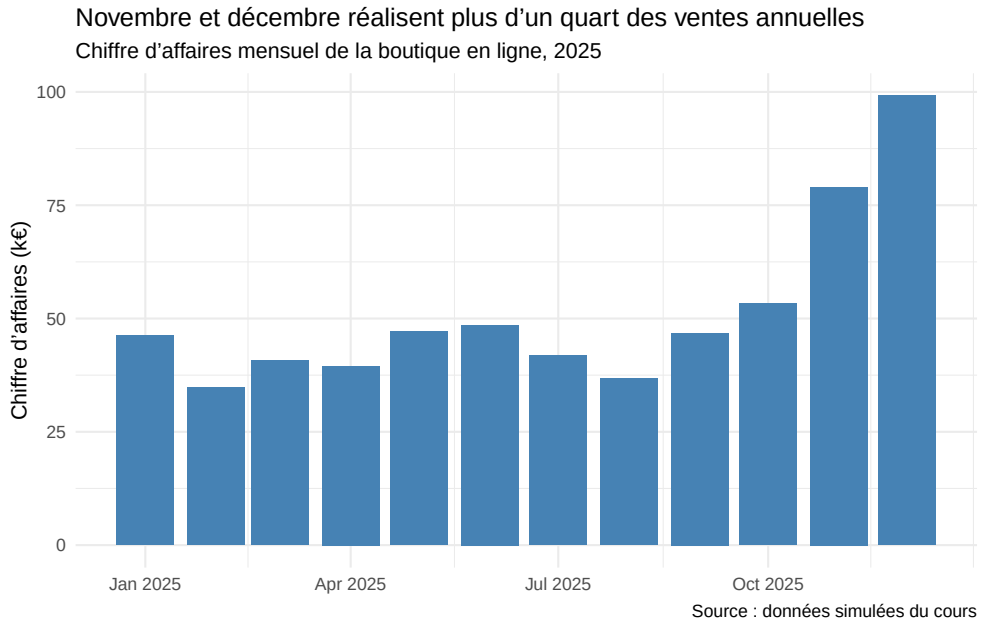
Les palettes `viridis` et certaines palettes ColorBrewer restent lisibles pour les personnes daltoniennes (`RColorBrewer::display.brewer.all()` les liste).

Tailles — `scale_size(range = c(min, max))`.

4.3.8 Titres et thèmes

`labs()` définit le titre, le sous-titre, la source et les noms des axes et légendes ; les thèmes (`theme_minimal()`, `theme_bw()`, `theme_classic()`…) règlent l'apparence générale, et `theme()` permet de modifier chaque élément.

```r
ggplot(ca_mensuel, aes(x = mois, y = ca / 1000)) +
  geom_col(fill = "steelblue") +
  labs(title = "Novembre et décembre réalisent plus d'un quart des ventes
   ↪  annuelles",
       subtitle = "Chiffre d'affaires mensuel de la boutique en ligne, 2025",
       x = NULL, y = "Chiffre d'affaires (k€)",
       caption = "Source : données simulées du cours") +
  theme_minimal()
```



Un graphique de présentation : un titre qui énonce le **message**, des axes avec unités, une source, des couleurs lisibles, aucun effet inutile. Enregistrer avec `ggsave("figure.png", width = 8, height = 5, dpi = 300)`.

Le package `esquisse` (menu *Addins* de RStudio, ou `esquisse::esquisser()`) permet de construire un graphique `ggplot2` à la souris, puis de récupérer le code correspondant : pratique pour explorer ou retrouver une syntaxe.

4.4 Régression linéaire simple

4.4.1 Le modèle

On cherche à expliquer une variable quantitative Y (la **réponse**) par une variable quantitative X (le **prédicteur**) :

$$Y_i = \beta_0 + \beta_1 X_i + \varepsilon_i, \qquad i = 1, \dots, n$$

- β_0 : ordonnée à l'origine ;
- β_1 : pente, c'est-à-dire la variation moyenne de Y quand X augmente d'une unité ;
- ε_i : erreur, supposée centrée, de variance constante σ^2, indépendante d'une observation à l'autre (et gaussienne pour les tests).

La méthode des **moindres carrés ordinaires** choisit $\hat\beta_0$ et $\hat\beta_1$ qui minimisent la somme des carrés des résidus $\sum_i (Y_i - \hat\beta_0 - \hat\beta_1 X_i)^2$. On obtient :

$$\hat{\beta}_1 = \frac{\text{cov}(X, Y)}{\text{var}(X)}, \qquad \hat{\beta}_0 = \bar{Y} - \hat{\beta}_1 \bar{X}$$

4.4.2 Ajuster le modèle avec `lm()`

Quel est le lien entre le prix pratiqué et les ventes ? La **formule** `Sales ~ Price` se lit « `Sales` expliqué par `Price` ». On passe le tableau avec l'argument `data` (plutôt que d'écrire `Carseats$Sales ~ Carseats$Price`, ce qui empêche ensuite d'utiliser `predict()`).

```
modele_1 <- lm(Sales ~ Price, data = Carseats)
summary(modele_1)
```

```
#>
#> Call:
#> lm(formula = Sales ~ Price, data = Carseats)
#>
#> Residuals:
#>    Min     1Q Median     3Q    Max
#> -6.522 -1.844 -0.146  1.650  7.511
#>
#> Coefficients:
#>             Estimate Std. Error t value Pr(>|t|)
#> (Intercept) 13.64192    0.63281   21.56   <2e-16 ***
#> Price       -0.05307    0.00535   -9.91   <2e-16 ***
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
#>
#> Residual standard error: 2.53 on 398 degrees of freedom
#> Multiple R-squared:  0.198,  Adjusted R-squared:  0.196
#> F-statistic: 98.2 on 1 and 398 DF,  p-value: <2e-16
```

Lire la sortie de `summary()`

- *Estimate* : $\hat{\beta}_1 \approx$ -0.053 ; chaque dollar supplémentaire est associé, en moyenne, à 53 unités vendues en moins.
- *Std. Error* : précision de l'estimation.
- *t value* et *Pr(>|t|)* : test de $H_0 : \beta_1 = 0$. Une p-valeur très petite indique que la relation observée serait très improbable si la pente réelle était nulle.
- *Multiple R-squared* (R^2) : part de la variance des ventes expliquée par le modèle, ici environ 20 % ; le prix seul n'explique qu'une petite partie des différences entre magasins.
- *Residual standard error* : estimation de σ, l'erreur typique de prédiction (en milliers d'unités).

```
coef(modele_1)
```

```
#> (Intercept)       Price
#>    13.64192    -0.05307
```

```
confint(modele_1)        # intervalles de confiance à 95 %
```

```
#>                2.5 %   97.5 %
#> (Intercept) 12.3978 14.88599
#> Price       -0.0636 -0.04255
```

4.4.3 Prédire

```
scenarios <- tibble(Price = c(100, 120, 140))
predict(modele_1, newdata = scenarios, interval = "confidence")   # ventes
 ↪  moyennes
```

```
#>     fit   lwr   upr
#> 1 8.335 8.035 8.634
#> 2 7.273 7.020 7.526
#> 3 6.212 5.855 6.568
```

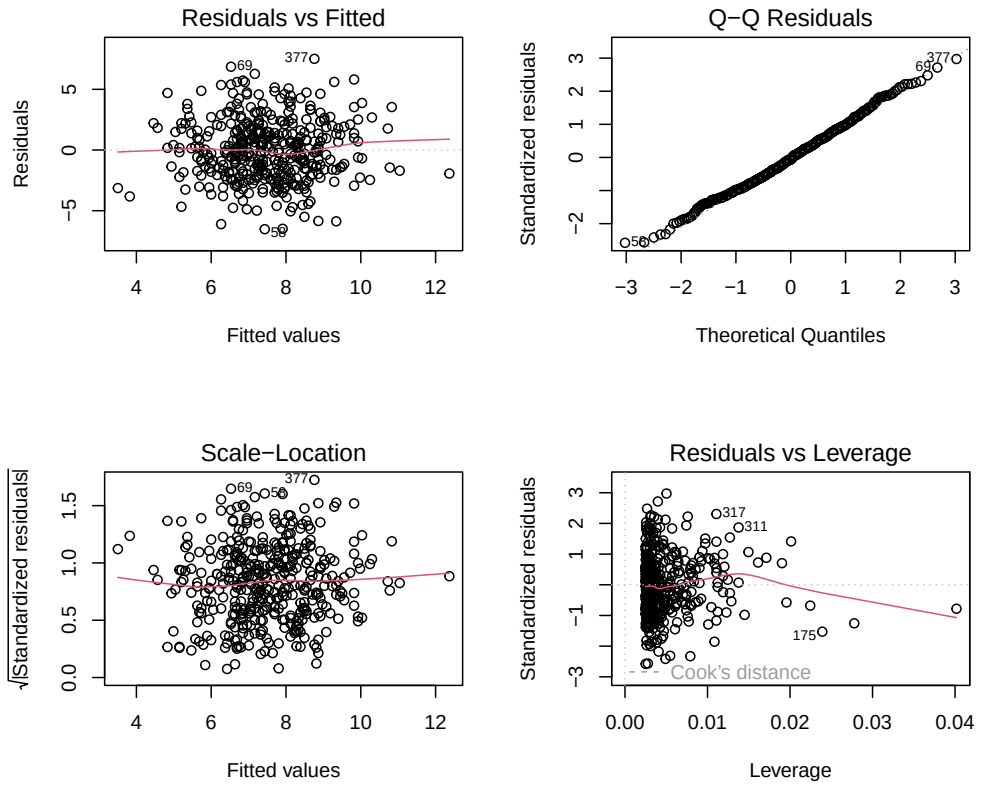
```
predict(modele_1, newdata = scenarios, interval = "prediction")   # pour un
 ↪  magasin
```

```
#>     fit   lwr   upr
#> 1 8.335 3.347 13.32
#> 2 7.273 2.288 12.26
#> 3 6.212 1.221 11.20
```

L'intervalle de **prédiction** (pour un magasin) est bien plus large que l'intervalle de **confiance** (pour la moyenne des magasins) : il inclut la variabilité d'un magasin à l'autre.

4.4.4 Vérifier les hypothèses : les résidus

```
par(mfrow = c(2, 2))
plot(modele_1)
```




```
par(mfrow = c(1, 1))
```

Graphique	Ce qu'on vérifie	Signe d'alerte
Residuals vs Fitted	linéarité	une courbure
Q-Q Residuals	normalité des résidus	points loin de la diagonale
Scale-Location	variance constante	un entonnoir
Residuals vs Leverage	points influents	points au-delà des courbes de Cook

Corrélation n'est pas causalité. Ces données sont **observationnelles** : les magasins n'ont pas reçu leur prix au hasard. Un magasin peut vendre cher *parce que* sa clientèle est aisée, ou parce que le concurrent est cher aussi. Pour mesurer l'effet causal d'un prix, l'idéal est une **expérience** (test A/B de prix). À défaut, il faut au moins tenir compte des autres facteurs : c'est le rôle de la régression multiple.

4.5 Régression linéaire multiple

4.5.1 Le modèle

$$Y_i = \beta_0 + \beta_1 X_{i1} + \cdots + \beta_p X_{ip} + \varepsilon_i$$

Chaque β_j s'interprète **toutes les autres variables étant fixées** (*ceteris paribus*).

```
modele_2 <- lm(Sales ~ Price + CompPrice + Advertising + ShelveLoc + Age +
 ↪  Income,
               data = Carseats)
summary(modele_2)
```

```
#>
#> Call:
#> lm(formula = Sales ~ Price + CompPrice + Advertising + ShelveLoc +
#>     Age + Income, data = Carseats)
#>
#> Residuals:
#>    Min     1Q Median     3Q    Max
#> -2.773 -0.695  0.028  0.673  3.329
#>
#> Coefficients:
#>                Estimate Std. Error t value Pr(>|t|)
#> (Intercept)     5.47523    0.50501   10.84   <2e-16 ***
#> Price          -0.09532    0.00267  -35.70   <2e-16 ***
#> CompPrice       0.09257    0.00412   22.45   <2e-16 ***
#> Advertising     0.11590    0.00772   15.01   <2e-16 ***
#> ShelveLocGood   4.83567    0.15250   31.71   <2e-16 ***
#> ShelveLocMedium 1.95199    0.12538   15.57   <2e-16 ***
#> Age            -0.04613    0.00318  -14.52   <2e-16 ***
#> Income          0.01578    0.00184    8.59   <2e-16 ***
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
#>
#> Residual standard error: 1.02 on 392 degrees of freedom
#> Multiple R-squared:  0.872,  Adjusted R-squared:  0.87
#> F-statistic:  381 on 7 and 392 DF,  p-value: <2e-16
```

Interprétation marketing

- `Price` : à prix du concurrent et autres facteurs égaux, chaque dollar de plus fait perdre environ 95 unités. L'effet est presque **deux fois plus fort** que dans le modèle simple : dans les données, les magasins chers sont souvent face à des concurrents chers (`CompPrice`), ce qui masquait une partie de l'effet du prix. Oublier une variable importante **biaise** les coefficients des autres.
- `CompPrice` : quand le concurrent augmente son prix d'un dollar, nos ventes augmentent d'environ 93 unités.
- `Advertising` : 1 000 $ de publicité locale en plus sont associés à environ 116 unités vendues en plus. Si chaque unité rapporte une marge de 5 $, la publicité ne serait pas rentable à la marge ($116 \times 5\ \$ < 1\ 000\ \$$) : un calcul de **retour sur investissement** simple, à confirmer par une expérience.
- `ShelveLoc` (variable qualitative) : R crée des **indicatrices** par rapport à la modalité de **référence** `Bad`. Un bon emplacement en rayon est associé à 4.8 milliers d'unités de plus qu'un mauvais. Pour changer de référence : `relevel()`.

Le R^2 passe de 0.2 à 0.87.

4.5.2 Comparer des modèles

- Le R^2 augmente **toujours** quand on ajoute une variable : il ne permet pas de comparer des modèles de tailles différentes. On utilise le R^2 **ajusté**, l'**AIC** (plus petit = meilleur) ou, mieux, l'erreur de prédiction sur un jeu de **validation**.
- Deux modèles emboîtés se comparent avec un test de Fisher (`anova()`).

```
anova(modele_1, modele_2)
```

```
#> Analysis of Variance Table
#>
#> Model 1: Sales ~ Price
#> Model 2: Sales ~ Price + CompPrice + Advertising + ShelveLoc + Age + Income
#>   Res.Df  RSS Df Sum of Sq   F Pr(>F)
#> 1    398 2552
#> 2    392  407  6      2145 344 <2e-16 ***
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
AIC(modele_1, modele_2)
```

```
#>          df  AIC
#> modele_1  3 1882
#> modele_2  9 1160
```

4.5.3 Interactions

L'effet du prix dépend-il de l'emplacement en rayon ? Le terme `Price:ShelveLoc` (ou `Price * ShelveLoc`, qui inclut aussi les effets principaux) laisse la pente varier selon l'emplacement :

```
modele_3 <- lm(Sales ~ Price * ShelveLoc + CompPrice + Advertising + Age +
 ↪  Income,
               data = Carseats)
anova(modele_2, modele_3)
```

```
#> Analysis of Variance Table
#>
#> Model 1: Sales ~ Price + CompPrice + Advertising + ShelveLoc + Age + Income
#> Model 2: Sales ~ Price * ShelveLoc + CompPrice + Advertising + Age + Income
```

```
#>   Res.Df RSS Df Sum of Sq    F Pr(>F)
#> 1    392 407
#> 2    390 407  2     0.869 0.42   0.66
```

L'interaction n'est pas significative : l'effet du prix est le même quel que soit l'emplacement, ce que suggéraient les droites parallèles du graphique.

4.5.4 Évaluer la prédictibilité (cadre PCS)

On mesure la qualité prédictive sur des données **non utilisées** pour l'ajustement, et on la compare à une **référence** naïve (prédire la moyenne).

```
set.seed(2026)
indices_train <- sample(nrow(Carseats), size = round(0.7 * nrow(Carseats)))
train <- Carseats[indices_train, ]
test  <- Carseats[-indices_train, ]

rmse <- function(observe, predit) sqrt(mean((observe - predit)^2))

m1 <- lm(Sales ~ Price, data = train)
m2 <- lm(Sales ~ Price + CompPrice + Advertising + ShelveLoc + Age + Income,
 ↪  data = train)
c(reference = rmse(test$Sales, mean(train$Sales)),
  simple    = rmse(test$Sales, predict(m1, test)),
  multiple  = rmse(test$Sales, predict(m2, test)))
```

```
#> reference    simple  multiple
#>     2.971     2.638     1.059
```

La racine de l'erreur quadratique moyenne (RMSE) est en milliers d'unités. Le modèle multiple divise l'erreur par plus de deux. Changez la graine : le classement des modèles reste-t-il le même ? C'est une vérification de **stabilité**.

4.6 Régression logistique

4.6.1 Quand la réponse est binaire

Pour prédire une variable **binaire** (achète / n'achète pas, résilie / reste), la régression linéaire ne convient pas : elle peut prédire des valeurs hors de $[0, 1]$. La régression logistique modélise la **probabilité** $p = P(Y = 1 \mid X)$ via la fonction logit :

$$\log \frac{p}{1-p} = \beta_0 + \beta_1 X_1 + \cdots + \beta_p X_p \quad \Longleftrightarrow \quad p = \frac{1}{1 + e^{-(\beta_0 + \beta_1 X_1 + \ldots)}}$$

Le rapport $p/(1-p)$ est la **cote** (*odds*). Les coefficients sont estimés par **maximum de vraisemblance** avec `glm(..., family = binomial)`.

4.6.2 Exemple : choix de marque

Le jeu `OJ` (package `ISLR`) décrit 1 070 achats **réels** de jus d'orange dans cinq magasins américains, issus de données de caisse utilisées dans l'ouvrage *Business Analysis Using Regression* (Foster, Stine et Waterman, 1998). À chaque achat, le client a choisi entre deux marques, Citrus Hill (`CH`) et Minute Maid (`MM`). Qu'est-ce qui fait choisir Minute Maid ? Ces données sont réutilisées au chapitre 5.

Variable	Description
`Purchase`	marque achetée : `CH` ou `MM` (**la cible**)
`WeekofPurchase`	semaine de l'achat (numéro de semaine)
`StoreID`, `STORE`, `Store7`	magasin de l'achat (identifiant ; codage alternatif ; magasin n° 7 ou non)
`PriceCH`, `PriceMM`	prix de référence de chaque marque ($)
`DiscCH`, `DiscMM`	remise appliquée à chaque marque ($)
`SalePriceCH`, `SalePriceMM`	prix de vente, après remise ($)
`PctDiscCH`, `PctDiscMM`	remise en pourcentage du prix de référence
`PriceDiff`	prix de vente de MM moins prix de vente de CH ($)
`ListPriceDiff`	prix de référence de MM moins prix de référence de CH ($)
`SpecialCH`, `SpecialMM`	mise en avant spéciale de la marque en magasin (1 = oui)
`LoyalCH`	fidélité du client à CH, entre 0 et 1, calculée sur ses achats passés

```
oj <- OJ |> mutate(achat_mm = as.integer(Purchase == "MM"))
mean(oj$achat_mm)
```

```
#> [1] 0.3897
```

```
modele_log <- glm(achat_mm ~ PriceDiff + LoyalCH + SpecialMM + SpecialCH,
                  data = oj, family = binomial)
summary(modele_log)
```

```
#>
#> Call:
#> glm(formula = achat_mm ~ PriceDiff + LoyalCH + SpecialMM + SpecialCH,
#>     family = binomial, data = oj)
#>
#> Coefficients:
#>             Estimate Std. Error z value Pr(>|z|)
#> (Intercept)    3.241      0.237   13.66  < 2e-16 ***
#> PriceDiff     -2.771      0.376   -7.37  1.7e-13 ***
#> LoyalCH       -6.354      0.386  -16.44  < 2e-16 ***
#> SpecialMM      0.190      0.248    0.77     0.44
#> SpecialCH     -0.394      0.261   -1.51     0.13
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
#>
#> (Dispersion parameter for binomial family taken to be 1)
#>
#>     Null deviance: 1430.85  on 1069  degrees of freedom
#> Residual deviance:  836.88  on 1065  degrees of freedom
#> AIC: 846.9
#>
#> Number of Fisher Scoring iterations: 5
```

Les coefficients sont sur l'échelle du **log de la cote**. Leur exponentielle est un **rapport de cotes** (*odds ratio*, OR) : OR > 1 augmente la cote d'acheter MM, OR < 1 la diminue.

```
round(exp(cbind(OR = coef(modele_log), confint.default(modele_log))), 3)
```

```
#>                OR  2.5 % 97.5 %
```

```
#> (Intercept) 25.548 16.050 40.668
#> PriceDiff    0.063  0.030  0.131
#> LoyalCH      0.002  0.001  0.004
#> SpecialMM    1.209  0.744  1.965
#> SpecialCH    0.674  0.405  1.124
```

- `PriceDiff` : quand MM devient plus cher que CH de 10 centimes, la cote d'acheter MM est multipliée par $e^{0,1\times\hat{\beta}} \approx 0.76$, soit une baisse d'environ 24 %.
- `LoyalCH` : la fidélité passée à CH est de loin le facteur le plus fort. En marketing, le comportement passé est souvent le meilleur prédicteur du comportement futur.
- Les mises en avant en magasin n'ont pas d'effet significatif ici.

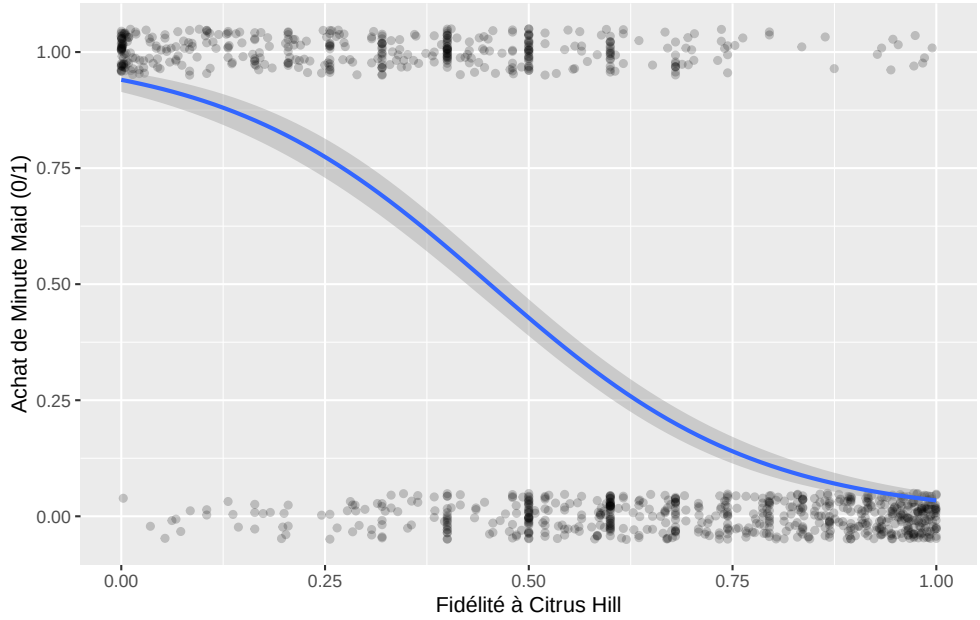
4.6.3 Prédire des probabilités

```
nouveaux <- tibble(PriceDiff = c(-0.2, 0.3), LoyalCH = 0.5, SpecialMM = 0,
 ↪  SpecialCH = 0)
predict(modele_log, newdata = nouveaux, type = "response")
```

```
#>      1      2
#> 0.6498 0.3170
```

Pour un client sans préférence (`LoyalCH = 0,5`), la probabilité d'acheter MM passe d'environ 65 % quand MM est 20 centimes moins cher à environ 32 % quand il est 30 centimes plus cher : une mesure de la **sensibilité au prix**.

```
ggplot(oj, aes(x = LoyalCH, y = achat_mm)) +
  geom_jitter(height = 0.05, alpha = 0.2) +
  geom_smooth(method = "glm", method.args = list(family = binomial), formula =
   ↪  y ~ x) +
  labs(x = "Fidélité à Citrus Hill", y = "Achat de Minute Maid (0/1)")
```



Transformer les probabilités en décisions (avec un **seuil**) et évaluer un classifieur (matrice de confusion, courbe ROC) est l'objet du chapitre 5.

4.7 Résumé

- Visualiser avant de modéliser (quartet d'Anscombe) ; un graphique a un message, des axes avec unités, une source.
- R de base : `plot()`, `hist()`, `boxplot()`, `barplot()` ; paramètres `col`, `pch`, `cex`, `main`, `xlab`… ; `par(mfrow = ...)`.
- `ggplot2` : `ggplot(données) + geom_*(aes(...))` ; mappages **dans** `aes()`, constantes **hors** de `aes()` ; `position` des barres ; `facet_wrap(vars())` ; `scale_*()` ; `labs()`, `theme_*()` ; `ggsave()`.
- Régression linéaire : `lm(y ~ x1 + x2, data)`, `summary()`, `coef()`, `confint()`, `predict()`, diagnostics avec `plot(modele)`.
- Interpréter chaque coefficient « toutes choses égales par ailleurs » ; oublier une variable importante biaise les autres ; association $\neq$ causalité.
- Comparer les modèles sur des données de **validation** (RMSE), face à une référence naïve.
- Régression logistique : `glm(y ~ ..., family = binomial)` ; `exp(coef())` donne les rapports de cotes ; `predict(type = "response")` les probabilités.

4.8 Exercices

Exercice 4.1 — `ggplot2` sur `rp2018`. On utilise le jeu `rp2018` (package `questionr`), qui contient des résultats du recensement de 2018 pour les communes de plus de 2 000 habitants, restreint à cinq départements :

```
data(rp2018)
rp <- filter(rp2018, departement %in% c("Oise", "Rhône", "Hauts-de-Seine",
                                        "Lozère", "Bouches-du-Rhône"))
```

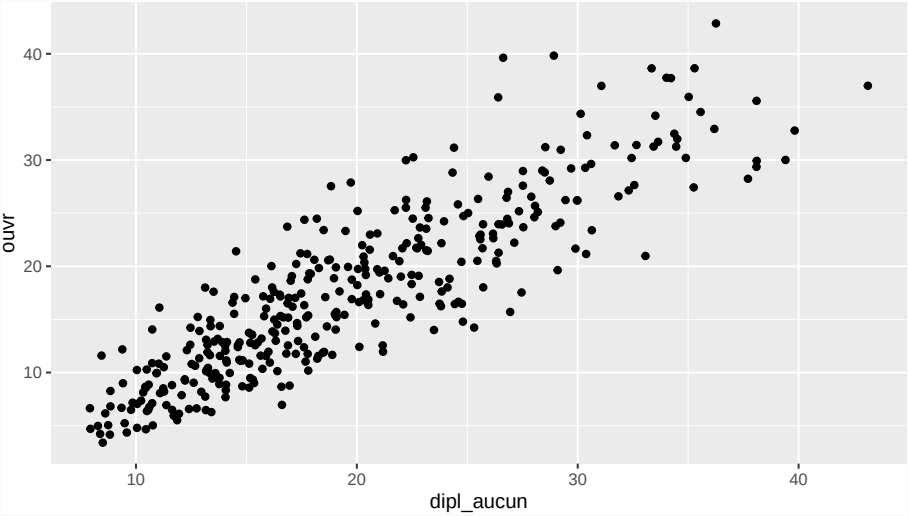
Variables utiles : `commune` (nom), `departement`, `pop_tot` (population totale), `pop_cl` (taille de la commune en classes), et des pourcentages : `dipl_aucun` (sans diplôme), `dipl_sup` (diplômés du supérieur), `ouvr` (ouvriers), `cadres`, `chom` (chômeurs), `proprio` (propriétaires).

1. Faire un nuage de points croisant le pourcentage de sans diplôme (`dipl_aucun`) et le pourcentage d'ouvriers (`ouvr`).
2. Refaire ce nuage avec des points rouges de transparence 0,2.
3. Représenter la répartition du pourcentage de propriétaires (`proprio`) selon la taille de la commune (`pop_cl`) sous forme de boîtes à moustaches.
4. Représenter le nombre de communes selon leur taille (`pop_cl`) sous forme de diagramme en bâtons.
5. Faire le nuage de points croisant le pourcentage de sans diplôme et le pourcentage d'ouvriers, en faisant varier la couleur selon le département.
6. Sur le même graphique, faire varier la taille des points selon la population totale (`pop_tot`).
7. Toujours sur le même graphique, rendre les points transparents (opacité 0,5).
8. Représenter la répartition du pourcentage de propriétaires selon la taille de la commune sous forme de boîtes à moustaches, en faisant varier la couleur de remplissage (`fill`) selon le département.
9. Représenter le nombre de communes selon leur taille sous forme de diagramme en bâtons empilés, avec une couleur différente par département.
10. Modifier le paramètre `position` pour afficher les barres les unes à côté des autres.
11. Modifier à nouveau `position` pour représenter, pour chaque taille de commune, les proportions de communes de chaque département.

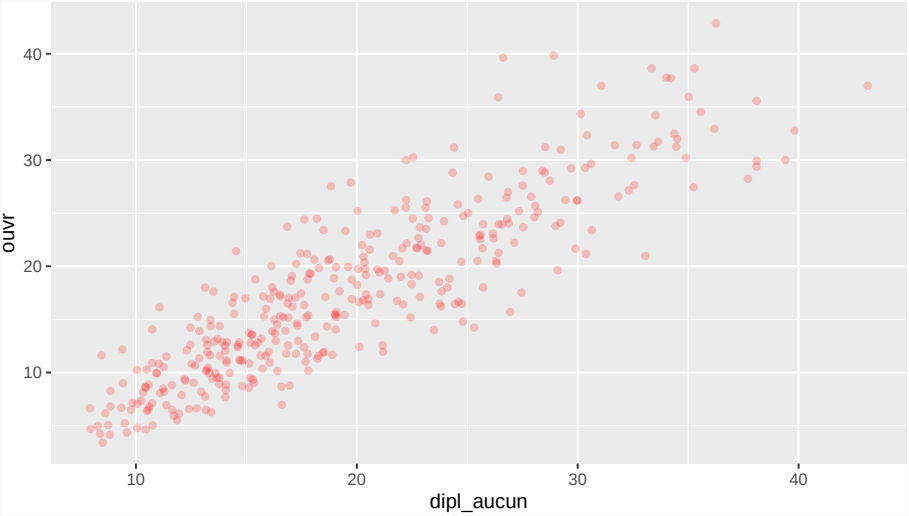
12. Faire un nuage de points du pourcentage de cadres (`cadres`, en abscisse) et du pourcentage de diplômés du supérieur (`dipl_sup`, en ordonnée), avec un graphique par département grâce à `facet_grid()`.
13. Sur le même graphique, faire varier la taille des points selon la population totale et rendre les points transparents.
14. Faire le nuage de points croisant le pourcentage de chômeurs (`chom`) et le pourcentage de sans diplôme, et y ajouter le nom des communes (`commune`), en rouge et en taille 2,5.
15. Dans le graphique précédent, n'afficher que le nom des communes ayant plus de 15 % de chômeurs.

Solution

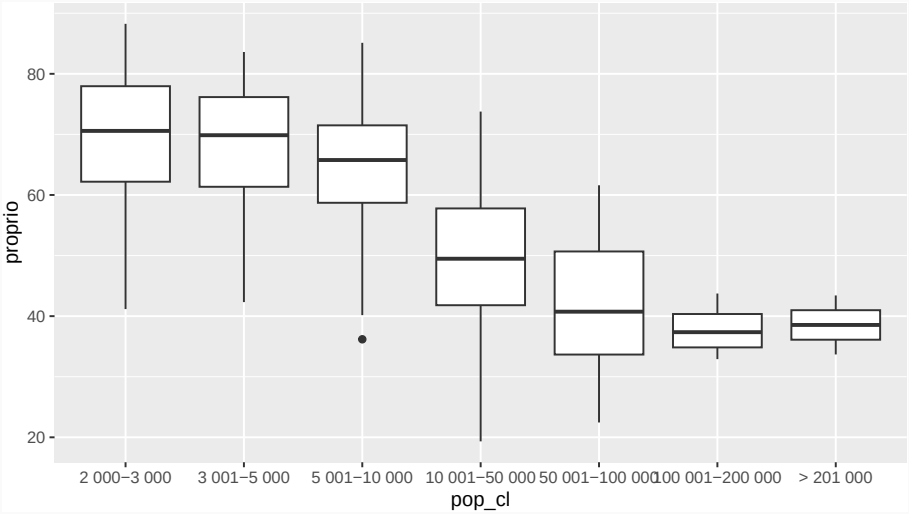
```
ggplot(rp) + geom_point(aes(x = dipl_aucun, y = ouvr))
 ↪  # 1
```



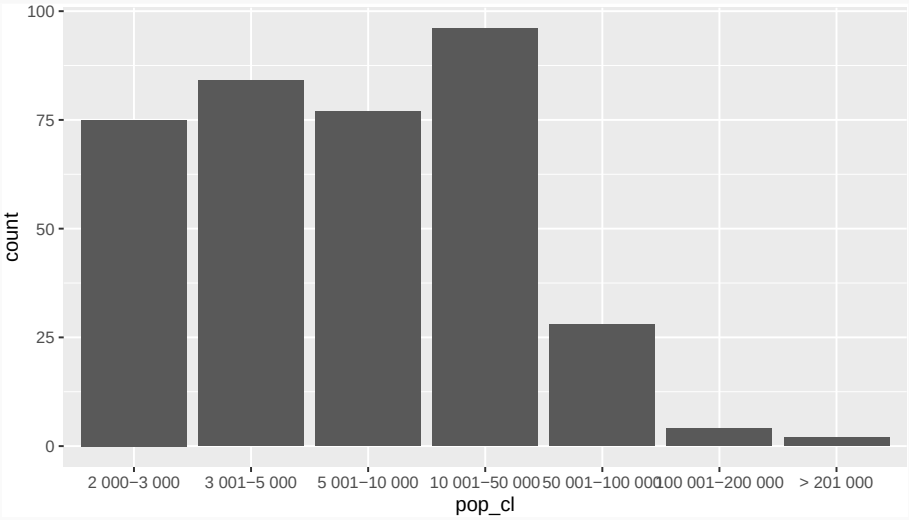

```
ggplot(rp) + geom_point(aes(x = dipl_aucun, y = ouvr), color = "red",
 ↪  alpha = 0.2)  # 2
```



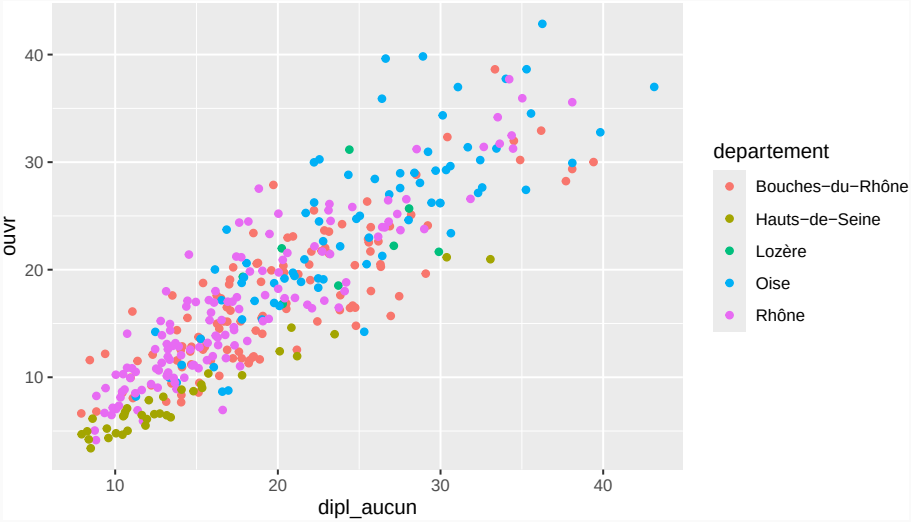

```
ggplot(rp) + geom_boxplot(aes(x = pop_cl, y = proprio))
 ↪  # 3
```



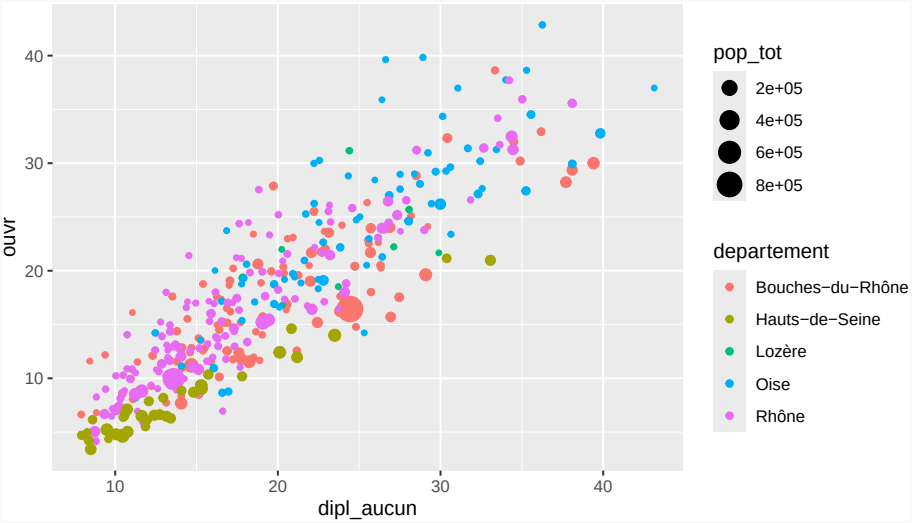

```
ggplot(rp) + geom_bar(aes(x = pop_cl))
↪  # 4
```



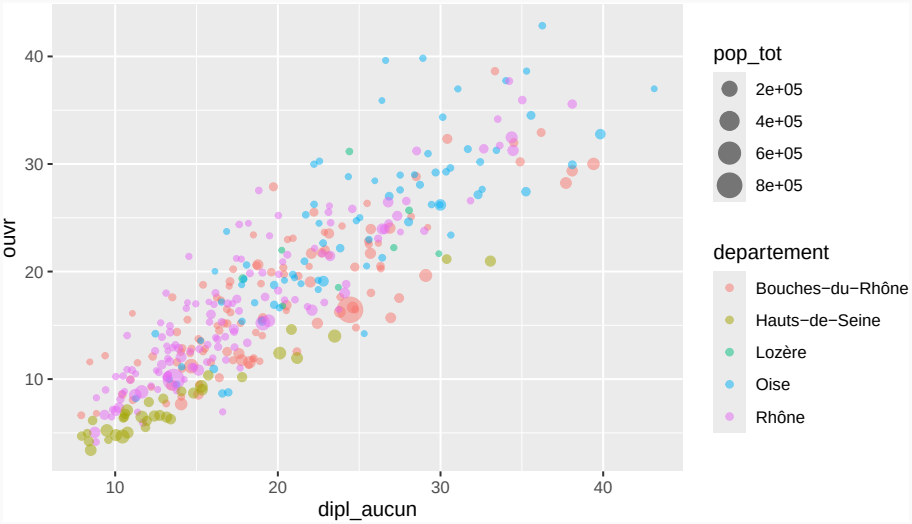

```
ggplot(rp) + geom_point(aes(x = dipl_aucun, y = ouvr, color =
↪  departement))    # 5
```



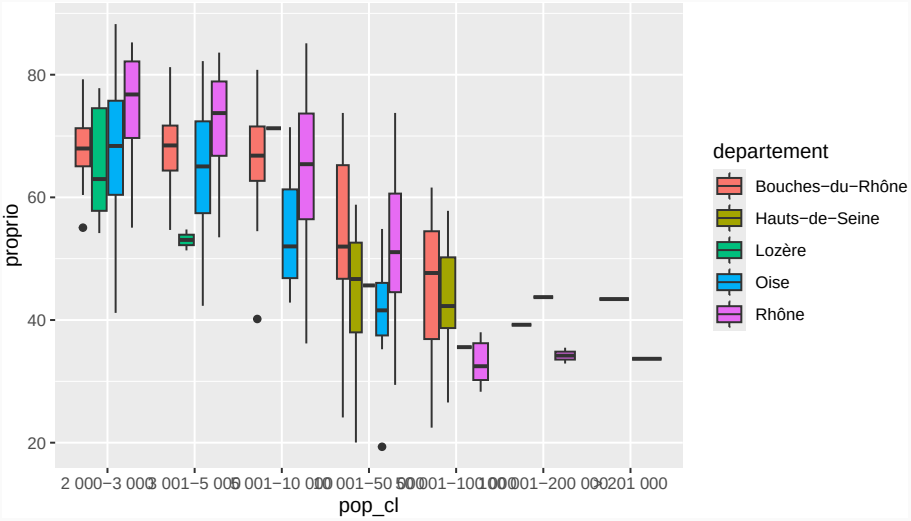
```
ggplot(rp) + geom_point(aes(x = dipl_aucun, y = ouvr, color = departement,
↪  # 6
                            size = pop_tot))
```



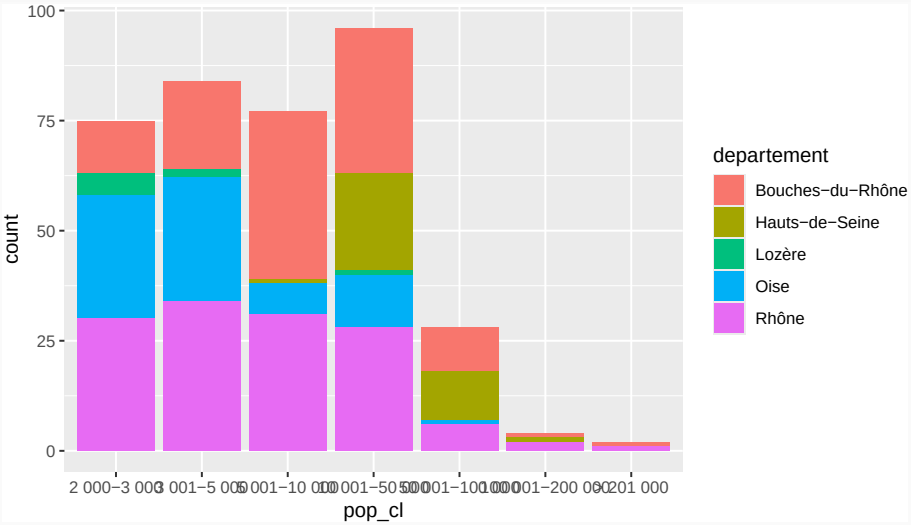

```
ggplot(rp) + geom_point(aes(x = dipl_aucun, y = ouvr, color = departement,
↪  # 7
                            size = pop_tot), alpha = 0.5)
```



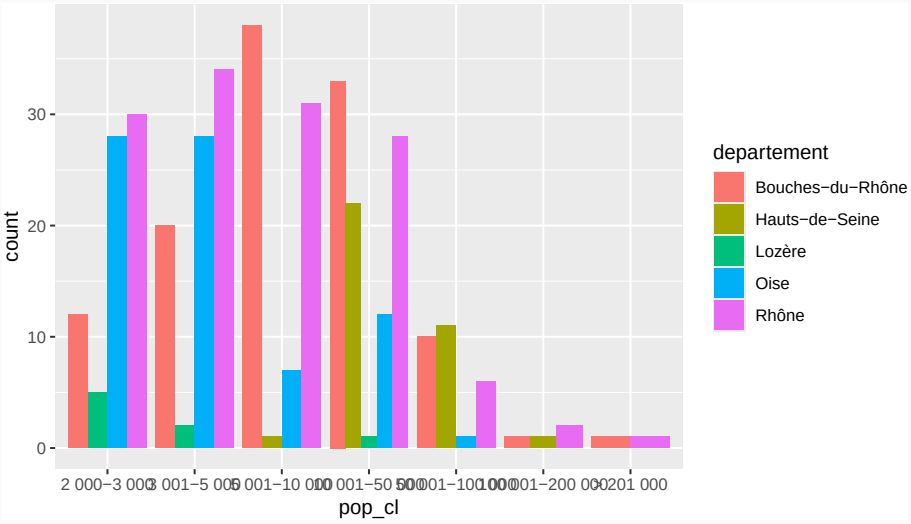

```
ggplot(rp) + geom_boxplot(aes(x = pop_cl, y = proprio, fill =
↪  departement))    # 8
```



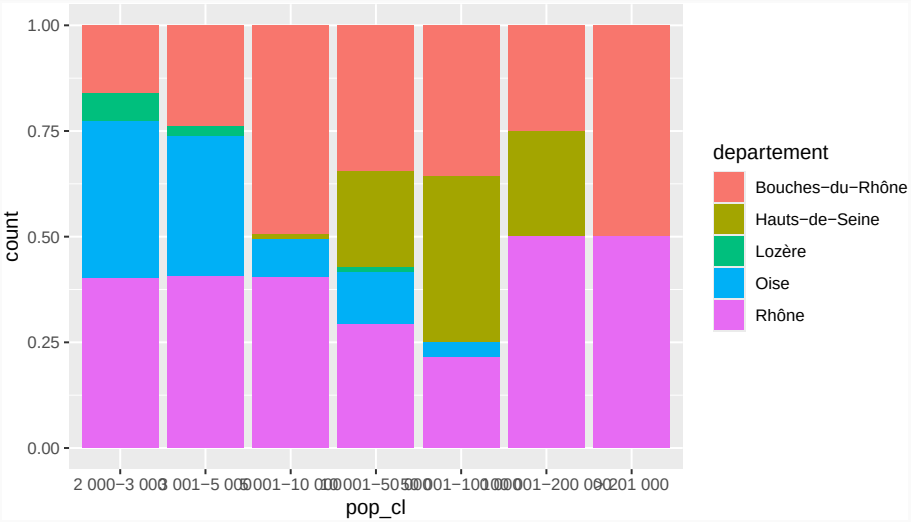

```
ggplot(rp) + geom_bar(aes(x = pop_cl, fill = departement))
↪  # 9
```



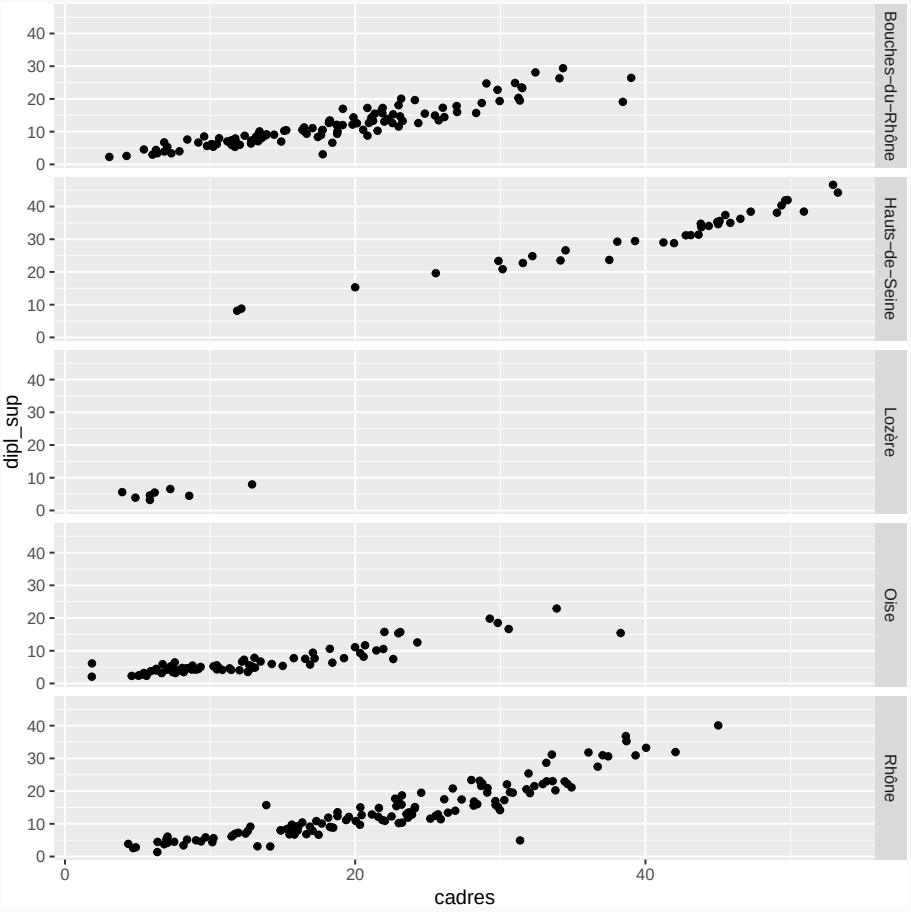

```
ggplot(rp) + geom_bar(aes(x = pop_cl, fill = departement), position =
↪  "dodge") # 10
```



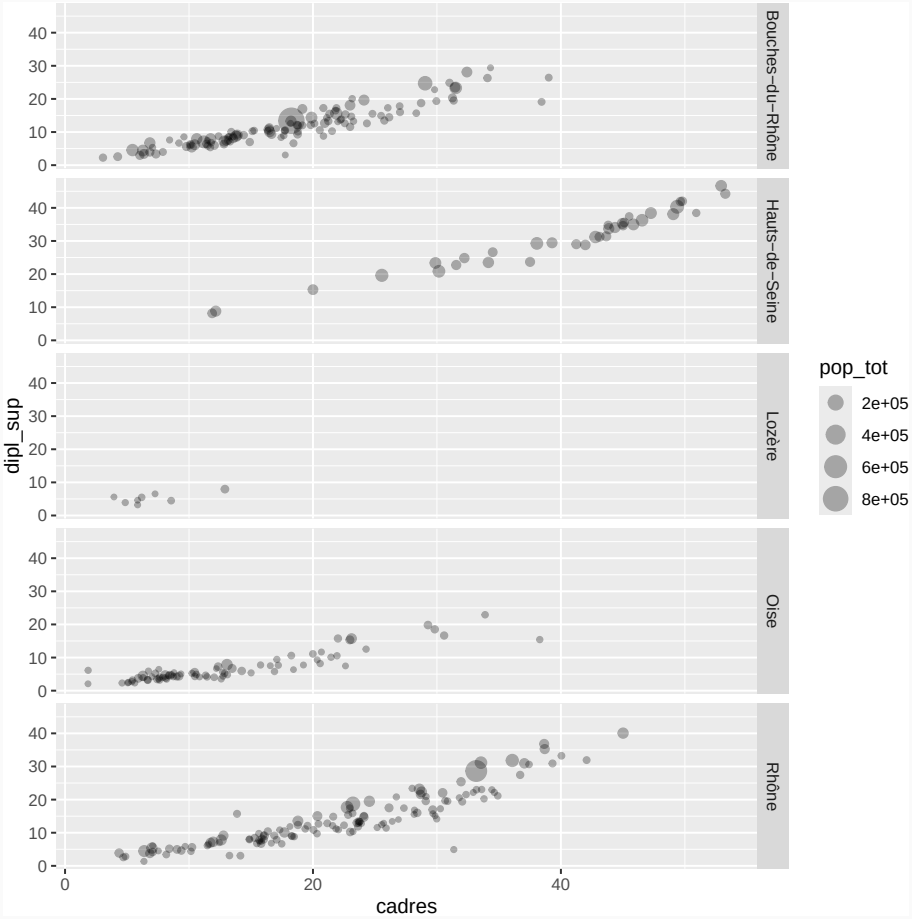
```
ggplot(rp) + geom_bar(aes(x = pop_cl, fill = departement), position =
 ↪  "fill")  # 11
```



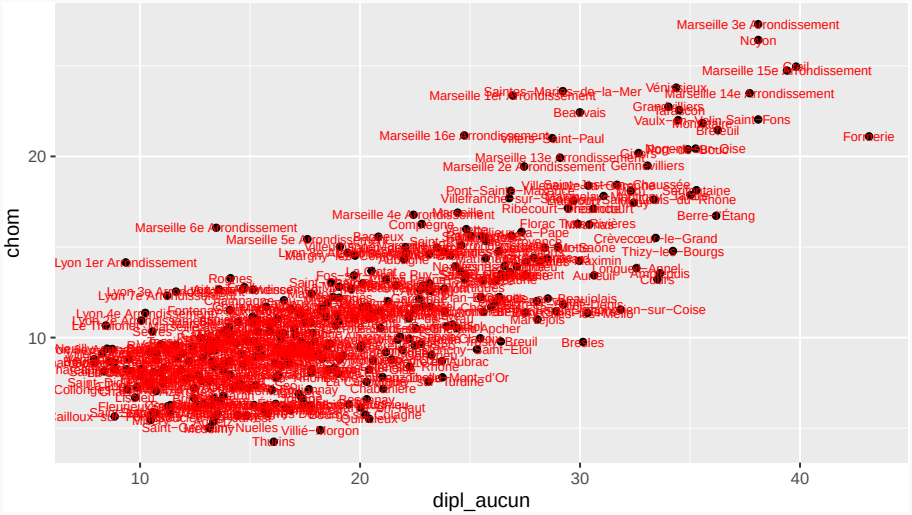

```
ggplot(rp) +
 ↪  # 12
  geom_point(aes(x = cadres, y = dipl_sup)) +
  facet_grid(rows = vars(departement))
```



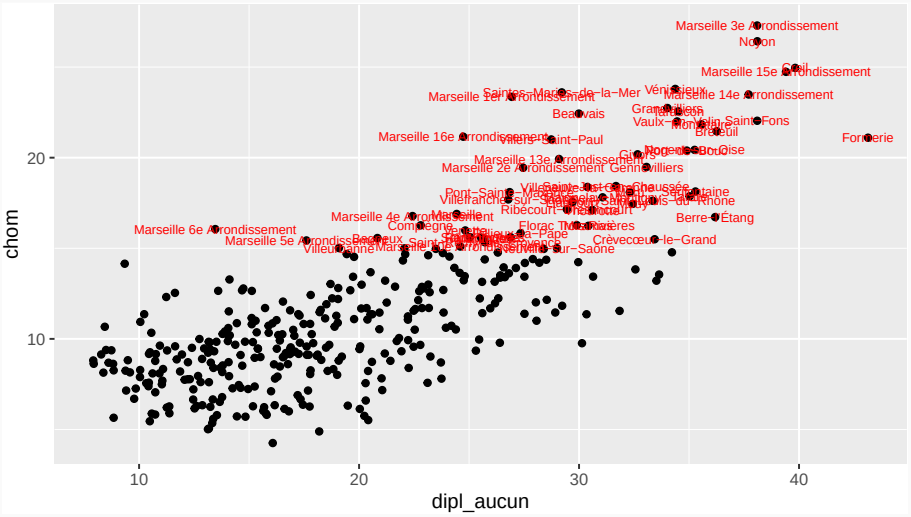
```
ggplot(rp) +
 ↪  # 13
  geom_point(aes(x = cadres, y = dipl_sup, size = pop_tot), alpha = 0.3) +
  facet_grid(rows = vars(departement))
```




```
ggplot(rp, aes(x = dipl_aucun, y = chom)) +
 ↪  # 14
  geom_point() +
  geom_text(aes(label = commune), color = "red", size = 2.5)
```



```
ggplot(rp, aes(x = dipl_aucun, y = chom)) +
  # 15
  geom_point() +
  geom_text(data = filter(rp, chom > 15), aes(label = commune),
            color = "red", size = 2.5)
```



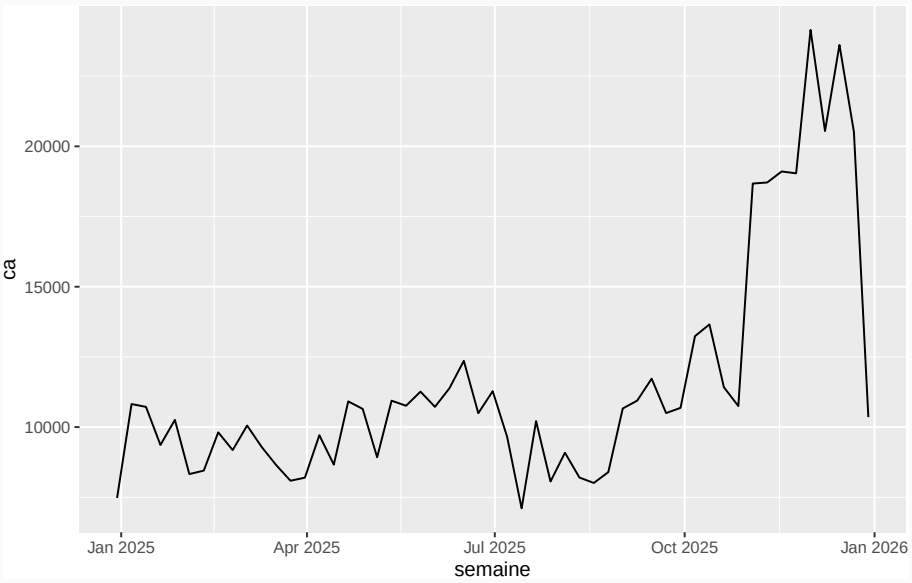
Au graphique 14, les étiquettes se superposent et deviennent illisibles : c'est pourquoi on n'affiche, en général, que les noms des points remarquables (graphique 15).

Exercice 4.2 — Tableau de bord de la boutique. Avec les tables de la boutique, produire :

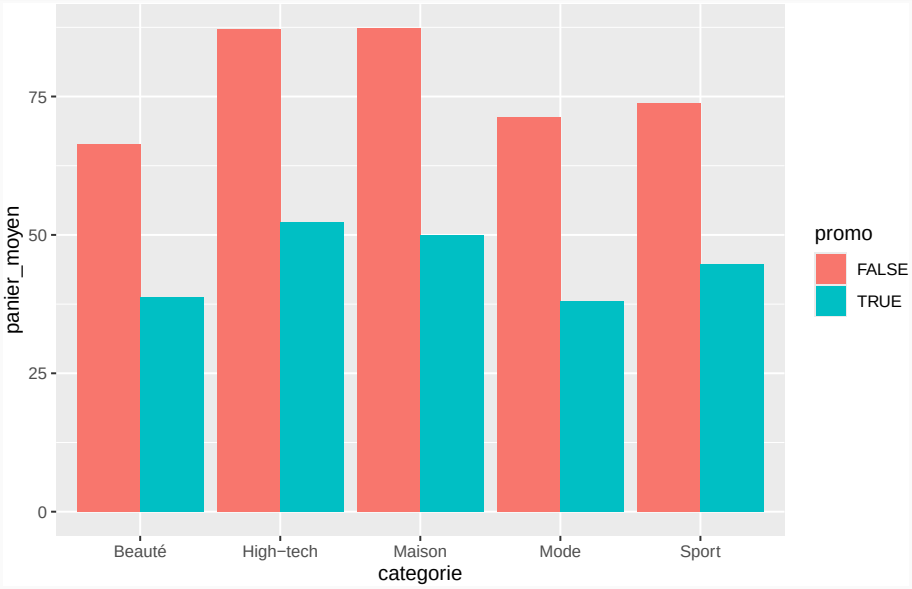
1. le chiffre d'affaires hebdomadaire de l'année (`floor_date(date, "week")`) ;
2. le panier moyen par catégorie et par statut promo, en barres côte à côte ;
3. la part des commandes en promotion par mois, en courbe ;
4. un graphique au choix, soigné (titre-message, axes, source), prêt à être présenté à la direction.

Solution (questions 1 à 3)

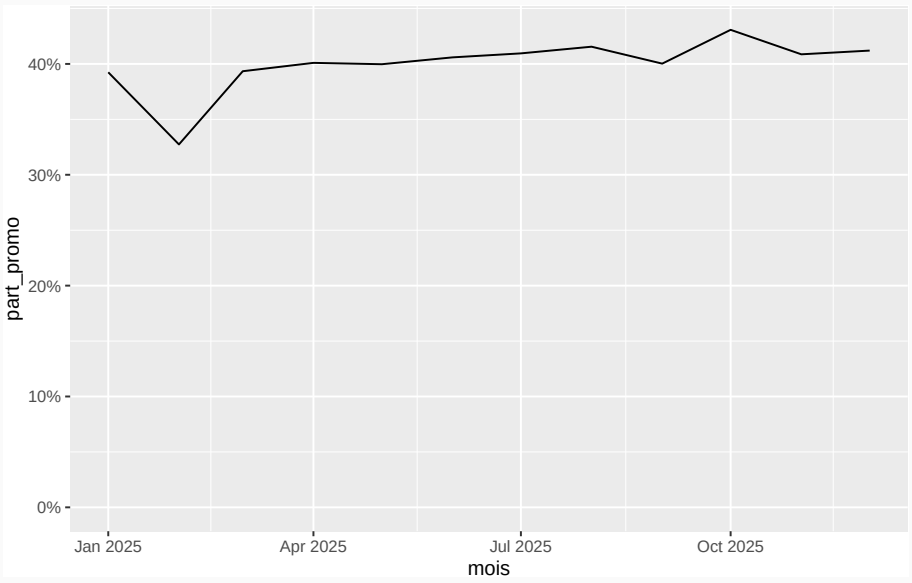
```
commandes |>
  mutate(semaine = floor_date(date, "week", week_start = 1)) |>
  group_by(semaine) |>
  summarise(ca = sum(montant)) |>
  ggplot(aes(semaine, ca)) +
  geom_line()
```




```
commandes |>
  group_by(categorie, promo) |>
  summarise(panier_moyen = mean(montant), .groups = "drop") |>
  ggplot(aes(x = categorie, y = panier_moyen, fill = promo)) +
  geom_col(position = "dodge")
```




```
commandes |>
  mutate(mois = floor_date(date, "month")) |>
  group_by(mois) |>
  summarise(part_promo = mean(promo)) |>
  ggplot(aes(mois, part_promo)) +
  geom_line() +
  scale_y_continuous(labels = scales::percent, limits = c(0, NA))
```



Exercice 4.3 — Régression sur `Carseats`.

1. Ajuster `Sales ~ Advertising` puis ajouter `Price`, `CompPrice` et `ShelveLoc`. Comment évolue le coefficient de `Advertising` ?
2. Prédire les ventes d'un magasin avec `Price = 110`, `CompPrice = 125`, `ShelveLoc = "Medium"` et un budget publicitaire de 0, 10 puis 20 k$.
3. Les magasins hors des États-Unis (`US = "No"`) ont-ils un budget publicitaire différent ? Utiliser un graphique et un test.

Solution

```
coef(lm(Sales ~ Advertising, data = Carseats))
```

```
#> (Intercept) Advertising
#>      6.7370      0.1144
```

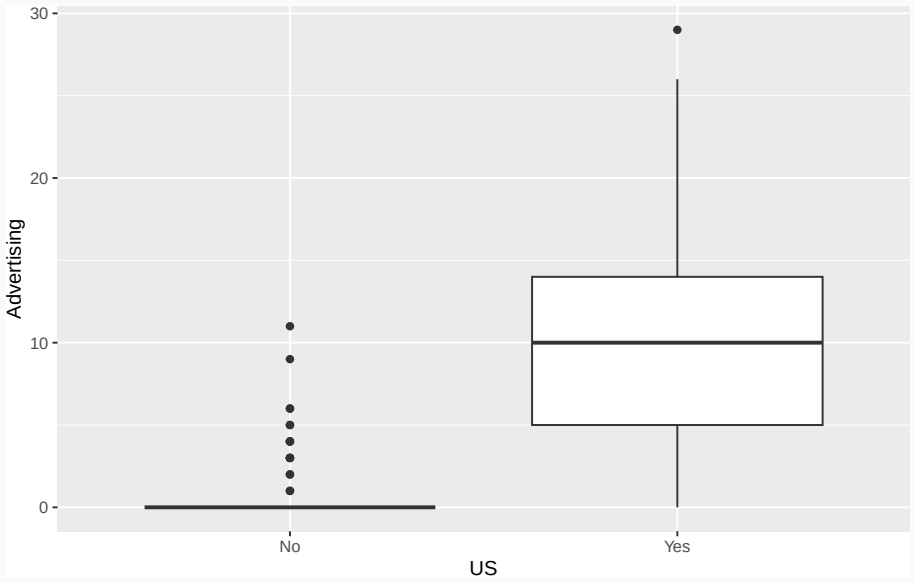
```
coef(lm(Sales ~ Advertising + Price + CompPrice + ShelveLoc, data =
 ↪  Carseats))
```

```
#>     (Intercept)     Advertising           Price       CompPrice
#>         3.84443         0.12028        -0.09359         0.09374
#>   ShelveLocGood ShelveLocMedium
#>         4.72932         1.77276
```

```
m <- lm(Sales ~ Advertising + Price + CompPrice + ShelveLoc, data =
 ↪  Carseats)
predict(m, newdata = tibble(Advertising = c(0, 10, 20), Price = 110,
                            CompPrice = 125, ShelveLoc = "Medium"))
```

```
#>     1     2     3
#> 7.040 8.243 9.446
```

```
ggplot(Carseats, aes(x = US, y = Advertising)) + geom_boxplot()
```




```
wilcox.test(Advertising ~ US, data = Carseats)
```

```
#>
#>  Wilcoxon rank sum test with continuity correction
#>
#> data:  Advertising by US
#> W = 2085, p-value <2e-16
#> alternative hypothesis: true location shift is not equal to 0
```

Hors des États-Unis, la grande majorité des magasins (125 sur 142) n'a aucun budget publicitaire.

Exercice 4.4 — Poids de naissance. Avec `phbirths` (`datasets/phbirths.dat`), modéliser le poids de naissance (`grams`) en fonction de la durée de gestation, puis ajouter le tabagisme de la mère (`smoke`). Interpréter le coefficient de `smoke`, puis ajuster une régression logistique du faible poids (moins de 2 500 g).

Solution

```
phbirths <- read.table("../datasets/phbirths.dat")
coef(lm(grams ~ gestate + smoke, data = phbirths))
```

```
#> (Intercept)     gestate   smokeTRUE
#>     -3027.0       162.0      -190.6
```

```
glm(I(grams < 2500) ~ gestate + smoke, data = phbirths, family = binomial)
 ↪  |>
  coef() |> exp() |> round(3)
```

```
#> (Intercept)     gestate   smokeTRUE
#>   1.061e+12   4.490e-01   2.349e+00
```

À durée de gestation égale, les bébés de mères fumeuses pèsent environ 190 g de moins en moyenne.

Pour aller plus loin

- J. Barnier, *Introduction à R et au tidyverse*, chapitre « Visualiser avec ggplot2 ».
- H. Wickham, *ggplot2: Elegant Graphics for Data Analysis*, 3e éd. (ggplot2-book.org).
- G. James et al., *An Introduction to Statistical Learning*, chapitres 3 (régression linéaire) et

4 (classification), en accès libre ; les jeux `Carseats` et `OJ` en sont issus.

Chapitre 5

Analyse en composantes principales, arbres de décision et forêts aléatoires

Objectifs du chapitre

- Comprendre le principe de l'analyse en composantes principales (ACP).
- Réaliser une ACP avec `FactoMineR`, lire le cercle des corrélations et le plan des individus.
- Utiliser des variables illustratives (supplémentaires) quantitatives et qualitatives pour interpréter les axes.
- Construire, élaguer et utiliser un arbre de décision avec `rpart`.
- Évaluer un classifieur : matrice de confusion, sensibilité, spécificité, courbe ROC et AUC.
- Comprendre le principe des forêts aléatoires, pour la classification et la régression, et les mettre en œuvre avec `randomForest`.

```
library(tidyverse)
library(FactoMineR)    # ACP
library(factoextra)    # graphiques des résultats d'ACP
library(rpart)         # arbres de décision
library(pROC)          # courbes ROC
library(randomForest) # forêts aléatoires
library(ISLR)          # jeux de données OJ et Caravan
source("R/boutique.R")
```

5.1 Analyse en composantes principales (ACP)

5.1.1 Le problème

On dispose de n individus décrits par p variables **quantitatives**. Avec $p = 2$, un nuage de points suffit pour tout voir ; avec $p = 10$, c'est impossible. L'ACP cherche à **résumer** le tableau par un petit nombre de nouvelles variables, les **composantes principales**, qui :

- sont des **combinaisons linéaires** des variables d'origine ;
- sont **non corrélées** entre elles ;
- conservent le **maximum de variabilité** (d'« information ») du nuage.

Géométriquement, on cherche le plan sur lequel la projection du nuage de points est la moins déformée possible, comme on choisirait l'angle de vue qui montre le mieux un objet en 3D sur une photo.

L'ACP répond à deux questions :

1. **Individus** : quels individus se ressemblent ? Quels profils se dégagent ?
2. **Variables** : quelles variables sont liées entre elles ? Peut-on résumer plusieurs variables corrélées par une seule dimension ?

En marketing, elle sert par exemple à résumer des dizaines d'indicateurs de comportement client en deux ou trois dimensions interprétables, à construire des **cartes perceptuelles** (positionnement des marques selon les notes données par des consommateurs), ou à préparer une segmentation (chapitre 6).

5.1.2 Principe mathématique

1. **Centrer-réduire** les variables : $z_{ij} = (x_{ij} - \bar{x}_j)/s_j$. C'est l'**ACP normée**, à utiliser dès que les variables ont des unités ou des ordres de grandeur différents (euros, jours, nombres de commandes).
2. Calculer la **matrice des corrélations** R ($p \times p$).
3. Diagonaliser R : ses **vecteurs propres** $u_1, \dots, u_p$ donnent les directions des axes, ses **valeurs propres** $\lambda_1 \geq \dots \geq \lambda_p$ la variance portée par chaque axe.
4. Coordonnées des individus sur l'axe k : $F_k = Zu_k$.

En ACP normée, la variance totale (l'**inertie**) vaut p, et la part d'inertie expliquée par l'axe k est λ_k/p.

```
# L'ACP « à la main » sur trois variables de mtcars
Z <- scale(mtcars[, c("mpg", "hp", "wt")])   # centrer-réduire
R <- cor(Z)
decomposition <- eigen(R)
decomposition$values                          # valeurs propres ; leur somme
 ↪  vaut 3
```

```
#> [1] 2.5382 0.3521 0.1097
```

```
decomposition$values / sum(decomposition$values)   # part d'inertie
```

```
#> [1] 0.84606 0.11736 0.03657
```

5.1.3 Exemple 1 : le décathlon

Le jeu `decathlon` (package `FactoMineR`) est l'exemple de référence de l'ACP : les performances de 41 athlètes aux dix épreuves du décathlon, lors de deux compétitions (Décastar et Jeux olympiques de 2004), ainsi que leur classement et leur nombre de points.

```
data(decathlon)
head(decathlon[, 1:6])
```

```
#>           100m Long.jump Shot.put High.jump  400m 110m.hurdle
#> SEBRLE   11.04      7.58    14.83      2.07 49.81       14.69
#> CLAY     10.76      7.40    14.26      1.86 49.37       14.05
#> KARPOV   11.02      7.30    14.77      2.04 48.37       14.09
#> BERNARD  11.02      7.23    14.25      1.92 48.93       14.99
#> YURKOV   11.34      7.09    15.19      2.10 50.42       15.31
#> WARNERS  11.11      7.60    14.31      1.98 48.68       14.23
```

Les dix épreuves sont les variables **actives** : elles construisent les axes. `Rank` et `Points` sont des variables **quantitatives illustratives**, `Competition` une variable **qualitative illustrative** : elles ne participent pas au calcul des axes, mais aident à les interpréter.

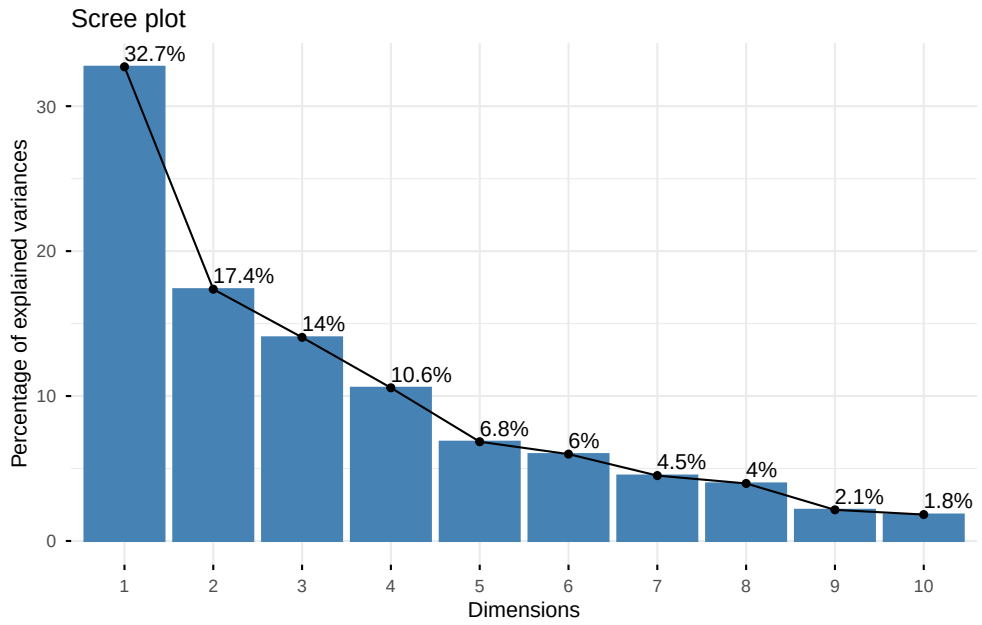
```
res_acp <- PCA(decathlon,
               quanti.sup = 11:12,    # Rank, Points
               quali.sup  = 13,       # Competition
               scale.unit = TRUE,     # ACP normée
               graph = FALSE)
```

5.1.3.1 Combien d'axes retenir ?

```
res_acp$eig[1:5, ]
```

```
#>        eigenvalue percentage of variance
#> comp 1     3.2719                 32.719
#> comp 2     1.7371                 17.371
#> comp 3     1.4049                 14.049
#> comp 4     1.0569                 10.569
#> comp 5     0.6848                  6.848
#>        cumulative percentage of variance
#> comp 1                             32.72
#> comp 2                             50.09
#> comp 3                             64.14
#> comp 4                             74.71
#> comp 5                             81.56
```

```
fviz_eig(res_acp, addlabels = TRUE)
```



Critères usuels (à combiner, pas à appliquer mécaniquement) :

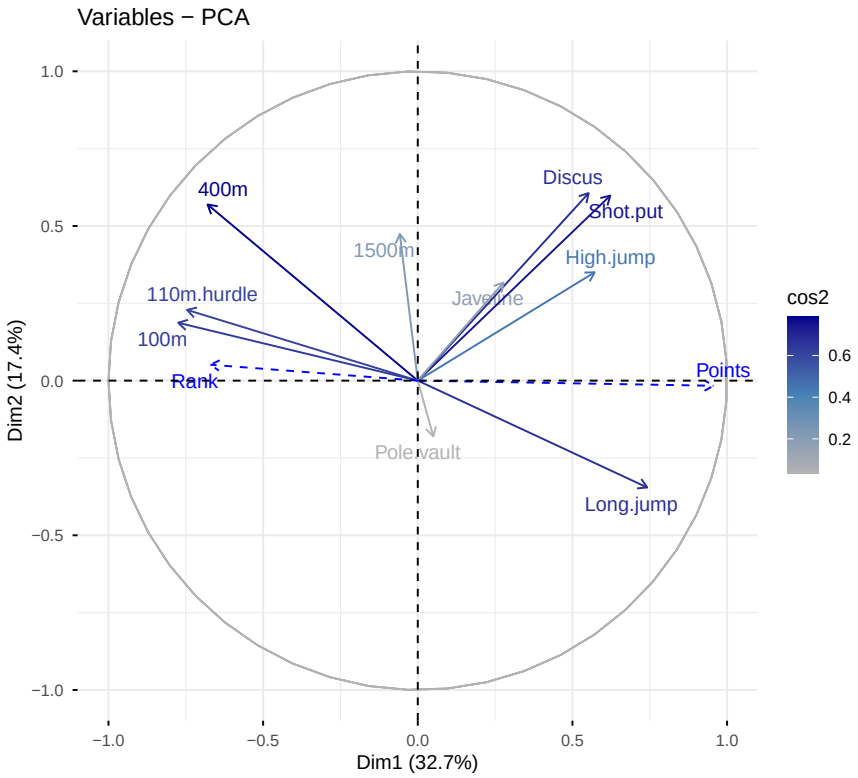
- le **coude** de l'éboulis des valeurs propres ;
- la **règle de Kaiser** (ACP normée) : garder les axes de valeur propre supérieure à 1, c'est-à-dire ceux qui résument plus qu'une variable d'origine ;
- un **pourcentage d'inertie cumulé** suffisant ;
- surtout, l'**interprétabilité** des axes.

Ici, les deux premiers axes expliquent environ 50 % de l'inertie. C'est modeste, ce qui est fréquent quand les variables sont nombreuses et peu corrélées.

5.1.3.2 Le cercle des corrélations

Chaque variable est représentée par une flèche dont les coordonnées sont ses **corrélations** avec les deux axes.

```
fviz_pca_var(res_acp, repel = TRUE, col.var = "cos2",
             gradient.cols = c("grey70", "steelblue", "darkblue"))
```



Règles de lecture

- Une flèche **proche du cercle** : variable bien représentée dans le plan. Une flèche courte est mal représentée ; on ne l'interprète pas.
- Deux flèches **proches** : variables corrélées positivement.
- Deux flèches **opposées** : corrélées négativement.
- Deux flèches **orthogonales** : non corrélées (si elles sont bien représentées).

Interprétation : le premier axe oppose les épreuves de **vitesse** (100 m, 400 m, 110 m haies, dont les temps faibles sont bons) au saut en longueur. Comme un bon sprinteur a un **temps faible**, les variables de course sont à l'opposé du saut en longueur, alors qu'il s'agit des mêmes qualités athlétiques. Le deuxième axe est porté par les **lancers** (poids, disque) mais aussi par le 400 m et le 1 500 m : il distingue les athlètes puissants mais plus lents sur les courses longues (en haut) des athlètes plus légers et endurants (en bas).

La variable illustrative `Points` est très corrélée au premier axe : celui-ci mesure donc la **performance globale**.

```
round(res_acp$var$contrib[, 1:2], 1)     # contribution (%) de chaque variable
 ↪  aux axes
```

```
#>             Dim.1 Dim.2
#> 100m         18.3   2.0
#> Long.jump    16.8   6.9
#> Shot.put     11.8  20.6
```

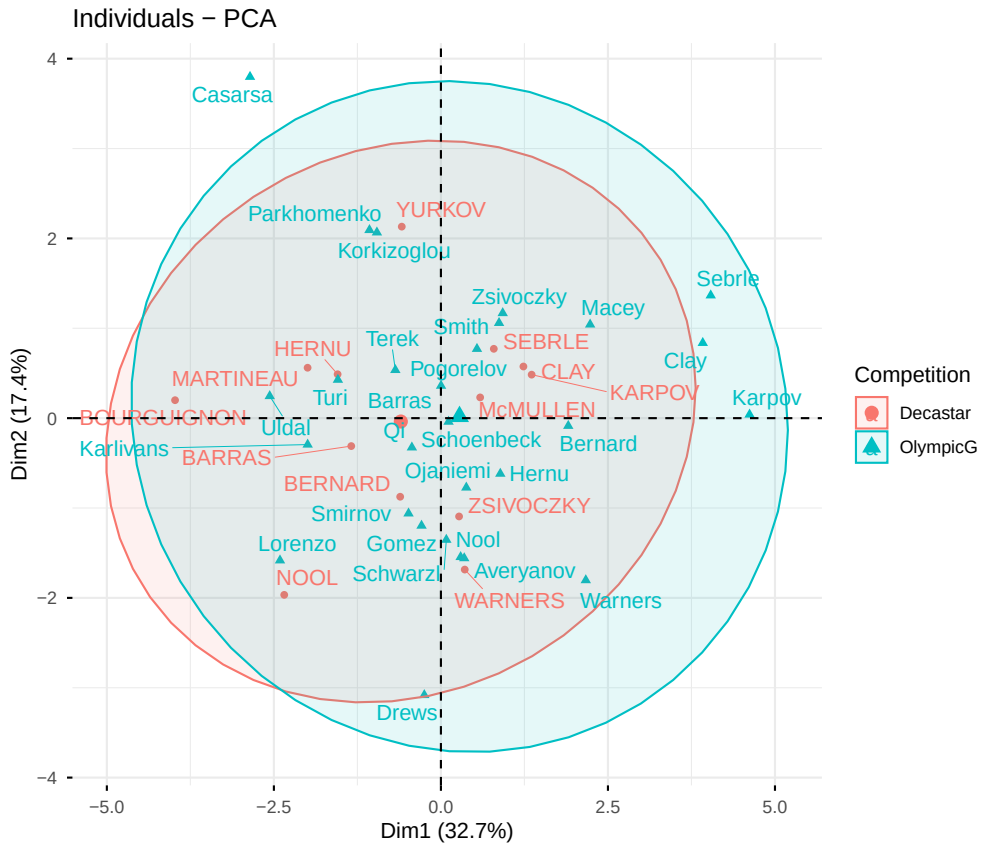
```
#> High.jump    10.0   7.1
#> 400m         14.1  18.7
#> 110m.hurdle  17.0   3.0
#> Discus        9.3  21.2
#> Pole.vault    0.1   1.9
#> Javeline      2.3   5.8
#> 1500m         0.1  12.9
```

```
round(res_acp$var$cos2[, 1:2], 2)        # qualité de représentation
```

```
#>             Dim.1 Dim.2
#> 100m         0.60  0.04
#> Long.jump    0.55  0.12
#> Shot.put     0.39  0.36
#> High.jump    0.33  0.12
#> 400m         0.46  0.32
#> 110m.hurdle  0.56  0.05
#> Discus       0.31  0.37
#> Pole.vault   0.00  0.03
#> Javeline     0.08  0.10
#> 1500m        0.00  0.22
```

5.1.3.3 Projection des individus

```
fviz_pca_ind(res_acp, repel = TRUE,
             habillage = "Competition",    # couleur selon la variable
              ↪  qualitative illustrative
             addEllipses = TRUE)
```



- Deux individus **proches** ont des profils semblables (s'ils sont bien représentés : `cos2` élevé).

- La position d'un individu s'interprète grâce au cercle des corrélations : un athlète à droite du plan est du côté des fortes valeurs de `Points` et de saut en longueur, donc des temps de course faibles.
- Les **contributions** repèrent les individus qui pèsent le plus dans la construction d'un axe ; un individu au profil extrême peut à lui seul déterminer un axe.

5.1.3.4 Caractériser les axes avec les variables illustratives

La variable qualitative `Competition` est projetée au **barycentre** des individus de chaque modalité. La fonction `dimdesc()` décrit automatiquement chaque axe par les variables (actives et illustratives) qui lui sont le plus liées, avec un test de significativité :

```
description <- dimdesc(res_acp, axes = 1:2)
description$Dim.1$quanti      # corrélations des variables quantitatives avec
 ↪  l'axe 1
```

```
#>             correlation   p.value
#> Points           0.9562 2.099e-22
#> Long.jump        0.7419 2.850e-08
#> Shot.put         0.6225 1.388e-05
#> High.jump        0.5719 9.362e-05
#> Discus           0.5525 1.802e-04
#> Rank            -0.6705 1.616e-06
#> 400m            -0.6796 1.028e-06
#> 110m.hurdle     -0.7462 2.137e-08
#> 100m            -0.7747 2.778e-09
```

```
round(res_acp$quali.sup$v.test[, 1:2], 2)   # position des modalités, en
 ↪  valeur-test
```

```
#>          Dim.1 Dim.2
#> Decastar -1.43 -0.12
#> OlympicG  1.43  0.12
```

La **valeur-test** (*v.test*) mesure l'écart entre la position moyenne d'une modalité et le centre du nuage ; au-delà de 2 en valeur absolue, l'écart est significatif (au seuil de 5 % environ). Les athlètes des Jeux olympiques sont plutôt du côté des bonnes performances sur l'axe 1, mais l'écart n'est **pas significatif** (v.test d'environ 1,4) : avec 41 athlètes, on ne peut pas conclure que les performances diffèrent selon la compétition.

5.1.4 Exemple 2 : le comportement des clients de la boutique

Appliquons la même démarche aux clients de la boutique en ligne. La fonction `indicateurs_clients()` (chapitre 3) calcule pour chaque client actif en 2025 ses indicateurs de comportement :

Variable	Description
`recence`	jours depuis la dernière commande
`frequence`	nombre de commandes en 2025
`montant_total`	chiffre d'affaires 2025 (€)
`panier_moyen`	montant moyen d'une commande (€)
`part_promo`	part des commandes passées avec un code promo
`nb_categories`	nombre de catégories de produits différentes achetées
`articles_par_commande`	nombre moyen d'articles par commande

Variable	Description
`anciennete`	années depuis l'inscription

```
indicateurs <- indicateurs_clients(lire_boutique("commandes"),
 ↪  lire_boutique("clients"))

clients_acp <- indicateurs |>
  drop_na(age) |>                       # on écarte les clients sans âge
   ↪  renseigné
  select(client_id, recence, frequence, montant_total, panier_moyen,
   ↪  part_promo,
         nb_categories, articles_par_commande, anciennete,     # variables
 ↪  actives
         age, canal_acquisition, newsletter) |>                # variables
 ↪  illustratives
  column_to_rownames("client_id")
dim(clients_acp)
```

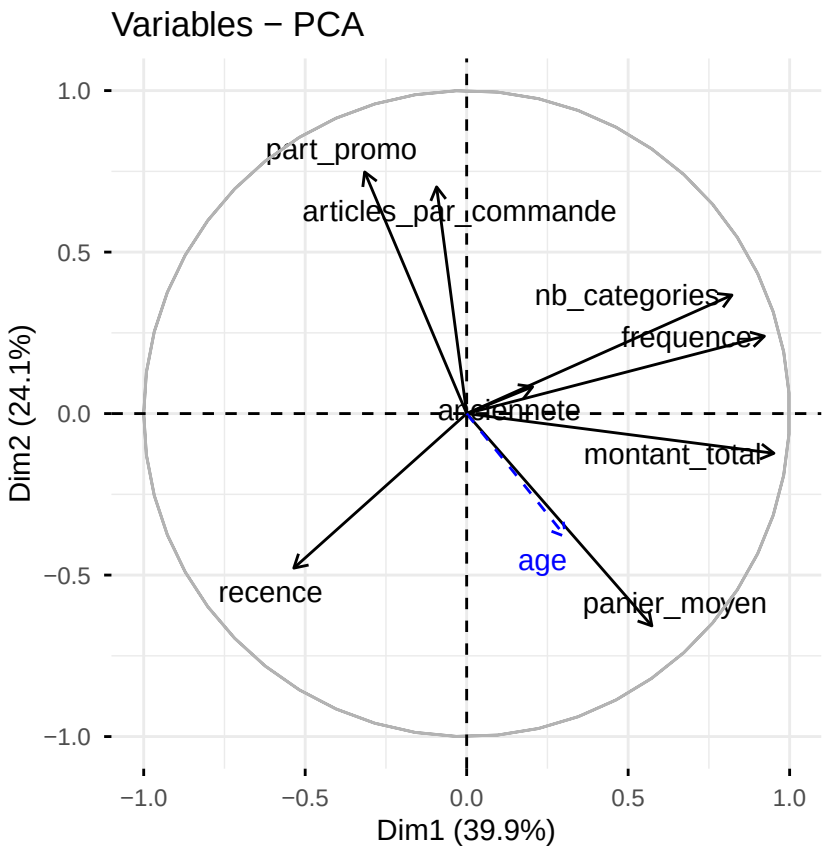
```
#> [1] 1688   11
```

Les huit indicateurs de comportement sont **actifs**. L'âge (quantitatif), le canal d'acquisition et l'abonnement à la newsletter (qualitatifs) sont **illustratifs** : on veut savoir s'ils sont liés aux profils de comportement, sans qu'ils construisent les axes.

```
acp_clients <- PCA(clients_acp, quanti.sup = 9, quali.sup = 10:11, graph =
 ↪  FALSE)
round(acp_clients$eig[1:4, ], 1)
```

```
#>        eigenvalue percentage of variance
#> comp 1        3.2                   39.9
#> comp 2        1.9                   24.1
#> comp 3        1.1                   13.3
#> comp 4        0.7                    8.3
#>        cumulative percentage of variance
#> comp 1                              39.9
#> comp 2                              64.0
#> comp 3                              77.3
#> comp 4                              85.6
```

```
fviz_pca_var(acp_clients, repel = TRUE)
```



Les deux premiers axes résument environ 64 % de l'inertie, et se lisent clairement :

- **Axe 1 — la valeur du client** : fréquence, montant total et nombre de catégories achetées, opposés à la récence (un client récent a une récence faible). À droite, les bons clients, actifs et fidèles ; à gauche, les clients peu actifs.
- **Axe 2 — le comportement promotionnel** : part des commandes en promotion et nombre d'articles par commande, opposés au panier moyen. En haut, des clients qui achètent beaucoup d'articles bon marché en promotion ; en bas, des clients aux paniers élevés, peu sensibles aux promotions.

L'ancienneté est mal représentée dans ce plan : elle porte à elle seule le troisième axe.

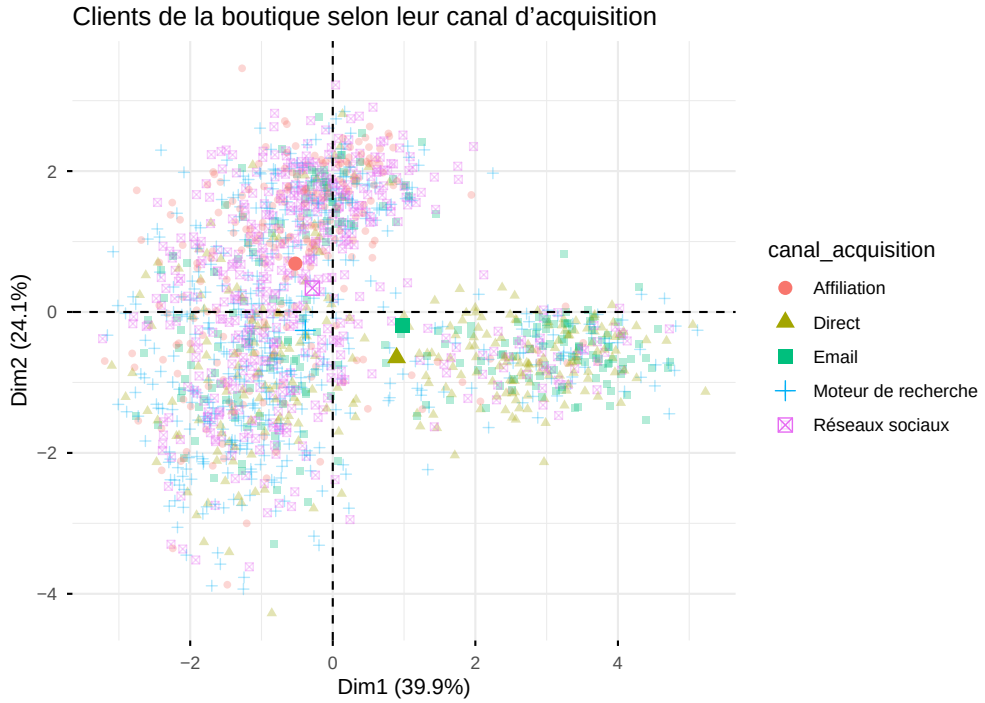
```
round(acp_clients$quanti.sup$coord[, 1:2], 2)     # âge
```

```
#> Dim.1 Dim.2
#>  0.31 -0.38
```

```
round(acp_clients$quali.sup$v.test[, 1:2], 1)      # canal et newsletter
```

```
#>                     Dim.1 Dim.2
#> Affiliation          -4.9   8.3
#> Direct                8.8  -8.2
#> Email                 8.7  -2.1
#> Moteur de recherche  -5.5  -4.8
#> Réseaux sociaux      -4.2   6.4
#> Non                  -9.0  -6.2
#> Oui                   9.0   6.2
```

```
fviz_pca_ind(acp_clients, geom = "point", habillage = "canal_acquisition",
             alpha.ind = 0.3, addEllipses = FALSE, mean.point = TRUE) +
  labs(title = "Clients de la boutique selon leur canal d'acquisition")
```



Les variables illustratives enrichissent l'interprétation :

- les clients venus en **direct** ou par **email** sont du côté des clients à forte valeur et peu sensibles aux promotions ;
- ceux venus par **affiliation** et **réseaux sociaux** sont du côté des promotions ;
- les **abonnés à la newsletter** sont plus engagés (axe 1) ;
- les clients **plus âgés** ont des paniers plus élevés et utilisent moins les promotions (corrélation négative avec l'axe 2).

Sur l'axe 1, toutes les valeurs-test des modalités dépassent 4 en valeur absolue : ces écarts ne sont pas dus au hasard.

Ces liens sont des **associations**. On ne peut pas en conclure que recruter par email « crée » de bons clients : les canaux attirent des publics différents. Rappelons aussi que ces données sont **simulées** ; sur des données réelles, les variables très asymétriques (montants) sont souvent transformées (logarithme) avant l'ACP, car quelques clients extrêmes peuvent sinon déterminer un axe à eux seuls.

Démarche d'une ACP

1. Choisir les variables actives (quantitatives) et illustratives.
2. ACP normée si les unités diffèrent.
3. Choisir le nombre d'axes (éboulis, Kaiser, interprétabilité).
4. Interpréter les axes avec le cercle des corrélations et les contributions.
5. Interpréter le plan des individus **à la lumière** du cercle.
6. Valider et enrichir l'interprétation avec les variables illustratives (`dimdesc()`, valeurs-test).
7. N'interpréter que les éléments bien représentés (`cos2`).

Pour explorer une ACP de façon interactive, le package `Factoshiny` propose une interface graphique : `Factoshiny::PCAshiny(decathlon)`.

5.2 Arbres de décision

5.2.1 Principe

Un arbre de décision prédit une variable cible en posant une **suite de questions binaires** sur les variables explicatives (« la fidélité du client à la marque est-elle supérieure à 0,48 ? »). C'est une méthode **supervisée** : on connaît la réponse sur les données d'apprentissage.

- **Classification** : la cible est qualitative (achète / n'achète pas) ; chaque feuille prédit la classe majoritaire.
- **Régression** : la cible est quantitative (montant) ; chaque feuille prédit la moyenne.

Construction (algorithme CART) : on part de tous les individus. À chaque nœud, on cherche la variable et le seuil qui séparent le mieux les individus en deux groupes **les plus homogènes possible** vis-à-vis de la cible. On répète dans chaque groupe (*partitionnement récursif*).

L'homogénéité d'un nœud se mesure par son **impureté**. Pour des proportions de classes $p_1, \dots, p_K$:

- indice de **Gini** : $G = 1 - \sum_k p_k^2$;
- **entropie** : $H = -\sum_k p_k \log p_k$.

Un nœud pur (une seule classe) a une impureté nulle. On choisit la coupure qui fait le plus baisser l'impureté.

Points forts : très lisible (on peut l'expliquer à une équipe commerciale), gère variables quantitatives et qualitatives, peu sensible aux échelles, détecte des interactions. **Points faibles** : instable (un petit changement des données peut changer tout l'arbre), tendance au **sur-apprentissage** si l'arbre est trop profond.

5.2.2 Les données : le choix d'une marque de jus d'orange

On reprend le jeu `OJ` (chapitre 4) : 1 070 achats de Citrus Hill (`CH`) ou de Minute Maid (`MM`), avec prix, remises, mises en avant et fidélité du client (`LoyalCH`). Objectif : prédire la marque achetée, par exemple pour **cibler** les clients à qui envoyer un bon de réduction MM.

On réserve un jeu de **test** dès le départ :

```
set.seed(2026)
indices_train <- sample(nrow(OJ), 800)
train <- OJ[indices_train, ]
test  <- OJ[-indices_train, ]
table(train$Purchase)
```

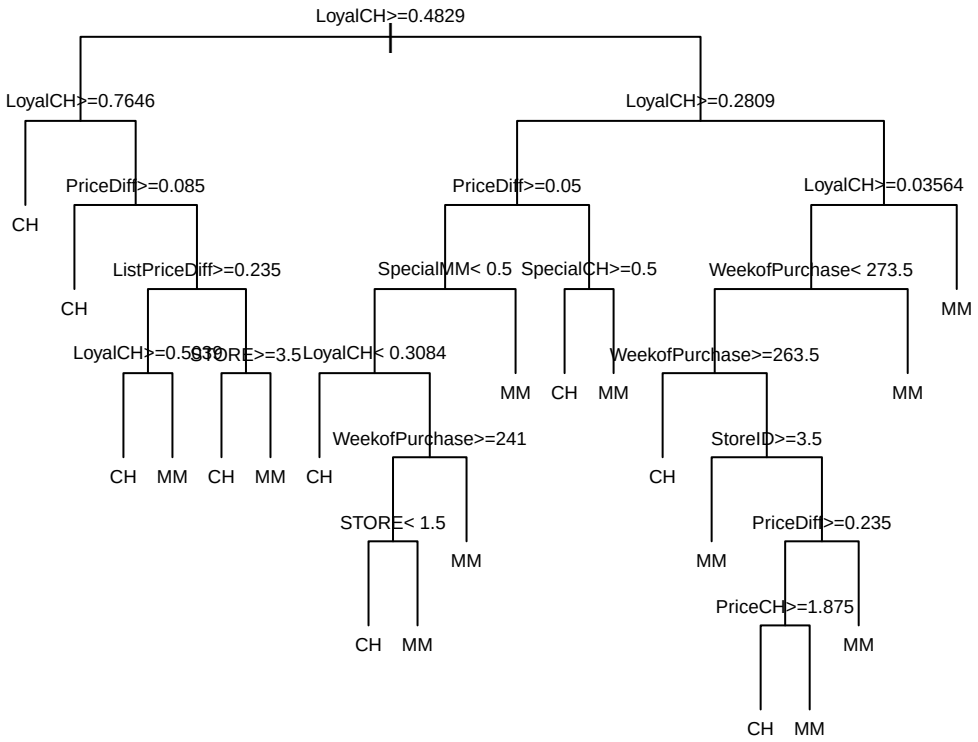
```
#>
#>  CH  MM
#> 485 315
```

5.2.3 Construire l'arbre

```
set.seed(2026)   # rpart utilise une validation croisée aléatoire
arbre <- rpart(Purchase ~ ., data = train, method = "class",
               control = rpart.control(cp = 0.001))
```

La formule `Purchase ~ .` signifie « `Purchase` expliqué par toutes les autres variables ». Avec un paramètre de complexité `cp` très faible, l'arbre est volontairement très détaillé :

```
plot(arbre, uniform = TRUE, margin = 0.05)
text(arbre, cex = 0.6)
```



Cet arbre apprend le bruit des données d'entraînement : il faut l'**élaguer**.

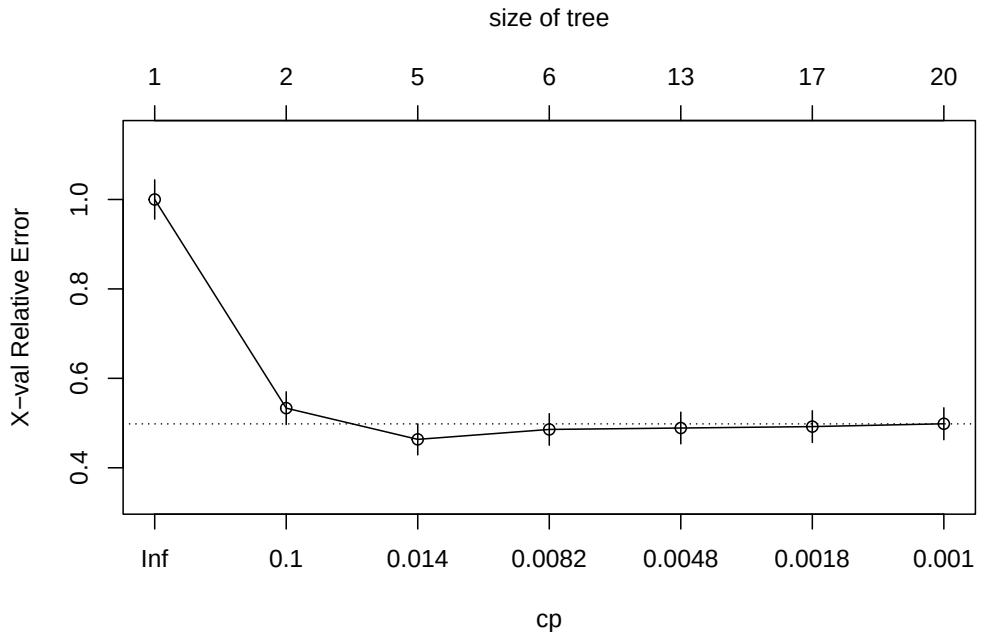
5.2.4 Élaguer l'arbre

Le **paramètre de complexité** `cp` pénalise les arbres trop grands. `rpart` estime l'erreur de chaque taille d'arbre par **validation croisée** (colonne `xerror`).

```
printcp(arbre)
```

```
#>
#> Classification tree:
#> rpart(formula = Purchase ~ ., data = train, method = "class",
#>     control = rpart.control(cp = 0.001))
#>
#> Variables actually used in tree construction:
#> [1] ListPriceDiff  LoyalCH        PriceCH        PriceDiff
#> [5] SpecialCH      SpecialMM      STORE          StoreID
#> [9] WeekofPurchase
#>
#> Root node error: 315/800 = 0.39
#>
#> n= 800
#>
#>       CP nsplit rel error xerror  xstd
#> 1 0.5175      0      1.00   1.00 0.044
#> 2 0.0201      1      0.48   0.53 0.037
#> 3 0.0095      4      0.42   0.46 0.035
#> 4 0.0071      5      0.41   0.49 0.035
#> 5 0.0032     12      0.36   0.49 0.035
#> 6 0.0011     16      0.34   0.49 0.035
#> 7 0.0010     19      0.34   0.50 0.036
```

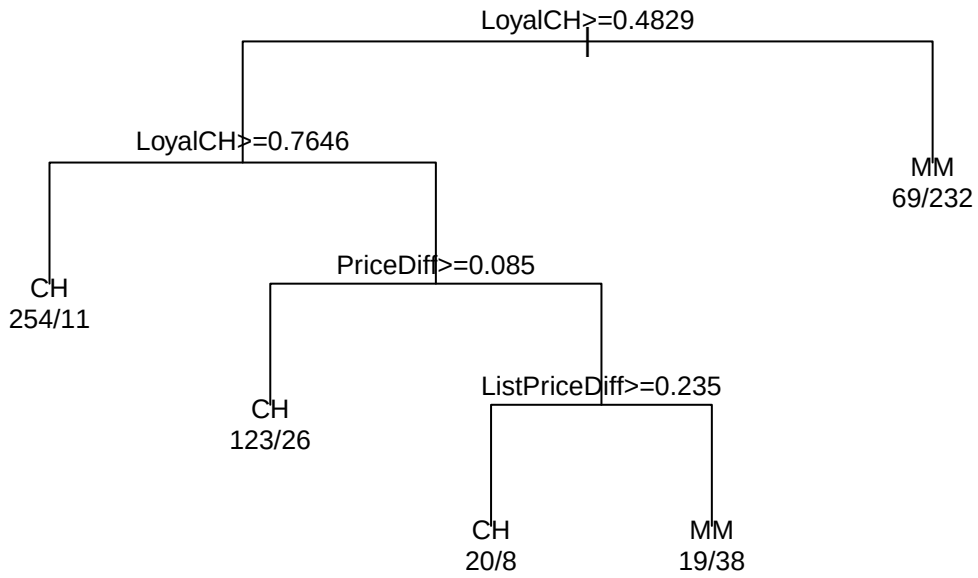
```
plotcp(arbre)
```



Règle usuelle : choisir le `cp` qui minimise `xerror`, ou le plus grand `cp` dont `xerror` reste à moins d'un écart-type du minimum (règle « 1-SE »), ce qui donne un arbre plus simple.

```
table_cp <- arbre$cptable
cp_optimal <- table_cp[which.min(table_cp[, "xerror"]), "CP"]
arbre_elague <- prune(arbre, cp = cp_optimal)

plot(arbre_elague, uniform = TRUE, margin = 0.08)
text(arbre_elague, use.n = TRUE, cex = 0.8, digits = 4)   # digits : affichage
 ↪  des effectifs
```



Lecture : chaque nœud indique une condition ; on va à **gauche** si elle est vraie. Chaque feuille affiche la classe prédite et les effectifs `CH/MM` d'entraînement. Le premier critère est la **fidélité** à Citrus Hill : les clients peu fidèles à CH achètent majoritairement MM. Pour les clients à fidélité moyenne, c'est la **différence de prix** qui fait basculer le choix. L'arbre se traduit directement

en règles commerciales.

```
round(arbre_elague$variable.importance, 1)
```

```
#>        LoyalCH    SalePriceMM      PriceDiff        StoreID
#>          154.8           14.9           14.6           12.8
#>        PriceMM  ListPriceDiff         DiscMM      PctDiscMM
#>          11.2           11.1           10.6           10.1
#> WeekofPurchase        PriceCH    SalePriceCH      SpecialCH
#>            5.6            4.7            1.9            1.4
#>         Store7
#>            1.2
```

5.2.5 Prédire

```
classe_predite <- predict(arbre_elague, newdata = test, type = "class")
proba_mm <- predict(arbre_elague, newdata = test, type = "prob")[, "MM"]
head(data.frame(observe = test$Purchase, predit = classe_predite,
                proba_mm = round(proba_mm, 2)))
```

```
#>    observe predit proba_mm
#> 1       CH     CH     0.17
#> 11      CH     CH     0.04
#> 13      CH     CH     0.04
#> 16      CH     CH     0.04
#> 22      CH     CH     0.17
#> 24      CH     CH     0.04
```

5.2.6 Matrice de confusion

On compare les classes prédites aux classes observées sur le jeu de **test**.

```
confusion <- table(observe = test$Purchase, predit = classe_predite)
confusion
```

```
#>        predit
#> observe  CH  MM
#>      CH 136  32
#>      MM  13  89
```

On appelle « positif » la classe d'intérêt, ici l'achat de `MM`.

	Prédit `CH` (négatif)	Prédit `MM` (positif)
Observé `CH`	vrais négatifs (VN)	faux positifs (FP)
Observé `MM`	faux négatifs (FN)	vrais positifs (VP)

Indicateur	Formule	Question
Taux de bonne classification (*accuracy*)	(VP + VN) / total	Quelle part est bien classée ?
Sensibilité (rappel)	VP / (VP + FN)	Parmi les acheteurs de MM, combien sont détectés ?
Spécificité	VN / (VN + FP)	Parmi les acheteurs de CH, combien sont reconnus ?

Indicateur	Formule	Question
Précision	VP / (VP + FP)	Parmi les clients ciblés, combien achètent vraiment MM ?

```
evaluer <- function(confusion, positif) {
  negatif <- setdiff(colnames(confusion), positif)
  vp <- confusion[positif, positif]; fn <- confusion[positif, negatif]
  vn <- confusion[negatif, negatif]; fp <- confusion[negatif, positif]
  c(accuracy    = (vp + vn) / sum(confusion),
    sensibilite = vp / (vp + fn),
    specificite = vn / (vn + fp),
    precision   = vp / (vp + fp))
}
round(evaluer(confusion, positif = "MM"), 3)
```

```
#>    accuracy sensibilite specificite   precision
#>       0.833       0.873       0.810       0.736
```

En marketing, le choix de l'indicateur dépend du **coût des erreurs** : envoyer un bon de réduction à un client qui n'en fera rien (faux positif) coûte le prix de l'envoi ; ne pas cibler un client réceptif (faux négatif) coûte une vente perdue. Une campagne bon marché (email) privilégie la sensibilité ; une campagne coûteuse (courrier, remise importante) privilégie la précision.

> Le taux de bonne classification est trompeur quand les classes sont **déséquilibrées**. Si seulement 2 % des clients répondent à une campagne, un modèle qui prédit « personne ne répond » a 98 % de bonnes classifications… et ne sert à rien. L'exercice 5.2 en donne un exemple.

5.2.7 Courbe ROC et AUC

Un classifieur produit une **probabilité** ; la classe dépend d'un **seuil** (0,5 par défaut). Baisser le seuil augmente la sensibilité mais diminue la spécificité. La **courbe ROC** trace la sensibilité en fonction de 1 − spécificité pour **tous les seuils** possibles.

- Un classifieur parfait passe par le coin supérieur gauche.
- Un classifieur aléatoire suit la diagonale.
- L'**AUC** (aire sous la courbe) résume la courbe : 0,5 pour le hasard, 1 pour un classifieur parfait. C'est la probabilité qu'un individu positif tiré au hasard reçoive un score plus élevé qu'un individu négatif tiré au hasard.

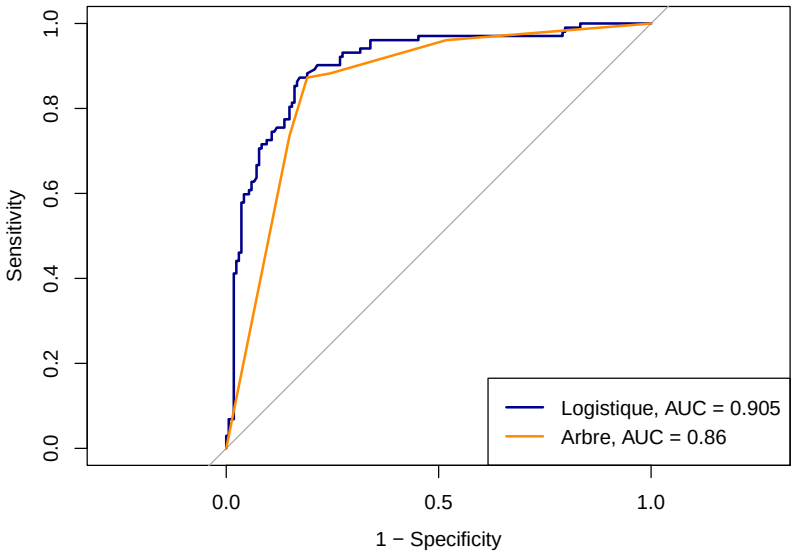
Comparons l'arbre à la régression logistique du chapitre 4 :

```
logistique <- glm(Purchase ~ PriceDiff + LoyalCH + SpecialMM + SpecialCH,
                  data = train, family = binomial)
proba_log <- predict(logistique, newdata = test, type = "response")

roc_arbre <- roc(test$Purchase, proba_mm, levels = c("CH", "MM"), direction =
 ↪  "<")
roc_log   <- roc(test$Purchase, proba_log, levels = c("CH", "MM"), direction =
 ↪  "<")

plot(roc_log, col = "darkblue", legacy.axes = TRUE)
```

```
plot(roc_arbre, col = "darkorange", add = TRUE)
legend("bottomright", col = c("darkblue", "darkorange"), lwd = 2,
       legend = c(paste("Logistique, AUC =", round(auc(roc_log), 3)),
                  paste("Arbre, AUC =", round(auc(roc_arbre), 3))))
```



La courbe de l'arbre est formée de segments : un arbre élagué n'a que quelques feuilles, donc quelques valeurs de probabilité possibles. Ici, la régression logistique discrimine un peu mieux ; l'arbre reste plus facile à expliquer. Ce compromis entre **performance** et **interprétabilité** est fréquent.

Choisir un seuil : la fonction `coords()` donne le seuil qui maximise sensibilité + spécificité (indice de Youden). En pratique, le seuil se choisit avec les équipes métier, en fonction du coût des erreurs et du budget de la campagne.

```
coords(roc_log, "best", ret = c("threshold", "sensitivity", "specificity"))
```

```
#>   threshold sensitivity specificity
#> 1     0.389      0.8725      0.8274
```

> Les indicateurs doivent être calculés sur un jeu de **test** jamais utilisé pour construire ni régler le modèle. Mesurés sur le jeu d'entraînement, ils sont trop optimistes (voir la fuite de données au chapitre 1).

5.3 Forêts aléatoires

5.3.1 Pourquoi aller au-delà d'un arbre ?

Un arbre de décision est lisible, mais **instable** : un léger changement des données d'apprentissage peut modifier la première coupure, et donc tout l'arbre. En termes statistiques, ses prédictions ont une **variance** élevée. Un arbre profond a peu de biais (il s'adapte finement aux données) mais beaucoup de variance ; l'élagage réduit la variance au prix d'un peu de biais.

Les **méthodes d'ensemble** combinent de nombreux modèles pour réduire cette variance. La **forêt aléatoire** (*random forest*, Leo Breiman, 2001) est la plus utilisée d'entre elles : c'est l'une des méthodes les plus performantes « clés en main » sur des données tabulaires, comme on en rencontre sans cesse en marketing (clients, transactions, campagnes).

5.3.2 Rappels théoriques

5.3.2.1 Le bagging

Le *bagging* (*bootstrap aggregating*, Breiman, 1996) procède en trois étapes :

1. tirer B échantillons **bootstrap** : chacun contient n observations tirées **avec remise** parmi les n observations d'apprentissage ;
2. construire un arbre **profond, non élagué** sur chaque échantillon : $\hat{f}_1, \dots, \hat{f}_B$;
3. **agréger** les B prédictions.

Pourquoi cela marche-t-il ? Si l'on moyenne B prédictions de même variance σ^2 et de corrélation deux à deux ρ, la variance de la moyenne vaut

$$\mathrm{Var}\left(\frac{1}{B}\sum_{b=1}^{B}\hat{f}_b(x)\right) = \rho\,\sigma^2 + \frac{1-\rho}{B}\,\sigma^2.$$

Le second terme disparaît quand B augmente : moyenner beaucoup d'arbres réduit la variance **sans augmenter le biais**. Mais le premier terme demeure : si les arbres se ressemblent trop (corrélation ρ forte), le gain est limité. Or, avec le bagging seul, les arbres se ressemblent : s'il existe une variable très prédictive (comme la fidélité `LoyalCH` du jeu `OJ`), presque tous les arbres la choisissent pour leur première coupure.

5.3.2.2 L'idée de la forêt aléatoire : décorréler les arbres

La forêt aléatoire ajoute une seconde source de hasard : **à chaque nœud** de chaque arbre, la meilleure coupure est cherchée parmi m variables **tirées au hasard** parmi les p variables explicatives (paramètre `mtry`), et non parmi toutes. Les variables moins dominantes ont ainsi leur chance, les arbres deviennent plus différents les uns des autres (ρ diminue), et la moyenne est plus efficace.

- Avec $m = p$, on retrouve le bagging.
- Valeurs par défaut de `randomForest` : $m = \lfloor\sqrt{p}\rfloor$ en classification, $m = \lfloor p/3\rfloor$ en régression.

5.3.2.3 Classification et régression

	Classification	Régression
Variable cible	qualitative (achète / n'achète pas)	quantitative (montant, ventes)
Critère de coupure dans chaque arbre	baisse de l'impureté de **Gini**	baisse de la **somme des carrés des résidus**
Taille minimale des feuilles (`nodesize`)	1 : arbres très profonds	5
`mtry` par défaut	$\lfloor\sqrt{p}\rfloor$	$\lfloor p/3\rfloor$
Prédiction de la forêt	**vote majoritaire** des B arbres ; la **probabilité** d'une classe est la part des arbres qui la prédisent	**moyenne** des prédictions des B arbres : $\hat{f}(x) = \frac{1}{B}\sum_b \hat{f}_b(x)$
Erreur mesurée	taux d'erreur, matrice de confusion, AUC	erreur quadratique moyenne (MSE, RMSE), part de variance expliquée

En **régression**, une forêt prédit une moyenne de valeurs observées dans les feuilles : elle **ne peut pas extrapoler**. Ses prédictions restent dans l'intervalle des valeurs de la cible vues à l'apprentissage, et elle prolonge mal une tendance (par exemple des ventes en croissance continue).

5.3.2.4 L'erreur *out-of-bag*

Un échantillon bootstrap ne contient pas toutes les observations : la probabilité qu'une observation donnée n'y figure pas est $(1 - 1/n)^n \approx e^{-1} \approx 0{,}37$. Environ un tiers des observations sont donc « hors du sac » (*out-of-bag*, OOB) pour chaque arbre.

Pour chaque observation, on peut agréger les prédictions des seuls arbres qui **ne l'ont pas vue** : on obtient l'**erreur OOB**, une estimation de l'erreur de prédiction comparable à une validation croisée, **sans jeu de validation séparé**. Elle sert notamment à régler `mtry`. Elle ne dispense pas de garder un jeu de **test** pour l'évaluation finale si l'on compare plusieurs méthodes.

5.3.2.5 Importance des variables

Une forêt de 500 arbres ne se lit pas comme un arbre unique. Deux mesures résument le rôle de chaque variable :

- **diminution moyenne de l'impureté** (`MeanDecreaseGini` en classification, `IncNodePurity` en régression) : total des baisses d'impureté obtenues grâce aux coupures sur cette variable, moyenné sur les arbres. Rapide à calculer, mais elle favorise les variables quantitatives ou à nombreuses modalités ;
- **importance par permutation** (`MeanDecreaseAccuracy` en classification, `%IncMSE` en régression) : on **mélange au hasard** les valeurs d'une variable dans les observations OOB et l'on mesure la **dégradation** de la prédiction. Si mélanger une variable ne change rien, elle n'est pas utile au modèle. Plus fiable, elle demande `importance = TRUE`.

L'importance mesure l'**utilité pour prédire**, pas un effet causal, et ne donne pas le **sens** de l'effet. Deux variables très corrélées se partagent l'importance : chacune peut paraître peu importante alors que l'information qu'elles portent l'est.

5.3.2.6 Paramètres à régler

Paramètre	Rôle	Conseil
`ntree`	nombre d'arbres B	500 par défaut ; augmenter ne provoque **pas** de sur-apprentissage, seulement un calcul plus long : vérifier que l'erreur OOB s'est stabilisée (`plot()`)
`mtry`	variables candidates à chaque nœud	le paramètre principal ; à régler avec l'erreur OOB
`nodesize`, `maxnodes`	taille des feuilles, nombre de feuilles	profondeur des arbres ; les valeurs par défaut conviennent souvent
`sampsize`, `strata`, `classwt`	taille et composition des échantillons bootstrap	utiles quand les classes sont **déséquilibrées** (exercice 5.5)

Points forts : très bonnes performances avec peu de réglages, variables quantitatives et qualitatives sans transformation, interactions et effets non linéaires détectés automatiquement, estimation OOB gratuite. **Points faibles** : modèle peu interprétable (une « boîte noire » qu'on explore avec l'importance des variables et les graphiques de dépendance partielle), calcul plus lourd qu'un arbre, pas d'extrapolation en régression, pas de valeurs manquantes acceptées par `randomForest` (les traiter avant, par exemple avec `na.roughfix()`). Pour de grands volumes de données, le package `ranger` est beaucoup plus rapide.

5.3.3 Classification : le choix de la marque de jus d'orange

On reprend les données `OJ` et le **même découpage** entraînement / test que pour l'arbre (`train` : 800 achats, `test` : 270 achats), afin de comparer les méthodes sur les mêmes observations.

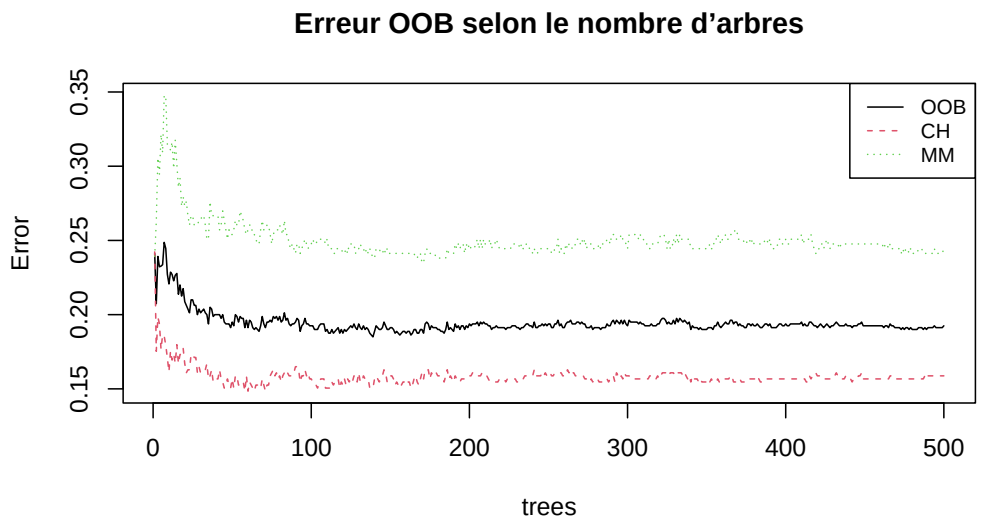
```
set.seed(2026)
foret <- randomForest(Purchase ~ ., data = train, ntree = 500, importance =
 ↪  TRUE)
foret
```

```
#>
#> Call:
#>  randomForest(formula = Purchase ~ ., data = train, ntree = 500,      importance =
 ↪  TRUE)
#>                Type of random forest: classification
#>                      Number of trees: 500
#> No. of variables tried at each split: 4
#>
#>         OOB estimate of  error rate: 19.25%
#> Confusion matrix:
#>     CH  MM class.error
#> CH 408  77      0.1588
#> MM  77 238      0.2444
```

La sortie indique le type de forêt (classification), le nombre d'arbres, la valeur de `mtry` (ici $\lfloor\sqrt{17}\rfloor = 4$), l'**erreur OOB** et la matrice de confusion OOB, calculées sur les données d'apprentissage sans jeu de validation.

L'erreur s'est-elle stabilisée ? `plot()` trace l'erreur OOB (et celle de chaque classe) en fonction du nombre d'arbres :

```
plot(foret, main = "Erreur OOB selon le nombre d'arbres")
legend("topright", colnames(foret$err.rate), col = 1:3, lty = 1:3, cex = 0.8)
```



Au-delà de 150 à 200 arbres, l'erreur ne fait plus qu'osciller légèrement : 500 arbres suffisent largement.

Régler `mtry` avec l'erreur OOB :

```
erreur_oob <- sapply(1:10, function(m) {
  set.seed(2026)
  modele <- randomForest(Purchase ~ ., data = train, mtry = m)
  modele$err.rate[modele$ntree, "OOB"]
})
round(setNames(erreur_oob, 1:10), 3)
```

```
#>     1     2     3     4     5     6     7     8     9    10
#> 0.229 0.202 0.189 0.189 0.198 0.194 0.199 0.201 0.208 0.202
```

L'erreur est la plus faible autour de $m = 3$ ou 4 ; la valeur par défaut convient. Avec $m = 1$, les coupures sont presque aléatoires ; avec m grand, on se rapproche du bagging.

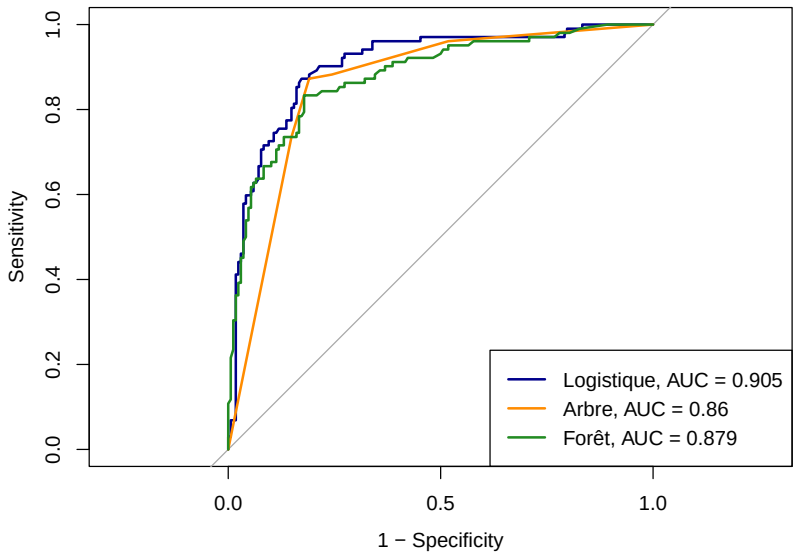
Évaluer sur le jeu de test et comparer à l'arbre élagué et à la régression logistique :

```
table(observe = test$Purchase, predit = predict(foret, newdata = test))
```

```
#>        predit
#> observe  CH  MM
#>      CH 138  30
#>      MM  20  82
```

```
proba_foret <- predict(foret, newdata = test, type = "prob")[, "MM"]
roc_foret <- roc(test$Purchase, proba_foret, levels = c("CH", "MM"), direction
 ↪  = "<")

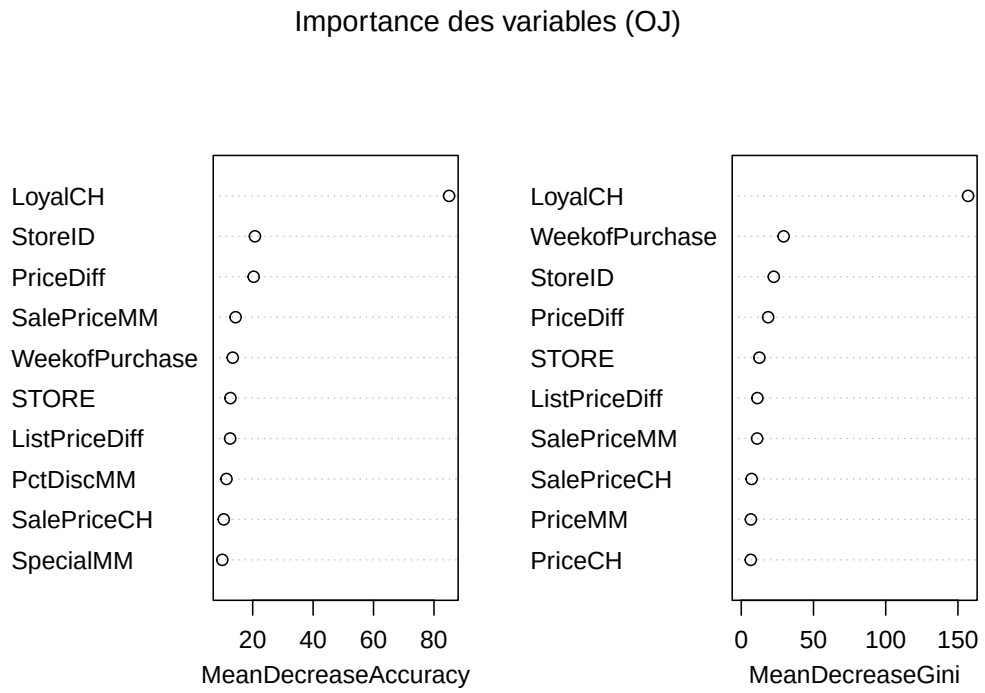
plot(roc_log, col = "darkblue", legacy.axes = TRUE)
plot(roc_arbre, col = "darkorange", add = TRUE)
plot(roc_foret, col = "forestgreen", add = TRUE)
legend("bottomright", col = c("darkblue", "darkorange", "forestgreen"), lwd =
 ↪  2,
       legend = c(paste("Logistique, AUC =", round(auc(roc_log), 3)),
                  paste("Arbre, AUC =", round(auc(roc_arbre), 3)),
                  paste("Forêt, AUC =", round(auc(roc_foret), 3))))
```



La forêt fait mieux que l'arbre unique : agréger 500 arbres stabilise les prédictions et donne des probabilités plus fines. Elle ne dépasse pourtant pas la régression logistique : dans ces données, l'essentiel de l'information tient dans deux variables (`LoyalCH` et `PriceDiff`) dont l'effet est presque linéaire sur l'échelle du logit, situation idéale pour un modèle logistique.

Importance des variables :

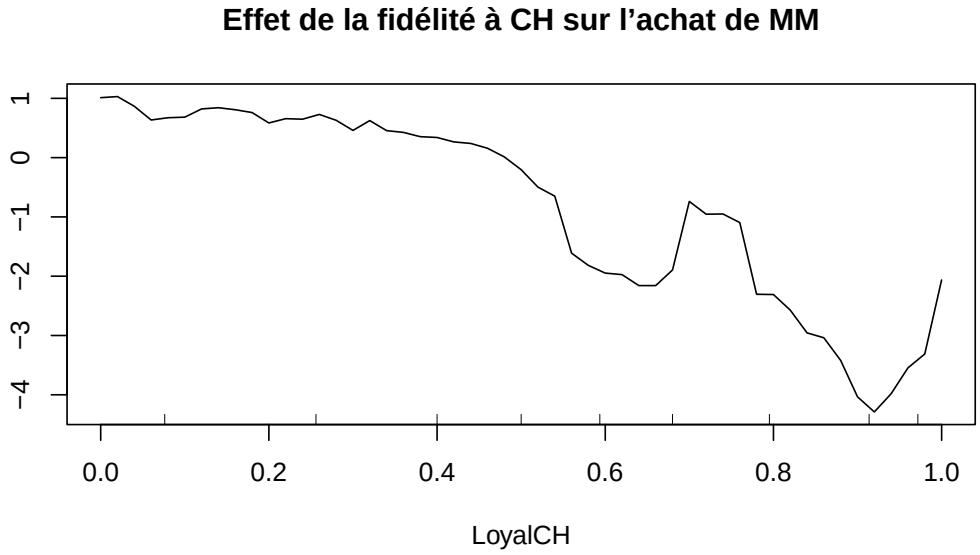
```
varImpPlot(foret, n.var = 10, main = "Importance des variables (OJ)")
```



La fidélité à la marque domine nettement, devant le magasin et les écarts de prix : on retrouve, sous une autre forme, les conclusions de l'arbre et de la régression logistique.

Dépendance partielle. Pour voir **comment** une variable influence la prédiction, `partialPlot()` trace la prédiction moyenne de la forêt quand on fait varier cette variable, les autres restant à leurs valeurs observées (l'axe vertical est sur l'échelle du logit de la probabilité de la classe choisie) :

```
partialPlot(foret, pred.data = as.data.frame(train), x.var = "LoyalCH",
            which.class = "MM", main = "Effet de la fidélité à CH sur l'achat
             ↪  de MM",
            xlab = "LoyalCH")
```



Plus un client est fidèle à Citrus Hill, moins la forêt prédit l'achat de Minute Maid, avec une chute marquée autour de 0,5. Les irrégularités de la courbe (bosse vers 0,7, remontée près de 1) ne doivent pas être surinterprétées : la forêt est une moyenne de fonctions en escalier, et certaines valeurs de `LoyalCH` sont rares dans les données. On lit la **tendance générale**, pas chaque variation.

5.3.4 Régression : les ventes de sièges auto

On reprend le jeu `Carseats` (chapitre 4) pour prédire les ventes `Sales`, avec le même découpage 70 % / 30 % qu'au chapitre 4.

```
set.seed(2026)
indices_train <- sample(nrow(Carseats), size = round(0.7 * nrow(Carseats)))
ventes_train <- Carseats[indices_train, ]
ventes_test  <- Carseats[-indices_train, ]

set.seed(2026)
foret_ventes <- randomForest(Sales ~ ., data = ventes_train, importance = TRUE)
foret_ventes
```

```
#>
#> Call:
#>  randomForest(formula = Sales ~ ., data = ventes_train, importance = TRUE)
#>                Type of random forest: regression
#>                      Number of trees: 500
#> No. of variables tried at each split: 3
#>
#>           Mean of squared residuals: 2.673
#>                     % Var explained: 64.83
```

En régression, la sortie donne l'erreur quadratique moyenne OOB (*Mean of squared residuals*) et la part de variance expliquée, l'équivalent d'un R^2 calculé hors échantillon. Ici, `mtry` vaut $\lfloor 10/3 \rfloor = 3$.

Comparons sur le jeu de test la référence naïve (la moyenne), un arbre de régression, la forêt et le modèle linéaire multiple du chapitre 4 :

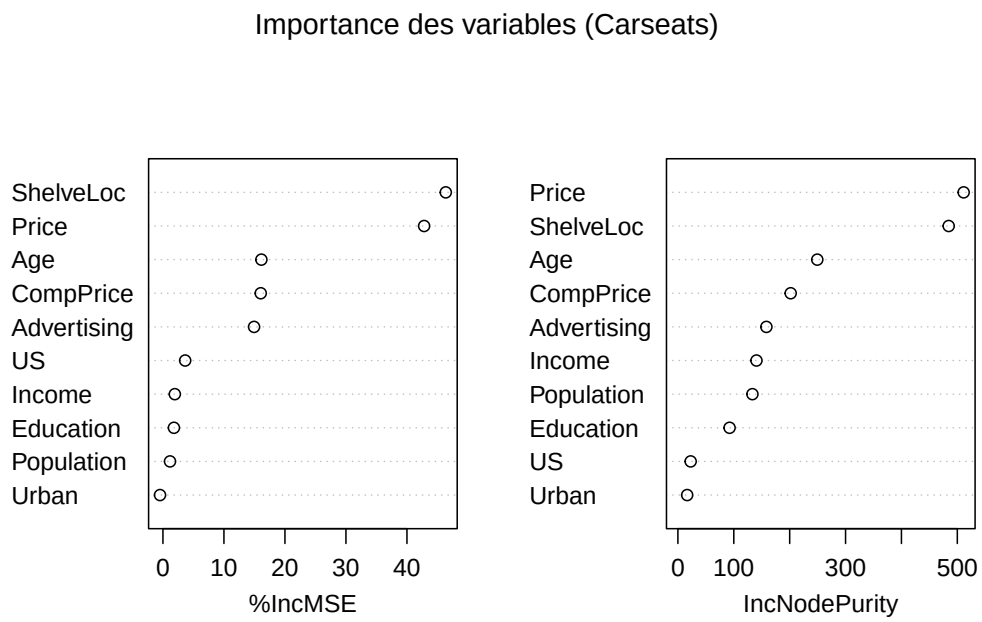
```
rmse <- function(observe, predit) sqrt(mean((observe - predit)^2))
arbre_ventes <- rpart(Sales ~ ., data = ventes_train)
```

```
lineaire <- lm(Sales ~ Price + CompPrice + Advertising + ShelveLoc + Age +
 ↪  Income,
               data = ventes_train)
c(reference = rmse(ventes_test$Sales, mean(ventes_train$Sales)),
  arbre     = rmse(ventes_test$Sales, predict(arbre_ventes, ventes_test)),
  foret     = rmse(ventes_test$Sales, predict(foret_ventes, ventes_test)),
  lineaire  = rmse(ventes_test$Sales, predict(lineaire, ventes_test)))
```

```
#> reference     arbre     foret  lineaire
#>     2.971     2.508     1.809     1.059
```

La forêt améliore nettement l'arbre unique, mais le **modèle linéaire** reste le meilleur. Les ventes de `Carseats` sont en effet construites comme une fonction presque **linéaire** du prix, de l'emplacement, de la publicité… Un modèle linéaire capture exactement cette structure ; une forêt doit l'approcher par une moyenne de fonctions en escalier.

```
varImpPlot(foret_ventes, main = "Importance des variables (Carseats)")
```



L'emplacement en rayon et le prix dominent, comme dans le modèle linéaire. Les variables que le modèle linéaire n'avait pas retenues (population, niveau d'études, zone urbaine) ont une importance **par permutation** (`%IncMSE`) quasi nulle ; leur diminution d'impureté (`IncNodePurity`) n'est pourtant pas négligeable, car cette mesure favorise les variables quantitatives, sur lesquelles les arbres trouvent toujours des coupures, même inutiles.

> **Une forêt aléatoire est un excellent point de départ, pas une garantie.** Comparez-la toujours, sur des données de test, à une référence naïve et à un modèle simple et interprétable (régression linéaire ou logistique). Si le modèle simple fait aussi bien, préférez-le : il s'explique plus facilement aux équipes métier et aux personnes concernées par la décision.

5.4 Résumé

- **ACP** : résumer p variables quantitatives par quelques axes non corrélés qui conservent le maximum d'inertie. ACP normée si les unités diffèrent.
- `PCA(données, quanti.sup = ..., quali.sup = ..., graph = FALSE)` ; `fviz_eig()`, `fviz_pca_var()`, `fviz_pca_ind()`, `dimdesc()`.
- Cercle des corrélations : angle entre flèches = corrélation ; n'interpréter que les variables bien représentées.
- Les variables **illustratives** ne construisent pas les axes ; elles aident à les interpréter (valeurs-test).
- **Arbre de décision** : partitionnement récursif minimisant l'impureté (Gini) ; `rpart()`, `printcp()`, `prune()`, `predict()`.
- Évaluer sur un jeu de **test** : matrice de confusion, sensibilité, spécificité, précision ; courbe ROC et AUC avec `pROC`. Choisir le seuil selon le coût des erreurs.
- **Forêt aléatoire** : moyenne de nombreux arbres profonds construits sur des échantillons bootstrap, avec m variables tirées au hasard à chaque nœud (`mtry`) pour décorréler les arbres. Vote majoritaire en classification, moyenne en régression. `randomForest()`, erreur OOB, `varImpPlot()`, `partialPlot()`. Toujours comparer à un modèle simple.

5.5 Exercices

Exercice 5.1 — ACP sur `USArrests`. Le jeu `USArrests` donne, pour les 50 États américains, le nombre d'arrestations pour 100 000 habitants pour meurtre, agression et viol, ainsi que le pourcentage de population urbaine.

1. Faut-il réaliser une ACP normée ? Justifier.
2. Réaliser l'ACP. Combien d'axes retenir ?
3. Interpréter les deux premiers axes à l'aide du cercle des corrélations.
4. Quels États ont les profils les plus extrêmes ?

Solution

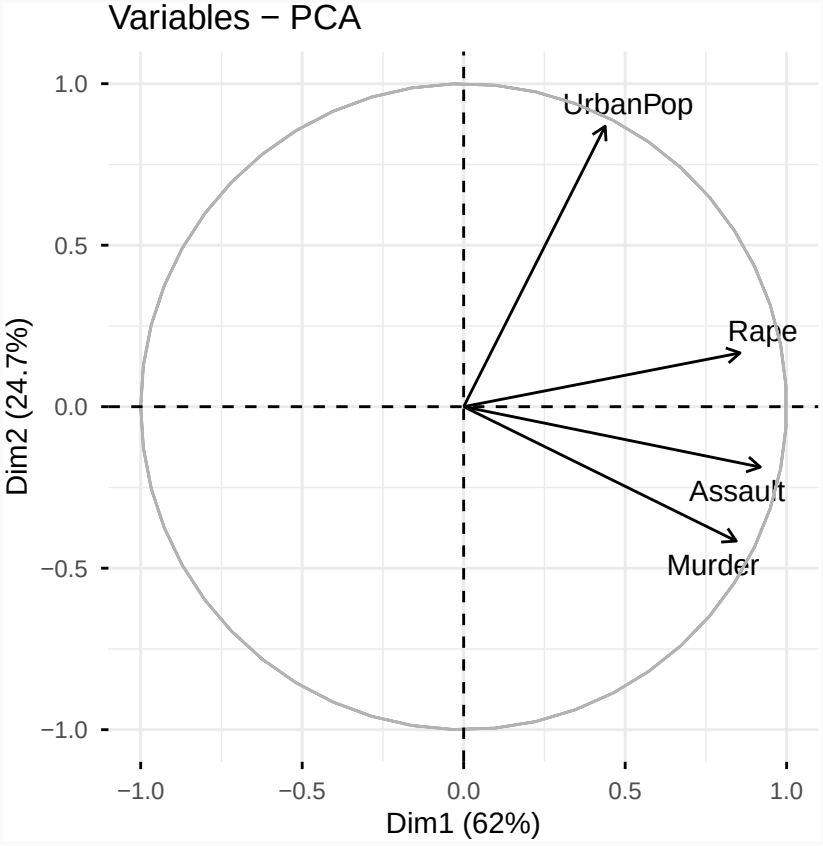
```
sapply(USArrests, var)        # variances très différentes : ACP normée
 ↪  nécessaire
```

```
#>   Murder  Assault UrbanPop     Rape
#>    18.97  6945.17   209.52    87.73
```

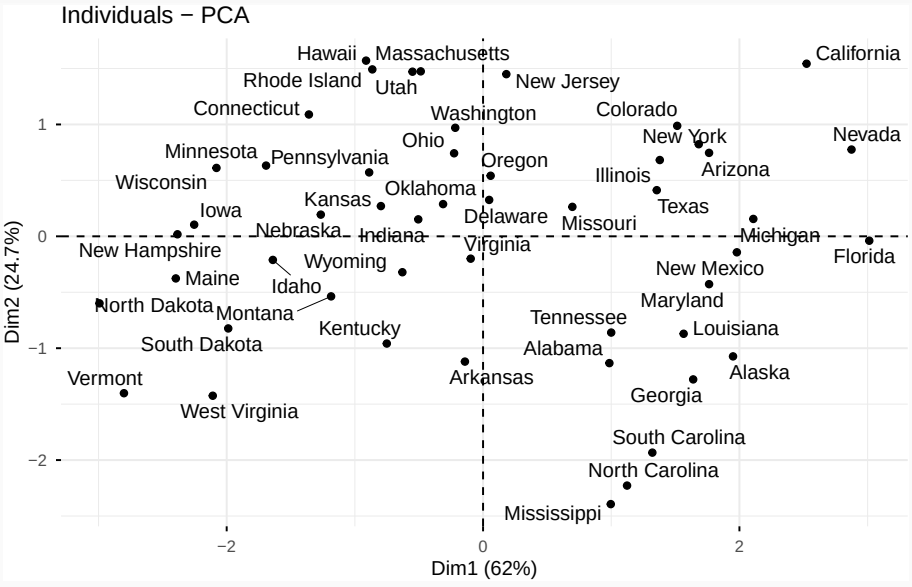
```
acp_us <- PCA(USArrests, graph = FALSE)
acp_us$eig
```

```
#>        eigenvalue percentage of variance
#> comp 1     2.4802                 62.006
#> comp 2     0.9898                 24.744
#> comp 3     0.3566                  8.914
#> comp 4     0.1734                  4.336
#>        cumulative percentage of variance
#> comp 1                             62.01
#> comp 2                             86.75
#> comp 3                             95.66
#> comp 4                            100.00
```

```
fviz_pca_var(acp_us, repel = TRUE)
```




```
fviz_pca_ind(acp_us, repel = TRUE)
```



L'axe 1 (environ 62 % de l'inertie) regroupe les trois types de crimes : c'est un axe de **criminalité globale**. L'axe 2 est porté par `UrbanPop` : il oppose les États **urbains** aux États **ruraux**.

Exercice 5.2 — Ciblage d'une campagne d'assurance. Le jeu `Caravan` (package `ISLR`) contient les données **réelles** de 5 822 clients d'un assureur néerlandais, diffusées lors du concours *CoIL Challenge 2000*. L'assureur voulait savoir quels clients cibler pour vendre

une assurance pour caravane (camping-car). Chaque client est décrit par 86 variables :

- 43 variables **socio-démographiques** (noms commençant par `M`, par exemple `MOSTYPE`, le type de client, ou `MKOOPKLA`, la classe de pouvoir d'achat). Elles ne décrivent pas le client lui-même mais **son quartier** : tous les clients d'un même code postal ont les mêmes valeurs. La plupart sont codées en classes de 0 à 9 (par exemple, part des habitants du quartier ayant tel revenu) ;
- 42 variables de **détention de produits** d'assurance : niveau des cotisations par type de contrat, codé en classes (noms commençant par `P`, par exemple `PPERSAUT` pour l'assurance automobile) et nombre de contrats (noms commençant par `A`, par exemple `APERSAUT`) ;
- la cible `Purchase` : le client a-t-il souscrit l'assurance caravane (`Yes` / `No`) ?

Les variables de quartier illustrent une pratique courante en marketing (le **géomarketing**) et ses limites : on attribue à chaque client les caractéristiques moyennes de son voisinage.

1. Quelle est la proportion de souscripteurs ? Que vaut le taux de bonne classification d'un modèle qui prédit toujours « No » ?
2. Découper les données (70 % / 30 %) et construire un arbre de classification. Que prédit-il sur le jeu de test ?
3. Utiliser plutôt les **probabilités** prédites : tracer la courbe ROC et calculer l'AUC.
4. L'assureur peut contacter 10 % des clients. En ciblant les 10 % de clients du jeu de test aux probabilités les plus élevées, quelle part des souscripteurs atteint-il ? Comparer au hasard (10 %).

Solution

```
mean(Caravan$Purchase == "Yes")          # environ 6 % : prédire "No"
 ↪  donne 94 % de bonnes réponses
```

```
#> [1] 0.05977
```

```
set.seed(1)
idx <- sample(nrow(Caravan), round(0.7 * nrow(Caravan)))
car_train <- Caravan[idx, ]
car_test  <- Caravan[-idx, ]

arbre_car <- rpart(Purchase ~ ., data = car_train, method = "class",
                   control = rpart.control(cp = 0.002))
table(observe = car_test$Purchase,
      predit = predict(arbre_car, car_test, type = "class"))
```

```
#>        predit
#> observe   No  Yes
#>     No  1629   13
#>     Yes  101    4
```

```
logit_car <- glm(Purchase ~ ., data = car_train, family = binomial)
```

```
#> Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
```

```
score <- predict(logit_car, car_test, type = "response")
auc(roc(car_test$Purchase, score, levels = c("No", "Yes"), direction =
 ↪  "<"))
```

```
#> Area under the curve: 0.729
```

```r
cibles <- score >= quantile(score, 0.9)          # les 10 % de meilleurs
↪ scores
sum(car_test$Purchase[cibles] == "Yes") / sum(car_test$Purchase == "Yes")
```

```
#> [1] 0.3238
```

Avec le seuil de 0,5, l'arbre ne prédit presque jamais « Yes » : son taux de bonne classification est élevé mais il ne sert à rien pour cibler. En classant les clients par probabilité et en ne contactant que les 10 % les mieux notés, on atteint une part des souscripteurs bien supérieure à 10 % : c'est ce gain (*lift*) qui justifie un modèle de ciblage. La régression logistique sur 85 variables peut produire des avertissements de convergence ; les modèles régularisés (hors programme) sont plus adaptés.

Exercice 5.3 — Questions sur les forêts aléatoires.

1. Pourquoi agréger de nombreux arbres réduit-il l'erreur de prédiction ? Quelle composante de l'erreur (biais ou variance) est réduite ?
2. Quelle est la différence entre le bagging et une forêt aléatoire ? Pourquoi cette différence améliore-t-elle les performances ?
3. Comment une forêt produit-elle sa prédiction en classification ? En régression ?
4. Quelle part des observations est « hors du sac » pour un arbre donné ? À quoi sert l'erreur OOB ?
5. Augmenter le nombre d'arbres peut-il conduire au sur-apprentissage ?
6. Une équipe marketing conclut d'une forte importance de la variable « nombre de visites du site » qu'augmenter les visites fera augmenter les achats. Qu'en pensez-vous ?

Éléments de réponse

1. Les arbres profonds ont un faible biais mais une forte **variance** : ils dépendent beaucoup de l'échantillon d'apprentissage. La moyenne de B prédictions a une variance $\rho\sigma^2 + (1-\rho)\sigma^2/B$, plus faible que celle d'un arbre, sans augmentation du biais.
2. Le bagging construit chaque arbre sur un échantillon bootstrap en considérant toutes les variables à chaque nœud ; la forêt aléatoire ne considère que m variables tirées au hasard. Les arbres sont alors moins corrélés (ρ plus faible), donc leur moyenne réduit davantage la variance.
3. Classification : vote majoritaire des arbres (et part des votes comme probabilité). Régression : moyenne des prédictions des arbres.
4. Environ $e^{-1} \approx 37\,\%$. L'erreur OOB, calculée en prédisant chaque observation avec les seuls arbres qui ne l'ont pas utilisée, estime l'erreur de prédiction sans jeu de validation ; elle sert à choisir `mtry` et à vérifier que le nombre d'arbres suffit.
5. Non : ajouter des arbres stabilise la moyenne, l'erreur se stabilise puis ne bouge plus. Seul le temps de calcul augmente.
6. L'importance mesure l'utilité d'une variable pour **prédire**, pas un effet **causal** : les clients qui visitent souvent le site sont déjà intéressés. Pour mesurer l'effet d'une action (augmenter les visites), il faut une expérience, par exemple un test A/B.

Exercice 5.4 — Forêt aléatoire en régression. Avec `phbirths` (`datasets/phbirths.dat` : poids de naissance `grams`, durée de gestation `gestate`, tabagisme de la mère `smoke`, origine `black`, années d'études `educ`) :

1. Découper les données en 70 % pour l'entraînement et 30 % pour le test.
2. Ajuster une forêt aléatoire de `grams` sur toutes les autres variables. Que vaut `mtry` ?

Quelle part de variance est expliquée (OOB) ?
3. Choisir `mtry` (de 1 à 4) avec l'erreur OOB.
4. Comparer sur le jeu de test le RMSE de la forêt retenue à celui d'une régression linéaire et à celui de la référence naïve (la moyenne). Conclure.

Solution

```
phbirths <- read.table("../datasets/phbirths.dat")
set.seed(1)
idx <- sample(nrow(phbirths), round(0.7 * nrow(phbirths)))
ph_train <- phbirths[idx, ]
ph_test  <- phbirths[-idx, ]

set.seed(1)
foret_ph <- randomForest(grams ~ ., data = ph_train)
foret_ph                                          # mtry = floor(4 / 3) = 1
```

```
#>
#> Call:
#>  randomForest(formula = grams ~ ., data = ph_train)
#>                Type of random forest: regression
#>                      Number of trees: 500
#> No. of variables tried at each split: 1
#>
#>           Mean of squared residuals: 222885
#>                     % Var explained: 42.7
```

```
erreur_oob <- sapply(1:4, function(m) {
  set.seed(1)
  tail(randomForest(grams ~ ., data = ph_train, mtry = m)$mse, 1)
})
round(setNames(erreur_oob, 1:4))
```

```
#>      1      2      3      4
#> 222885 192152 199950 205937
```

```
set.seed(1)
foret_ph <- randomForest(grams ~ ., data = ph_train, mtry =
↪  which.min(erreur_oob))
lineaire_ph <- lm(grams ~ ., data = ph_train)
c(reference = rmse(ph_test$grams, mean(ph_train$grams)),
  lineaire  = rmse(ph_test$grams, predict(lineaire_ph, ph_test)),
  foret     = rmse(ph_test$grams, predict(foret_ph, ph_test)))
```

```
#> reference  lineaire     foret
#>     655.4     431.7     452.1
```

Le réglage de `mtry` améliore la forêt, mais la régression linéaire reste meilleure. Avec seulement quatre variables explicatives et une relation essentiellement linéaire, une forêt n'a pas d'avantage : la régression linéaire, plus simple et interprétable, est à préférer.

Exercice 5.5 — Forêt aléatoire et classes déséquilibrées. On reprend le jeu `Caravan` et le découpage de l'exercice 5.2 (`car_train`, `car_test`).

1. Ajuster une forêt aléatoire de `Purchase` sur toutes les variables. Examiner la matrice de confusion OOB : combien de souscripteurs sont détectés ?
2. Calculer l'AUC sur le jeu de test et la part des souscripteurs atteinte en ciblant les 10

% de clients aux probabilités les plus élevées. Comparer à la régression logistique de l'exercice 5.2.

3. Refaire la forêt en construisant chaque arbre sur un échantillon **équilibré** de 200 non-souscripteurs et 200 souscripteurs (`sampsize = c(No = 200, Yes = 200), strata = car_train$Purchase`). Comparer la matrice de confusion sur le jeu de test et l'AUC.

Solution

```
set.seed(1)
foret_car <- randomForest(Purchase ~ ., data = car_train)
foret_car$confusion                          # presque aucun souscripteur
 ↪  détecté
```

```
#>       No Yes class.error
#> No  3798  34    0.008873
#> Yes  242   1    0.995885
```

```
score_foret <- predict(foret_car, car_test, type = "prob")[, "Yes"]
auc(roc(car_test$Purchase, score_foret, levels = c("No", "Yes"), direction
 ↪  = "<"))
```

```
#> Area under the curve: 0.688
```

```
cibles <- score_foret >= quantile(score_foret, 0.9)
sum(car_test$Purchase[cibles] == "Yes") / sum(car_test$Purchase == "Yes")
```

```
#> [1] 0.2952
```

```
set.seed(1)
foret_equilibree <- randomForest(Purchase ~ ., data = car_train,
                                 sampsize = c(No = 200, Yes = 200),
                                 strata = car_train$Purchase)
table(observe = car_test$Purchase, predit = predict(foret_equilibree,
 ↪  car_test))
```

```
#>        predit
#> observe   No  Yes
#>     No  1370  272
#>     Yes   56   49
```

```
score_eq <- predict(foret_equilibree, car_test, type = "prob")[, "Yes"]
auc(roc(car_test$Purchase, score_eq, levels = c("No", "Yes"), direction =
 ↪  "<"))
```

```
#> Area under the curve: 0.719
```

Avec les réglages par défaut, chaque arbre voit très peu de souscripteurs (environ 6 %) et vote presque toujours « No » : la forêt détecte à peine un souscripteur, et son classement est moins bon que celui de la régression logistique. En équilibrant les échantillons bootstrap, chaque arbre voit autant de souscripteurs que de non-souscripteurs : la forêt en détecte alors une bonne partie, au prix de nombreux faux positifs, et son AUC s'améliore. Le choix entre ces deux comportements dépend, encore une fois, du coût d'un contact inutile face à celui d'une vente manquée.

Pour aller plus loin

- F. Husson, S. Lê et J. Pagès, *Analyse de données avec R*, chapitre ACP ; vidéos de F. Husson sur YouTube (chaîne « François Husson »), en français.

- G. Saporta, *Probabilités, analyse des données et statistique*, chapitre 7.
- G. James et al., *An Introduction to Statistical Learning*, chapitres 8 (arbres, bagging et forêts aléatoires) et 12 (ACP).
- L. Breiman, « Random Forests », *Machine Learning*, 45, 2001, p. 5-32.

Chapitre 6

Classification automatique, éthique et R Shiny

Objectifs du chapitre

- Distinguer apprentissage supervisé et non supervisé.
- Segmenter des clients avec les k-moyennes et la classification ascendante hiérarchique (CAH).
- Choisir un nombre de classes, décrire et nommer les classes obtenues.
- Identifier les enjeux éthiques et légaux d'un projet de science des données, en particulier en marketing.
- Comprendre la structure d'une application web R Shiny.

```
library(tidyverse)
library(FactoMineR)
library(factoextra)
library(cluster)     # silhouette
source("R/boutique.R")
```

6.1 Classification non supervisée

6.1.1 Supervisé ou non supervisé ?

- **Supervisé** (chapitres 4 et 5) : on connaît la variable à prédire (la marque achetée) et on apprend à la prédire.
- **Non supervisé** : il n'y a pas de variable cible. On cherche une **structure** dans les données. La **classification automatique** (*clustering*) forme des groupes d'individus tels que les individus d'un même groupe se ressemblent et que les groupes soient différents entre eux.

En marketing, c'est la **segmentation** de clientèle : regrouper les clients selon leurs comportements pour adapter les offres, les messages et les canaux à chaque segment. Attention au vocabulaire : en français, « classification » désigne souvent ce regroupement non supervisé, alors qu'en anglais *classification* désigne la prédiction supervisée d'une classe.

6.1.2 Distances et standardisation

Regrouper des individus « qui se ressemblent » suppose de mesurer leur **distance**. La plus utilisée est la distance **euclidienne** :

$$d(x_i, x_{i'}) = \sqrt{\sum_{j=1}^{p} (x_{ij} - x_{i'j})^2}$$

> Avec la distance euclidienne, une variable de grande variance (un montant en euros, de 16 à 1 800) écrase une variable de petite variance (une part de commandes en promotion, entre 0 et 1). Il faut presque toujours **centrer-réduire** les variables avant de calculer des distances : `scale()`.

6.1.3 Les données : les clients de la boutique

On reprend les indicateurs de comportement des clients actifs en 2025 (chapitres 3 et 5) :

```
indicateurs <- indicateurs_clients(lire_boutique("commandes"),
 ↪  lire_boutique("clients"))

variables <- c("recence", "frequence", "montant_total", "panier_moyen",
               "part_promo", "nb_categories", "articles_par_commande",
                ↪  "ancienne te")
x <- scale(indicateurs[, variables])     # centrer-réduire
rownames(x) <- indicateurs$client_id
dim(x)
```

```
#> [1] 1741    8
```

Les données ont été simulées avec quatre segments de clients « cachés ». Nous allons voir si les méthodes de classification les retrouvent, et ce qu'elles nous disent pour les nommer.

6.2 Les k-moyennes (*k-means*)

6.2.1 Algorithme

On fixe le nombre de classes k à l'avance.

1. Choisir k centres initiaux au hasard.
2. **Affecter** chaque individu au centre le plus proche.
3. **Recalculer** chaque centre comme la moyenne des individus de sa classe.
4. Répéter les étapes 2 et 3 jusqu'à ce que les affectations ne changent plus.

L'algorithme diminue à chaque étape l'**inertie intra-classe** (somme des carrés des distances des individus au centre de leur classe). Il converge vers un **minimum local** qui dépend des centres initiaux : on le lance plusieurs fois (argument `nstart`) et on garde la meilleure solution.

6.2.2 Décomposition de l'inertie

$$\underbrace{I_{\text{totale}}}_{\text{totss}} = \underbrace{I_{\text{intra}}}_{\text{tot.withinss}} + \underbrace{I_{\text{inter}}}_{\text{betweenss}}$$

Une bonne partition a une inertie intra-classe faible (classes homogènes) et une inertie inter-classe forte (classes bien séparées). Le rapport $I_{\text{inter}}/I_{\text{totale}}$ (un « R^2 » de la partition) mesure la part de l'information conservée par la partition.

```
set.seed(1)
km <- kmeans(x, centers = 4, nstart = 25)
```

```
km$size                                    # effectif de chaque classe
```

```
#> [1] 370 596 423 352
```

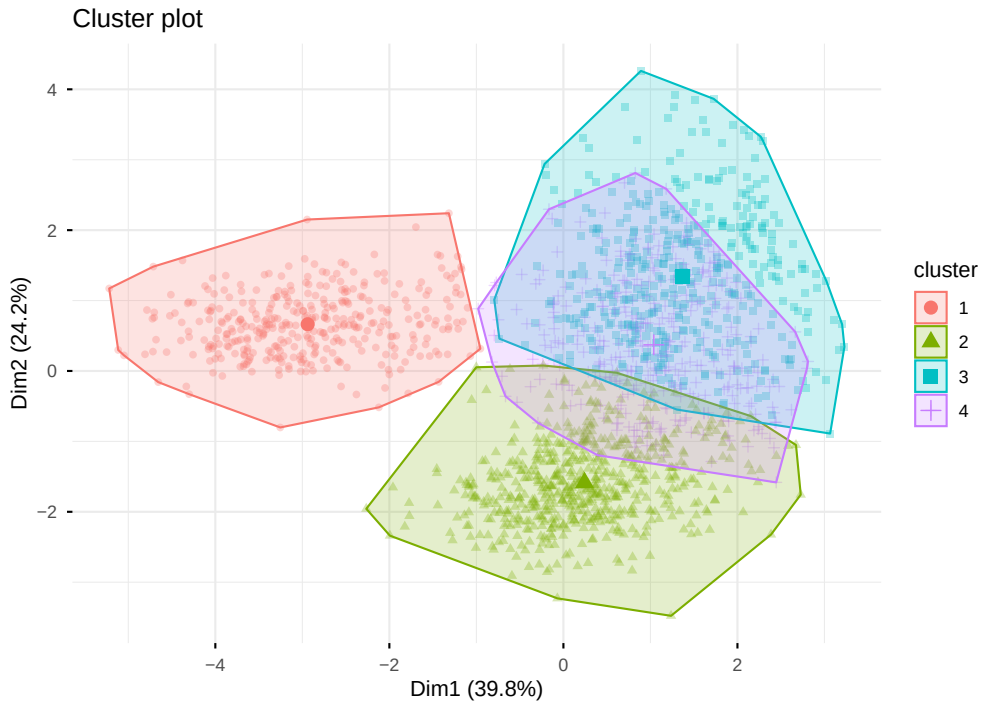
```
round(km$centers, 2)                       # centres (sur données réduites)
```

```
#>   recence frequence montant_total panier_moyen part_promo nb_categories
#> 1   -0.60      1.42          1.75         1.27      -0.77          1.10
#> 2   -0.39      0.16         -0.33        -0.82       0.86          0.32
#> 3    1.31     -0.88         -0.60         0.24      -0.45         -0.90
#> 4   -0.29     -0.71         -0.56        -0.22      -0.10         -0.62
#>   articles_par_commande anciennete
#> 1                 -0.45       0.26
#> 2                  0.87       0.28
#> 3                 -0.46       0.40
#> 4                 -0.45      -1.23
```

```
km$betweenss / km$totss                    # part d'inertie expliquée
```

```
#> [1] 0.5633
```

```
fviz_cluster(km, data = x, geom = "point", alpha = 0.3, ggtheme =
 ↪  theme_minimal())
```



`fviz_cluster()` représente les classes sur le premier plan d'une ACP : c'est une façon naturelle de visualiser une partition en plus de deux dimensions.

6.3 Classification ascendante hiérarchique (CAH)

6.3.1 Algorithme

1. Au départ, chaque individu forme une classe.
2. On **fusionne** les deux classes les plus proches.
3. On recommence jusqu'à n'avoir plus qu'une seule classe.

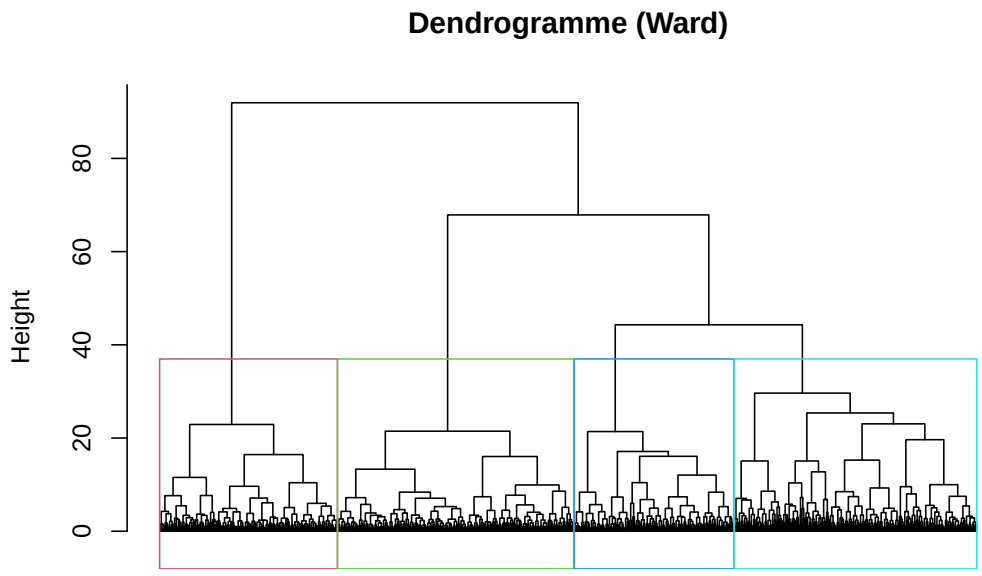
On obtient une hiérarchie de partitions emboîtées, représentée par un **dendrogramme**. Il n'est pas nécessaire de fixer k à l'avance : on « coupe » l'arbre après coup.

La **distance entre deux classes** (critère d'agrégation) se définit de plusieurs façons :

Critère	Distance entre classes	Effet
lien simple (`single`)	plus petite distance entre deux individus	classes allongées, effet de chaîne
lien complet (`complete`)	plus grande distance	classes compactes
lien moyen (`average`)	distance moyenne	compromis
Ward (`ward.D2`)	augmentation de l'inertie intra-classe due à la fusion	classes compactes, de tailles proches ; **le plus utilisé**

Le critère de Ward est cohérent avec les k-moyennes : tous deux minimisent l'inertie intra-classe.

```
distances <- dist(x, method = "euclidean")
cah <- hclust(distances, method = "ward.D2")
plot(cah, labels = FALSE, hang = -1, main = "Dendrogramme (Ward)", xlab = "",
 ↪  sub = "")
rect.hclust(cah, k = 4, border = 2:5)
```



Lecture : la **hauteur** d'une fusion mesure la perte d'homogénéité qu'elle provoque. Un grand saut de hauteur indique qu'on fusionne des classes très différentes : c'est un bon endroit pour couper. Avec 1 741 clients, les feuilles sont illisibles ; seule la partie haute de l'arbre compte.

```
classes_cah <- cutree(cah, k = 4)
table(classes_cah)
```

```
#> classes_cah
#>   1   2   3   4
#> 379 504 517 341
```

```
table(cah = classes_cah, kmeans = km$cluster)    # comparaison des deux
 ↪  partitions
```

```
#>    kmeans
#> cah   1   2   3   4
#>   1 368   2   2   7
#>   2   0 500   0   4
#>   3   2  49 407  59
#>   4   0  45  14 282
```

Les numéros de classes sont arbitraires ; ce qui compte est de savoir si les **mêmes individus** sont regroupés ensemble. Les deux méthodes donnent ici des partitions proches : c'est un premier signe de **stabilité**.

6.3.2 k-moyennes ou CAH ?

	k-moyennes	CAH
Nombre de classes	fixé à l'avance	choisi après coup, sur le dendrogramme
Taille des données	grandes tailles possibles (millions de clients)	coûteuse au-delà de quelques dizaines de milliers d'individus (matrice des distances)
Résultat	dépend de l'initialisation	déterministe
Usage combiné	consolider une CAH	choisir k et initialiser les k-moyennes

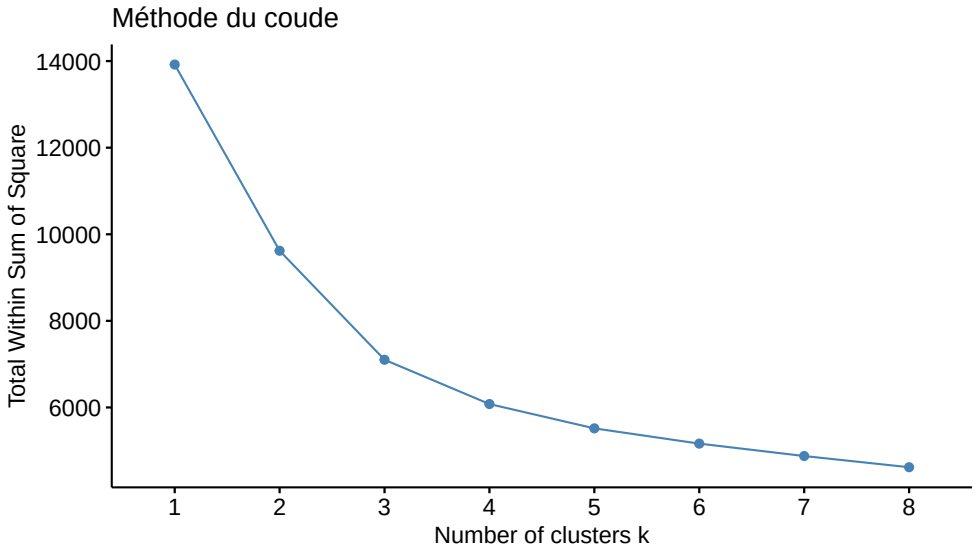
6.4 Choisir le nombre de classes

Aucun critère n'est parfait : on en combine plusieurs, puis on vérifie que les classes sont **interprétables** et **utiles** pour la question posée. Une équipe marketing peut gérer quatre ou cinq segments, rarement quinze.

6.4.1 Méthode du coude

On trace l'inertie intra-classe en fonction de k. Elle diminue toujours ; on cherche le k à partir duquel le gain devient faible.

```
fviz_nbclust(x, kmeans, method = "wss", nstart = 25, k.max = 8) +
  labs(title = "Méthode du coude")
```



6.4.2 Indice de silhouette

Pour chaque individu i, on compare $a(i)$, sa distance moyenne aux individus de sa classe, à $b(i)$, sa distance moyenne aux individus de la classe voisine la plus proche :

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))} \in [-1, 1]$$

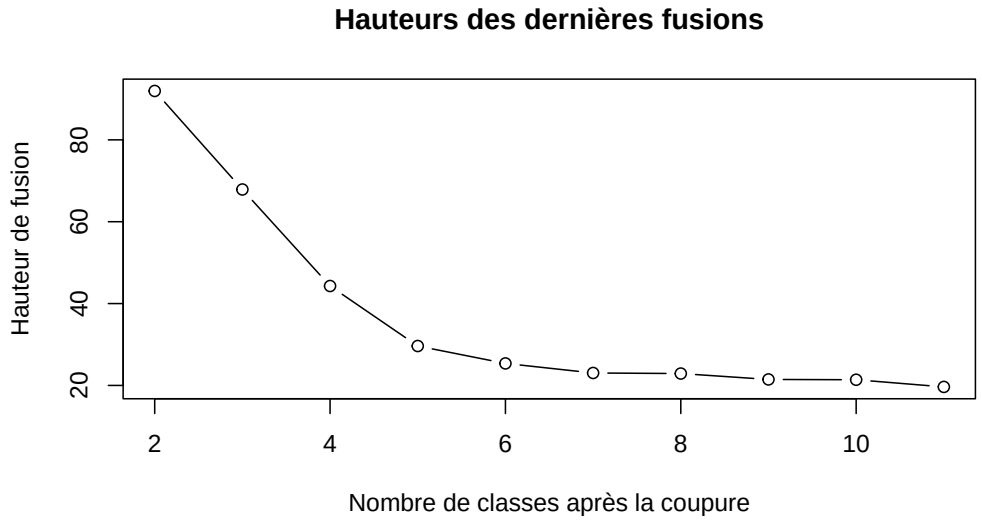
Une valeur proche de 1 : l'individu est bien classé ; proche de 0 : il est entre deux classes ; négative : il est probablement mal classé. On cherche un k qui donne une **silhouette moyenne** élevée.

```
silhouette_moyenne <- sapply(2:7, function(k) {
  set.seed(1)
  classes <- kmeans(x, centers = k, nstart = 25)$cluster
  mean(silhouette(classes, distances)[, "sil_width"])
})
round(setNames(silhouette_moyenne, 2:7), 3)
```

```
#>     2     3     4     5     6     7
#> 0.333 0.310 0.303 0.296 0.298 0.250
```

6.4.3 Le saut d'inertie du dendrogramme

```
hauteurs <- sort(cah$height, decreasing = TRUE)[1:10]
plot(1:10 + 1, hauteurs, type = "b", xlab = "Nombre de classes après la
 ↪  coupure",
     ylab = "Hauteur de fusion", main = "Hauteurs des dernières fusions")
```



On coupe juste avant un grand saut de hauteur.

Bilan. Les critères ne sont pas d'accord : la silhouette moyenne est un peu plus élevée pour $k = 2$, mais les écarts entre $k = 2, 3, 4$ sont faibles ; le coude et les hauteurs du dendrogramme suggèrent 3 ou 4 classes. Une partition en deux classes (« bons clients » / « autres ») serait trop grossière pour adapter les actions marketing. On retient $k = 4$, à condition que les classes soient interprétables, ce que l'on vérifie maintenant.

6.5 Décrire les classes

Une classification n'est utile que si l'on sait **décrire** et **nommer** ses classes.

6.5.1 Moyennes par classe

```
profils <- indicateurs |>
  mutate(classe = factor(km$cluster)) |>
  group_by(classe) |>
  summarise(effectif = n(),
            across(all_of(variables), \(v) round(mean(v), 2)))
profils
```

```
#> # A tibble: 4 x 10
#>   classe effectif recence frequence montant_total panier_moyen
#>   <fct>     <int>   <dbl>     <dbl>         <dbl>        <dbl>
#> 1 1           370    16.6     11.3          1034.         91.8
#> 2 2           596    35.5      6.13          224.         36.8
#> 3 3           423   182.       1.83          118.         64.8
#> 4 4           352    43.6      2.53          135.         52.6
#> # i 4 more variables: part_promo <dbl>, nb_categories <dbl>,
#> #   articles_par_commande <dbl>, anciennete <dbl>
```

On compare chaque classe à la moyenne générale pour la nommer. Par exemple :

Profil observé	Nom proposé	Action possible
commandes très fréquentes, gros montants, peu de promotions	**Fidèles premium**	programme de fidélité, avant-premières

Profil observé	Nom proposé	Action possible
beaucoup d'articles, petits paniers, forte part de promotions	**Chasseurs de promos**	ventes privées, mais attention à la marge
dernière commande lointaine, une ou deux commandes	**Occasionnels endormis**	campagne de réactivation
inscrits récemment, quelques commandes	**Nouveaux clients**	parcours d'accueil, deuxième commande

(Les numéros de classes dépendent de l'initialisation : associez chaque nom au bon numéro en lisant le tableau des profils.)

Les variables **qui n'ont pas servi** à construire les classes aident aussi à les décrire, comme les variables illustratives d'une ACP :

```
indicateurs |>
  mutate(classe = factor(km$cluster)) |>
  group_by(classe) |>
  summarise(age_moyen = round(mean(age, na.rm = TRUE)),
            part_newsletter = round(mean(newsletter == "Oui"), 2),
            canal_principal = names(which.max(table(canal_acquisition))))
```

```
#> # A tibble: 4 x 4
#>   classe age_moyen part_newsletter canal_principal
#>   <fct>      <dbl>           <dbl> <chr>
#> 1 1             46            0.69 Direct
#> 2 2             32            0.57 Réseaux sociaux
#> 3 3             41            0.31 Moteur de recherche
#> 4 4             33            0.38 Réseaux sociaux
```

6.5.2 Les classes retrouvent-elles la structure simulée ?

Les données étant simulées, on connaît les vrais segments : lors de la simulation, ils ont été tirés au hasard juste après `set.seed(2026)`, avec les proportions ci-dessous. On peut donc les reconstituer et les comparer aux classes obtenues.

```
set.seed(2026)
segment_simule <- sample(c("Fidèles", "Chasseurs de promos", "Occasionnels",
 ↪  "Nouveaux"),
                         2000, replace = TRUE, prob = c(0.20, 0.30, 0.35,
                          ↪  0.15))
verite <- tibble(client_id = sprintf("C%04d", 1:2000), segment_simule)

indicateurs |>
  mutate(classe = km$cluster) |>
  left_join(verite, by = "client_id") |>
  count(classe, segment_simule) |>
  pivot_wider(names_from = segment_simule, values_from = n, values_fill = 0)
```

```
#> # A tibble: 4 x 5
#>   classe Fidèles Occasionnels `Chasseurs de promos` Nouveaux
#>    <int>   <int>        <int>                 <int>    <int>
#> 1      1     369            1                     0        0
#> 2      2       2           33                   561        0
```

```
#> 3       3        4           402                    15         2
#> 4       4        8           106                    23       215
```

Chaque classe correspond majoritairement à un segment simulé. Les erreurs se concentrent entre « Occasionnels » et « Nouveaux », qui ont peu de commandes dans les deux cas. Sur des données réelles, on ne connaît **jamais** cette vérité : c'est pourquoi on vérifie la stabilité et l'interprétabilité des classes.

6.5.3 ACP puis CAH : la fonction `HCPC()`

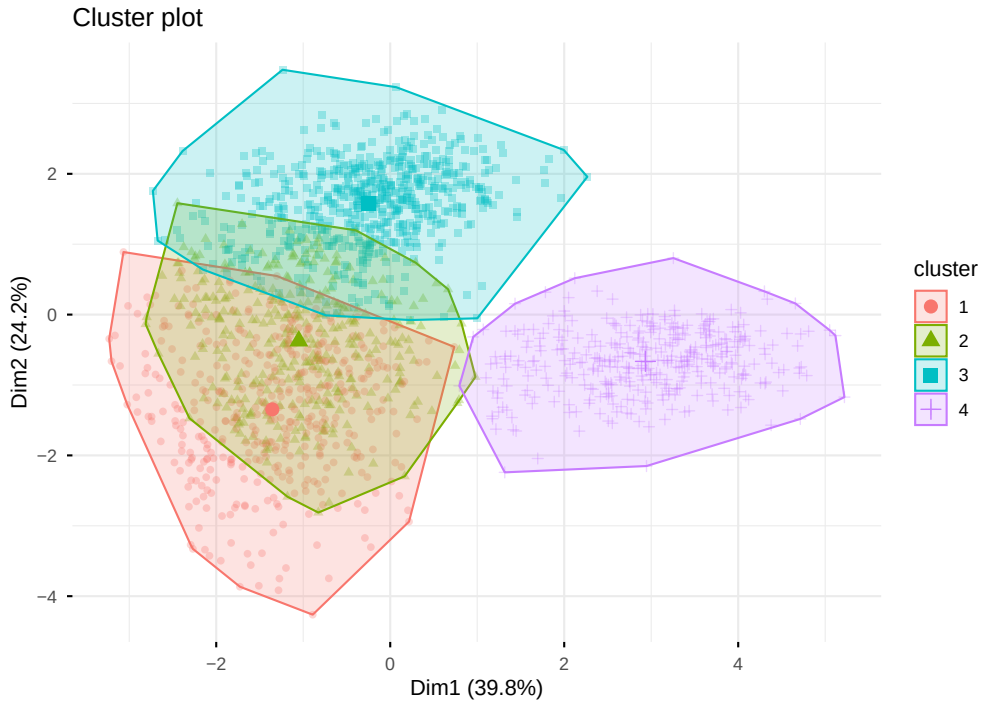
`FactoMineR` enchaîne naturellement une ACP, une CAH (Ward) sur les coordonnées factorielles, puis une consolidation par k-moyennes. Elle propose un nombre de classes (selon le saut d'inertie) et **décrit automatiquement** les classes.

```
acp <- PCA(as.data.frame(indicateurs[, variables]), graph = FALSE)
res_hcpc <- HCPC(acp, nb.clust = -1, graph = FALSE)   # -1 : coupure
 ↪  automatique
table(res_hcpc$data.clust$clust)
```

```
#>
#>   1   2   3
#> 718 652 371
```

La coupure automatique propose ici **trois** classes : elle fusionne les nouveaux clients et les occasionnels. Rien n'empêche d'imposer le nombre de classes retenu pour des raisons métier :

```
res_hcpc <- HCPC(acp, nb.clust = 4, graph = FALSE)
fviz_cluster(res_hcpc, geom = "point", alpha = 0.3, ggtheme = theme_minimal())
```




```
res_hcpc$desc.var$quanti$`1`      # variables qui caractérisent la classe 1
```

```
#>                    v.test Mean in category Overall mean
#> recence            30.830         181.1247      68.7898
#> anciennete          9.460           3.2304       2.6916
#> panier_moyen        6.003          65.1346      58.4648
```

```
#> part_promo            -10.671           0.2625       0.4254
#> articles_par_commande -10.916           1.9645       2.3432
#> montant_total         -14.208         118.8460     352.5338
#> frequence             -20.791           1.8353       5.4612
#> nb_categories         -21.400           1.6118       2.7501
#>                       sd in category Overall sd    p.value
#> recence                      98.3288    86.3748 1.049e-208
#> anciennete                    1.1062     1.3501  3.092e-21
#> panier_moyen                 24.2732    26.3397  1.942e-09
#> part_promo                    0.3557     0.3619  1.388e-26
#> articles_par_commande         0.7968     0.8224  9.680e-28
#> montant_total                70.2266   389.8943  8.182e-46
#> frequence                     0.9107     4.1342  5.256e-96
#> nb_categories                 0.6910     1.2610 1.328e-101
```

Pour chaque classe, la sortie compare la moyenne de la classe (*Mean in category*) à la moyenne générale (*Overall mean*) et teste l'écart (*v.test* : au-delà de 2 en valeur absolue, l'écart est significatif). La fonction `catdes()` fait la même chose pour n'importe quelle variable qualitative. `res_hcpc$desc.ind$para` donne les **parangons** : les clients les plus proches du centre de chaque classe, utiles pour présenter chaque segment par un exemple concret (un « persona »).

Démarche d'une segmentation

1. Choisir les variables (comportement d'achat, pas de variables sensibles) et les standardiser.
2. Choisir une distance (euclidienne en général).
3. CAH (Ward) pour explorer et choisir k ; k-moyennes pour consolider.
4. Choisir k : coude, silhouette, saut d'inertie, **interprétabilité** et **utilité métier**.
5. Décrire chaque classe (moyennes, `catdes()`, parangons) et la nommer.
6. Vérifier la **stabilité** : obtient-on les mêmes classes avec une autre graine, un autre sous-échantillon, une autre méthode ?

6.6 L'éthique dans la science des données

Les données décrivent souvent des **personnes**. Les décisions prises à partir de modèles (crédit, recrutement, prix, publicité, santé, justice) ont des conséquences réelles sur leur vie. Un modèle techniquement excellent peut être inacceptable.

6.6.1 Quelques cas marquants

- **COMPAS** (États-Unis, 2016) : un logiciel d'évaluation du risque de récidive utilisé par des tribunaux. Une enquête de ProPublica a montré que les prévenus noirs non récidivistes étaient presque deux fois plus souvent classés « à haut risque » que les prévenus blancs non récidivistes. Le débat qui a suivi a montré que plusieurs définitions mathématiques de l'équité ne peuvent pas être satisfaites en même temps.
- **Outil de recrutement d'Amazon** (abandonné en 2018) : entraîné sur dix ans de CV majoritairement masculins, il pénalisait les CV contenant le mot « women's ». Les données historiques reproduisaient une discrimination passée.
- **Cambridge Analytica** (2018) : les données de dizaines de millions de profils Facebook ont été collectées sans consentement éclairé et utilisées pour du ciblage politique. Cet exemple concerne directement le marketing numérique.

6.6.2 Enjeux propres au marketing

- **Ciblage et discrimination** : une segmentation fondée sur le code postal peut servir d'**indicateur indirect** (*proxy*) de l'origine ou du revenu, et exclure certains quartiers d'une offre ou d'une publicité.
- **Prix personnalisés** : faire payer plus cher les clients jugés « peu sensibles au prix » pose des questions de loyauté envers le consommateur.
- **Exploitation des vulnérabilités** : cibler des personnes endettées avec des offres de crédit, ou des joueurs compulsifs avec des paris.
- **Interfaces trompeuses** (*dark patterns*) : rendre le refus des cookies ou la résiliation d'un abonnement délibérément difficiles.
- **Données sensibles** : santé, opinions, orientation sexuelle… peuvent être **inférées** à partir d'achats apparemment anodins ; le RGPD encadre très strictement leur traitement.

6.6.3 Sources de problèmes éthiques

Étape	Risque
Formulation	la question elle-même est discriminatoire ou intrusive
Collecte	absence de consentement ; données non représentatives (biais de sélection)
Préparation	variables qui servent de **proxy** à un attribut protégé (code postal ↔ origine)
Modélisation	modèle qui reproduit ou amplifie les biais historiques ; opacité
Évaluation	performance mesurée globalement mais très inégale selon les groupes
Communication	résultats présentés comme plus certains qu'ils ne le sont
Déploiement	boucle de rétroaction : les décisions du modèle modifient les données futures

6.6.4 Le cadre légal en Europe

- Le **RGPD** (Règlement général sur la protection des données, 2018) s'applique à toute donnée permettant d'identifier une personne, directement ou indirectement. Principes clés : **base légale** du traitement (dont le consentement), **limitation des finalités**, **minimisation** des données, durée de conservation limitée, droits d'accès, de rectification et d'effacement, encadrement des décisions entièrement automatisées. Toute personne peut **s'opposer** à tout moment à l'utilisation de ses données à des fins de prospection commerciale, y compris au profilage lié à cette prospection. En France, la **CNIL** veille à son application ; elle a notamment sanctionné à plusieurs reprises des entreprises pour le dépôt de cookies publicitaires sans consentement valable.
- L'**AI Act** européen (règlement sur l'intelligence artificielle, adopté en 2024 et appliqué progressivement) classe les systèmes d'IA par niveau de risque et impose des obligations renforcées aux systèmes « à haut risque » (recrutement, crédit, éducation…). Il interdit certaines pratiques, comme les techniques de manipulation qui exploitent les vulnérabilités des personnes.

Pseudonymiser n'est pas anonymiser. Remplacer un nom par un identifiant (comme `C0042` dans la boutique) laisse les données personnelles : en croisant âge, code postal et

historique d'achats, on peut souvent réidentifier une personne. Des données pseudonymisées restent soumises au RGPD.

6.6.5 Questions à se poser dans tout projet

- Ai-je le droit d'utiliser ces données pour **cette** finalité ? Les clients ont-ils été informés ?
- Ai-je besoin de **toutes** ces variables (minimisation) ?
- Mes données représentent-elles toutes les populations concernées par la décision ?
- Mon modèle a-t-il des performances comparables selon les groupes (sexe, âge, origine…) ?
- Puis-je **expliquer** une décision à la personne concernée ?
- Qui est responsable si le modèle se trompe ? Une personne peut-elle contester la décision ?

Le cadre PCS du chapitre 1 y contribue : un résultat instable ou non prédictible ne devrait pas servir à prendre des décisions sur des personnes.

6.7 Interfaces graphiques avec R Shiny

6.7.1 Qu'est-ce que Shiny ?

Shiny est un package R qui permet de créer des **applications web interactives** sans écrire de HTML ni de JavaScript : tableaux de bord, outils d'exploration de données, démonstrations de modèles. L'utilisateur modifie des paramètres (curseurs, menus) et les résultats se mettent à jour automatiquement. C'est un moyen efficace de mettre une analyse entre les mains d'une équipe marketing, sans qu'elle ait à écrire de code.

6.7.2 Structure d'une application

Une application Shiny comporte deux parties :

- l'**interface** (`ui`) : la mise en page, les **entrées** (*inputs*) et l'emplacement des **sorties** (*outputs*) ;
- le **serveur** (`server`) : une fonction qui calcule les sorties à partir des entrées.

Entrées (`ui`)	Sorties (`ui`)	Rendu (`server`)
`sliderInput()`	`plotOutput()`	`renderPlot()`
`selectInput()`	`tableOutput()`	`renderTable()`
`numericInput()`	`textOutput()`	`renderText()`
`checkboxInput()`	`verbatimTextOutput()`	`renderPrint()`
`fileInput()`	`DT::DTOutput()`	`DT::renderDT()`

Chaque entrée a un identifiant (`inputId`) que le serveur lit avec `input$identifiant` ; chaque sortie a un identifiant que le serveur remplit avec `output$identifiant <- render...()`.

6.7.3 Un exemple complet : explorer la segmentation des clients

Enregistrez le code suivant dans un fichier `app.R` placé dans le dossier `06-notes`, puis cliquez sur *Run App* dans RStudio (ou exécutez `shiny::runApp("app.R")`).

```
library(shiny)
library(tidyverse)
source("R/boutique.R")
```

```
indicateurs <- indicateurs_clients(lire_boutique("commandes"),
 ↪  lire_boutique("clients"))
variables <- c("recence", "frequence", "montant_total", "panier_moyen",
               "part_promo", "nb_categories", "articles_par_commande",
                ↪  "anciennete")

ui <- fluidPage(
  titlePanel("Segmentation des clients de la boutique"),
  sidebarLayout(
    sidebarPanel(
      sliderInput("k", "Nombre de segments", min = 2, max = 8, value = 4),
      checkboxGroupInput("vars", "Variables de segmentation", choices =
       ↪  variables,
                          selected = variables),
      selectInput("var_x", "Abscisse", variables, selected = "frequence"),
      selectInput("var_y", "Ordonnée", variables, selected = "part_promo")
    ),
    mainPanel(
      plotOutput("nuage"),
      tableOutput("profils")
    )
  )
)

server <- function(input, output) {
  # Expression réactive : recalculée seulement quand k ou les variables
   ↪  changent
  segments <- reactive({
    req(length(input$vars) >= 2)
    set.seed(1)
    kmeans(scale(indicateurs[, input$vars]), centers = input$k, nstart =
     ↪  20)$cluster
  })

  output$nuage <- renderPlot({
    ggplot(indicateurs, aes(.data[[input$var_x]], .data[[input$var_y]],
                            color = factor(segments()))) +
      geom_point(alpha = 0.5) +
      labs(color = "Segment") +
      theme_minimal()
  })

  output$profils <- renderTable({
    indicateurs |>
      mutate(segment = segments()) |>
      group_by(segment) |>
      summarise(clients = n(), across(all_of(variables), mean))
  })
}

shinyApp(ui = ui, server = server)
```

6.7.4 La réactivité

Shiny recalcule une sortie **seulement** quand une entrée dont elle dépend change. Une expression `reactive()` sert à calculer une seule fois un résultat partagé par plusieurs sorties : ici, la segmentation est utilisée par le graphique et par le tableau, mais n'est recalculée que lorsque `k` ou les variables changent, pas lorsqu'on change d'axe sur le graphique. `req()` interrompt le calcul tant qu'une condition n'est pas remplie (au moins deux variables cochées).

6.7.5 Partager une application

- en local : `shiny::runApp()` ;
- en ligne : shinyapps.io (offre gratuite limitée) via le package `rsconnect`, ou un serveur Shiny Server / Posit Connect de l'entreprise. Attention : publier une application qui contient des données clients revient à les diffuser.

Pour aller plus loin, la galerie shiny.posit.co/r/gallery et le livre *Mastering Shiny* de H. Wickham (mastering-shiny.org) présentent de nombreux exemples.

6.8 Résumé

- Classification **non supervisée** : former des groupes homogènes sans variable cible ; en marketing, c'est la **segmentation** ; **standardiser** avant de calculer des distances.
- **k-moyennes** : `kmeans(x, centers = k, nstart = 25)` ; minimise l'inertie intra-classe ; dépend de l'initialisation.
- **CAH** : `hclust(dist(x), method = "ward.D2")`, dendrogramme, `cutree()`.
- Choisir k : coude, silhouette, saut d'inertie, interprétabilité et utilité métier.
- Décrire les classes : moyennes par classe, variables illustratives, `HCPC()`, `catdes()`, parangons ; les **nommer**.
- **Éthique** : consentement, minimisation, *proxies*, équité entre groupes, explicabilité ; RGPD (dont le droit d'opposition à la prospection) et AI Act.
- **Shiny** : `ui` (entrées, emplacements des sorties) + `server` (`render*()`) ; `reactive()` pour les calculs partagés.

6.9 Exercices

Exercice 6.1 — Segmentation RFM simplifiée.

1. Segmenter les clients avec les seules variables `recence`, `frequence` et `montant_total`, après les avoir passées au logarithme (`log1p()`) puis standardisées.
2. Comparer, pour $k = 3$ à 6, la silhouette moyenne.
3. Décrire et nommer les classes obtenues pour le k retenu.
4. Comparer cette segmentation à celle du cours (tableau croisé). Les classes sont-elles stables quand on change les variables ?

Solution

```
x_rfm <- indicateurs |>
  transmute(recence = log1p(recence), frequence = log1p(frequence),
            montant_total = log1p(montant_total)) |>
  scale()

sapply(3:6, function(k) {
  set.seed(1)
  cl <- kmeans(x_rfm, centers = k, nstart = 25)$cluster
  mean(silhouette(cl, dist(x_rfm))[, "sil_width"])
})
```

```
#> [1] 0.3761 0.3347 0.3502 0.3508
```

```
set.seed(1)
km_rfm <- kmeans(x_rfm, centers = 4, nstart = 25)
indicateurs |>
  mutate(classe_rfm = km_rfm$cluster) |>
  group_by(classe_rfm) |>
  summarise(n = n(), recence = median(recence), frequence =
    ↪  median(frequence),
            montant = median(montant_total))
```

```
#> # A tibble: 4 x 5
#>   classe_rfm     n recence frequence montant
#>        <int> <int>   <dbl>     <dbl>   <dbl>
#> 1          1   514    13           6   228.
#> 2          2   359   127           1    56.5
#> 3          3   390    11.5        11  1019.
#> 4          4   478    79.5         3   151.
```

```
table(rfm = km_rfm$cluster, cours = km$cluster)
```

```
#>    cours
#> rfm   1   2   3   4
#>   1  12 388  13 101
#>   2   0  27 211 121
#>   3 354  35   0   1
#>   4   4 146 199 129
```

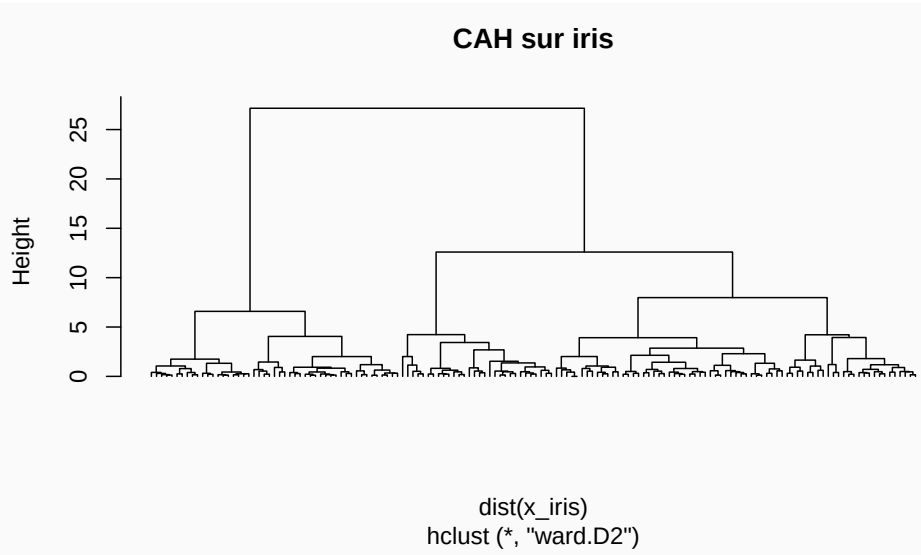
La segmentation RFM retrouve bien les clients à forte valeur, mais distingue moins bien les comportements promotionnels, absents de ses variables : le **choix des variables** détermine les segments que l'on peut trouver.

Exercice 6.2 — Classification de `iris`. On oublie la variable `Species`.

1. Réaliser une CAH (Ward) sur les quatre mesures standardisées et tracer le dendrogramme. Combien de classes suggère-t-il ?
2. Réaliser une partition en 3 classes avec les k-moyennes et la croiser avec `Species`. Quelles espèces sont confondues ?

Solution

```
x_iris <- scale(iris[, 1:4])
cah_iris <- hclust(dist(x_iris), method = "ward.D2")
plot(cah_iris, labels = FALSE, hang = -1, main = "CAH sur iris")
```




```
set.seed(1)
km_iris <- kmeans(x_iris, centers = 3, nstart = 25)
table(classe = km_iris$cluster, espece = iris$Species)
```

```
#>       especе
#> classe setosa versicolor virginica
#>      1      0         39        14
#>      2      0         11        36
#>      3     50          0         0
```

setosa est parfaitement isolée ; *versicolor* et *virginica* se recouvrent en partie.

Exercice 6.3 — Éthique. Une enseigne veut utiliser la segmentation de ses clients, leur code postal et leur âge pour proposer des **prix personnalisés** sur son site.

1. Quels risques éthiques et légaux identifiez-vous ?
2. Quelles questions poseriez-vous avant de commencer ?
3. Quelles précautions techniques et organisationnelles proposeriez-vous ?

Exercice 6.4 — Shiny. Modifier l'application d'exemple pour :

1. afficher la silhouette moyenne de la partition ;
2. ajouter un bouton de téléchargement de la liste des clients d'un segment choisi (`downloadButton()` et `downloadHandler()`) ;
3. ajouter un onglet (`tabsetPanel()`) montrant le chiffre d'affaires mensuel par segment.

Pour aller plus loin

- F. Husson, S. Lê et J. Pagès, *Analyse de données avec R*, chapitre « Classification ».
- C. O'Neil, *Algorithmes : la bombe à retardement*, Les Arènes, 2018 (traduction de *Weapons of Math Destruction*).
- CNIL, recommandations sur la prospection commerciale et les cookies : cnil.fr.
- H. Wickham, *Mastering Shiny*, O'Reilly, 2021.

Chapitre 7

Projet et évaluation des acquis

La dernière séance est consacrée à la **présentation des projets** et à l'**évaluation des acquis**. Le projet compte pour 50 % de la note du module : c'est l'occasion de mettre en œuvre l'ensemble du cycle de vie de la science des données sur un vrai jeu de données (AAV7).

Ce chapitre propose une **méthode de travail** pour le projet et une **fiche de révision**. Les modalités précises (sujet, constitution des groupes, dates et format de rendu, barème) sont communiquées par l'enseignant.

7.1 Déroulé d'un projet : le DSLC en pratique

Étape	Questions à se poser	Outils du cours
1. Formulation	Quelle est la question ? Pour qui ? Quelle décision éclairera-t-elle ? Les données permettent-elles d'y répondre ?	chapitre 1
2. Données	D'où viennent-elles ? Que représente chaque ligne, chaque variable ? Y a-t-il des valeurs manquantes, aberrantes, des doublons ?	`read_csv()`, `glimpse()`, `summary()`, `count()`
3. Découpage	Faut-il un jeu de test ? Quel découpage (aléatoire, temporel, par groupe) ?	`sample()`, `set.seed()`
4. Nettoyage	Quelles transformations ? Lesquelles sont des **choix** à documenter ?	`dplyr`, `tidyr`
5. Exploration	Distributions, relations entre variables, premiers constats	analyses uni- et bivariées, `ggplot2`
6. Structure	Peut-on résumer les variables ? Existe-t-il des groupes d'individus ?	`PCA()`, `kmeans()`, `hclust()`, `HCPC()`
7. Modélisation	Quelle méthode, et pourquoi ? Quel modèle de référence simple ?	`lm()`, `glm()`, `rpart()`, `randomForest()`
8. Évaluation PCS	Le résultat tient-il sur le jeu de test ? Avec d'autres choix de nettoyage ? Une autre graine ?	RMSE, matrice de confusion, ROC/AUC
9. Communication	Quel message principal ? Quelles limites ? Quelles recommandations ?	Quarto / R Markdown, graphiques soignés, Shiny

> Commencez toujours par un **modèle de référence** très simple (prédire la moyenne, ou la classe majoritaire). Un modèle complexe n'a d'intérêt que s'il fait nettement mieux que cette référence.

7.2 Le rendu : un notebook reproductible

Le rapport doit pouvoir être **régénéré entièrement** à partir des données brutes en cliquant sur *Render* (Quarto) ou *Knit* (R Markdown). Une structure possible, en Quarto :

````
---
title: "Titre qui énonce la question"
author: "Noms des membres du groupe"
date: today
format:
  html:
    toc: true
    code-fold: true
---

```{r}
#| include: false
library(tidyverse)
source("R/fonctions.R")
```

# Résumé
Trois à cinq phrases : question, données, méthode, résultat principal, limite
↪ principale.

# 1. Contexte et question
# 2. Données : origine, signification des variables, qualité
# 3. Préparation : choix de nettoyage et leur justification
# 4. Exploration
# 5. Modélisation / analyse
# 6. Évaluation : prédictibilité et stabilité
# 7. Conclusions, limites et recommandations
# Annexe : sessionInfo()
````

7.3 Grille d'auto-évaluation

Avant de rendre votre projet, vérifiez chaque point.

Question et données

- ☐ La question est précise et liée à une décision réelle.
- ☐ L'origine des données, leur mode de collecte et la signification de chaque variable sont décrits.
- ☐ Les limites des données (représentativité, valeurs manquantes) sont discutées.
- ☐ Les questions éthiques et légales ont été examinées (données personnelles ? biais possibles ?).

Code

- ☐ Le notebook s'exécute de bout en bout sur un autre ordinateur (projet RStudio, chemins relatifs, packages listés).
- ☐ Les données brutes ne sont jamais modifiées à la main.
- ☐ Le code répété est factorisé en fonctions aux noms explicites.
- ☐ Les tirages aléatoires sont reproductibles (`set.seed()`).

Analyse

- ☐ Chaque méthode est justifiée par rapport à la question et au type des variables.
- ☐ Les graphiques ont un titre, des axes nommés avec unités et une légende lisible.
- ☐ Les performances sont mesurées sur des données **non utilisées** pour l'apprentissage, et comparées à une référence simple.
- ☐ Au moins une vérification de **stabilité** est présentée (autre graine, autre traitement des valeurs manquantes, autre méthode).

Communication

- ☐ Le message principal tient en une phrase.
- ☐ Les résultats sont interprétés dans les termes du domaine (en marketing : clients, ventes, campagnes), pas seulement en chiffres.
- ☐ Les limites et l'incertitude sont énoncées honnêtement.
- ☐ Les recommandations sont concrètes et réalisables.

7.4 Réussir la soutenance

- Commencer par la **question** et le **message principal**, pas par le code.
- Une idée par diapositive ; des graphiques lisibles de loin.
- Montrer le cheminement, y compris les impasses et les choix difficiles.
- Prévoir une démonstration courte (par exemple une application Shiny) si elle apporte quelque chose.
- Préparer les questions : « Pourquoi cette méthode ? », « Ce résultat est-il stable ? », « Que feriez-vous avec plus de temps ou de données ? ».

7.5 Où trouver des jeux de données

Marketing et commerce

- *Bank Marketing* (UCI) : résultats de campagnes de télémarketing d'une banque portugaise ;
- *Online Retail II* (UCI) : deux ans de transactions d'un site de vente en ligne britannique, idéal pour une analyse RFM et une segmentation ;
- *Telco Customer Churn* (IBM, disponible sur Kaggle) : résiliation d'abonnés d'un opérateur ;
- les jeux du package `ISLR` utilisés dans ces notes (`Carseats`, `OJ`, `Caravan`) et la boutique simulée du cours (`datasets`).

Données publiques

- data.gouv.fr et INSEE : données publiques françaises ;
- Eurostat : statistiques européennes ;
- Kaggle Datasets et UCI Machine Learning Repository.

Vérifiez toujours la **licence** et la **documentation** d'un jeu de données avant de l'utiliser.

7.6 Fiche de révision

7.6.1 Le langage R (chapitre 2)

Besoin	Code
assigner	`x <- 3`
type, structure	`typeof(x)`, `class(x)`, `str(x)`
vecteur, facteur	`c(1, 2)`, `factor(c("a", "b"))`
liste, data frame	`list(a = 1, b = "x")`, `data.frame(x = 1:3)`
indexer	`x[2]`, `x[x > 0]`, `df[i, j]`, `df$col`, `liste[[1]]`
condition	`if (cond) { } else { }`, `ifelse(cond, a, b)`
boucle	`for (i in seq_along(x)) { }`, `while (cond) { }`
fonction	`f <- function(x, y = 1) { ... }`
valeurs manquantes	`is.na(x)`, `mean(x, na.rm = TRUE)`
packages	`install.packages("p")`, `library(p)`, `p::f()`

7.6.2 Manipulation (chapitre 3)

Besoin	Code
lire, écrire	`read_csv()`, `read_csv2()`, `read_excel()`, `write_csv()`, `saveRDS()`
décrire	`summary()`, `quantile()`, `table()`, `freq()`
croiser	`table(x, y)`, `lprop()`, `cprop()`, `chisq.test()`, `t.test()`, `cor()`
lignes	`filter()`, `slice_max()`, `arrange(desc())`, `distinct()`
colonnes	`select()`, `rename()`, `mutate()`, `case_when()`
résumer	`group_by() |> summarise()`, `count()`
joindre	`left_join(x, y, by = c("a" = "b"))`, `anti_join()`
remodeler	`pivot_longer()`, `pivot_wider()`
générer	`seq()`, `rep()`, `rnorm()`, `sample()`, `set.seed()`

7.6.3 Graphiques et régression (chapitre 4)

Besoin	Code
base R	`plot()`, `hist()`, `boxplot()`, `barplot()`
ggplot2	`ggplot(df) + geom_point(aes(x, y, color = g)) + labs()`
barres	`geom_bar(position = "fill")`, `geom_col()`
petits multiples	`facet_wrap(vars(g))`

Besoin	Code
régression linéaire	`lm(y ~ x1 + x2, data = df)`, `summary()`, `predict()`
régression logistique	`glm(y ~ ., data = df, family = binomial)`, `exp(coef())`

7.6.4 ACP, arbres et forêts aléatoires (chapitre 5)

Besoin	Code
ACP	`PCA(df, quanti.sup = , quali.sup = , graph = FALSE)`
graphiques ACP	`fviz_eig()`, `fviz_pca_var()`, `fviz_pca_ind()`
décrire les axes	`dimdesc()`
arbre	`rpart(y ~ ., data, method = "class")`, `printcp()`, `prune()`
forêt aléatoire	`randomForest(y ~ ., data, ntree = 500, mtry = , importance = TRUE)`
interpréter une forêt	`varImpPlot()`, `partialPlot()`
évaluer	`table(observe, predit)`, `roc()`, `auc()`

7.6.5 Classification (chapitre 6)

Besoin	Code
standardiser	`scale(df)`
k-moyennes	`kmeans(x, centers = k, nstart = 25)`
CAH	`hclust(dist(x), method = "ward.D2")`, `cutree(cah, k)`
choisir k	`fviz_nbclust(x, kmeans, method = "silhouette")`
ACP + CAH	`HCPC(res_acp, nb.clust = -1)`
Shiny	`ui <- fluidPage(...)`, `server <- function(input, output) {...}`

7.7 Questions de révision

1. Pourquoi un résultat issu de données est-il une preuve plutôt qu'une démonstration ?
2. Quelle est la différence entre les jeux de validation et de test ? Pourquoi découper avant d'explorer ?
3. Quel découpage choisir pour prédire les ventes du mois prochain ? Pour prédire le comportement des clients d'un nouveau magasin ?
4. Donnez un exemple de fuite de données dans un projet de prédiction de l'attrition.
5. Quelle différence entre `&` et `&&` ? Entre `[ ]` et `[[ ]]` sur une liste ?
6. Pourquoi regarder à la fois la moyenne et la médiane d'un montant d'achat ?
7. Dans un test A/B, que conclure d'une différence non significative ?

8. Que fait `left_join()` si la clé n'est pas unique dans la seconde table ? Quelle jointure permet de trouver les clients sans commande ?
9. Qu'est-ce qu'un jeu de données *tidy* ?
10. Quand une variable va-t-elle dans `aes()`, et quand hors de `aes()` ?
11. Comment interpréter un coefficient de régression multiple ? Pourquoi peut-il changer quand on ajoute une variable ? Comment interpréter un rapport de cotes ?
12. Pourquoi réduire les variables avant une ACP ou un calcul de distances ?
13. Comment lire l'angle entre deux flèches du cercle des corrélations ? À quoi sert une variable illustrative ?
14. Pourquoi élaguer un arbre de décision ? En quoi une forêt aléatoire diffère-t-elle d'un arbre, et que mesure son erreur OOB ?
15. Pourquoi le taux de bonne classification est-il trompeur sur des classes déséquilibrées ? Que représente l'AUC ?
16. Quelles différences entre k-moyennes et CAH ? Comment choisir le nombre de classes d'une segmentation ?
17. Citez trois risques éthiques d'un projet de ciblage marketing et une précaution pour chacun.
18. Quel est le rôle de `reactive()` dans une application Shiny ?